

强化学习：从算法到基础模型

Reinforcement Learning: From Algorithms To Foundation Models

作者：Zihan Ding - arXiv:2607.17560v1 - 2026 年 7 月

本 PDF 是基于公开 arXiv 源文件的中文机器翻译阅读版。正文页序与源文档对齐，数学表达式按文本抽取结果尽量保留，图表以占位标记表示；页 290-319
的参考文献保留英文。该稿已完成自动页标记、文本完整性和渲染检查，但未经过人工逐句学术校对。

源文件：2607.17560v1.pdf (319 页)

原文第 1 页

强化学习:来自 基础模型的算法

丁子涵

arXiv:2607.17560v1 [cs.AI] 2026 年 7 月 20 日 一篇论文 提交给学院 普林斯顿大学 学位候选人 哲学博士

推荐验收 由该部 电气计算机工程 指导老师:池进教授

2026年5月

原文第 2 页

(C) 丁子涵版权所有,2026。 保留所有权利。

摘要

强化学习 (RL) 为顺序决策提供了通用框架 在明确的目标下。在其经典形式中,强化学习研究智能体应该如何行动 在动态环境中最大化长期回报。然而,在更丰富的环境中, 问题超出了单一代理和固定环境:智能行为 可能需要与其他主体进行战略互动、适应不确定性以及 对复杂、高维世界进行推理的能力。本论文研究强化 从两个互补的角度学习:游戏中的强化学习算法和游戏中的强化学习 基础模型时代。第一部分重点介绍游戏中的多智能体强化学习。它检查 激励、政策和均衡概念如何在竞争性和一般性方面相互作用 和环境,涵盖规范的两人零和游戏、大型视频游戏 领域,以及具有更一般战略结构的多人游戏设置。在一起,这些 作品研究了多智能体系统中学习的算法基础和 强化学习方法在复杂交互环境中的实际行为。第二部分研究具有生成和基础模型的强化学习 els,其动机是广泛的先验知识可以极大地丰富序列 决策。在这种情况下,预训练的生成模型和学习的世界模型 不仅可以作为表示工具,还可以作为规划、控制、 和政策优化。该论文开发了基于扩散的世界模型,研究了 用于高效视频生成的强化学习,探索表达生成模式 EL 作为政策课程,并研究互动视频世界模型,其中行动塑造未来的观察。它通过架构进一步解决了长视野顺序建模的问题。 将记忆融入世界建模的结构。总而言之,这些贡献提出了 强化学习的统一观点 作为复杂序列领域中目标驱动适应的研究。从策略- 从 gic 游戏到生成世界模型,论文强调了强化学习如何 连接算法决策、环境建模和新兴功能

大型基础模型的有效性,提供了更广泛的原理视角 潜在的智能行为。

致谢

普林斯顿是一个既深邃又优雅的地方----外表安静,但无尽的思想丰富。这是一个为学习、为反思、为慢慢塑造的地方一个人的性格。四年来,见证的不仅仅是我的博士学位。日记- 不仅如此,也是一个远远超出学术范畴的成长时期。是什么让这次旅程真正一路上遇到的人都很有意义。和他们一起,我不仅分享了 研究的需求,还有对话、友谊和生活本身。他们的支持和陪伴支撑着我走过了这条漫长且充满挑战的道路。我也是 深深感谢电气与计算机系的行政人员 工程和整个普林斯顿校园。他们用耐心、善良和关怀,多年来默默地支持我们,就像忠实的园丁照料土地一样 其他人可能会成长。我对普林斯顿最珍贵的记忆之一是在周围度过的夜晚 餐桌,吃饭既是休息的场所,也是思考的场所。我们 互相带来谜语、脑筋急转弯、半成形的想法和突然的好奇心, 最初的随意聚会常常演变成活跃的思想交流和交流 想象力。那些时刻提醒我,智力生活不仅仅属于 办公室、教室或报纸----它也存在友谊、笑声和谈话中, 在共同思考的简单快乐中。 谨向我的导师迟进教授致以最深的谢意,是我博士期间最重要的导师。旅行。他与我们一起度过每一个 研究阶段:从最早的头脑风暴,到实验设计、项目 组织,以及帮助我们完成提交的多轮论文润色 最后期限。从他那里,我了解到研究中真正的严谨意味着什么----不仅在广泛的领域 愿景,但体现在每个细节的精确度,直至单个参数的值 或手稿中单个单词的选择。他为我打下了坚实的基础 强化学习和优化,同时不断开辟有希望的方向 用于探索。更重要的是,他教会了我如何辨别好的味道 研究以及如何判断问题的重要性。在他的指导下,我的路

思维能力从理论理解发展为更广泛的研究能力 包括概念深度和经验判断。同时,他给了我 独立探索的自由,并成长为我自己的研究风格。他的 乐观、慷慨和善良也塑造了我们集团持久的精神 方式。我无法完全表达我对他的指导和支持的感激之情。我也深深感谢我的论文承诺的其他成员 - T 恤---Benjamin Eysenbach 教授、Pramod Viswanath 教授和教授 王梦迪.他们每个人都是各自领域的顶尖学者,我有 从他们的工作、见解和反馈中学到了很多东西。我感觉很 很幸运在整个论文过程中得到他们的支持和指导 过程。我还要感谢我的好朋友:刘清华、小马、周露、王元浩、Ahmed Khaled、李文哲、Seth Karten、陈一晓、聂雨绮、Andy 苏,任艾伦,郑宇恒,金可欣,李勤勤南,李安建,尚正元,唐亚谦、王首达、刘新浩、林玉斌、林勇、吉向、张子旭、Cindy Wu、张强、王佳辰、黄凯旋、蔡天乐、Eshaan Nichani、王宏杰、吴童、高成、侯继凯、倪成卓、刘苏林、查立涵、梁凯曲、赵雨来、韩北宁、唐雅倩、刘雨辰、袁志扬、浩宇 吴、赵浩宇、杨吉米、王润哲、李一凡、吴越、张学舟、朱晓楠,石成帅,梁欣然,杨子然,严沁欣。谢谢你 为了我们在普林斯顿度过的时光,为了你们的友谊,为了我们的许多谈话, 交流、互相鼓励的时刻让这些年变得更加丰富 否则他们就会这样。最后,也是最重要的,我要感谢我的父母。谢谢你养育我,塑造我,用坚定不移的爱支持我。你的影响力已经存在 我所做的一切,你们的支持是我每一步的基础这次旅程。无论我取得什么成就,我首先要归功于你们。

原文第 7 页

人类最大的奥秘不在于智力,但在人性和意识中。

摘要。 3 致谢。

5

我排除16

1 强化学习简介 17 1.1 AGI时代的强化学习。 17 号 1.2 单智能体强
化学习。 19 1.2.1 马尔可夫决策过程。 21 1.2.2 策略和价值函数。 28 1.2.3 值迭代。 32 1.2.4 政策
梯度。 34 1.3 博弈论多智能体强化学习。 38 1.3.1 零和博弈。 41 1.3.2 马尔可夫博弈。 43 1.3.3 二人零和马尔可夫博弈。 46 1.3.4 马尔可夫势博弈。 48

2 组织 41

II 游戏中的强化学习 43

3 两人零和博弈 44 3.1 简介。 46 3.2 相关作品。 48

3.3 预备工作。	49
3.4 方法论。	52
3.4.1 纳什-DQN。	52
3.4.2 纳什-DQN-利用者。	56
3.4.3 理论论证。	56
3.5 实验。	57
3.5.1 基线。	57
3.5.2 表格马尔可夫博弈。	58
3.5.3 两人视频游戏。	60
3.6 范式博弈纳什求解器的比较。	63
3.6.1 乘法权重更新。	63
3.6.2 比较。	66
3.7 表格马尔可夫游戏算法。	66
3.7.1 纳什算法子程序。	68
3.7.2 纳什 Q 学习。	68
3.7.3 自玩。	71
3.7.4 虚构的自我对弈。	71
3.7.5 双神谕。	72
3.7.6 纳什-VI。	72
3.7.7 纳什-VI-剥削者。	73
3.7.8 纳什级数算法的比较。	75
3.7.9 定理30的证明。	75
4 零和视频游戏	76
4.1 简介。	76
4.2 相关工作。	79
4.3 多智能体强化学习。	81
4.4 格斗天梯。	82
4.4.1 场景。	83
4.4.2 状态和观察。	83

4.4.3 动作空间。	84
4.4.4 奖励。	86
4.4.5 特点。	86
4.5 评估指标。	88
4.6 FightLadder-基线。	90
4.7 结果。	90
4.7.1 单人完整视频游戏。	91
4.7.2 两人基线算法的性能。	95
4.7.3 训练有素的代理的不可剥削性。	97
4.8 格斗天梯详情。	97
4.8.1 密集奖励。	97
4.8.2 游戏设置。	97
4.8.3 MARL游戏平台比较。	97
4.9 FightLadder的基线算法-Baselines。	98
4.9.1 培训细节。	100
4.10 个人 Elo 结果。	100
5 多人一般和游戏	106
5.1 简介。	108
5.2 相关工作。	111
5.3 方法。	113
5.3.1 问题描述。	114
5.3.2 分配公平性。	115
5.3.3 博弈论框架。	115
5.3.4 分布式方法。	119
5.4 评价。	123
5.5 随机马尔可夫模型。	123
5.6 分配公平性分析。	124

III 基础模型中的强化学习 129

6 扩散世界模型	130	6.1 简介。	130	6.2 相关工作。	134	6.3 预备知识。	137	6.4 扩散世界模型。	138	6.4.1 条件扩散模型。	140	6.4.2 具有扩散世界模型的基于模型的强化学习。	140	6.5 实验。	142	6.5.1 DWM 与 DWM 一步动力学模型。	144	6.5.2 DWM 与 DWM 决策扩散器。	145	6.5.3 DWM 与 DWM 无模型对应物。	147	6.5.4 消融研究。	147	6.6 实施细节。	148	6.6.1 扩散世界模型。	151	6.6.2 一步动力学模型。	153	6.6.3 基于扩散世界模型的离线强化学习方法。	154	6.7.1 DWM-TD3BC: 带有扩散世界模型的 TD3+BC。	155	6.7.2 DWM-IQL: 具有扩散世界模型的 IQL。	157	6.7.3 DWM-PQL: 具有扩散世界模型的悲观 Q 学习。	159	6.8 离线强化学习算法的训练和评估细节。	161	6.8.1 常用设置。	162	6.8.2 无模型算法。	159	6.8.3 基于模型的算法。	161	6.9 附加实验。	162	6.9.1 使用 DWM 进行长期规划的详细结果。	161	6.9.2 世界建模: 预测误差分析。	162	6.9.3 基于 Transformer 的世界模型。	162	6.9.4 附加基线。	163
----------	-----	---------	-----	-----------	-----	-----------	-----	-------------	-----	---------------	-----	---------------------------	-----	---------	-----	--------------------------	-----	------------------------	-----	-------------------------	-----	-------------	-----	-----------	-----	---------------	-----	----------------	-----	--------------------------	-----	------------------------------------	-----	-------------------------------	-----	----------------------------------	-----	-----------------------	-----	-------------	-----	--------------	-----	----------------	-----	-----------	-----	---------------------------	-----	---------------------	-----	-----------------------------	-----	-------------	-----

6.9.5 附加环境:稀疏奖励任务。	165
6.9.6 消融:训练和推理的扩散步骤数。	166
6.9.7 消融:扩散世界模型的序列长度。	170
6.9.8 消融:OOD 评估 RTG 值。	171
6.9.9 消融:A-返回值估计。	174
6.9.10 消融:RTG 重新标记和模型微调。	176
7 一致性模型作为强化学习策略	178
7.1 简介。	178
7.2 相关作品。	180
7.3 预备知识。	182
7.3.1 离线和在线强化学习。	182
7.3.2 一致性模型。	182
7.4 一致性模型作为强化学习策略。	183
7.5 实验评估。	186
7.5.1 离线强化学习:行为克隆。	187
7.5.2 离线强化学习:一致性 Actor-Critic。	189
7.5.3 离线到在线和在线强化学习。	192
7.6 一致性模型训练和推理细节。	195
7.7 离线强化学习实验详细信息。	196
7.7.1 计算时间。	196
7.7.2 消融研究。	200
7.8 离线到在线和在线 RL 详细信息。	200
7.8.1 算法。	200
7.8.2 计算时间。	201
8 少步视频生成中的强化学习	203
8.1 简介。	203
8.2 相关工作。	206
8.3 方法论。	209

8.3.1 扩散模型。	209	8.3.2 浓度蒸馏。	
210	8.3.3 变分分数蒸馏。		
211	8.3.4 潜在奖励微调。	212	8.4 多目标蒸馏。
214	8.5 实验。		
215	8.5.1 实施。		
216	8.5.2 与 SOTA 方法和模型的比较。	216	8.5.3 与像素空间奖励的比较。
220	8.5.4 蒸馏方法的消融。	221	8.5.5 消融研究。
222	8.6 美元方法详情。		
223	8.6.1 伪代码。		
224	8.6.2 扩散模型训练和推理。	224	8.6.3 实施细节。
225	8.6.4 学生-教师参数化。		
226	8.6.5 推导。	227	8.6.6 推理时间分析。
229	8.7 奖励模型微调。		
230	8.7.1 微调效果的证据。	230	8.7.2 直接奖励梯度。
230	8.7.3 不同奖励类型的潜在奖励模型。		
231	8.7.4 潜在奖励模型训练。	234	8.7.5 潜在奖励模型微调。
235	8.7.6 去噪扩散策略优化。	236	
9	视频世界模型	239	9.1 简介。
239	9.2 相关作品。	241	9.3 方法论。
243			

9.3.1 初步:潜视频扩散模型。 243 9.3.2 交互式长视频生成。 246 9.3.3 检索具有全局状态的增强视频世界模型。 248 9.3.4 长上下文扩展基线。 250 9.4 实验。 250 9.4.1 数据集和评估协议。 251 9.4.2 培训细节。 251 9.4.3 世界一致性结果。 252 9.4.4 复合误差结果。 253 9.4.5 VBench 评估。 254 9.4.6 扩展:真实世界设置。 255 9.4.7 消融:VRAG 的记忆和训练。 257 9.5 基线方法详细信息。 260 9.6 附加实验。 260 9.6.1 复合误差评估指标分析。 260 9.6.2 Vanilla 长上下文扩展与 YaRN。 262 9.6.3 对主要结果的更多讨论。 264 9.6.4 预测的全局状态。 264 9.6.5 内存和时间开销。 265 10 世界模型记忆 267 10.1 简介。 267 10.2 相关工作。 269 10.3 预备知识。 271 10.3.1 循环神经网络。 271 10.3.2 视频扩散模型。 272 10.4 方法论。 273 10.4.1 循环自回归扩散。 274 10.4.2 逐块和逐帧自回归。 276 10.4.3 并行注意力的隐藏状态预取。 277

10.5 实验。 279 10.5.1 数据集和
评估协议。 279 10.5.2 迷宫结果。 280 10.5.3 我的世界结果。 280 10.5.4 VBench 结果。 284 10.5.5
计算资源分析。 284 10.6 消融研究。 285 10.6.1 内存帧的噪声水平。 286 10.6.2 跨步分块自回归。 286 10.6.3 动作条件设计。 287 10.7 逐帧
RNN 中注意力的高效并行化。 288

参考书目 290

原文第 16 页

第一部分

排除

第一章

强化介绍 学习

1.1 AGI时代的强化学习

强化学习(RL)为设计提供了通用的数学框架 激励驱动的决策系统。这种观点对于 通用人工智能(AGI),智能系统必须随着时间的推移而行动 明确目标,适应不断变化的环境,并与复杂的环境相协调 环境。在概念层面上,强化学习也与人类生理学研究密切相关。 生物学和神经科学,多巴胺相关信号通常被解释为携带 有关奖励、动机和学习的信息。这些联系表明 基于激励的适应不仅是一个工程原理,也是一个基本原理 智能行为机制。RL 的中心思想是指定智能体应该想要什么,而不是预先指定智能体想要什么。 准确地描述它应该如何表现。代理期望的结果被编码为 quan- 最好是通过奖励函数来评估行为的后果。代理 本身遵循一个决策规则,称为策略,学习包括改进 这项政策的目的是为了最佳地实现由重新制定的目标 病房。在最简单的单代理环境中,最优性通常意味着最大化预期

世界模型 P, R

更新规划

代理在环境中的行动

政策 $\pi(a|s)$ 在 s 下, 奖励 $R(s,a)$

下一个状态 s' , 奖励 $R(s,a)$ 经验

图 1.1: 通用强化学习系统: 代理与环境交互

环境 优化其奖励最大化的策略, 同时可选地使用学习的 用于预测动态和奖励、支持规划和改进政策的世界模型 优化。

累积奖励。然而, 在更丰富的多代理环境中, 相关概念 最优性可以采用纳什均衡或其他均衡概念的形式, 取决于不同主体如何相互作用以及激励如何耦合。 尽管奖励函数可能很简单, 但从中得出的策略可以是 非常重要。这种复杂性很大程度上是由问题的动态性质驱动的: 行动影响未来状态, 环境可能随机演变, 并且在多智能体中 设置其他代理的行为也会影响决策环境。结果, 环境建模成为一个中心问题。在经典强化学习中, 这是通过 状态转换模型, 描述系统如何响应动作而演化 和不确定性。基础模型的最新进展为强化学习开辟了新的方向 将广泛的先验知识注入决策过程。而不是学习---- 完全从头开始, 代理可以利用预训练的表示和结构化的表示 了解世界, 以便更明智地探索并优化其政策 更多样本 有效且具有更好的泛化能力。从这个角度来看, 世界建模可以

被视为通过基础模型的功能丰富的环境建模。一个强大的世界模型可以支持规划、提高样本效率并更好地协调运营与代理人的目标同步。同时,这些好处关键取决于世界模型本身的准确性和计算效率,这使得设计适当的架构和算法是一个核心挑战。

1.2 单智能体强化学习

通过反复试验,强化学习代理尝试找到最优策略来最大化它的长期回报。该过程由马尔可夫决策过程制定。

1.2.1 马尔可夫决策过程

定义 1(马尔可夫决策过程)。一个 MDP 可以用一个 key 元组来描述

元素 $\mathbf{x}, \mathbf{S}, \mathbf{A}, \mathbf{P}, \mathbf{R}, \gamma$:

* $\mathbf{S}$: 环境状态集。

* $\mathbf{A}$: 代理可能采取的行动的集合。

* $\mathbf{P}: \mathbf{S} \times \mathbf{A} \times \mathbf{S} \rightarrow \Delta(\mathbf{S})$: 对于每个时间步 $t \in \mathbb{N}$, 给定代理的动作 $a \in \mathbf{A}$,

从状态 $s \in \mathbf{S}$ 到下一个时间步长 $s' \in \mathbf{S}$ 的状态的转移概率。* $\mathbf{R}: \mathbf{S} \times \mathbf{A} \times \mathbf{S} \rightarrow \mathbb{R}$: the reward function that returns a scalar value to the agent for 由于动作 a 而从 s 转移到 s' 。奖励具有绝对价值 一致受 $R_{\max}$ 限制。

* $\gamma \in [0, 1]$ 是代表时间价值的折扣因子。

在每个时间步 t , 环境都有一个状态 s_t 。学习代理观察这个 state 并在 s_t 处执行一个动作。该动作使环境转变为

下一个状态 $s_{t+1} \in \mathbf{S}$, 在 s_{t+1} 处, 新环境返回立即奖励

如果智能体不能完全观察环境状态, 那么它就变成部分可观察的 设置, 缩写为 POMDP。

at s_t to the agent. The reward function can be also written as $R : S \times A \rightarrow \mathbb{R}$,

which is interchangeable with $R : S \times A \times S \rightarrow \mathbb{R}$.

1.2.2 策略和价值函数

鉴于上述 MDP 设置,智能体的目标是解决 MDP:找到最优的 随着时间的推移最大化奖励的政策。从数学上讲,一个共同的目标是 让代理找到马尔可夫(即输入仅取决于当前状态)并且

平稳(即函数形式与时间无关)策略函数 $\pi : S \rightarrow \Delta(A)$,其中

$\Delta(A)$ 表示概率单纯形。

形式上,马尔可夫策略仅取决于当前状态,可以写为 $\pi : S \rightarrow \Delta(A)$ (1.1)

对于离散随机情况或 $\pi : S \rightarrow A$ 在确定性情况下。相比之下,一般或非马尔可夫政策可能取决于时间 h 可用的整个历史。如果我们

将历史表示为 $H_h = \{s_1, a_1, r_1, \dots, s_h, a_h, r_h, s_{h+1}\}$, (1.2)

那么一般策略就是一个映射 $\pi_h : H_h \rightarrow \Delta(A)$, (1.3)

或等效 $\pi_h(a_h | H_h)$ 为随机策略。

只要转移函数和奖励存在,最优马尔可夫策略就存在

函数都是马尔可夫且平稳的,因此默认情况下我们使用 π 来表示马尔可夫政策没有进一步明确。

该策略采取连续的行动,使得折扣累积奖励为 在此无限范围 MDP 中最大化:

$$\begin{aligned} & \text{« ff} \ddot{y} \text{ } v \\ & J_{\pi} = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid s_0 \right] \quad (1.4) \\ & v \\ & t \leftarrow 0 \end{aligned}$$

MDP 的另一个常见数学目标是最大化时间平均 有限范围 MDP 的奖励:

$$\begin{aligned} & \text{« ffT } '1 \\ & J_{\pi} = \lim_{T \rightarrow \infty} \frac{1}{T} \mathbb{E}_{\pi} \left[\sum_{t=0}^{T-1} R(s_t, a_t) \mid s_0 \right] \quad (1.5) \\ & v \\ & tt \leftarrow 0 \end{aligned}$$

我们将默认采用无限范围 MDP 设置。

基于方程的目标函数。(1.4),在给定策略 π 下,我们可以定义

状态动作函数(即 Q 函数,它决定预期回报 在状态 s 中采取动作 a)和价值函数(决定回报) 与状态 s 中的政策相关联为:

$$\begin{aligned} & \text{« ff} \ddot{y} \text{ } v \\ & Q_{\pi}(s, a) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid s_0 = s, a_0 = a \right] \quad (1.6) \\ & v \\ & t \leftarrow 0 \end{aligned}$$

$$\begin{aligned} & \text{« ff} \ddot{y} \text{ } v \\ & V_{\pi}(s) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid s_0 = s \right] \quad (1.7) \\ & v \end{aligned}$$

where $\mathbb{E}_{\pi}$ is the expectation under the probability measure P_{π} over the set of infinitely

long state-action trajectories $\tau = (s_0, a_0, s_1, a_1, \dots)$, where P_{π} is the probability measure

induced by the transition probabilities P , policy π , initial state s and initial action a (in

the case of Q functions). The relationship between Q and V functions is $V_{\pi}(s) = \mathbb{E}_{\pi} [Q_{\pi}(s, \pi(s)) \mid s_0 = s]$ and $Q_{\pi}(s, a) = \mathbb{E}_{\pi} [R(s, a) + \gamma V_{\pi}(\pi(s)) \mid s_0 = s, a_0 = a]$.

1.2.3 值迭代

基于价值的方法

对于所有具有有限状态和动作的 MDP,至少存在一个确定性状态: 的最优政策。有了已知的转移函数 $P(s'|s, a)$,这就变成了

算法1 值迭代

1:输入:转换模型 P 、奖励函数 R 、折扣因子 γ 、容忍度 ϵ

2:对所有 $s \in \mathcal{S}$ 任意初始化 $V_0(s)$ 3: 对于 $k = 0, 1, 2, \dots$ 做 r4: $V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V_k(s')]$, @s $\in \mathcal{S}$ 5: if $\|V_{k+1} - V_k\| \leq \epsilon$ then

6:打破r7: Return: V_{k+1} and $\pi^* \text{ psq } P \arg \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V_{k+1}(s')]$

动态规划问题。最基本的动态规划之一 计算最优值的方法是值迭代如Alg。 1. 从 任意初始值函数 V_0 ,值迭代重复应用贝尔曼优化 恶意运算符:

$\tilde{V}_k(s) \leftarrow \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V_{k-1}(s')]$. (1.8) 一个 $s \in \mathcal{S}$

收敛后,可以通过贪婪行为来恢复最优策略 收敛值函数:

$\tilde{\pi}^* \text{ psq } P \arg \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^*(s')]$. (1.9)

一个 $s \in \mathcal{S}$

在不知道转换函数 P 的情况下,引入 Q 学习来寻找最优解

mal Q 函数 Q^* 最大化方程。 (1.6)。相应地,最优策略可以为derived from the Q -function by taking the greedy action $\pi^* \text{ psq } \arg \max_a Q^*(s, a)$. The 经典的 Q 学习算法通过 Q 近似 Q^* ,并通过时间更新其值

差异学习: '-

$Q_{p,t}(s, a) \leftarrow Q_{p,t}(s, a) + \alpha [R_t + \gamma \max_{a'} Q_{p,t}(s, a') - Q_{p,t}(s, a)]$. (1.10)

$\alpha \in (0, 1)$

研究人员应用神经网络作为 Q 函数的函数逼近器

更新方程(1.10)。具体来说,DQN 优化了以下方程: "-2j

$\min_{\theta} \mathbb{E}_{(s, a, R, s')} [D_t(s, a, \theta)^2] = \mathbb{E}_{(s, a, R, s')} [R + \gamma \max_{a'} Q_{\theta}(s', a') - Q_{\theta}(s, a)]^2$. (1.11)

$\theta \in \mathbb{R}^n$

神经网络参数 θ 通过画i.i.d来拟合。重播缓冲区中的样本D and then updating in a supervised learning fashion. $Q_{\theta'}$ is a slowly updated target 有助于稳定训练的网络。

行李员接线员

使用上面介绍的 MDP 表示法,贝尔曼算子映射一个值函数 其在转换核 P 和奖励函数 R 下的一步前瞻更新。

对于固定策略 π ,贝尔曼期望算子定义为 $B_{\pi} V(q) = E_{\pi} [r + \gamma V(q') | s, a]$ (1.12)

$\tilde{y} = \pi(a | s) P(s' | s, a) R(s, a, s')$ (1.13)

一个 s_1

其简写形式为 $B_{\pi} V(q) = \sum_{s'} P(s' | s, \pi) R(s, \pi(s), s') + \gamma \sum_{s'} P(s' | s, \pi) V(q(s'))$. In particular, 上一小节中定义的值函数满足贝尔曼定点

方程 $V_{\pi} = B_{\pi} V_{\pi}$. (1.14)

为了实现最优控制,我们用行动最大化代替策略平均值, 获得贝尔曼最优算子

$\tilde{y} = B V(q) = \max_a \sum_{s'} P(s' | s, a) R(s, a, s') + \gamma \sum_{s'} P(s' | s, a) V(q(s'))$, (1.15) 一个 s_1

这与后来写成简写形式 $B V(q) = \max_a \sum_{s'} P(s' | s, a) R(s, a, s') + \gamma \sum_{s'} P(s' | s, a) V(q(s'))$ 的操作符相同 r_1

$\gamma \sum_{s'} P(s' | s, \pi) V(q(s'))$ 。同样,对于动作值,我们有贝尔曼最优算子: $\tilde{y} = B^{\pi} Q(q)$

$\tilde{y} = B^{\pi} Q(q) = \sum_{s'} P(s' | s, \pi) R(s, \pi(s), s') + \gamma \sum_{s'} P(s' | s, \pi) Q(q(s'))$. (1.16) 一个 s_1

其简写形式为 $B^{\pi} Q(q) = \sum_{s'} P(s' | s, \pi) R(s, \pi(s), s') + \gamma \sum_{s'} P(s' | s, \pi) Q(q(s'))$.

Bellman算子的收敛性

定义 2(度量空间)。度量空间 (X, d) 是定义了度量 d 的集合 X 。度量集合 X 中任意两个元素之间的距离。距离度量在度量空间中需要满足以下性质:

- * 身份: $d(x, x) = 0, \forall x \in X$;
- * 非负性: $d(x, y) \geq 0, \forall x, y \in X$;
- * 对称性: $d(x, y) = d(y, x), \forall x, y \in X$;
- * 三角不等式: $d(x, z) \leq d(x, y) + d(y, z)$ 。

定义 3(柯西序列)。在度量空间 (X, d) 中,我们取一个子集(序列) $\{x_n\} \subset X$, if $\forall \epsilon > 0, \exists N \in \mathbb{N}, \forall n, m \geq N, d(x_n, x_m) < \epsilon$, then the sequence $\{x_n\}$ is a Cauchy sequence.

定义 4(完备度量空间)。度量空间 (X, d) 是完备的,如果每个柯西序列收敛到属于 X 的元素。即元素的每个柯西序列的收敛极限集合在于集合本身。

定理 5(承包商/收缩映射)。函数 f 在度量空间 (X, d) 上定义, and satisfies for $\forall x_1, x_2 \in X, 0 < \alpha < 1, d(f(x_1), f(x_2)) \leq \alpha d(x_1, x_2)$, then the function (operator) f is a contraction mapping or contractor.

定理 6(巴拿赫不动点定理)。令 (X, d) 为完备度量空间并且

函数 $f: X \rightarrow X$ 是承包商,则 f 有唯一不动点 $x^* \in X$ (即, $f(x^*) = x^*$) such that the sequence $x, f(x), f^2(x), \dots, f^n(x), \dots$ converges to x^* .

引理 7(贝尔曼最优算子)。

$\forall V \in \mathcal{B}(X), \exists \gamma \in [0, 1]$ 附注 1.17 一个 s_1

证明值迭代的收敛性等价于证明贝尔曼最优算子满足 Banach 不动点定理。在这种情况下,度量空间 (X, d) 有 $X = \{V \mid V \text{ 是 } S \times A \rightarrow \mathbb{R} \text{ 的函数}\}$ 和 $d(V, W) = \max_{s \in S} |V(s) - W(s)|$ 。很容易看出这一点

度量空间是完备的。Theorem 8 (Convergence of Bellman Optimal Operator- V^*). Optimal V^* value is a fixed

贝尔曼收缩算子 B 的点,应用于广义 $V : S \times A \rightarrow \mathbb{R}$ 。

运营商是超规范的承包商,即 $\|BV_1 - BV_2\| \leq \gamma \|V_1 - V_2\|$ (1.18)

证明。

$$\|BV_1 - BV_2\| \leq \max_{s \in S} |B V_1(s) - B V_2(s)|$$

$$= \max_{s \in S} |\sum_{a \in A} p(s,a) [R(s,a) + \gamma V_1(s,a) - R(s,a) - \gamma V_2(s,a)]|$$

一个附注 1.19 (萨阿 1.19) 萨阿 1.19

$$\|BV_1 - BV_2\| \leq \gamma \|V_1 - V_2\| \quad (1.20)$$

s, a

$$\|BV_1 - BV_2\| \leq \gamma \|V_1 - V_2\| \quad (\text{by triangular inequality}) \quad (1.21)$$

s, a

$$\|BV_1 - BV_2\| \leq \gamma \|V_1 - V_2\|$$

$\|V_1 - V_2\|$ (将概率 1 分配给 $\max_a Q(s,a)$) (1.22)

$$\|BV_1 - BV_2\| \leq \gamma \|V_1 - V_2\| \quad (1.23)$$

$$\|BV_1 - BV_2\| \leq \gamma \|V_1 - V_2\| \quad (1.24)$$

Theorem 9 (Convergence of Bellman Optimal Operator- Q^*). Optimal Q^* value is a fixed

应用于一般 Q 的贝尔曼收缩算子 B 的点: $S \times A \rightarrow \mathbb{R}$

as:(为了简单起见,忽略下面的 “ $*$ ”)

一个 1.25 一个 1.25

运营商是超规范的承包商,即 $\|BQ1 - BQ2\|_8 \leq \gamma \|Q1 - Q2\|_8$ (1.26)

证明。

 $\ddot{y}$

$$\frac{1}{\sqrt{2}} \left(|1\rangle_1 |1\rangle_2 - |1\rangle_1 |1\rangle_2 \right) = \frac{1}{\sqrt{2}} \left(|BQ1\rangle |BQ2\rangle - |8\rangle_{\max} |Pss1\rangle |pRss1\rangle |q\rangle_{\max Q1ps}, a1 q\rangle |Rss1\rangle |q\rangle_{\max Q2ps}, a2 qq|1\rangle_s, a1\rangle_a2\rangle_s \right)$$

$$(1.27) \quad \ddot{y}$$

$$1 \ 1 \ 1 \ 1'' \max \gamma | \ P_{ss} \ 1 \ p_{\max} \ Q_1 \ p_s, \ a_1 \ q' \ \max \ Q_2 \ p_s, \ a_2 \ q q|$$

$$1 \ 1 \ (1.28) \ s, a \ a1 \ a2 \ s1 \ \ddot{y}$$

$$1 \leq |d| \max\{\gamma_1, \gamma_2\} \leq P_{ss} + \max\{Q_1, Q_2\} \leq P_{ss} + \max\{Q_1, Q_2\} + \max\{Q_1, Q_2\} \quad (\text{by triangular inequality})$$

1 1 s,a a1 a2 s1

$$(1.29) \quad \ddot{y}$$

$$1 \ 1 \ 1 \ 1 d' \max \gamma \ Pss \ 1 \max |Q1 \ ps, a \ q \ ' Q2 \ ps, a \ q| \text{ (by convexity of max function)}$$

s,aa s1

$$(1.30)d' \max \gamma \max$$

$|Q_1| p \hat{s} 1, a | q' Q_2| p \hat{s} 1, a | q|$ (将概率 1 分配给 $\max q s, a s l$

$$(1.31) d' \max_{\gamma} \gamma \|Q_1 - Q_2\|_8 \quad (1.32)$$

$$s, a'' \gamma || Q1 \quad ' \quad Q2 || 8 \quad (1.33)$$

不仅最优贝尔曼算子具有收缩性质,而且生成普通行李员接线员。证明类似,此处不再赘述。

备注 10. 具体来说,在正常贝尔曼算子的收缩中,如果其中之一 value function is the unbiased value function, e.g., $V_{2\pi}$ “ $BV_{2\pi}$ or $Q_{\pi 2}$ “ $BQ_{\pi 2}$, another one is the estimation ($V_{1\pi}$ or $Q_{\pi 1}$), the convergence theorems say that the estimated value 函数将收敛到真值函数。

Q-Learning 的融合

定理 11(Q 学习的收敛性)。考虑有限贴现 MDP 有界奖励和 Q-learning 更新

$$Q_{t+1}(s, a) = (1 - \alpha) Q_t(s, a) + \alpha (r_t + \gamma \max_{a'} Q_t(s', a'))$$

对于任意 (s, a) , (1.34) 一个

而对于 $(s, a) \in \mathcal{S} \times \mathcal{A}$, 在 q 处我们保留

$$Q_{t+1}(s, a) = Q_t(s, a). \quad (1.35)$$

假设:

* 每个状态-动作对 (s, a) 被无限频繁地访问;

* 奖励是统一有限的;

* 学习率满足 Robbins-Monro 条件

$$\sum_{t=0}^{\infty} \alpha_t = \infty, \quad \sum_{t=0}^{\infty} \alpha_t^2 < \infty, \quad \alpha_t \leq \alpha_{t-1}. \quad (1.36)$$

然后迭代 Q_t 几乎肯定会收敛到最优动作值函数 Q^* :

$$Q_t(s, a) \xrightarrow{a.s.} Q^*(s, a), \quad \forall (s, a) \in \mathcal{S} \times \mathcal{A}, \quad (1.37)$$

其中

其中 Q^* 是 Bellman 最优算子的唯一不动点

$$Q^*(s, a) = (1 - \gamma) R(s, a) + \gamma \max_{a'} \sum_{s'} P(s' | s, a) Q^*(s', a'). \quad (1.38)$$

备注 12. 该定理表明 Q-learning 是一个随机逼近过程 用于求解贝尔曼最优性定点方程。B 担保的收缩

Q^* 的唯一性,而采样转换引起的鞍差噪声 由递减的步长控制。

1.2.4 政策梯度

基于策略的方法

基于策略的方法旨在直接搜索策略空间以找到最优 policy π^* 。One can parameterise the policy expression $\pi^* \leftarrow \pi_\theta^*$ and update the parameter θ in the direction that maximises the cumulative reward $J(\theta) = \mathbb{E} \sum_{t=0}^{\infty} \gamma^t r_t$ by $\nabla_{\theta} J(\theta)$ to find the optimal policy. 然而,梯度将取决于未知的影响 国家分配政策的变化。著名的政策梯度(PG)定理[Sutton et al., 2000] 推导了一个不涉及状态分布的解析解, 即:

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right] \quad (1.39)$$

where μ_{θ} is the state occupancy measure under policy π_{θ} and $\nabla \log \pi_{\theta}(a | s)$ is the updating policy's score function. 当策略是确定性的并且操作集是连续的时, 获得确定性策略梯度 (DPG) 定理 [Silver et al., 2014] 为

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right] \quad (1.40)$$

PG 定理的一个经典实现是 REINFORCE [Williams, 1992], 其中 uses a sample return $R_t = \sum_{i=t}^{\infty} \gamma^i r_i$ to estimate $Q_{\pi_{\theta}}$. Alternatively, one can use a model of Q_{ω} (also called critic) to approximate the true $Q_{\pi_{\theta}}$ and update the parameter ω through TD learning. 这种方法催生了著名的演员评论家方法 [Konda 和齐齐克利斯, 2000; 彼得斯和沙尔, 2008]。演员批评家方法的重要变体 包括信赖域方法 [Schulman et al., 2015a, 2017]、软演员批评家方法 [Haarnoja et al., 2018], 以及深度确定性策略梯度 (DDPG) 方法 [Lillicrap 等人, 2015]。

政策梯度定理

在本小节中,我们将政策目标写为 $J(\pi_\theta) = \mathbb{E}_s \sum_{t=0}^{\infty} \gamma^t V(\pi_\theta) - \mathbb{E}_s \sum_{t=0}^{\infty} \gamma^t Q(s, \pi_\theta)$. (1.41)

Here μ_π denotes the discounted state occupancy measure induced by policy π , and $\pi_\theta(a|s)$ denotes a stochastic policy parameterized by θ . For the reparameterization form, we assume that an action can be generated by first sampling $\epsilon \sim p(\epsilon)$ from a fixed 噪声分布,然后应用可微变换 $a = f_\theta(\epsilon; s)$. (1.42)

The conditional density induced by this sampling procedure is just the policy $\pi_\theta(a|s)$.

假设 1. S 和 A 是封闭且有界的。Assumption 2. $P(s'|s, a, \epsilon; \theta)$, $f_\theta(\epsilon; s)$, $\pi_\theta(a|s, \epsilon)$, $R(s, a, s')$, $V(s)$ and their derivatives are continuous in all variables s, a, s', θ , and ϵ .

备注 13. 上述两个假设允许我们交换导数和积分,以及多重积分的顺序,使用富比尼定理和莱布尼兹积分规则。

定理 14(随机策略梯度 (SPG) 定理)。假设 MDP 满足

满足假设 1 和假设 2, 则 $\nabla_\theta J(\pi_\theta) = \mathbb{E}_s [\mu_\pi(s) \cdot \nabla_\theta \log \pi_\theta(a|s) Q(s, \pi_\theta)]$

$= \mathbb{E}_s [\mu_\pi(s) \cdot \nabla_\theta \log \pi_\theta(a|s) \cdot \sum_{s'} P(s'|s, a) \gamma V(s') - V(s)]$.

推论 15(基线减法随机政策梯度)。 $\nabla_\theta J(\pi_\theta) = \mathbb{E}_s [\mu_\pi(s) \cdot \nabla_\theta \log \pi_\theta(a|s) \cdot \sum_{s'} P(s'|s, a) \gamma V(s') - V(s)]$

where $A_{\pi\theta}(\mathbf{p}_s, \mathbf{a}_q) - Q_{\pi\theta}(\mathbf{p}_s, \mathbf{a}_q) = V_{\pi\theta}(\mathbf{p}_s)$ is the advantage function. Both of these are 无偏,但基线减法减少了梯度估计的方差。

定理 16(重新参数化策略梯度定理)。假设 MDP

满足假设1和2,则” v_i

 π

$\nabla \theta \text{ Jp} \theta q \text{ " Es"} \mu \text{p} \theta , \varepsilon \text{ "p} \nabla \theta \text{ f} \theta \text{ p} \varepsilon ; \text{sq} \nabla \text{a Q ps, aq a "f p} \varepsilon ; \text{sq} .$

 $\theta \quad v$ θ

证明。根据策略梯度定理,我们有 z

∇θ Jpθq “ μpθ psqπθ pa | sqQπθ ps, aq∇θ log πθ pa | sq da ds.

因此 $\nabla_{\theta} J_{p_{\theta}q}$

Z

“μπθ psqπθ pa | sqQπθ ps, aq∇θ log πθ pa | sq da ds

$$Z \wedge Z'.$$

$\pi\theta$ $\pi\theta$

“ μ psq Q ps, aq∇θ πθ pa | sq da ds

$$z^{\prime} z z^j$$

$\pi\theta$ $\pi\theta$ $\pi\theta$

“ μ psq ∇θ pQ ps, aqπθ pa | sqq da ' πθ pa | sq∇θ Q ps, aq da ds

$$z''^{\Lambda_Z} z_j$$

$\pi\theta$ $\pi\theta$ $\pi\theta$

“ μ psq ∇θ Q ps, aqπθ pa | sq da ' πθ pa | sq∇θ Q ps, aq da ds

$$z''^{\wedge} z \cdot z_j$$

$\pi\theta$ $\pi\theta$ $\pi\theta$

V

“ μ psq ∇θ ppeqQ ps, fθ pε; sqq dε ' ppeq∇θ Q ps, aqva“f pε;s q dε ds

 θ

(通过重新参数化) $z = z'$

 $\pi\theta \quad \pi\theta$
$$v \propto \pi\theta$$

V

“ μ psq ppeq ∇θ fθ pε; sq∇a Q ps, aq a“f pε;sq ` ∇θ Q ps, aq a“f pε;sq dε

V V

 $\theta \quad \theta$ z_j

V

' ppeq∇θ Qπθ ps, aqva“f pε;sq dε ds

 θ

Z

V

“ μπθ psq ppeq∇θ fθ pε; sq∇a Qπθ ps, aqva“f pε;sq δε ds.

 θ

备注 17. 该定理是重新参数化对梯度的直接应用 政策目标。它被理解为已知但未正式呈现或衍生

任何现有工作中的政策梯度。我们将其包含在这里是为了完整性以及 一个有用的理论工具。

推论 18(确定性政策梯度定理)。假设 MDP 满足 Assumption 1 and 2, for a deterministic policy μ_θ psq, we have

$$\nabla_{\theta} J(\mu_{\theta}) = \mathbb{E}_{\mu_{\theta}} \left[\nabla_{\theta} Q^{\mu_{\theta}}(s, a) \right]$$

$$\mu_{\theta}$$

$$\nabla_{\theta} J(\mu_{\theta}) = \mathbb{E}_{\mu_{\theta}} \left[\nabla_{\theta} Q^{\mu_{\theta}}(s, a) \right]$$

$$\theta \nabla$$

$$\theta$$

Proof. Let δ be the delta function. Thus $\delta(x) = \int \delta(x - y) dy$. Furthermore, let

$$\nabla_{\theta} J(\mu_{\theta}) = \mathbb{E}_{\mu_{\theta}} \left[\nabla_{\theta} Q^{\mu_{\theta}}(s, a) \right]$$

$$\nabla_{\theta} J(\mu_{\theta}) = \mathbb{E}_{\mu_{\theta}} \left[\nabla_{\theta} Q^{\mu_{\theta}}(s, a) \right]$$

$$\mu_{\theta} \nabla_{\theta}$$

$$\nabla_{\theta} J(\mu_{\theta}) = \mathbb{E}_{\mu_{\theta}} \left[\nabla_{\theta} Q^{\mu_{\theta}}(s, a) \right]$$

$$\nabla_{\theta} J(\mu_{\theta}) = \mathbb{E}_{\mu_{\theta}} \left[\nabla_{\theta} Q^{\mu_{\theta}}(s, a) \right]$$

$$\theta$$

$$\nabla_{\theta} J(\mu_{\theta}) = \mathbb{E}_{\mu_{\theta}} \left[\nabla_{\theta} Q^{\mu_{\theta}}(s, a) \right]$$

$$\nabla_{\theta} J(\mu_{\theta}) = \mathbb{E}_{\mu_{\theta}} \left[\nabla_{\theta} Q^{\mu_{\theta}}(s, a) \right]$$

$$\nabla_{\theta} J(\mu_{\theta}) = \mathbb{E}_{\mu_{\theta}} \left[\nabla_{\theta} Q^{\mu_{\theta}}(s, a) \right]$$

$$\theta$$

$$\nabla_{\theta} J(\mu_{\theta}) = \mathbb{E}_{\mu_{\theta}} \left[\nabla_{\theta} Q^{\mu_{\theta}}(s, a) \right]$$

$$\nabla_{\theta} J(\mu_{\theta}) = \mathbb{E}_{\mu_{\theta}} \left[\nabla_{\theta} Q^{\mu_{\theta}}(s, a) \right]$$

$$\nabla_{\theta} J(\mu_{\theta}) = \mathbb{E}_{\mu_{\theta}} \left[\nabla_{\theta} Q^{\mu_{\theta}}(s, a) \right]$$

$$\theta$$

备注19. DPG和DDPG都是基于这个定理提出的,首先 Silver 等人证明了。 [2014]。它仅限于确定性策略,因此可以推断 作为定理 16 的推论,它也适用于随机策略。

推论 20(熵正则化重参数化策略梯度定理)。考虑熵正则化值 $\mathcal{H}(\mu_{\theta})$

$$\nabla_{\theta} J(\mu_{\theta}) = \mathbb{E}_{\mu_{\theta}} \left[\nabla_{\theta} Q^{\mu_{\theta}}(s, a) \right]$$

$$\nabla_{\theta} J(\mu_{\theta}) = \mathbb{E}_{\mu_{\theta}} \left[\nabla_{\theta} Q^{\mu_{\theta}}(s, a) \right]$$

$$\nabla_{\theta} J(\mu_{\theta}) = \mathbb{E}_{\mu_{\theta}} \left[\nabla_{\theta} Q^{\mu_{\theta}}(s, a) \right]$$

$$\nabla_{\theta} J(\mu_{\theta}) = \mathbb{E}_{\mu_{\theta}} \left[\nabla_{\theta} Q^{\mu_{\theta}}(s, a) \right]$$

$$\nabla_{\theta} J(\mu_{\theta}) = \mathbb{E}_{\mu_{\theta}} \left[\nabla_{\theta} Q^{\mu_{\theta}}(s, a) \right]$$

$$\nabla_{\theta} J(\mu_{\theta}) = \mathbb{E}_{\mu_{\theta}} \left[\nabla_{\theta} Q^{\mu_{\theta}}(s, a) \right]$$

是概率密度函数 $p(\mathbf{x}|\mathbf{q})$ 的微分熵, α 是正反函数

站着。假设 MDP 满足假设 1 和假设 2, 则 $\forall i$

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{s \sim \mu} [\nabla_{\theta} \log \pi_{\theta}(a|s) \nabla_{\theta} Q(s, a; \theta)]$$

θ

1.3 博弈论多智能体强化学习

单代理强化学习研究针对动态环境的决策制定 固定的, 并且不会策略性地响应学习者的行为。在多智能体强化学习中, 通过相反, 部分环境被其他学习或决策主体取代。因此, 每个智能体的返回不仅取决于状态转换模型, 自己的政策, 但也取决于其他代理选择的政策。这种相互依存 自然地产生了博弈论结构。从这个角度来看, 多智能体强化学习可以被视为将 MDP 扩展到交互 多个参与者、联合行动和可能不同的奖励的主动决策过程 功能。与等式中的单代理目标相比。(1.4), 其中优化

针对被动环境的单一策略, 多代理目标是在 tuple of policies $\pi_1, \dots, \pi_M$ 。For player i , a general discounted-return objective can be 写成 $\mathbb{E}_{\pi_1, \dots, \pi_M} [r_i]$

$$J_i(\pi_1, \dots, \pi_M) = \mathbb{E}_{\pi_1, \dots, \pi_M} [r_i]$$

(1.43)

The optimization problem is therefore no longer simply $\max_{\pi} J(\pi)$, but rather a strategic

每个参与者在考虑政策的同时优化自己的目标的问题 他人的情况。根据奖励结构, 人们研究解决方案概念, 例如 均衡策略、零和博弈中的极小极大策略或团队中的合作最优 设置。在本节中, 我们首先介绍极小极大优化作为数学 竞争互动的核心, 然后回顾零和博弈, 然后是它们的顺序 对马尔可夫博弈的初步推广, 最后简单讨论一般和与合作 设置。

极小极大优化 零和博弈解决的数学核心是迷你 最大优化。在最基本的形式中,我们用两个优化目标 $f(x, y)$ 竞争变量 $x \in P$ 和 $y \in Q$:

最小最大 $f(x, y)$ (1.44) $x \in P, y \in Q$

其中 $f(x, y) : P \times Q \rightarrow \mathbb{R}$ 不一定是凸凹的。当 f 对 x 是凸的,对于每个固定的 y 和 f 对 y 是凹的,对于每个固定的 x , f 是凹的,该问题称为凸-凹的。许多博弈论学习问题,特别是在多智能体强化方面 学习,可以理解为寻找这种极小极大结构的稳定解决方案。

1.3.1 零和博弈

零和博弈提供了典型的博弈论设置,其中极小极大结构 方程中的真值。 (1.44) 自然产生。他们模拟了完全竞争性的互动,其中一个 玩家的收益恰好是另一个玩家的损失,因此总效用之和为零。

对于两人零和游戏,让 μ 表示最大化玩家的策略,

ν 表示最小化玩家的策略,令 $R(\mu, \nu)$ 表示对

在策略对 (μ, ν) 下最大化玩家。一个中心结果是极小极大定理

[Von Neumann 和 Morgenstern, 1945], 其中指出 " " " " " "

$$\max_{\mu} \min_{\nu} E[R, \mu, \nu] = \min_{\nu} \max_{\mu} E[R, \mu, \nu]. \quad (1.45)$$

$$\mu^* \nu^* \nu^* \mu^*$$

极小极大定理意味着博弈有一个明确定义的值,并且纳什方程 librium 是一个自然的解决方案概念:当玩家使用均衡策略时,对手无法通过单方面偏差来改进它。

定义 21(纳什均衡)。对于两人零和博弈,策略对

如果两个参与者都不能通过单向改进其目标,则 μ^*, ν^* 被称为纳什均衡

时间偏差,即

$$\forall \mu, \nu \in \mathcal{P}(\mathcal{S}), \forall q \in \mathcal{Q}, \forall v \in \mathcal{V}, \quad (1.46)$$

1.3.2 马尔可夫博弈

定义 22(马尔可夫博弈)。有限视野马尔可夫博弈可以被视为阶段游戏的顺序扩展,玩家在 H 步骤上重复交互。环境状态按照马尔可夫转移规则演化。一般来说,多人情况下,马尔可夫博弈由下式指定

$$\{H, S, \{A_i\}_{i=1}^n, P, \{R_i\}_{i=1}^n\} \quad (1.47)$$

其中 H 是地平线, S 是状态空间, A_i 是玩家 i 的动作空间, P

$$P: S \times A_1 \times \dots \times A_n \rightarrow \mathcal{P}(S) \quad (1.48)$$

$$R_i: S \times A_1 \times \dots \times A_n \rightarrow \mathbb{R} \quad (1.49)$$

1.3.3 二人零和马尔可夫博弈

作为马尔可夫博弈的一个特例,我们将两人零和马尔可夫博弈定义如下。

定义 23(两人零和马尔可夫博弈)。它由元组指定

$$(H, S, A, B, P, R) \quad (1.50)$$

其中 A 和 B 分别是最大玩家和最小玩家的动作空间,

P 是过渡内核的集合 $P_h : S \times A \times B \rightarrow \Delta(S)$, (1.51)

R 是最大玩家的奖励函数 $R_h : S \times A \times B \rightarrow \mathbb{R}$. (1.52)

最小玩家收到奖励 R_h , 因此每一步的总奖励总和为零。

策略和价值函数 最大玩家的(马尔可夫)策略 μ 是以下集合的集合

函数 $\mu_h : S \rightarrow \Delta(A)$, 类似地, 最小玩家的策略 ν 是一个集合 $\nu_h : S \rightarrow \Delta(B)$. 我们写 $\mu_h(a|s)$ 和 $\nu_h(b|s)$ 为相应的操作

步骤 h 的概率。We use $V_{\mu,\nu} : S \rightarrow \mathbb{R}$ to denote the value function at step h under policy μ and ν ,

so that $V_{\mu,\nu}$ gives the expected cumulative rewards received under policy μ and ν ,

从步骤 h 的 s 开始:

$H \geq 0$

ν

$V_{\mu,\nu} : \mathbb{E}_{\mu,\nu} [R_{h+1} | s_h = s, a_h, b_h] = V_{\mu,\nu}(s)$.

ν (1.53)

$h \geq 0$

我们还定义 $Q_{\mu,\nu} : S \times A \times B \rightarrow \mathbb{R}$ to be the action-value function at step h so that

$Q_{\mu,\nu}$

$h \geq 0, a, b$ 给出了在策略 μ 和 ν 下收到的累积奖励,从

步骤 h 处的 s, a, b : $Q_{\mu,\nu}$

$h \geq 0, a, b$

$H \geq 0$

(1.54): $\mathbb{E}_{\mu,\nu} [R_{h+1} | s_h = s, a_h = a, b_h = b] = Q_{\mu,\nu}(s, a, b)$.

$h \geq 0$

为了简单起见,我们定义转移运算符

$T_{\mu,\nu} : \mathbb{R}^S \rightarrow \mathbb{R}^S$ 为 $(T_{\mu,\nu} V)(s) = \mathbb{E}_{\mu,\nu} [R_{h+1} + \gamma V(s_{h+1}) | s_h = s]$ (1.55)

对于任何值函数 V 。我们还使用 $rD_{\mu}h^{Vh} Q_{\mu}^{psq} := E_{a''}[\mu h^{p'} |sq, b''^{Vh} p' |sq] Q_{\mu}^{ps, a, bq}$ (1.56)

对于任何动作-价值函数 Q 。根据价值函数的定义,我们有 Bellman

等式 $Q_{\mu, v}^{\mu, v}$

$h^{ps, a, bq} = pRh^{Ph} Vh^{l} q_{ps, a, bq},$

(1.57) $V_{\mu, v}^{psq} = pD_{\mu}^{Vh} Q_{\mu, v}^{psq},$

,

哈哈哈哈哈 μ, v

for all $ps, a, b, hq P S^A A^B^r Hs$, and at the pH^{1qth} step we have $VH^{1} psq = 0$ for

全部都是 $P S$ 。

最佳响应和纳什均衡。对于任何最大玩家 μ 的策略,有 $\mu, v : p\mu q$

存在最小玩家的最佳响应,这是满足 Vh^{psq} 的策略 $v : p\mu q$ $\mu, v : p\mu q$

$\inf v Vh_{\mu, v}^{psq}$ for any $ps, hq P S^A r Hs$. We denote $Vh_{\mu, :} := Vh$. According to the

前面的贝尔曼方程(3.3),我们进一步得到了动态定义的最佳响应

编程方式(实验部分采用): $Q_{\mu, :}^{\mu, :}$

$h^{ps, a, bq} = pRh^{Ph} Vh^{l} q_{ps, a, bq}$ (1.58)

$Vh_{\mu, :}^{psq} = \inf pD_{\mu}^{Vh} Q_{\mu, :}^{psq},$

h^{qpsq} (1.59) v

通过对称性,我们还可以定义 $\mu : p\nu q$ 和 $Vh_{:, v}$ 。据进一步了解,存在

在某种意义上,相对于对手的最佳反应而言最优的策略 $\mu <, v <$

那个 $\mu_{<, :}^{\mu, :}$

$\&Vh^{psq} = \sup Vh^{psq},$

μ

$\%V_{:, v} \leftarrow psq = \inf V_{:, v}^{psq},$ $'$ $'$ $h h v$

for all $p, q \in \mathcal{P}(\mathcal{S} \times \mathcal{H})$. We call these optimal strategies μ^*, ν^* the Nash equilibrium of 马尔可夫博弈, 满足以下极小极大方程 2:

$$\mu^* \leftarrow \arg \sup_{\mu} \inf_{\nu} V(\mu, \nu) \quad \nu^* \leftarrow \arg \inf_{\nu} \sup_{\mu} V(\mu, \nu)$$

直观上, 纳什均衡给出了一个解决方案, 其中没有玩家可以获得任何收益 只改变她自己的政策。

学习目标。我们衡量任何一对一般政策的次优性 μ^*, ν^* 使用它们的性能与最优策略性能之间的差距

分别对抗最佳响应时的 $V(\mu^*, \nu^*)$ (即纳什均衡): $V(\mu^*, \nu^*) - V(\mu^*, \nu)$ 和 $V(\mu^*, \nu^*) - V(\mu, \nu^*)$

"我"我" $V(\mu^*, \nu^*) - V(\mu^*, \nu)$ 和 $V(\mu^*, \nu^*) - V(\mu, \nu^*)$

定义 24(α -近似纳什均衡)。一对一般策略 μ, ν 是 α -approximate Nash equilibrium, if $V(\mu, \nu) - V(\mu^*, \nu^*) \leq \alpha$. 在带有模拟器的马尔可夫游戏的值迭代中, 估计的转移模型 是:

$\hat{P} = \frac{1}{n} \sum_{i=1}^n P_i$ 我 " 1

纳什均衡策略 μ^* 可以通过以下公式导出:

由贝尔曼最优方程, $Q^* = \max_{\mu} \min_{\nu} Q(\mu, \nu)$ 我 " 1

Q^* 我 " 1

$$Q^* = \min_{\nu} \max_{\mu} Q(\mu, \nu) \quad (1.62)$$

Q^* 我 " 1

甲、乙

这里的极小极大定理与矩阵博弈的定理不同, 即 $\max_{\mu} \min_{\nu} \mu^T A \nu \neq \min_{\nu} \max_{\mu} \mu^T A \nu$ for any matrix A , since here $V(\mu, \nu)$ is in general not bilinear in

纳什均衡 p^*, q^* 可以通过以下公式计算:

$$\hat{y}p\hat{u}h\hat{p}^{\top}|sq, \hat{v}h\hat{p}^{\top}|sqq \leftarrow \arg \min \arg \max \mu h pa|sqvh pb|sqQ^h ps, a, bq \quad (1.63)$$

vh μh

甲、乙

对于每个 ps、总部。

定理 25. 对于两人零和马尔可夫模拟器中的值迭代 (VI) 游戏, 其中 $n \in \mathbb{N}$, $\epsilon \in (0, 1)$, $t \leq \log \frac{2HSA}{\epsilon}$

p , with probability at least $1 - p$, the policy $\hat{p}, \hat{q}$ resulting

from VI will be ε 'optimal.

$$V1; \hat{v} \text{ ps1 } q \text{ ' } V1\hat{\mu}; \text{ ps1 } q \text{ d' } \varepsilon \quad (1.64)$$

1.3.4 马尔可夫势博弈

马尔可夫势博弈构成了一般和马尔可夫的一个重要的结构化子类 游戏。虽然一般和博弈可能涉及参与者之间相互冲突的目标, 势游戏承认单个标量势函数可以跟踪每个玩家的参与 单方面偏差的激励。这种结构很有用,因为它将战略转变为 平衡问题转化为势函数的优化问题。

潜在的游戏

我们首先回顾一下舞台游戏的概念。设 N 为玩家人数, A_i 为 action space of player i , and write the joint action as a "pair" $a = (a_1, a_2, \dots, a_N)$.

对于固定状态 s , 如果单方参与, 则状态依赖阶段博弈称为潜在博弈。玩家奖励的变化与共同潜力的变化完全匹配功能。

定义 26(阶段潜力博弈)。对于具有奖励功能的阶段游戏 $\text{trips}, \text{aquN} \in \mathbb{N}$, 如果存在函数 $\Phi_{\text{ps}, \text{aq}}$ 使得对于

每个玩家 i、每个状态 s、每个 a_i 、 $\bar{a}_i \in A_i$ 和每个固定 $a_{-i}, R_i(p_s, \bar{a}_i), a_{-i} q' R_i(p_s, a_i), a_{-i} q'' \Phi(p_s, \bar{a}_i), a_{-i} q' \Phi(p_s, a_i), a_{-i} q$. (1.65)

马尔可夫势博弈

动态对应是通过马尔可夫扩展阶段势博弈获得的状态转换,以便一步支付比较被替换为比较政策引发的长期价值函数。

定义 27(马尔可夫势博弈)。考虑一般和马尔可夫博弈players $i \in \{1, \dots, N\}$, policies $\pi = (\pi_1, \dots, \pi_N)$, and player-wise value functions $V_i(\pi)$. The 如果存在势函数 $\Phi(\pi)$ 这样的博弈称为马尔可夫势博弈that for every player i , every state s , every $\pi_i, \tilde{\pi}_i \in \Pi_i$, and every fixed profile $\pi_{-i} \in \Pi_{-i}$,

$$V_i(\pi_i, \pi_{-i}) - V_i(\tilde{\pi}_i, \pi_{-i}) = \Phi(\pi_i, \pi_{-i}) - \Phi(\tilde{\pi}_i, \pi_{-i}). \quad (1.66)$$

这个定义意味着每个玩家单方面偏离的动机是完美的 与相同标量势的变化一致。因此,最大化

潜力与达到纳什均衡是相容的。Proposition 28. If π^* maximizes the potential function $\Phi(\pi)$, then π^* is a Nash equilibrium of the Markov potential game.

定理 29. 对于有限马尔可夫势博弈,至少存在一个确定性 纳什均衡。

充分的条件。几个结构条件表明马尔可夫博弈是 马尔可夫势博弈。典型例子包括:

- * 每个状态阶段博弈都是一个潜在博弈,并且转换结构保留 各州之间的这种一致性;
- * 可分离的奖励以及与状态无关的转换;
- * 独立于动作的转换与状态方面的潜在博弈奖励相结合 结构。

这些条件说明了马尔可夫潜在博弈背后的主要设计原则: 战略互动可能在参与者之间分散,但诱导的激励可以 仍然可以用单个全局势函数来概括。

组织机构

本论文分为两个技术部分,它们发展了一个共同的主题: 互补的方向。第一部分研究游戏中的强化学习算法, 其中的核心挑战是多代理环境中的战略决策。 第二部分研究基础模型时代的强化学习,其中预训练 生成模型和学习世界模型为顺序决策提供了丰富的先验 制作。这两部分共同连接了强化的算法基础 学习世界建模在大规模生成和交互中的新兴作用 系统。 第一部分从第三章两人零和博弈开始,该章发展了强化- 在典型的对抗性环境中进行学习算法,并建立了基本的 激励、政策和均衡行为之间的联系。接下来是 4 零和视频游戏,将这些想法带入更丰富的视频游戏环境中 环境并研究竞争性强化学习在大规模实践中的表现。 该部分最后以5多人一般和博弈结束,该章移动 超越纯粹的竞争到更一般的战略互动,其中相关概念最优性不再是单一赢家,而是由各方激励形成的均衡 多个代理。 第二部分在这个想法的推动下转向使用基础模型的强化学习 广泛的先验知识可以使优化更加有效并更好地与

复杂的目标。6 扩散世界模型研究基于扩散的生成如何 迭代模型可以作为规划和控制的学习环境模型。章- 7 强化学习策略的一致性模型探索表达生成 将类建模为策略,展示了生成建模的进步如何直接实现 丰富强化学习的行动空间。接下来,8 RL 的少步视频 然后 Generation 研究强化学习如何提高视频生成的效率 当生成器本身是一个强大的预训练模型时。9 视频世界模型进一步发展了交互环境中的世界建模视角,其中 世界模型必须反映行动如何塑造未来的视觉观察。最后,10 带内存的世界模型解决了长视野序列建模问题,提供了架构 支持更强大的世界模型以进行连续生成的结构工具 互动。总而言之,这些章节提出了强化学习的统一观点: 从算法问题研究目标驱动的顺序决策 战略游戏到基于基础模型的系统,这些系统依赖于学习的世界模型 表示、预测和优化复杂环境。

原文第 43 页

第二部分

游戏中的强化学习

第三章

两人零和博弈

本节基于论文 “ A Deep Reinforcement Learning Approach for 在两人 Atari 游戏中寻找不可利用的策略 ” [Ding 等人, 2022]与苏迪佳、刘庆华和金池合作撰写。

3.1 简介

多智能体系统中的强化学习 (RL) 已成功解决了许多具有挑战性的问题 任务,包括围棋 [Silver et al., 2017]、捉迷藏 [Baker et al., 2019]、星际争霸 [Vinyals 等人,2017]、Dota [Berner 等人,2019]、扑克 [Heinrich 和 Silver,2016;棕色 和桑德霍尔姆,2019; Zha 等人,2021 年] 和棋盘游戏 [Lanctot 等人,2019 年;塞里诺 等人,2019]。除了扑克系统之外,这些作品中大量都衡量了 他们在对抗固定代理人、普通人类玩家或 几张照片中的专家。游戏的一个显着特点是对手可以 进一步模拟学习者的行为,调整他们的策略,并利用学习者的 弱点。目前尚不清楚这些多主体中的许多人发现的政策是否 系统仍然可以对抗对手的对抗性剥削。在本文中,我们考虑两人零和马尔可夫博弈(MG),以及我们的目标 目标是找到纳什均衡(NE)[Nash et al., 1950]。根据定义, NE 策略是一个固定点,没有玩家有动力偏离当前策略 策略。由于极小极大定理,一个玩家的 NE 策略也是最好的

面对对手的最佳反应时的解决方案。也就是说 NE 是自然数 不受敌对对手利用的解决方案。纳什均衡和不可利用性的概念在《纳什均衡》中得到了很好的研究。学习广泛形式游戏 (EFG) 的社区,例如扑克 [Heinrich 和 Silver, 2016年;布朗和桑德霍姆,2019;查等人,2021;麦卡利尔等人,2021]。区别于 EFG 具有树形结构的过渡动态特性,不能有效地表示 过去的多个状态可能导致未来相同的状态的游戏, 本文重点关注具有一般转移结构的 MG,并利用马尔可夫 结构。先前的另一系列作品 [Heinrich 和 Silver,2016;兰托特等人,2017] 直接结合基于最佳响应的算法在正常形式的博弈中寻找 NE, 例如虚构游戏 [Brown, 1951] 和双重预言 [McMahan et al., 2003],以及单预言 代理深度强化学习算法,例如 DQN [Mnih et al., 2013] 和 PPO [Schulman et al., 2017]寻找最佳反应。虽然这些方法可以应用于 MG,除了将其视为正常形式的游戏外,他们并没有利用 MG 的精细结构,这导致学习效率显着低下(如表格和 Atari 所示) 本文的实验)。 本文提出了两种新颖的端到端深度强化学习算法 用于学习 两人零和马尔可夫博弈的纳什均衡--Nash Deep Q-Network (Nash-DQN) 及其变体 Nash Deep Q-Network with Exploiter (Nash-DQN-利用者)。 Nash-DQN 结合了学习 Nash 的最新理论进展 表格马尔可夫博弈的均衡 [Hu 和 Wellman,2003;白和金,2020;刘等人, 2021 年; Jin et al., 2021b],采用著名的单代理 DRL 算法 DQN [Mnih et al., 2013]用于解决连续状态空间和函数逼近。纳什-DQN- Exploiter 是 Nash-DQN 的一个变体,它在 它的学习。剥削对手刺激了主力的探索。两者都 算法是理论算法的实际变体,保证 在基本表格设置中收敛到纳什均衡。 实验评估是在表格马尔可夫游戏和双 玩家视频游戏,以展示所提出算法的有效性和鲁棒性。

(1).史莱姆齐射 (2)。拳击 (3).双扣篮 (4)。乒乓球 (5)。网球 (6)。环绕声

图 3.1:六款两人视频游戏的屏幕截图。

如图 3.1 所示,我们实验中的视频游戏包括五个两人 Atari PettingZoo 库中的游戏 [Terry et al., 2021b; Bellemare et al., 2013]和一个长凳- 标记环境史莱姆排球 [Ha,2020]。由于计算能力的限制 资源,我们考虑基于 RAM 的 Atari 游戏版本,并截断长度 每场比赛的步数为 300 步。我们通过训练对抗性对手来测试性能 使用直接利用学习器策略的 DQN。我们在两种设置下的实验表明我们的算法明显优于 MARL in-中的标准算法 包括神经虚拟自我博弈 (NFSP) [Heinrich 和 Silver, 2016] 和政策空间 响应 Oracle (PSRO) [Lanctot et al., 2017],在针对广告的鲁棒性方面 社会剥削。

3.2 相关工作

在本节中,我们回顾相关工作。合作游戏中的 MARL。MARL 丰富的文献集中于 合作环境,所有玩家都有相同的目标,并寻求最大化这一目标 共同目标。已经开发了许多经验或理论算法,例如 VDN [Sunehag 等人,2017]、COMA [Foerster 等人,2018]、QMIX [Rashid 等人,2018]、MADDPG [Lowe 等人,2017]、MAPPO [Yu 等人,2021]。相比之下,我们的论文重点关注 竞争环境 和关键挑战是找到不可利用的政策。大多数为合作游戏设计的算法没有机制 处理对手的对抗性剥削。IIEFG 中的 MARL。这一系列工作 [Lanctot et al., 2009;海因里希等人,2015a; 布朗和桑德霍姆,2019;法里纳等人,2020;麦卡利尔等人,2021;科祖诺等人,

2021 年; Bai et al., 2022] 专注于学习不完美信息扩展型博弈 (IIEFG)。这一系列结果的重点是寻找纳什均衡,这是非 可利用的。然而,这些结果大部分集中在设置和应用程序上 (例如扑克) 状态空间是离散的。更重要的是,与 MG 相比, IIEFG 模型对转换做出了强有力的假设,该转换必须是树形的。的 IIEFG 的结果通常与信息数量呈超线性关系 集,通常随着游戏的时间长度呈指数增长。零和 MG 中的 MARL。之前并没有太多的实证研究 设计专门解决零和 MG 的算法。然而,有一个富裕阶层 可以直接应用于此设置的经验算法。这些算法 涉及结合单代理强化学习算法,例如深度 Q 网络(DQN)[Mnih et al., 2013] 或近端策略优化 (PPO) [Schulman et al., 2017],最好- 用于正态博弈的基于响应的纳什均衡发现算法,包括 虚构游戏 (FP) [Brown, 1951]、双重预言 (DO) [McMahan et al., 2003] 以及许多 其他人。其他一些例子,例如神经虚拟自我游戏(NFSP)[Heinrich 和 Silver, 2016],政策空间响应预言机(PSRO)[Lanctot et al., 2017],在线双 预言机 [Dinh et al., 2021] 和优先虚拟自我对战 [Vinyals et al., 2019] 也在一般属于此类算法或其变体。这些算法称为单 代理 RL 算法计算当前 “元策略” 的最佳响应,以及 然后使用基于最佳响应的正常形式游戏算法来计算新的 “元策略”。然而,这些算法本质上将 MG 视为正常形式的游戏,不能有效地利用 MG 内的精细结构。具体来说,对于范式 游戏中,FP 的收敛速度是动作数量的指数,而 DO 是 线性。然而,直接将马尔可夫博弈转换为正规形式博弈将会生成 地平线中的新行动空间指数,状态数量和行动数量 原始马尔可夫博弈。正如我们的实验所示,这会导致效率显著低下 随着 MG 尺寸的扩大。另一方面,对于二人零和马尔可夫博弈已有丰富的研究 从理论角度来看。其中许多作品 [Hu 和 Wellman, 2003; 白和

金,2020;白等人,2020;刘等人,2021; Jin et al., 2021a] 专注于表格 设置,要求状态和动作的数量是有限的。这些算法 被证明在许 多样本中收敛于纳什均衡策略 状态、动作、范围(或贴现系数)数量的多项式,以及 目标精度。其中,Nash Q-learning [Hu and Wellman, 2003] 是其中之一 沿着这条研究路线的最早的作品,它被证明收敛于 NE 的一般性 假设 NE 对于每个阶段博弈都是 唯一的,对博弈求和 学习过程。另一方面,Golf-with-Exploiter [Jin et al., 2021b] 是另一种 两人零和马尔可夫的可证明多项式收 敛的理论工作 游戏。我们的纳什 DQN 算法是基于可证明的表格算法设计的 纳什值迭代 [Liu et al., 2021],这是值迭代的自然 扩展 算法从单智能体设置到多智能体设置。为了更好地理解,我们提供了纳什Q-learning的异同详细比较,纳什 第二节中的 价值迭代、Golf-with-Exploiter 和 Nash DQN。3.7.8。[谢等人,2020] 考虑具有线性函数近似的马尔可夫博弈。有几个理论上的 致力于研究具有一般函数逼近的零和马尔可夫博弈[Jin 等人,2021b; Huang et al., 2021],其中包括神经网络函数逼近 作为特 殊情况。然而,这些算法具有样本效率,但计算效率不高 高效。他们需要解决具有复杂信心集的乐观政策:的限制,在实际中 无法运行。单代理深度强化学习。深度强化学习算法 有效地证明了在单代理环境中优化策略。政策外 像深度 Q 网络 (DQN) [Mnih et al., 2013] 及其变体这样的算法已经表明 在实时 Atari 视频游戏上的超人表演 [Badia et al., 2020]。在策略算法, 如近端策略优化 (PPO) [Schulman et al., 2017] 已广泛应用于具有离散和连续动作的复杂任务 空间。我们建议读者参考 [Sutton 和 Barto,2018;阿鲁库马兰等人,2017;利利克拉普 等,2015; Dong et al., 2020] 以及其中的参考资料以获取更多算法和详细 信息 关于单智能体深度强化学习。

3.3 预备知识

在本文中,我们考虑马尔可夫博弈[MGs,Shapley,1953;利特曼,1994],其中 将标准马尔可夫决策过程 (MDP) 推广到多人游戏设置中。每个玩家有自己的效用并优化其策略以最大化效用。我们考虑一个 MG 中的特殊设置称为两人零和游戏,它具有竞争关系 两名球员之间的关系。 更具体地说,考虑两人零和游戏的无限期折扣版本 MG,记为 MGpS、A、B、P、 r 、 γ q。S 是状态空间,A 和 B 是 分别为最大玩家和最小玩家的动作空间。P 是状态转移

分布, P_{p-s} , a , b_q 是给定当前状态的下一个状态的分布 s and action pair p_a , b_q . $r : S \times A \times B \rightarrow \mathbb{R}$ is the reward function. In the zero-sum 设置,奖励是最大玩家的收益和最小玩家的损失

到零和收益结构。 $\gamma \in [0,1]$ 是贴现因子。在每一步中,

两个玩家将观察状态 $s \in S$ 并选择他们的行动 $a \in A$ 和 $b \in B$ 独立且同时。可以在之后观察对手的行动 他们采取行动,并且将收到奖励 $r_p(s, a, b_q)$ (r 代表最大玩家, r 对于最小玩家)。然后环境转移到下一个状态 $s_1 \in S$ P_{p-s} , a , b_q 。

政策、价值函数。我们为每个参与者定义策略和价值函数。对于

对于最大玩家,(马尔可夫)策略是一个映射 $\mu : S \rightarrow \Delta A$ 。这里我们只考虑离散的动作空间,所以 ΔA 是动作集 A 上的概率单纯形。同样,策略

最小玩家为 $\nu : S \rightarrow \Delta B$ 。 $V_{\mu,\nu} : S \rightarrow \mathbb{R}$ represents the value function evaluated with policies μ and ν , which can 扩大为从状态 s 开始的预期累积奖励,

" 8

$\gamma V_{\mu,\nu}$

$\mu, \nu \in \mathcal{H}^1$

ν

$V_{\mu,\nu} = \mathbb{E}_{\mu,\nu} [\sum_{t=0}^{\infty} \gamma^t r_p(s_t, a_t, b_t) | s_0 = s]$ (3.1)

h " 1

Correspondingly, $Q_{\mu,v} : S \times A \times B \rightarrow \mathbb{R}$ is the state-action value function evaluated with strategy μ and v ,也可以扩展为预期累积奖励:

" 8

$$\gamma \sum_{h=1}^{\infty} \gamma^{h-1}$$

v

$$Q_{\mu,v}(s, a, b) = E_{\mu,v} [r + \gamma V_{\mu,v}(s') | s, a, b] \quad (3.2)$$

h “ 1

为了方便起见,本文中我们还使用了简化的表示法, $rP V_{\mu,v}(s, a, b)$: “

$E_{\mu,v}[r + \gamma V_{\mu,v}(s') | s, a, b]$,其中 P 作为过渡函数可以被视为算子。Similarly, we denote $rD_{\mu,v}(s, a, b) = E_{\mu,v}[r + \gamma Q_{\mu,v}(s', a, b) | s, a, b]$ for any state-action value function $Q_{\mu,v}$. 这样,两人MG的贝尔曼方程可以写为: $Q_{\mu,v}(s, a, b) = r + \gamma P V_{\mu,v}(s, a, b) = r + \gamma \sum_{s'} P(s'|s, a, b) Q_{\mu,v}(s', a, b)$ (3.3)

for all $s \in S, a \in A, b \in B$.

最佳响应和纳什均衡。在两人游戏中,如果另一位玩家 发挥固定马尔可夫策略的方式,优化学习者的策略与优化相同 关于 MDP 中单一代理的政策(与其他参与者的政策作为 en- 的一部分)

环境)。对于两个玩家的情况,给定最大玩家的策略 μ ,存在一个最佳的response of the min-player, which is a policy $v : \mu \mapsto v$ satisfying $V_{\mu,v} \geq V_{\mu,\mu'}$ for any $\mu' \in \Pi$. 类似地,对于最小玩家的策略 v ,存在一个最佳的response of the max-player, which is a policy $\mu : v \mapsto \mu$ satisfying $V_{\mu,v} \leq V_{\mu',v}$ for any $\mu' \in \Pi$.

for any $s \in S$. We simplify the notation as: $V_{\mu,v} = V_{\mu,v}$. Similar best response for a given min-player's policy v also exists as $\mu : v \mapsto \mu$ satisfying $V_{\mu,v} \leq V_{\mu',v}$ for any $\mu' \in \Pi$. By leveraging the Bellman optimality equation (3.3), best response can be derived through dynamic programming.

贝尔曼方程的老化(3.3),最佳响应可以通过动态导出

$$Q_{\mu,v}(s, a, b) = r + \gamma P V_{\mu,v}(s, a, b) = r + \gamma \sum_{s'} P(s'|s, a, b) Q_{\mu,v}(s', a, b) \quad (3.4)$$

v

纳什均衡 (NE) 被定义为一对策略 μ^*, v^* 作为

针对对手的最佳反应的最佳结果,表明:

$$V_{\mu^*,v^*} \geq V_{\mu,v^*} \geq V_{\mu,v} \geq V_{\mu',v^*} \geq V_{\mu',v} \geq V_{\mu',v^*} \quad (3.5)$$

μ^*, v^*

对于所有 $s \in \mathcal{S}$ 。NE 的存在已由之前的工作证明 [Filar 和 Vrieze, 2012]。此外,NE策略满足以下极小极大方程:

$$V^* = \min_{\mu} \max_{\nu} V(\mu, \nu) = \max_{\nu} \min_{\mu} V(\mu, \nu) \quad (3.6)$$

与普通博弈类似,但没有收益的双线性结构 矩阵。NE 策略是指没有参与者有动力改变自己的策略 策略。 μ^* 、 ν^* 的值函数记为 V^* 、 Q^* ,满足

$$\begin{aligned} (3.7) \quad & V^* = \min_{\mu} \max_{\nu} \sum_{s \in \mathcal{S}} p(s) V^*(\mu, \nu) \\ & Q^* = \max_{\nu} \min_{\mu} \sum_{s \in \mathcal{S}} p(s) Q^*(\mu, \nu) \end{aligned}$$

学习目标。策略 μ^* , ν^* 的可利用性可以定义为

当对抗他们的最佳反应时,与纳什策略相比,价值的有效性。Formally, the exploitability of the max-player can be defined as $V^* - V(\mu^*, \nu^*)$ while 最小玩家的可利用性定义为 $V(\mu^*, \nu^*) - V^*$ 。我们定义总数

μ^* , ν^* 的次优性简单地作为两个参与者的可利用性的总和

$$\text{Exploitability} = V^* - V(\mu^*, \nu^*) + V(\mu^*, \nu^*) - V^* = 0 \quad (3.8)$$

这个量在 MG 的文献中也被称为对偶间隙,可以表示为 被视为纳什均衡的距离度量。我们注意到纳什的二元性差距 平衡等于零。此外,我们进行实验的所有视频游戏都是 与两个玩家对称,这意味着 $V^* - V(\mu^*, \nu^*) = 0$ 。

纳什-VI 和纳什 Q-学习。MG 基于模型的表格算法---- Nash-VI ,通过执行 Bellman 最优性更新来计算接近最优的策略 等式。(3.7) 将 p 、 q 替换为使用样本的经验估计。经验的 转移矩阵中条目 $P_{s|s',a,b}$ 的估计由多少次计算 $p(s,a,b,s')$ 被访问次数除以 $p(s,a,b)$ 被访问次数。请参阅算法

12秒。了解更多详情。类似于Q-learning和VI之间的关系 单代理设置,Nash Q-Learning [Hu and Wellman, 2003] 是无模型版本

Nash-VI ,执行增量更新 $Q_{ps}, a, b, q \leftarrow p_1 - \alpha q_{ps}, a, b, q \leftarrow \alpha p_1 + \gamma \tilde{V}$ Nash $Q_{ps1}, \tilde{V}, \tilde{q}$
 每当观察到新样本 ps, a, b, r, s_1, q 时。

3.4 方法论

为了学习两人零和马尔可夫博弈的纳什均衡,本文提出 两种新颖的端到端深度 MARL 算法 - Nash-DQN 和 Nash-DQN-Exploiter。 Nash-DQN 将单智能体 DQN [Mnih et al., 2013] 与 Nash-VI [Liu et al., 2013] 结合起来 [2021]----一种可证明的表格马尔可夫博弈算法。 Nash-DQN-Exploiter 是一个变体 通过在学习阶段明确训练对抗性对手来实现 Nash-DQN 鼓励学习代理的探索。讨论 Nash-VI 的细节 在秒。 3.7.6。

3.4.1 纳什-DQN

我们在算法 2 中描述了 Nash-DQN,它将神经网络纳入 用于逼近 Q 值函数的表格 Nash-VI [Liu et al., 2021] 算法。 与单代理 DQN 类似,Nash-DQN 在训练中维护两个网络 process:Q网络及其目标网络,由 ϕ 和 ϕ_{target} 参数化, 分别。在每一集中, Nash-DQN 执行以下两个主要步骤:

1. 数据收集:Nash-DQN 采用 ϵ -贪婪策略进行探索。在每个
 状态 st , 概率为 ϵ , 双方随机采取行动;否则,他们将采样

动作 pat, bt, q 来自其 Q 值矩阵的纳什均衡(即 Nash $Q_{\phi_{\text{st}}}, j, j, q$, 见(3.9))。之后,我们将收集到的数据添加到经验重放缓冲区D中。

算法 2 纳什深度 Q 网络 (Nash-DQN) 1: 初始化重放缓冲区 $D \leftarrow \emptyset$, 计数器 $i \leftarrow 0$, Q 网络 $Q \leftarrow \Phi$ 2: 初始化目标网络参数: $\Phi_{\text{target}} \leftarrow \Phi$ 3: 对于第 k 集 “ 1, ..., K 做 4: 重置环境并观察 s_1 。 5: 对于 $t \leftarrow 1, \dots, H$ 做

6: % 收集数据 #Uniform $\mathbf{A}^B$ with probability ϵ

7: 示例动作 $\mathbf{a}_t, \mathbf{b}_t$ 来自 $\mathbf{p}_t, \mathbf{v}_t \leftarrow \text{NashpQ}(\Phi, \mathbf{p}_t, \mathbf{v}_t)$ 否则。 8: 执行动作 $\mathbf{a}_t, \mathbf{b}_t$, 观察奖励 r_t , 下一个状态 s_{t+1} 。 9: 将数据样本 $\mathbf{p}_t, \mathbf{a}_t, \mathbf{b}_t, r_t, s_{t+1}$ 存储到 D 中 10: % 更新 Q 网络 11: 随机采样小批量 $M \sim \mathcal{U}(D)$ 。 12: 对于所有 $j \in M$ 所做的 13: compute $\mu^j, \mathbf{v}^j \leftarrow \text{NashpQ}(\Phi_{\text{target}}, \mathbf{p}^j, \mathbf{v}^j)$

14: set $y_j \leftarrow r_j + \gamma \mu^j, \mathbf{v}^j \leftarrow \Phi_{\text{target}}(\mathbf{p}^j, \mathbf{v}^j)$.

15: 对损失 $J(\Phi) = \sum_j \|\Phi(\mathbf{p}^j, \mathbf{v}^j) - y_j\|^2$ 执行 m 步 GD 来更新 Φ 。 16: % 更新目标网络 17: $\Phi_{\text{target}} \leftarrow \Phi$ 如果 $i \% N = 0$: $\Phi_{\text{target}} \leftarrow \Phi$ 。

, 模型更新: 我们首先从重播缓冲区中随机采样一批数据 $M \subset D$, 然后使用下面的损失执行 m 步梯度下降来更新 Φ

$J(\Phi) = \sum_j \|\Phi(\mathbf{p}^j, \mathbf{v}^j) - y_j\|^2$,

其中目标 y_j 根据第 14 行计算。我们采用设置约定 $\Phi_{\text{target}}(\mathbf{p}^j, \mathbf{v}^j) = 0$ for all terminal state $\mathbf{s}^j$ 。

这里, NashpQ 是正常形式博弈的 NE 子例程, 它采用支付矩阵 $\mathbf{A} \in \mathbb{R}^{A \times B}$ as input and outputs one of its Nash equilibria $\mu^*, \mathbf{v}^* \in \Delta$. In math, we have:

$\mu^*, \mathbf{v}^* \in \Delta$ is a Nash equilibrium if and only if $\forall \mu, \mathbf{v} \in \Delta, \mu^* \succeq \mu, \mathbf{v}^* \succeq \mathbf{v}$. (3.9)

有几个现成的库可以实现这个纳什子例程。COM 后 比较几种不同实现的性能(详细信息请参见第 3.6 节), 我们发现 ECOS 库 [Domahidi et al., 2013] 效果最好, 从现在开始 作为我们算法中的默认选择。

关于目标值更新的选择(第14行),可以将其视为蒙特

卡洛估计
$$r_{ps_j} + \gamma E_{\pi} [P_{\pi} | s_j, a_j, b_j] q_{\max} - \min_{\pi} \hat{V}^{\pi} P_{\Delta B} \hat{\mu}^T Q_{\phi} \text{target}_{ps_1}, \dots, q_{\psi s}. \quad (3.10)$$

直观上,我们的目标是通过 Q_{ϕ} 来近似纳什均衡 Q 的 Q 值函数

网络 Q 。回想一下, Q_{ϕ} 是贝尔曼最优性方程的唯一解: $@ps, a, bq, Q_{\phi}(ps, a, bq) = r_{ps, a, bq} + \gamma E_{\pi} [P_{\pi} | s, a, bq] \max_{\pi} \hat{V}^{\pi} P_{\Delta B} \hat{\mu}^T Q_{\phi}(ps_1, \dots, q_{\psi s}). \quad (3.11)$

因此,通过对 ϕ 执行梯度下降来最小化平方损失,如下所示 如图 15 所示,随着样本数量的增加, Q_{ϕ} 将减小其贝尔曼误差,并最终收敛到 $Q_{\phi} \approx Q$ 被收集。最后,我们注意到目标纳什 Q 网络 (Q_{ψ}^{target}) 已更新 以延迟方式作为 DQN 来稳定训练过程。

3.4.2 纳什-DQN-利用者

Nash-DQN 依赖于 ϵ 贪婪策略进行探索。为提高探索能力

为了提高效率,我们提出了 Nash-DQN 的变体 -- Nash-DQN-Exploiter, 另外 在训练过程中引入剥削者。通过不断地利用弱点作为主要代理人,剥削者强迫主要代理人扮演她所扮演的角色 还是不擅长,从而帮助主力改进,发现更有效的 策略。我们在算法 3 中描述了 Nash-DQN-Exploiter。我们让主代理维护一个 Q 网络 Q_{ϕ} 并让剥削者维护一个单独的价值网络 Q_{ψ} , 两者都是 ps 、 a 、 bq 的函数。我们对 Nash-DQN 进行了两个关键修改。首先,在数据上

收集阶段,在状态 st , 我们不再选择 μ_t 、 ν_t 两者作为纳什均衡

由 Q 计算得出。相反,我们只选择 μ_t 作为 Q_{ϕ} 的纳什策略,但选择

利用者的策略 ν_t 是利用者 Q 网络下 μ_t 的最佳响应

算法 3 具有利用者的纳什深度 Q 网络 (Nash-DQN-Exploiter) 1:初始化重放缓冲区 D “ H , 计数器 i ” 0, Q 网络 Q_ψ , 利用者网络 $Q_{\tilde{\psi}}$ 。 2:初始化目标网络参数: $\psi_{\text{target}} \leftarrow \psi, \tilde{\psi}_{\text{target}} \leftarrow \tilde{\psi}$ 。 3:对于第 k 集 “ $1, \dots, K$ 做 4:重置环境并观察 s_1 。 5:对于 t “ $1, \dots, H$ 做

6: % 收集数据 #Uniform $p_A \wedge B_q$ w.p. ϵ

7:示例动作 p_{at}, b_{tq} 来自 $p_{\mu t}, v_{tq}$ 否则根据 (3.12) 计算。 8:执行动作 p_{at}, b_{tq} , 观察奖励 r_t , 下一个状态 s_{t+1} 。 9: 将数据样本 $p_{st}, a_t, b_t, r_t, s_{t+1}$ 存储到 D 中。 10: % 更新 Q 网络和剥削者网络 11:随机采样小批量 $M \sim \mathcal{U}(D)$ 。 12:对于所有 $j \in M$ 所做的 13: compute $p_{\mu^j}, v_{q^j} \leftarrow \text{Nash } Q_\phi \text{ target } p_{sj}^1, \dots, v_{qj}$

14: set $y_j \leftarrow r_j + \gamma \hat{J} Q_\phi \text{ target } p_{sj}^1, \dots, v_{qj}$.

15: set $\tilde{y}_j \leftarrow r_j + \gamma \min_b P_B \hat{J} Q_\psi \text{ target } p_{sj}^1, \dots, b_{qj}$

16:对损失 $j \in M$ $\|Q_\phi p_{sj}^1 - y_j\|^2$ 执行 m_1 步 GD 来更新 Q_ϕ 。 17:对损失 $j \in M$ $\|Q_\psi \tilde{p}_{sj}^1 - \tilde{y}_j\|^2$ 执行 m_2 步 GD 以更新 Q_ψ 。 18: %更新目标网络 19: $i \leftarrow i + 1$; 如果 $i \% N = 0$: $\psi_{\text{target}} \leftarrow \psi, \tilde{\psi}_{\text{target}} \leftarrow \tilde{\psi}$ 。

$Q_{\tilde{\psi}}$ 正式地,

$p_{\mu t}, i, q \leftarrow \text{Nash } Q_\phi p_{st}, i, i, q, q$ (3.12)

$v_{tq} \leftarrow \argmin_{\mu} J_t Q_{\tilde{\psi}} p_{st}, i, i, q, v$ 。

在模型更新阶段, Nash-DQN-Exploiter 遵循与 Nash-DQN-Exploiter 完全相同的规则 DQN 更新 Q 和 $Q_{\tilde{\psi}}$, 即主代理的 Q 网络。然而, 对于 更新利用者网络, Nash-DQN-Exploiter 利用不同的回归目标 获取第 15 行指定的损失函数。可以将目标视为蒙特卡罗 的估计 ” 我

1 $\wedge T$ 1 (3.13) $\|p_{sj}^1 - y_j\|^2$ $\|Q_\psi \tilde{p}_{sj}^1 - \tilde{y}_j\|^2$ $\min_b P_B \hat{J} Q_\psi \text{ target } p_{sj}^1, \dots, b_{qj}$

乙

我们以这种方式设定目标是因为我们的目标是近似 Q_{μ^j} , 这是

使用我们的开发者网络, 主要参与者 μ 的当前政策与其最佳响应相悖

Q_ψ . 回想一下, Q_μ 满足以下贝尔曼方程以获得最佳响应: $Q_\mu = \sum_{\mathbf{s}, \mathbf{a}} p(\mathbf{s}, \mathbf{a}) \sum_{\mathbf{s}', \mathbf{a}'} \gamma V^{\mu}(\mathbf{s}', \mathbf{a}') + \sum_{\mathbf{s}, \mathbf{a}} p(\mathbf{s}, \mathbf{a}) R(\mathbf{s}, \mathbf{a})$. (3.1)

因此, 通过对 ψ 执行梯度下降来最小化平方损失, 如第 17 行所示,

Q_ψ 将减少其(最佳响应版本)贝尔曼误差, 并近似 Q_μ .

3.4.3 理论依据

通过特殊选择 Q 网络架构 Q_ϕ , 小批量大小 $|M|$ 和数量 对于 GD 的步骤, 我们的算法 Nash-DQN 和 Nash-DQN-Exploiter 都简化为 标准算法 Nash-VI [Liu et al., 2021] 和 Nash-VI-Exploiter 的 ϵ -贪婪版本 [Jin et al., 2021b] 用于学习表格马尔可夫游戏, 其中状态数和 行动是有限且微小的。请参阅第 2 节。3.7 有关连接的详细讨论 这些算法。

当用乐观探索代替 ϵ -贪婪探索时(通常在

构建置信上限的形式), Nash-VI 和 Nash-VI-Exploiter 都是 保证在表格设置中有效地找到 MG 的纳什均衡。

定理 30 ([Liu et al., 2021; Jin et al., 2021b])。对于表格马尔可夫游戏,

Nash-VI 和 Nash-VI-Exploiter 的乐观版本都可以找到 ϵ -近似纳什 equilibria in polypS , $A, B, p_1, \gamma, \epsilon, \log p_1$ steps with probability at least $1 - \delta$.

S 是状态的大小, A, B 分别是两个玩家动作的大小, γ 是

折扣系数。

我们将定理 30 的证明推迟到第 2 节。3.7. 我们强调, 相比之下, 现有 深度 MARL 算法, 例如 NFSP [Heinrich and Silver, 2016] 或 PSRO [Lanctot et al., 2017] 无法通过多项式收敛有效学习纳什均衡 表格马尔可夫游戏的速率。我们的模拟结果表明它们确实 非常高 寻找纳什均衡的效率低下。

3.5 实验

实验评估是在随机生成的表格马尔可夫上进行的 Slime Volley-Ball [Ha, 2020] 和 PettingZoo 上的游戏和两人视频游戏 雅达利 [Terry 等人, 2021b]。我们测试了所提出方法的性能以及 (a) 没有函数近似的基本表格形式的基线算法化(仅在表格环境中); (b) 具有深度神经网络的完整版本(在 表格环境和视频游戏)。对于 (a), 我们测量可利用性 通过使用地面实况转换和奖励计算确切的最佳响应 功能。这仅在表格环境中可行。对于 (b), 我们测量 通过根据学习到的策略训练单代理 DQN(开发者)来实现可利用性 直接利用它。

3.5.1 基线

出于基准测试的目的,我们有以下深度神经网络基线 可扩展测试的函数近似:

- * 自我对战(SP):每个智能体学习针对对手采取最佳响应策略 或者固定对手策略,即迭代最佳响应。
- * 虚拟自我对弈 (FSP) [Heinrich et al., 2015a]:每个智能体学习最佳的 针对对手历史策略的情景平均值的应对策略 设置,并将其保存到自己的策略集中。
- * 神经虚拟自我对弈 (NSFP) [Heinrich 和 Silver, 2016]:一种神经网络 FSP 的工作近似,明确维护一个策略网络来模仿 代理的历史行为,学习者学习针对代理的最佳反应 它。
- * 政策空间响应预言机 (PSRO) [Lanctot et al., 2017]:我们采用 基于双预言机(DO)的版本,每个智能体都会学习针对特定情况的最佳响应

对手策略集的元纳什策略,并将学习到的策略添加到 有自己的策略集。

对于没有函数逼近的表格情况,实现了 SP、FSP 和 DO 以 Q-learning 作为寻找最佳响应的基础学习代理。对于表格情况 通过函数逼近和视频游戏,所有四种基线方法都使用 DQN 作为 用于学习最佳响应策略的基本代理。算法的伪代码 SP、FSP、DO 在第 2 节中提供。3.7.

3.5.2 表格马尔可夫博弈

没有函数近似的表格形式。我们首先评估方法 (1) SP、(2) FSP、(3) PSRO、(4) Nash-DQN 和 (5) Nash-DQN-Exploiter 无功能 表格马尔可夫游戏的近似(即,没有神经网络)。他们减少 方法 (1) SP、(2) FSP、(3) DO、(4) 纳什值迭代 (Nash-VI) 和 (5) Nash 相应地,利用利用者(Nash-VI-Exploiter)进行值迭代。对于 SP、FSP 和 DO,我们采用 Q-learning 作为寻找最佳响应策略的子程序。作为

我们的深度 MARL 算法的表格版本、Nash-VI 和 Nash-VI-Exploiter 也 use Nash subroutine for calculating NE in normal-form games, with $\epsilon = 0.5$ for ϵ -greedy 勘探。第 2 节中提供了 Nash-VI 和 Nash-VI-Exploiter 的伪代码。3.7. 我们随机生成表格马尔可夫游戏,它具有离散状态空间

S , 两个玩家的离散动作空间 A 、 B 和地平线 H 1 状态转换 probability function $T: S \times A \times S \rightarrow [0, 1]$, $s, h \in S$ and reward function $R: S \times A \times S \rightarrow \mathbb{R}$:

$s, a, b \in S \times A \times B$, $s, h \in S$ are both i.i.d sampled uniformly over their ranges. As

如图 3.2 所示,我们在两个随机生成的不同大小的马尔可夫游戏上进行了测试: I. $|S| = 10, |A| = 2, |B| = 2, H = 3$; 二. $|S| = 100, |A| = 2, |B| = 2, H = 6$. 可利用性为 根据方程计算(3.8)式可以用动态规划求解 具有已知转换和奖励函数的表格案例。我们提出的两种算法 我们将视野编码到状态空间中,以便使用为折扣设计的算法 设置。

图 3.2:两个随机生成的马尔可夫博弈的表格案例实验。

没有函数逼近的情况下,可显着加速并降低可利用性 与其他基线相比,特别是对于更大的环境 (II)。这对齐 我们的理论依据如第 3.4.3 节所示。用神经网络作为函数逼近。在这组实验中---- 化,我们添加神经网络作为函数逼近器。我们评估 Nash-DQN 并 Nash-DQN-Exploiter 在相同的表格 MG 环境 I 和 II 上。我们设置了 基线方法的神经网络版本 - FSP、NFSP 和 PSRO,使用同一组 超参数和相同的训练配置。在训练过程中,模型 检查点保存在每种方法的不同阶段,并重新加载以供利用

测试。每种方法总共训练 5×10^4 集以获得最终模型,

作为漏洞利用测试,漏洞利用者接受了 3×10^4 集的训练。剥削者是

使用与基础代理相同的超参数从头开始训练的 DQN 代理(对于 寻找最佳响应)在 FSP、NFSP、PSRO 中。我们凭经验测量 可利用性 学习者策略如下:我们首先计算累积效用的平滑版本 开发者在每一集中实现的(平滑是通过对以上进行平均来进行的) 一小组相邻的剧集);然后我们报告最高平滑累积 剥削者的效用作为可剥削性的近似值。我们还测试了效果 使用 DQN 开发者通过针对 DQN 开发者进行训练来衡量可利用性 预言纳什策略(即真实纳什均衡)。两个表格的结果 环境如表3.1所示。如表 3.1 所示,Nash-DQN 和 Nash-DQN-Exploiter 明显优于所有其他方法。负面的 Oracle Nash 策略的可利用性值表明基于 DQN 的利用者

无法找到确切的理论最佳响应。尽管如此,它近似于 最好的反应很好。据报道,Oracle Nash 的可利用性非常接近于零,这证明了使用 DQN 进行开发测试的有效性。

表 3.1:两个表格马尔可夫博弈中的近似可利用性(越低越好)

方法 Nash Nash DQN Oracle SP FSP NFSP PSRO 环境 DQN 开发者纳什

表格环境 I 0.448 0.379 0.317 0.134 0.096 0.020 '0.027

表格环境 II 1.239 0.694 0.379 0.569 0.017 0.071 '0.082

3.5.3 两人视频游戏

为了评估所提出方法的可扩展性和鲁棒性,我们检查了所有的 PettingZoo li 中五个两人 Atari 环境中的算法 [Bellemare et al., 2013] brary [Terry 等人,2021b](拳击-v1、Double Dunk-v2、Pong-v2、网球-v2 2、环绕- v1)和环境 SlimeVolley-v0 中名为 Slime 的公共可用基准测试 排球[Ha,2020],如图3.1所示。为此设置测试的算法包括: (1) SP、(2) FSP、(3) NFSP、(4) PSRO、(5) Nash-DQN 和 (6) Nash-DQN-Exploiter。至 加快实验速度,每个环境被截断为每集 300 个步骤 培训和剥削。对于 Atari 游戏,观察基于 RAM 和 在 r0, 1s 范围内标准化。 与函数逼近的表格环境中的实验类似, 进行利用测试(使用单代理 DQN)来评估学习的模型。理想情况下,如果一个代理学习了完美的纳什均衡策略,那么根据定义,我们 应期望代理是完全不可利用的(即,即使是最强大的前任) 掠夺者只能在对称游戏中实现最多零的累积效用)。为了进行实验,我们首先训练了 5×10^4 个 episode 的所有方法。AF- 当方法经过充分训练后,我们采用它们的最终模型或某些分布 历史策略(FSP 为统一策略,PSRO 为元纳什策略),并且火车分开 剥削者与这些学到的策略进行对抗。我们将 DQN 代理实例化为 ex- ploiter 使用相同的一组超参数和网络架构进行学习 PettingZoo 中原始的 Tennis-v2 环境不是零和的,奖励包装器应用于 做到了。

刮擦固定训练的检查点。由此产生的学习曲线在explo- 化测试说明了剥削的程度。剥削者奖励大于零 表明由于博弈是对称的,代理已被利用。型号 剥削者奖励较低意味着更难被剥削(因此更好)。

图 3.3:六个两人零和游戏的漏洞利用测试的漏洞利用者学习曲线

视频游戏。

表 3.2:六款两人视频游戏的近似可利用性(越低越好)。

方法 SP FSP NFSP PSRO 纳什 DQN 纳什 DQN 剥削者

环境SlimeVolley	'0.099	0.316	0.643	'0.762	'0.396	'0.821
Boxing	99.475	93.683	24.544	66.891	'90.254	28.271
双倍的3.801	5.920	0.445	6.960	'0.539	1.367	
扣篮Pong	4.207	5.196	4.238	4.693	'3.336	'1.524
Tennis	4.176	2.355	2.643	2.943	'0.425	0.069
环绕	1.782	1.574	1.594	1.603	0.904	1.462

图3.3和表3.2显示了所有算法和基线的开发结果。 每种方法都针对五个随机种子进行训练,每个随机种子的模型为 在三个随机初始化下,使用 DQN 漏洞利用程序进一步利用了 104 集。 对于每种方法和环境,表 3.2 显示了性能最佳的模型及其 相应的可利用性。完整的结果在第 2 节中提供。秒。 ??。 价值观 表3.2中是利用测试中平滑利用者奖励的最大值

图 3.3。 基线方法 SP、FSP、NFSP、PSRO 在大多数游戏中表现不佳。

更好的模型

剥削者 剥削者

初始位置子选项。型号

剥削者 剥削者 次优。模型子选项。型号

(一) (二) (三) (四)

图 3.4:Boxing 开发测试中的关键帧:(a-c) 显示了次优模型

被剥削者剥削。(d) 显示我们提出的算法学习难以利用的策略 对剥削者的扭转策略具有鲁棒性。

这表明寻找近似纳什均衡策略的挑战 游戏。Nash-DQN 在所有方面都表现出优于其他方法的显着优势 六场比赛。除 Surround 环境外,Nash-DQN 实现非正向利用 五个环境的奖励,这表明了政策的不可利用性 通过 Nash-DQN 学习。Nash-DQN-Exploiter 还显示出无法利用的性能 SlimeVolley 和 Pong 环境。不同的环境表现出不同的水平 很难找到不可利用的模型。SlimeVolley 相对容易,几乎可以解决所有问题 实现可利用性接近于零的方法。环绕声通常很难解决,因为 游戏固有的复杂性。我们相信解决环绕声问题需要更多 先进的勘探技术可提高其性能。有趣的是,我们观察到,在 Boxing 环境的开发测试中,基线 SP 等方法有时会产生始终停留在角落的策略 地面。如图3.4所示,黑色代理使用SP学习到的次优模型 算法,它试图避免与白人对手(a-b)发生任何接触。这样的政策 (总是躲在角落里)在与普通玩家或人工智能对战时还不错 政策可能没有考虑到这种极端情况,因此甚至无法找到 黑特工。然而,这一政策很容易受到利用。一旦被剥削者 探索接触黑色特工的方法,(c) 我们的剥削者学会了严重剥削 这样的政策短时间内。相反,我们的算法 Nash-DQN 和 Nash-

DQN-Exploiter 永远不会学习如此容易被利用的策略。学习的模型 Nash-DQN 和 Nash-DQN-Exploiter 通常会积极接近利用者 并直接与之对抗,这在游戏中更难利用。此外,我们的政策对于剥削者的扭转策略更加稳健,因为 如图3.4(d)所示。为了解决我们模型上利用不足的可能性,我们利用了mod-

对于表 3.2 中的方法,使用较长时间(5×10^4 集)进行 els 处理,结果为

如图3.5所示。除了 Double Dunk 环境外,Nash-DQN 和 Nash- DQN-Exploiter 模型即使长期在四种环境中仍然难以被利用 足够的剥削。 Double Dunk 的难点在于每个智能体需要控制一个 两名玩家组成的团队通过团队协作进行竞争,这可能需要更长的时间 培训时间进一步完善所学政策。

图 3.5:对五款视频游戏进行较长时间利用测试的利用者学习曲线。

3.6 范式纳什求解器的比较

游戏

在本节中,我们将展示纳什均衡求解器(即纳什求解子例程 Nash) 对于零和范式博弈,这是所提出的算法的一个重要子程序 节奏。由于零和范式博弈的 NE 可以通过线性规划求解,一些涉及线性编程函数的软件包,如 ECOS、PuLP 和 Scipy 可以 被杠杆化。我们还实现了一个基于迭代算法的求解器----乘法 权重更新(MWU),详见第 2 节。秒。3.6.1.

3.6.1 乘法权重更新

MWU 算法 [Bailey 和 Piliouras,2018; Daskalakis 和 Panageas,2018] 不是- 在线学习环境中的遗憾,可用于解决两人零和中的NE问题 正常形式的游戏。给定一个收益矩阵 A (从最大玩家的角度来看), NE策略将通过迭代应用MWU来解决。具体来说,对于第 n 次迭代, 状态 s 和动作 pai , $b_j q$ 分别代表最大玩家 μ 和最小玩家 ν , i, j 是 离散动作空间的入口索引,则动作概率的更新规则为 两名球员是:

$$p_{nq}^T$$

$$\eta p_{nq}^T A_{pq} \mu_{n-1} q_{pai} |sq \leftarrow \mu_{nq} p_{pai} |sq r \quad p_{nq} p_{a-1} ; sq \eta p_{nq}^T A_{pq} \nu_{n-1} q_{pbj} |sq$$

$$(3.15) i=1 \dots \mu_i$$

$$p_{nq}^T A_{pq} \nu_{n-1} q_{pbj} |sq \leftarrow \nu_{nq} p_{pbj} |sq r \quad p_{nq}^T A_{pq} \mu_{n-1} q_{pai} |sq$$

$$\nu_{pn} l_{qpbj} |sq \leftarrow \nu_{pnq} p_{bj} |sq r \quad p_{nq}^T A_{pq} \mu_{n-1} q_{pai} |sq$$

$$(3.16) j=1 \dots \nu_{pnq} p_{bj} |sq ; sq \eta p_{nq}^T A_{pq} \mu_{n-1} q_{pai} |sq$$

其中 η 是学习率。通过迭代更新策略的每个动作条目 就支付矩阵而言,MWU 可证明收敛于 NE。

3.6.2 比较

我们进行实验来比较 NE 子程序的不同求解器,包括 Nashpy (对于单一纳什均衡或所有纳什均衡)、ECOS、MWU(单一或并行)、CVXPY、PuLP、Scipy、Gurobipy。该测试在 Dell XPS 15 笔记本电脑上进行 仅CPU计算。代码将在审核后发布(匿名

审查过程中)。实验在 6×6 随机矩阵上进行评估并求平均值

超过1000个样品。保证生成的马尔可夫博弈的零和性质 通过生成每个随机矩阵作为一个玩家的收益并将负值作为 对手的回报。如表所示。3.3、可解性表示求解器能否解决所有可能的问题 随机生成的矩阵(零和)。用于解决所有均衡的 Nashpy 无法汉化 一些简并矩阵。Scipy 和 Gurobipy 也无法解决一些回报 矩阵。其他求解器可以求解我们测试中的所有随机支付 矩阵,但有不同之处:

表 3.3: 针对零和矩阵博弈(6×6 随机矩阵)的不同纳什求解器比较。

求解器 单次样本耗时(s) 可解性 Nashpy (equilibria) 1 0.751 全部可解 Nashpy (equilibrium) 0.0016 对某些退化矩阵不可解 ECOS 0.0015 全部可解 MWU (single) 0.008 全部可解(但精度较低) MWU (parallel) 速度快,取决于批大小 3 全部可解(但精度较低) CVXPY 0.009 全部可解 PuLP 0.020 全部可解

Scipy 对某些矩阵不可解

Gurobi 0.01 对某些矩阵不可解 Nashpy (<https://github.com/dvinceknight/Nashpy>) 可用于实现两种版本的 NE 求解器:一种返回单个 NE,另一种针对给定的支付矩阵返回所有纳什均衡。ECOS (<https://github.com/embotech/ecos>) 是一个用于求解凸二阶锥规划的软件包。MWU 求解器是我们自行实现的,用于求解单个支付矩阵或并行求解一批矩阵。CVXPY (<https://github.com/cvxpy/cvxpy>) 是一个用于凸优化的 Python 软件包。PuLP (<https://github.com/coin-or/pulp>) 是一个基于 Python 的线性规划软件包。Scipy (`scipy.optimize.linprog()`) 是一个用于数值运算的通用软件包。Gurobi (<https://www.gurobi.com/>) 是一个用于线性和二次优化的软件包。

不同的求解速度和精度。可解性对本程序至关重要,因为由于应用了函数近似,支付矩阵中的数值可能是任意的。这些结果支持了我们选择基于 ECOS 的求解器作为所提出算法默认纳什求解子程序的决定,原因在于它在求解随机矩阵时兼具速度和鲁棒性。ECOS 最初是为求解凸二阶锥规划而构建的,这类问题涵盖了线性规划(LP)问题。它会尝试将输入矩阵转换为 Scipy 稀疏矩阵,从而加快求解过程。通过将正规形式博弈上的 NE 求解表述为 LP,我们可以接入 ECOS 求解器来获得解。诸如非负性和常数和这样的约束需要谨慎处理。其他求解器,如 Nashpy (equilibrium) 和 MWU (parallel),可以达到相近的计算耗时水平,但因为无法求解某些矩阵或结果精度较低而不太受青睐。具体而言,在批大小较大而 MWU 内循环迭代次数较少的情形下,MWU 可能比 ECOS 更快。然而,MWU 的精度取决于更新的迭代次数 [Bailey and Piliouras, 2018; Daskalakis and Panageas, 2018]。更多的迭代次数会带来更精确的近似,但也会增加 MWU 方法的计算时间。

经验上,我们发现子程序求解器的精度对于我们所提出的采用函数近似的算法至关重要,尤其是在具有长时间跨度的视频游戏中。此外,尽管我们已经通过比较选出了最佳求解器,但每次推断或更新步骤中所用求解器的计算耗时仍占相当大的比例。这为运行时效率的提升留下了一定空间。

3.7 表格型马尔可夫博弈上的算法

在本节中,我们进一步详细阐述我们的算法与表格型算法 Nash-VI 和 Nash-VI-Exploiter 之间的联系(见 3.7.8 节),并在 3.7.9 节中证明后两种算法的理论保证。随后,我们给出本文所用的所有子程序和算法的伪代码。具体而言,我们在 3.7.1 节介绍重要子程序的伪代码,随后依次给出以下几种算法的伪代码:自博弈(self-play, SP, 3.7.3)、虚拟自博弈(fictitious self-play, FSP, 3.7.4)、双重神谕法(double oracle, DO, 3.7.5)、纳什值迭代(Nash value iteration, Nash-VI, 3.7.6)以及带利用者的纳什值迭代(Nash value iteration with exploiter, Nash-VI-Exploiter, 3.7.7)。SP、FSP 和 DO 是实验比较中的基线方法,而 Nash-VI 和 Nash-VI-Exploiter 则分别是我们所提出算法 Nash-DQN 和 Nash-DQN-Exploiter 的表格型版本,不涉及函数近似。

3.7.1 纳什算法子程序

算法 4 Meta-Nash:元纳什均衡求解子程序

输入:两个策略集合 μ, ν ; 评估迭代次数 N

- 1: 初始化支付矩阵: $M_{i,j} \leftarrow 0, i \in \mathcal{P} | \mu | s, j \in \mathcal{P} | \nu | s$
- 2: for $\mu_i \in \mathcal{P} | \mu | s$ do
- 3: for $\nu_j \in \mathcal{P} | \nu | s$ do 4: for episodes $k \leftarrow 1, \dots, N$ do 5: 展开(rollout)策略 μ_i, ν_j 以获得单局回报 r_k $M_{i,j} \leftarrow \frac{1}{N} \sum_{k=1}^N r_k$ 6: $k \leftarrow k + 1$ 7: $\rho_\mu, \rho_\nu \leftarrow \text{Nash}(M)$
- 8: 返回 ρ_μ 或 ρ_ν

在介绍每个算法的伪代码之前,我们先总结在这些算法中所使用的若干子程序---Nash、Meta-Nash、Best-Response 和 Best-Response-Value。这些子程序在本节及后续各节中均以品红色标出。Nash:作为正规形式博弈的 NE 求解子程序,它在给定支付矩阵作为输入的情况下返回 NE 策略。具体而言,它使用第 3.6 节介绍的求解器,而 ECOS 是我们实验中的默认选择。

Meta-Nash:作为元纳什求解子程序(算法 4),它在给定两个策略集合 $\mu = (\mu_1, \dots, \mu_i, \dots, \mu_n)$, $v = (v_1, \dots, v_i, \dots, v_n)$ 的情况下,返回单边的元 NE 策略。对每

Best-Response:作为最优反应子程序,它返回满足式(3.4)的给定策略的最优反应策略。需要注意的是,我们这里所讨论的最优反应,是针对策略集合 μ_0, μ_1

算法 5 Best-Response I: 马尔可夫博弈中的最优反应子程序(已知转移、奖励函数)

输入: 混合策略 μ , 作为在 $\mu_0, \mu_1, \dots, \mu_n$ 上的一个分布

1: 初始化非马尔可夫策略 $\hat{\mu} \leftarrow \mu$, $\hat{v} \leftarrow v^0$, $h \leftarrow h^0$, $\mu_h \leftarrow pS^A \wedge Bqph^1 q^A$
 $S^A \wedge \tilde{N} r_0, 1s, v_h \leftarrow pS^A \wedge Bqph^1 q^A S^A \wedge \tilde{N} r_0, 1s$

2: 初始化非马尔可夫策略 $\hat{\mu}$ 、 $\hat{v}$ 的 Q 表, $Q \leftarrow tQh u, h \leftarrow h^0$, $Q_h \leftarrow pS^A \wedge$
 $Bqh \tilde{N} r_0, 1s$

3: 初始化非马尔可夫策略 $\hat{\mu}$ 、 $\hat{v}$ 的 V 表, $V \leftarrow tVh u, h \leftarrow h^0$, $V_h \leftarrow pS^A \wedge$
 $Bqph^1 q^A S^A \tilde{N} r_0, 1s$

4: for $h \leftarrow 1, \dots, H$ do

5: 对于所有 τh^1 :

$\tilde{y} \leftarrow \hat{\mu}$:

$Q_{\mu} \leftarrow$

$h \leftarrow \tau h^1, sh, ah, bh \leftarrow Ph psh^1 | sh, ah, bh \leftarrow qrrh psh, ah, bh \leftarrow Vh^1 p \tau h, sh^1 q s^1 PS \quad (3.17)$

$Vh \hat{\mu} \leftarrow prh^1, sh \leftarrow \min \hat{\mu} p^* | \tau h^1, sh \leftarrow qQ \hat{\mu} \leftarrow \hat{A} T$

$h \leftarrow prh^1, sh, \tau, \tilde{y} qvh p^* | \tau h^1, sh \leftarrow q \quad (3.18)$

$v \leftarrow h$

$\hat{v} h \leftarrow prh^1, sh \leftarrow \arg \min \hat{\mu} p^* | \tau h^1, sh \leftarrow qQ \hat{\mu} \leftarrow \hat{A} T$

$h \leftarrow prh^1, sh, \tau, \tilde{y} qvh p^* | \tau h^1, sh \leftarrow q \quad (3.19)$

$v \leftarrow h$

其中

$h^1 i$

$r i$

$i \leftarrow \mu h \leftarrow psh \leftarrow qppiq \leftarrow \Pi t^1 \leftarrow 1 \leftarrow \mu t^1 \leftarrow pat^1 \leftarrow st^1 \leftarrow q$

$\hat{\mu} h \leftarrow psh \leftarrow \tau h^1, sh \leftarrow q \leftarrow r \leftarrow h^1 j$

$j \leftarrow \rho \leftarrow pjq \leftarrow \Pi t^1 \leftarrow 1 \leftarrow \mu t^1 \leftarrow pat^1 \leftarrow st^1 \leftarrow q$

$\hat{\mu} \leftarrow$

6: 返回 v^* 或 $V^1 p s^1 q$

% $\hat{\mu}$ 是非马尔可夫混合策略 μ 的后验策略, $\hat{v}$ 是其最优反应

Best-Response-Value: 它的执行流程与作为最优反应子程序的 Best-Response 相同, 但会返回给定策略 $\hat{\mu}$ 下初始状态的平均值, 即式(3.8)中的 $V^1 \hat{\mu} \leftarrow ps^1 q$.

3.7.2 纳什 Q 学习

纳什 Q 学习(Nash Q-Learning)以及带利用者的纳什 Q 学习的伪代码分别见下方的算法 7 和算法 8。

3.7.3 自博弈

自博弈的伪代码见算法 9。

算法 6 Best-Response II: 马尔可夫博弈中的最优反应子程序(基于 Q 学习, 未知转移、奖励函数)

输入: 混合策略 ρ_μ , 作为在 $\mu_0, \mu_1, \dots, \mu_n$ 上的一个分布; 最优反应 Q 学习迭代次数 N; 软更新系数 α

1: 为最优反应玩家初始化 Q “ $tQ_h | h P r Hsu P R | S | ^h | B |$ 表, 2: for episodes k “ 1, ..., N do

3: 采样策略 μ_k “ ρ_μ

4: for t “ 1, ..., H do 5: % 收集数据

6: 采样贪心动作 at “ $\mu_k p | st q$

7: 以概率 ϵ , 采样随机动作 bt;

8: 否则, 根据 Q 采样贪心动作 bt “ $v p | st q$ 9: 展开环境以获得样本 pst, at, bt, rt, done, st¹ q (rt 是针对学习者玩家的) 10: % 更新最优反应 Q 值 11: if not done then 12: $Q_{target} t pst, bt q$ “ $rt + \gamma V t^1 pst^1 q$ 13: 其中 $V t^1 pst^1 q$ “ $\max_{b1} Q t^1 pst^1, b1 q$ 14: else 15: $Q_{target} t pst, bt q$ “ rt 16: $Q t pst, bt q \leftarrow Q_{target} t pst, bt q + \alpha (Q_{target} t pst, bt q - Q t pst, bt q)$ 17: if done then 18: break 19: 将 Q 表示为一个贪心策略 v^* 20: 返回 v^*

算法 7 纳什 Q 学习

1: 初始化 Q : $S^A \times B^N \rightarrow R$, 给定 ϵ, γ, α .

2: for k “ 1, ..., K do 3: for t “ 1, ..., H do 4: % 收集数据

5: 以概率 ϵ , 采样随机动作 at, bt;

6: 否则, at “ $\mu p | st q, bt$ “ $v p | st q, p \mu p | st q, v p | st q$ “ NashpQpst, “, “ qq

7: 展开环境以获得样本 pst, at, bt, rt, done, st¹ q 8: % 更新 Q 值 9: if not done then 10: 计算 $p \mu^*, v^* q$ “ NashpQpst¹, “, “ qq

11: 设 $Q_{target} pst, at, bt q$ “ $rt + \gamma \hat{V} J Qpst^1, “, “ q \hat{v}$.

12: else 13: 设 $Q_{target} pst, at, bt q$ “ rt 14: $Qpst, at, bt q \leftarrow Q_{target} pst, at, bt q + \alpha (Q_{target} pst, at, bt q - Qpst, at, bt q)$ 15: if done then 16: break

算法 8 带利用者的纳什 Q 学习

初始化 $Q, Q: S \times A \times B \rightarrow \mathbb{R}$, 给定 ϵ, γ, α 。

for $k = 1, \dots, K$ do for $t = 1, \dots, H$ do

以概率 ϵ , 采样随机动作 a_t, b_t ;

否则, $a_t = \arg \max_a \sum_b p(b|a) Q(a, b, s_t)$, $b_t = \arg \max_b \sum_a p(a|b) Q(a, b, s_t)$, 其中 $p(b|a) = \frac{1}{|B|} \mathbf{1}(b \in B)$, $p(a|b) = \frac{1}{|A|} \mathbf{1}(a \in A)$ 。

展开环境以获得样本 $pst, at, bt, rt, done, st+1$ 。纳什 Q 值更新 if not done then $Qtarget(pst, at, bt) \leftarrow rt + \gamma V Nash(pst+1, q)$, 其中 $V Nash(pst+1, q) = \max_{a,b} Q(pst+1, a, b, q)$ 。

$Qtarget(pst, at, bt) \leftarrow (1 - \alpha) Q(pst, at, bt) + \alpha Qtarget(pst, at, bt)$ 。

其中 $p, v, q = NashQ(pst+1, q)$, else $Qtarget(pst, at, bt) \leftarrow (1 - \alpha) Qtarget(pst, at, bt) + \alpha Qtarget(pst, at, bt)$ if done then break

注: Nash 是一个 NE 求解子程序

算法 9 马尔可夫博弈的自博弈

1: 初始化策略 $\mu_0 = \arg \max_{\mu} \sum_{\nu} \mu(\nu) V(\mu, \nu)$, $\nu_0 = \arg \max_{\nu} \sum_{\mu} \mu(\nu) V(\mu, \nu)$

2: 初始化策略集合: $\mu = \arg \max_{\mu} \sum_{\nu} \mu(\nu) V(\mu, \nu)$, $\nu = \arg \max_{\nu} \sum_{\mu} \mu(\nu) V(\mu, \nu)$

3: 初始化元策略: $\rho_{\mu} = \mathbf{1}_s, \rho_{\nu} = \mathbf{1}_s$

4: for $t = 1, \dots, T$ do 5: if $t \% 2 = 0$ then 6: $\nu = \arg \max_{\nu} \sum_{\mu} \rho_{\mu}(\nu) V(\mu, \nu)$

7: $\nu = \arg \max_{\nu} \sum_{\mu} \rho_{\mu}(\nu) V(\mu, \nu)$ 8: $\rho = \arg \max_{\rho} \sum_{\mu} \rho(\mu) V(\mu, \nu)$ 9: else 10: $\mu = \arg \max_{\mu} \sum_{\nu} \rho_{\nu}(\mu) V(\mu, \nu)$

11: $\mu = \arg \max_{\mu} \sum_{\nu} \rho_{\nu}(\mu) V(\mu, \nu)$

12: $\rho_{\mu} = \arg \max_{\rho} \sum_{\mu} \rho(\mu) V(\mu, \nu)$ 13: 可利用性 = Best-Response-Value $\rho_{\mu}, \mu, \rho_{\nu}, \nu$

14: 返回 μ, ν

3.7.4 虚拟自博弈

虚拟自博弈的伪代码见算法 10。我们用 $\text{uniform}(\cdot)$ 表示策略集合上的均匀分布。

算法 10 马尔可夫博弈的虚拟自博弈

```

1: 初始化策略  $\mu_0 \leftarrow \text{tph } u, v \leftarrow \text{tvh } u, h \in P \text{ rHs}$ 
2: 初始化策略集合:  $\mu \leftarrow \text{tp0 } u, v \leftarrow \text{tv0 } u$ 
3: 初始化元策略:  $\rho\mu \leftarrow r1.s, \rho v \leftarrow r1.s$ 
4: for  $t \leftarrow 1, \dots, T$  do 5: if  $t \% 2 \leftarrow 0$  then 6:  $v \leftarrow \text{Best-Response } \rho$ 
    $\check{T} \leftarrow \mu, \mu q$ 
7:  $v \leftarrow v \text{ t } v \text{ u}$  8:  $\rho \leftarrow \text{Uniform } \mu q$  9: else 10:  $\mu \leftarrow \text{Best-Response } \rho \check{T} \leftarrow v, v q$ 
   11:  $\mu \leftarrow \mu \text{ t } \mu \text{ u}$ 
   12:  $\rho\mu \leftarrow \text{Uniform } \mu q$ 
   13: 可利用性 =  $\text{Best-Response-Value } \rho\mu, \mu q \text{ - Best-Response-Value } \rho v, v q$ 
   14: 返回  $\mu, \rho\mu, v, \rho v$ 
```

算法 11 马尔可夫博弈的双重神谕法

```

1: 初始化策略  $\mu_0 \leftarrow \text{tph } u, v \leftarrow \text{tvh } u, h \in P \text{ rHs}$ 
2: 初始化策略集合:  $\mu \leftarrow \text{tp0 } u, v \leftarrow \text{tv0 } u$ 
3: 初始化元策略:  $\rho\mu \leftarrow r1.s, \rho v \leftarrow r1.s$ 
4: for  $t \leftarrow 1, \dots, T$  do 5: if  $t \% 2 \leftarrow 0$  then 6:  $v \leftarrow \text{Best-Response } \rho$ 
    $\check{T} \leftarrow \mu, \mu q$ 
7:  $v \leftarrow v \text{ t } v \text{ u}$  8:  $\rho \leftarrow \text{Meta-Nash } v, \mu q$  9: else 10:  $\mu \leftarrow \text{Best-Response } \rho \check{T} \leftarrow v, v q$ 
   11:  $\mu \leftarrow \mu \text{ t } \mu \text{ u}$ 
   12:  $\rho\mu \leftarrow \text{Meta-Nash } v, \mu q$ 
   13: 可利用性 =  $\text{Best-Response-Value } \rho\mu, \mu q \text{ - Best-Response-Value } \rho v, v q$ 
   14: 返回  $\mu, \rho\mu, v, \rho v$ 
```

3.7.5 双重神谕法

双重神谕法的伪代码见算法 11。

3.7.6 Nash-VI

纳什值迭代(Nash-VI)的伪代码见算法 12。与 Nash-DQN(算法 2)不同的是,对于表格型马尔可夫博弈,Q 网络被替换为 Q 表,并以表格方式进行更新(如算法 12 第 13 行),给定估计的转移函数 $P^{\sim}$ 和奖励函数 $r^{\sim}$ 。此处不使用目标 Q。由于 Nash-VI 被应用于表格型马尔可夫博弈,我们在此以不含奖励折扣因子的分episode(episodic)设定来给出其伪代码,这与第 3.4.1 节略有不同。

算法 12 纳什值迭代(Nash-VI, ϵ -贪心采样版本)

```

1: 初始化  $Q \leftarrow tQ_h u, h \in \mathcal{H}$ ,  $Q_h : \mathcal{S} \times \mathcal{A} \times \mathcal{B} \rightarrow \mathbb{R}$ , 缓冲区  $D \leftarrow \emptyset$ , 给定  $\epsilon$ 、更新间隔  $p_0$ 。
2: for  $k \leftarrow 1, \dots, K$  do 3: for  $t \leftarrow 1, \dots, H$  do 4: % 收集数据
5:   以概率  $\epsilon$ , 采样随机动作  $a_t, b_t$ ;
6:   否则,  $a_t \leftarrow \arg \max_a Q_t(s_t, a, b_t)$ ,  $b_t \leftarrow \arg \max_b Q_t(s_t, a, b)$ ;
7: 展开环境以获得样本  $p_{st}, a_t, b_t, r_t, done, s_{t+1}$  并存入  $D$ 。 8: % 更新 Q 值 9: if  $|D| \geq p_0$  then
10:   for  $(s, a, b, s')$  in  $D$  do
估计  $P^{\sim}_{h|s} \leftarrow \frac{1}{|D|} \sum_{(a,b,s')} p_{st} \delta_{a,b,s'}$ ,  $ah \leftarrow a, bh \leftarrow b, q \leftarrow q_{st}$ 。 估计  $r^{\sim}_{h|s} \leftarrow \frac{1}{|D|} \sum_{(a,b,s')} r_t \delta_{a,b,s'}$ 。
 $q \leftarrow \frac{1}{|D|} \sum_{(a,b,s')} q_{st}$ 。
 $\hat{\mu}_{h|s}, \hat{v}_{h|s}$ 
13:  $Q_h(s, a, b) \leftarrow r^{\sim}_{h|s} + \gamma \sum_{s'} P^{\sim}_{h|s}(s', a, b) Q_h(s', a, b)$ 。 14: 其中  $\mu_{h|s}, v_{h|s} \leftarrow \arg \max_{a,b} Q_h(s, a, b)$ 。 15: if
done then 16: break

```

3.7.7 Nash-VI-Exploiter

带利用者的纳什值迭代(Nash-VI-Exploiter)的伪代码见算法 13。与 Nash-DQN-Exploiter(算法 3)不同的是,对于表格型马尔可夫博弈,Q 网络和利用者网络 $Q^{\sim}$ 被替换为 Q 表,并以表格方式进行更新(如算法 13 第 14 行和第 16 行),给定估计的转移函数 $P^{\sim}$ 和奖励函数 $r^{\sim}$ 。此处不使用目标 Q 和目标 $Q^{\sim}$ 。由于 Nash-VI-Exploiter 被应用于表格型马尔可夫博弈,我们在此

以不含奖励折扣因子的分episode(episodic)设定给出其伪代码,这与第 3.4.2 节略有不同。

算法 13 带利用者的纳什值迭代(Nash-VI-Exploiter, ε -贪心采样版本)

```

1: 初始化  $Q \leftarrow tQ_h u$ ,  $Q \leftarrow tQ_h u$ ,  $h P rHs$ ,  $Q_h$ ,  $Q_h : Sh \wedge Ah \wedge Bh \tilde{N} R$ , 缓冲区  $D \leftarrow \varnothing$ ,
   给定  $\varepsilon$ 、更新间隔  $p_0$ .
2: for  $k \leftarrow 1, \dots, K$  do 3: for  $t \leftarrow 1, \dots, H$  do 4: % 收集数据
   5: 以概率  $\varepsilon$ , 采样随机动作  $a_t$ 、 $b_t$ ;
6: 否则,  $a_t \leftarrow \mu^t p^{\pi^*} | s_t q$ ,  $b_t \leftarrow v^{\pi^*} | s_t p^{\pi^*} | s_t q$ ,
   7:  $p_{ut} p^{\pi^*} | s_t q$ ,  $v_{t p^{\pi^*} | s_t q} \leftarrow NashQ_{pst}, \pi^*, \pi^* q q, \tilde{v}_{t p^{\pi^*} | s_t q} \leftarrow \arg \min_v \mu^{\wedge T t} p^{\pi^*} | s_t q Q_t p_{st}, \pi^*, \pi^* q v$ .
8: 展开环境以获得样本  $p_{st}, a_t, b_t, r_t, done, s_{t+1} q$  并存入  $D$ 。 9: % 更新  $Q$  值 10: if  $|D| \% p_0 \leftarrow 0$  then
   11: for  $@ps, a, b, s_1 q P Sh \wedge Ah \wedge Bh \wedge Sh_{t+1}, h P rHs$  do
估计  $P^{\pi^*} h psh_{t+1} \leftarrow s_1 | sh_{t+1} \leftarrow s, ah_{t+1} \leftarrow a, bh_{t+1} \leftarrow bq_{t+1} ni_{t+1} \leftarrow l psh_{t+1} \leftarrow r$  12:  $sli q, ps, a, b, sli q P D$ 。 估计  $r^{\pi^*} h psh_{t+1} \leftarrow s, ah_{t+1} \leftarrow a, bh_{t+1} \leftarrow bq_{t+1} m_1 m r$  13:  $i_{t+1} \leftarrow l ri ps, a, bq, ps, a, b, ri q P D$ 。
    $\hat{\mu} h_{t+1}, \hat{v} h_{t+1}$ 
14:  $Q_h ps, a, bq_{t+1} \leftarrow r^{\pi^*} h ps, a, bq_{t+1} p P^{\pi^*} h V h_{t+1} q ps, a, bq_{t+1} Irs_1$  为非终止态, 15: 其中  $p \mu^{\wedge} h_{t+1}, v^{\wedge} h_{t+1} q \leftarrow Nash p Q h_{t+1} q$ 。
Exploit 16:  $Q^{\pi^*} h ps, a, bq_{t+1} \leftarrow r^{\pi^*} h ps, a, \# bq_{t+1} p P^{\pi^*} h V h_{t+1} q ps, a, bq_{t+1} Exploit 1 min b_1 P B h_{t+1} \mu^{\wedge} h_{t+1} ps_1 q^{\wedge T} Q^{\pi^*} h_{t+1} ps_1, \pi^*, b_1 q$ , 若
 $s_1$  为非终止态 17: 其中  $V h_{t+1} ps q \leftarrow 0$  若  $s_1$  为终止态 18: if done then 19: break

```

3.7.8 纳什系列算法的比较

Nash-DQN、Nash-DQN-Exploiter 与表格型算法的联系 我们首先指出, Nash-VI 和 Nash-VI-Exploiter 算法的 ε -贪心版本(如第 3.7.6、3.7.7 节所示), 在应用中将我们的算法 Nash-DQN(算法 2)和 Nash-DQN-Exploiter(算法 3)与 Nash-VI(算法 12)和 Nash-VI-Exploiter(算法 13)的 ε -贪心版本相比较, 我们注意到, 除了分episode设定之间的微小差异

相对于无限时间折扣设定,后两种算法是前两种算法在以下情形下的特例:

1. 将神经网络结构特化为表示每个状态-动作对的值表(即将两种算法都特化到表格化设定);
2. 令小批量 M 包含所有先前数据(即使用完整批次 D);
3. 令梯度步数 m 足够大,使梯度下降(GD)能找到目标函数的最小值;
4. 令 $N \rightarrow \infty$,即在每次迭代时都更新目标网络。

我们指出,使用较小的小批量规模和较少的梯度步数是为了在表格化设定之外加速实际训练。目标网络的延迟更新则用于稳定训练过程。

Nash 系列算法 我们将从四个方面详细阐述 Nash-VI、Nash Q-Learning、Golf-with-Exploiter 和 Nash-DQN 这四种算法之间的本质异同:是否基于模型/是否为模型无关方法、更新方式、回放缓冲区、以及探索方法。

* Nash-VI:基于模型;使用完整批次更新,不采用软更新;存在一个包含目前所有样本的缓冲区;采用 ϵ -贪心探索。

* Nash Q-Learning:模型无关;对每个样本使用随机梯度更新,采用软更新 $Q \leftarrow (1 - \alpha)Q + \alpha Q_{\text{target}}$;无回放缓冲区;采用 ϵ -贪心探索。

* Golf-with-Exploiter:基于模型;在一个置信集内以乐观的方式更新策略与利用者(exploiter);存在一个包含目前所有样本的缓冲区;探索所用的行为策略与 ϵ -贪心探索不同。

* Nash-DQN:模型无关;采用小批量随机梯度更新,使用均方误差(Mean Squared Error(Q, Q_{target}))进行基于梯度的更新;存在一个包含目前所有样本的缓冲区;采用 ϵ -贪心探索。

从这些异同中可以看出,三种理论算法 Nash-VI、Nash Q-learning 和 GOLF-with-exploiter 在细节上略有差异,而 Nash-DQN 可以视为对 Nash-VI 和 Nash Q-learning 二者的一种实用近似。

3.7.9 定理 30 的证明

[Liu et al., 2021] 中乐观 Nash-VI 算法的结果,以及 [Jin et al., 2021b] 中 Golf-with-Exploiter 算法的结果(当特化到表格化设定时)已经证明,乐观版本的 Nash-VI 为将该分幕式结果转换为本文所考虑的无限时间折扣设定,我们可以简单地将无限时间博弈截断至 $H = \frac{1}{1-\gamma} \ln \frac{1}{\epsilon}$ 步,使得剩余的累计奖励至多为

$$\begin{aligned} & \frac{1}{1-\gamma} \rho(1-\gamma^H) \epsilon \\ & \gamma^H \frac{1}{1-\gamma} \epsilon \\ & \gamma^H \frac{1}{1-\gamma} \epsilon \\ & \frac{1}{1-\gamma} \gamma^H \epsilon \end{aligned}$$

该值小于目标精度。为进一步处理分幕式设定中价值函数与策略的非平稳性问题(这要求价值函数与策略不仅依赖于状态,还依赖于所处的步数),我们可以将状态空间 s 扩展为 ps, hq ,以在状态空间中包含步数信息(直至截断点 H)。

现在,我们便可以将该分幕式结果应用于无限时间折扣设定,从而说明乐观版本的 Nash-VI 与 Nash-VI-Exploiter 都能够在 $\text{polypS}, A, B, \rho(1-\gamma)^H, \epsilon, \log$

第 4 章

零和电子游戏

本节内容基于论文《FightLadder: A Benchmark for Competitive Multi-Agent Reinforcement Learning》[Li et al., 2024d],该论文由 Wenzhe Li、Seth Karten 与 Chi Jin 合作完成,此前已发表于 ICML 2024。

4.1 引言

作为人工智能(AI)的一个活跃分支,深度强化学习(DRL)已在多个领域取得了显著成功,包括但不限于策略性博弈 [Silver et al., 2016; Li et al., 2020; Moravčík et al., 2017; Vinyals et al., 2019; Berner et al., 2019]、机器人控制 [Lillicrap et al., 2015; Andrychowicz et al., 2020b; Brohan et al., 2022],以及大语言模型对齐 [Ouyang et al., 2022]。支撑这些快速进展的不仅是样本高效强化学习算法的发展,还包括设计良好的基准测试(benchmark)的出现。这些基准测试提供了环境平台,统一了评估协议,使得对最先进方法的比较成为可能,激励了更优解决方案的产生,并指导了实际应用。举例而言,近端策略优化(policy proximal optimization,PPO)[Schulman et al., 2017] 在不同的单智能体强化学习基准测试中展现出了卓越的性能,因而被视为最广泛采用的单智能体强化学习算法之一 [Andrychowicz et al., 2020a]。在多智能体强化学习(MARL)领域,尽管也有一系列

图 4.1: FightLadder 目前支持多种跨平台格斗类电子游戏:街头霸王 II(Genesis 平台)、街头霸王 III(Arcade 平台)、饿狼传说 2(Genesis 平台)、真人快打(Genesis 平台),以及拳皇 '97(Neo Geo 平台)。

基准测试被提出,但其中大多数都聚焦于完全协作的设定。在竞争性环境方面,一些平台通过表格化表示和相对简单的动态来模拟博弈,例如棋类游戏,而另一些平台则基于复杂的游戏引擎,需要大量的计算资源与专家知识,例如星际争霸 II 与 DOTA。为了推动竞争性多智能体强化学习(MARL)研究的发展,并将博弈论成果转化为实际应用,亟需一个在复杂性、效率与通用性之间取得恰当平衡的完全竞争性游戏平台。众所周知,由于与其他玩家交互引入了额外的非平稳性,多智能体博弈比单智能体博弈更具挑战性。在不同类型的交互中,完全竞争性设定可能相当困难。人们长期以来一直在设计和玩竞争性游戏,并致力于构建强大的 AI 对手,使游戏更具挑战性从而更加引人入胜。此前的 AI 研究已经探讨了使用强化学习解决竞争性博弈的方案,但大多针对小规模游戏,如西洋双陆棋(Backgammon)[Tesauro et al., 1995] 或其他棋类游戏 [Schrittwieser et al., 2020; Brown and Sandholm, 2018, 2019]。此外,这类工作大多使用状态向量作为输入,这可以说比直接从原始像素输入进行学习更为容易,

而原始像素输入正是大多数流行电子游戏中常见的输入形式。相比之下,本文考虑的是格斗类游戏,其特征在于丰富的策略空间以及策略层面的显著深度----包括捕捉特定时机、通过利用对手的僵直状态进行反击、管理能量资源等。此外,这些游戏还拥有数量相当庞大且招式各异的角色,这为 AI 智能体掌握游戏又增添了一层复杂性。因此,我们希望构建一个面向一系列格斗类游戏的平台,该平台以图像作为输入并具有复杂的格斗动态,以此为广泛的 AI 研究界提供一个具有挑战性的竞争性多玩家平台。除了游戏平台本身,针对特定游戏设定的评估标准和基准结果对于推动该领域发展也至关重要。过去几年中, MARL 在解决多玩家博弈方面已从理论与实证两个角度得到了广泛研究。针对特定设定,大量算法已被提出 [Sunehag et al., 2017; Yu et al., 2022; Lowe et al., 2017; Silver et al., 2018; Lanctot et al., 2017; Vinyals et al., 2019; Ding et al., 2022]。然而,对于竞争性游戏设定,目前仍缺乏统一的评估标准以对不同方法进行全面比较。在本工作中,我们提出了 FightLadder,一个竞争性双人博弈基准测试。我们的贡献包括三个方面:我们构建了 FightLadder 平台以支持五种双人格斗类游戏,并易于在未来扩展到其他游戏。这些游戏支持涉及渲染图像的多种观测空间。基于前人的工作,我们提供了求解这些竞争性博弈中最流行算法的实现,包括 AlphaStar 联赛训练算法(league training algorithm)[Vinyals et al., 2019] 以及策略空间响应神谕(policy space response oracle)[Lanctot et al., 2017]。此外,我们还提供了一个统一的评估框架,配合游戏平台和算法库进行 Elo 评分与可利用性(exploitability)测试。我们报告了使用上述工具包得到的实验结果,以作为双人竞争性博弈设定的基线。MARL 的一个重要挑战在于其多样化的性质,其中包括协作性博弈、竞争性博弈、双人博弈以及多人博弈,这些设定具有相当不同的问题结构、性质与解概念。尽管开发一个能够统一处理所有这些设定的方案颇具前景,但在本工作中,

我们通过实证证明,在某种程度上,现有方法在结合视觉输入、丰富的策略空间以及缺乏大量人类示范数据的情况下,单纯求解竞争性双人零和博弈本身仍存在局限性。我们希望 FightLadder 能够特别聚焦于这一基础性的双人设定,为研究界提供一个垫脚石,帮助其先开发出有效的自博弈(self-play)风格算法来应对这一设定,再进而处理更为复杂的场景,并启发涉及更多类型交互的未来研究方向。

4.2 相关工作

MARL 环境。根据博弈的收益结构,MARL 环境可分为三种类型:完全协作型、完全竞争型以及一般型。现有的完全协作型博弈环境针对多种场景设计,包括模拟游戏如 MAMuJoCo [Peng et al., 2021]、卡牌游戏如 Hanabi [Bard et al., 2020]、视频游戏如小规模星际争霸 SMAC [Samvelyan et al., 2019] 和 Google Research Football [Kurach et al., 2020],以及实际场景,如网格世界中的 Traffic Junction [Sukhbaatar et al., 2016]、面向铁路网络的 Flatland [Mohanty et al., 2020]、网络负载均衡 [Yao and Ding, 2022],以及面向城市交通的 CityFlow [Zhang et al., 2019]。协作性环境的特点是所有智能体共享单一的奖励函数,这使其与竞争性博弈明显不同。另一方面,完全竞争型博弈基准测试的发展相对滞后。此前的竞争性环境要么是低维或离散状态空间的博弈,例如 Pommerman [Resnick et al., 2018] 与棋类游戏 [Tesauro et al., 1995; Schrittwieser et al., 2020; Brown and Sandholm, 2018];要么是需要大量计算资源、以图像作为输入的复杂游戏,例如星际争霸 II [Vinyals et al., 2019] 或 DOTA [Berner et al., 2019]。本文提出的格斗类游戏环境在复杂性、效率

与通用性之间取得了恰当的平衡。此前也有少量工作探索过格斗类游戏:[Go et al., 2023] 专注于为单一格斗游戏----街头霸王----开发算法,而本文则提供了一个支持多种格斗类游戏的环境。[Palmas, 2022] 提供了一个面向格斗类游戏的平台,但其大部分工作聚焦于单智能体设定。它缺乏针对具有适应性对手的双人场景的明确标准,也没有提供评估现有竞争性 MARL 算法的基准测试。[Khan et al., 2022] 聚焦于盲人设定下的格斗类游戏,其中智能体必须依赖声学输入进行游玩。最后,还存在一些同时具有协作与竞争特性的通用多智能体博弈环境,包括 MPE [Mordatch and Abbeel, 2018]、MAgent [Zheng et al., 2018]、Hide-and-Seek [Baker et al., 2019]、DMLab2D [Beattie et al., 2020]、Arena [Song et al., 2020b]、Smarts [Zhou et al., 2020]、Neural MMO [Suarez et al., 2021]、PettingZoo [Terry et al., 2021a]、MATE [Pan et al., 2022] 等。通用的多智能体一般和博弈相当难以评估----甚至最优解概念本身也仍不明确。相比之下,本文所考虑的完全竞争性设定呈现出清晰的博弈论性质以及明确定义的解概念。我们还指出,尽管上述平台中的一些支持若干完全竞争型博弈,但它们并未为竞争性 MARL 算法提供精心设计的评估工具包以及广泛的基线方法。

MARL 算法与评估。为了解决多智能体学习任务,研究者们提出了多种算法,并构建了便于使用和评估的库。PyMARL [Samvelyan et al., 2019] 是为解决 SMAC 任务而构建的一个早期 MARL 库,而 PyMARL2 [Hu et al., 2021] 则在 PyMARL 的基础上引入了 QMIX [Rashid et al., 2020] 进行扩展。EPyMARL [Papoudakis et al., 2020] 同样是 PyMARL 的一个扩展,是面向协作性博弈的统一库,支持包括集中式与去中心化学习、价值分解等不同的学习范式。MARLlib [Hu et al., 2023b] 包含了主要的协作性 MARL 算法,如 VDN [Sunehag et al., 2017]、MAPPO [Yu et al., 2022]、MADDPG [Lowe et al., 2017] 等。更近期的库包括 Pantheonrl [Sarkar

et al., 2022]、MALib [Zhou et al., 2023] 等。这些库主要支持面向协作性博弈的 MARL 算法,缺乏对求解竞争性博弈的支持。另一方面,也存在一系列使用如下算法求解竞争性博弈的研究:自博弈(self-play)[Silver et al., 2018]、虚拟博弈(fictitious play)[Brown, 1951]、Nash Q-learning [Hu and Wellman, 2003; Ding et al., 2022]、双神谕法(double oracle)[McMahan et al., 2003]、策略空间响应神谕(PSRO)[Lanctot et al., 2017] 以及联赛训练(league training)[Vinyals et al., 2019]。目前仍缺乏一个统一的基准测试,以在同一组任务上比较和评估这些算法的效率,尤其是在与深度强化学习结合时。本文在完全竞争性设定下解决了这一问题。我们专注于双人零和博弈,并提出了一个面向格斗风格完全竞争性博弈的平台,同时提供了流行算法的基线实现与评估。

4.3 多智能体强化学习

FightLadder 旨在为 MARL 与博弈论领域中面向完全竞争性双人博弈的新算法提供研究动力。马尔可夫博弈(Markov Games, MGs)[Shapley, 1953] 将单玩家马尔可夫决策过程(MDP)推广到多玩家设定。每个玩家拥有各自的效用,并优化自身策略以最大化该效用。MG 中的双人零和设定代表了两个玩家之间的竞争关系。通过塑形密集奖励,该博弈可以推广到一般和设定。我们将一个有限时间跨度的双人一般和部分可观测马尔可夫博弈记为

POMGpS, $\mathcal{O}, \mathcal{A}, \mathcal{B}, \mathcal{P}, \mathcal{O}, \text{tr} \mathcal{O}^i, \mathcal{H}_q$ 。S 为状态空间,该空间可以是部分可观测的,并通过观测发射函数 $\mathcal{O}: \mathcal{S} \rightarrow \mathcal{O}$ 转换为观测空间 $\mathcal{O}$ 。A 与 B 分别为两个玩

状态 s 出发的期望累计奖励,

$$V_{\mu,v}^s := E_{\mu,v} [r_t | s_t = s]$$

在零和博弈中,我们有 $V_{1,\mu}^s = -V_{2,\mu}^s, \forall s \in S$, 并为简便起见定义 $V_{\mu,v}^s = V_{1,\mu}^s$ 。

定义 31(最优响应)。对于第一个玩家的任意策略 μ , 第二个玩家总存在一个针对该策略的最优响应(best response, BR), 即满足

$$V_{\mu,v} \geq V_{\mu,v'} \quad \forall v' \in \mathcal{V}$$

我们将 $V_{\mu,v}$ 记作简化形式。 $V_{2,\mu,v}$

v 为第二个玩家的价值函数。针对第二个玩家的最优响应可以类似地定义。

定义 32(Nash 均衡)。零和设定下的 Nash 均衡(NE)定义为一对满足以下极小极大方程的策略 μ^*, v^* :

$$\mu^*, v^* = \arg \max_{\mu} \min_{v} V_{\mu,v} = \arg \min_{v} \max_{\mu} V_{\mu,v}$$

定义 33(可利用性)。第一个玩家策略 μ 的可利用性(exploitability)定义为

$V_{\mu,v^*} - V_{\mu^*,v^*}$, 即其最优响应策略 v^* 的价值, 或者说与 NE 值之间的次优性差距。另一方玩家策略 v 的可利用性可类似地定义。需要注意的是, NE 策略总是会导致零可利用性, 因此追求不可利用策略是该博弈中一个合理的目标。

4.4 FightLadder

在本节中, 我们介绍 FightLadder 的技术细节。在接下来的部分中, 我们首先介绍 FightLadder 的不同游戏设定, 随后阐述与该环境相对应的 MG 各元素, 并以强调本基准测试的特点作为结尾。

4.4.1 场景设置

FightLadder 在现代游戏模拟器 [Murphy, 2013; Nichol et al., 2018] 和算法开发者之间提供了灵活的接口。得益于这种灵活性, FightLadder 能够支持过去几十年间的多种经典格斗游戏, 包括 Street Fighter、Mortal Kombat、Fatal Fury 和 The King of Fighters, 其中一些游戏至今仍非常流行。图 4.1 展示了 FightLadder 提供的若干格斗游戏截图。凭借这一多样化的游戏集合, 我们可以在背景、角色和运动动力学各不相同的多种格斗场景上对算法进行基准测试, 这进一步推动了通用而非在单一游戏上过拟合的新型算法的发展。为了提高可读性和清晰度, 本文余下部分将以 Street Fighter 为例进行说明和评估。其他格斗游戏与之非常相似, 读者可参阅第 4.8.2 节了解更多细节。我们以 [游戏别名]_[左侧角色]_vs_[右侧角色] 的形式为每个场景命名, 例如 Street Fighter 中的 sf_ryu_vs_ryu。

尽管 FightLadder 主要关注竞争性的双人设置, 但格斗游戏的本质使其能够无缝部署到单人场景中, 即智能体的任务是与内置的游戏 AI 对抗(例如 sf_ryu_vs_ryu(cpu))。在这种单人设置下, 用户可以选择角色并设置脚本化 AI 对手的难度。此外, 我们的基准测试还支持在更具挑战性的全流程游戏场景(例如 sf_ryu_full_game)中进行训练, 该场景要求智能体依次击败由计算机控制的全部 12 个角色, 且难度逐步提升。正如我们在后续实验中将看到的, 该场景也可以作为一项健全性检验, 用以验证基线算法能否从环境中学习到有效的行为。

4.4.2 状态与观测

我们将状态空间 S 定义为游戏模拟器在每一步动作之后存储的完整属性集合。与人类玩家一样, 智能体不允许访问

底层的完整状态,而只能访问由像素构成的观测空间 O ,即对应渲染屏幕的 128×100 RGB 图像。该图像包含双方玩家的位置和移动信息,以及屏幕顶部的血量。尽管我们默认使用像素作为观测,但我们也为用户提供了访问游戏状态附加信息的接口,包括双方智能体的位置、血量以及精确的倒计时数字。用户可以利用这些属性更好地理解智能体的行为,或增强特征表示。更多细节见第 4.8.2 节。

4.4.3 动作空间

在格斗游戏中,两名玩家共享同一动作空间 A 。原生的人类动作空间 A_{human} 旨在模拟街机游戏的摇杆控制,是一个 12 维的二值空间(['B', 'A', 'MODE', 'START', 'UP', 'DOWN', 'LEFT', 'RIGHT', 'C', 'Y', 'X', 'Z']),每一维表示某个按键是否被按下。需要注意的是,由于格斗游戏引擎的性质,该空间中包含许多冗余的、实际上无效的动作,例如同时朝相反方向移动,或同时移动与攻击。为了过滤掉这些冗余动作并构建一个更加结构化的空间,我们通过编码函数 $F: A_{human} \rightarrow A_{trans}$ 开发了一个经过分类转换的动作空间 A_{trans} 。具体而言, A_{trans} 是方向移动集 $A_{motion} = \{\text{防御、前进、跳跃、蹲下、后翻、前翻、进攻蹲、防御蹲}\}$ 与攻击移动集 $A_{attack} = \{\text{轻拳、中拳、重拳、轻脚、中脚、重脚}\}$ 的联合集合,如图 4.2 所示。每个动作会依据用户的配置保持一定数量的帧。游戏中还包含被称为近身攻击的特殊技巧,即投技(Throws)和擒抱(Holds),它们只能在靠近对手的特定区域内使用。

除了标准的动作集之外,格斗游戏的一个标志性元素是必杀技(special moves),这是一种强力的攻击或招式,需要玩家遵循

图 4.2:格斗游戏的动作与攻击动作空间。图像改编自《街头霸王 II》说明书。

图 4.3:角色 Ryu 在《街头霸王 II》中的必杀技示例(从左到右):波动拳(Fireball)、升龙拳(Dragon Punch)、龙卷旋风脚(Hurricane Kick)。图像改编自《街头霸王 II》说明书。

特定的动作序列(即按键的顺序组合,或按键保持与按键按下的组合),图 4.3 展示了一个示例。这些招式通常具有特殊属性(例如无敌帧、更大的攻击覆盖范围等),在游戏的策略性和深度中扮演着关键角色。它们对于更高水平的对局尤为重要,玩家可以由此创造出复杂的连招并压制对手。然而,我们观察到,让基线算法从零开始学会施展必杀技颇具挑战性,因为这需要智能体记住前几步的帧和动作,并在必杀技的动作序列中准确地做出下一步动作。此外,必杀技会因角色而异,这进一步增加了游戏的难度。因此,为了缓解这一挑战,我们也将硬编码的必杀技列表纳入动作空间的一部分,使智能体能够通过单一动作直接施展必杀技。

4.4.4 奖励

稀疏奖励(Sparse Reward)。双方智能体的目标都是在每一回合的比赛中最大化其胜率。稀疏奖励 r_{sparse} 在每一回合结束时给胜者赋值 +1,给败者赋值 -1。在稀疏奖励设置下,所有格斗游戏都是双人零和博弈,理论上保证至少存在一个纳什均衡[Filar and Vrieze, 2012],这直接导出了一对不可被利用的策略。

胜率(Win Rate)。对于两名玩家 A 和 B,策略 π_A 在单场比赛中战胜策略 π_B 可以定义为奖励关系

$$A \succ B$$

$$r_{sparse}(\pi_A, \pi_B) > r_{sparse}(\pi_B, \pi_A).$$

A B

其中 r_{sparse} 和 r_{sparse} 分别是零和设置下玩家 A 和 B 的稀疏奖励。胜率定义为获胜的概率 $p(\pi_A \succ \pi_B)$ 。

塑形密集奖励(Shaped Dense Reward)。虽然稀疏奖励便于评估,但我们发现基线算法难以从这种稀疏信号中有效地学习出良好的行为。为解决这一问题,

$$A \succ B$$

$r_{dense}(\pi_A, \pi_B) > r_{dense}(\pi_B, \pi_A)$ 。密集奖励还提供了一定的灵活性,用户可以通过配置奖励函数中的加权比例来控制智能体的进攻性。

4.4.5 特性

我们对所提出基准测试的以下特性进行说明,这些特性可能有利于 MARL 研究。

丰富的策略空间。我们基准测试的一个关键特性是格斗游戏本质所带来的丰富策略空间,这对博弈论算法的发展尤为有利。举例而言,格斗游戏要求玩家考虑:(a)角色多样性:每个角色都拥有独特的技能组合,各有优劣,因此需要掌握针对所有可能对手的策略与反策略,甚至在可以自由选择 and 排列角色时还需推理如何选择与排序;(b)机制的复杂性:格斗游戏设计有诸如无敌帧、命中判定框和连招系统等精巧机制,这对角色的微操构成了挑战;以及(c)对抗性对手:对手可能逐步适应玩家的策略,因此找到不可被利用的策略对于精通格斗游戏至关重要。

多样的难度等级。FightLadder 提供了几类场景:与单个 CPU 玩家对抗的单人模式(例如 `sf_ryu_vs_ryu(cpu)`)、单人全流程游戏模式(例如 `sf_ryu_full_game`)、双人模式(例如 `sf_ryu_vs_ryu`)、以及团队模式(在部分游戏中支持,例如 `The King of Fighters`)。难度等级按此顺序递增,因为双人模式(无 CPU)引入了额外的非平稳性(对手可能具有自适应性),而团队模式则提供了更丰富的策略空间。此外,FightLadder 支持为玩家及其对手指定任意的 CPU 难度等级和任意角色。这丰富了我们平台的特性以及策略空间的多样性。

计算效率。FightLadder 在使用上也享有高效的计算性能,与其他几种流行游戏环境的比较见表 4.1。其帧率比 SMACv2 快 13 倍,而内存占用仅为四分之一。虽然 FightLadder 的效率不及 PettingZoo Atari,但它提供了更高的游戏复杂度。复杂度与低计算成本之间的平衡对于大规模评估算法非常重要。

表 4.1:若干开源平台的 FPS 和内存使用情况。

环境	速度 (FPS)	内存 (MB)	FightLadder (Ours)	1935.76	195.46	SMACv2	146.72	876.96	PettingZoo	Atari	6268.18	32.13	DMLab2D
				1144.27	47.41								

逼真度与流行度。FightLadder 允许在完整长度的格斗游戏中测试智能体,其接口与人类感知方式相似,从而为竞争性 RL 算法提供了高逼真度的评估。此外,格斗游戏自发布以来一直日益流行,这使得将习得的 RL 智能体与人类专家玩家进行对抗测试变得更加容易。

开源与兼容性。FightLadder 面向广泛的 RL 研究社区设计,因此我们致力于提升其易用性,使所有潜在用户都能方便地使用。它与 Gym [Brockman et al., 2016] 接口兼容,用户可以直接利用现有的 RL 算法实现。

定制化、扩展性与灵活性。FightLadder 在配置和扩展方面极为灵活。在定制化方面,用户可以自定义动作空间(人类动作/转换后动作)、奖励函数(稀疏奖励/可调节的塑形密集奖励)、每步观测的帧数,以及访问有助于训练的附加信息。此外,我们的平台构建于流行的现代游戏模拟器之上,因此很容易扩展到未提供的其他游戏。具体而言,它支持 Gym Retro 和 MAMEToolkit,而这两者本身已经支持了大量的游戏。这种对多样化游戏的扩展能力,由我们的平台以最小的工程开发成本提供。更多细节请查阅我们的开源项目¹。

4.5 评估指标

对战内置游戏 AI。直接与游戏内置 AI 对战提供了一种衡量策略性能的直接方式。通常,格斗
<https://sites.google.com/view/fightladder/home>

游戏提供分级的难度结构,使玩家能够调整难度设置(例如,Street Fighter 具有八个不同的等级)。这种结构使得可以针对游戏的脚本化 AI,在不同难度挑战下对策略进行实证评估。然而,需要承认的是,硬编码对手所固有的局限性限制了该指标反映策略真实能力的程度。为简洁起见,我们将此类智能体统称为 CPU。

Elo 评分。智能体的水平可以通过 FIDE 评分系统 [Elo and Sloan, 1978] 进行排名,这是一种增量学习系统,会提高胜者的 Elo 并降低负者的 Elo。玩家 A 与 B 之间的 Elo 差异越大,Elo 更高的玩家 A 战胜 Elo 更低的玩家 B 的概率就越高。Elo 分数的计算遵循以下步骤:

首先,估计玩家 A 获胜的概率为:

$Elo_B - Elo_A$

$$p_A := \frac{1}{1 + 10^{\frac{Elo_B - Elo_A}{400}}}$$

然后,玩家 A 的 Elo 评分 Elo_A 将按以下公式更新:

$$Elo_A \leftarrow Elo_A + k(p_{\text{winner}} - p_A)$$

其中 k 为更新速率常数。该更新对玩家 B 以及排名系统中的其他任何玩家都是对称的。

对战 AI 剥削者(AI Exploiters)。如第 4.3 节所述,可剥削性(如定义 33)衡量的是某策略与该博弈纳什均衡之间的距离。具体而言,策略 μ 的可剥削性由其最佳

$$V_2(\mu) := \max_{\nu} \sum_{q \in \mathcal{Q}} p(q) (V_1(\nu, q) - V_1(\mu, q))$$

在实践中,我们可以使用任意单智能体深度 RL 算法作为剥削者(exploiter),以近似学习最优反应策略 $\nu^* : p \rightarrow \mu$ 。为了公平比较,我们应使用同一种一致的剥削者(相同的 RL 算法及相同配置)来评估不同基线方法的可剥削性。

对战人类玩家。尽管定义 33 是衡量可剥削性的通用指标,但它可能受限于所使用深度 RL 算法本身的能力。因此,我们还提供了一个接口,使人类玩家能够方便地与任何已训练好的模型进行对战。这一功能将直接、直观地展示智能体的优势与劣势,并激励开发者改进其算法,使其在总体上更加不可被剥削。鉴于现代 RL 算法在多种视频游戏中已展现出超越人类专家玩家的卓越成就 [Mnih et al., 2013; Vinyals et al., 2019; Berner et al., 2019],我们相信 FightLadder 将成为广大竞争性 MARL 社区一个前景可期的平台,研究者们最终将能够以显著更少的工程开发成本,构建出能够在更丰富的策略性游戏集合中击败世界冠军的 AI 智能体。

4.6 FightLadder-基线

为了方便社区在 FightLadder 平台上评估现有方法和新算法,我们开源了若干最先进(SOTA)竞争性 MARL 算法的实现,包括独立学习 [de Witt et al., 2020]、双时间尺度学习 [Daskalakis et al., 2020]、虚拟自我博弈 [Heinrich et al., 2015b]、策略空间响应先知(policy-space response oracle) [Lanctot et al., 2017] 以及联盟训练(league training) [Vinyals et al., 2019]。我们的代码库支持跨多个 GPU 的去中心化学习,且构建于 Stable-Baselines3 [Raffin et al., 2021] 之上,用户可以直接利用现成的 RL 算法实现。在我们的实验中,我们选择近端策略优化(PPO)[Schulman et al., 2017] 作为骨干策略优化算法。基线算法的更多细节见第 4.9 节。

4.7 结果

在本节中,我们在 FightLadder 所选定的游戏---Street Fighter 上提供基准测试结果。我们希望通过基准测试回答以下问题:(a) 现有的 RL 算法能否在单人场景下解决完整的视频游戏?(b)

第 1 级 (Guile) 第 2 级 (Ken) 第 3 级 (Chun-Li) 第 5 级 (Zangief) 第 6 级 (Dhalsim) 第 7 级 (Ryu) 1.0 1.0

调度分布 调度分布 胜率 0.5 0.5

0.0 0.0 第 9 级 (E. Honda) 第 10 级 (Blanka) 第 11 级 (Balrog) 第 13 级 (Vega) 第 14 级 (Sagat) 第 15 级 (M. Bison) 1.0 1.0

胜率 0.5 0.5

0.0 0.10 20 0 10 20 0 10 20 0 10 20 0 10 20 0.0 轮次 轮次 轮次 轮次 轮次 轮次

图 4.4:通过所提出的带课程学习的 PPO,在 sf_ryu_full_game 中的胜率曲线与调度分布柱状图。不同角色的对手以不同等级标注。第 4、8 和 12 级为无需战斗的奖励关卡,故省略。

最先进基线算法在双人竞争性设置下的性能表现如何比较?以及(c)多智能体训练是否有助于提升不可剥削性?

4.7.1 单人全流程游戏

为回答问题(a),我们在场景 sf_ryu_full_game 中以人类动作空间对 PPO 的性能进行了评估,作为可行性检验。如第 4.4 节所述,该场景要求智能体学习一个可泛化的策略,以应对难度逐步提升的所有不同角色。我们采用课程学习(curriculum learning)的方式,从易到难对策略进行训练。此外,为提高学习效率,我们开发了一个课程调度器(curriculum scheduler),用于在每个 epoch 之后为学习者匹配对手采样。更具体地说,对于当前策略为 π_L 的学习者 L ,我们从整个角色集合 C 中按照以下逆权重调度分布对其对手 C 进行采样:

$$C \propto \Delta p_{Cq} \pi_C q,$$

其中 $\pi_C q$ 是学习者对战对手的胜率, Δp_{Cq} 为单纯形(simplex)。直观而言,这样的课程会促使智能体更专注于最难对付的对手,这与优先经验回放(priority experience replay)类似。

图4.4展示了我们所提出方法在训练过程中的性能表现。经过20个epoch的训练(每个epoch包含1000万训练步,与从课程调度器中并行采样的对手进行对抗),该智能体能够以接近1的胜率击败每个等级的角色。除了以高概率击败每个角色外,训练后的策略还能以超过0.6的胜率完成整个游戏,超越了拥有数小时游戏经验的人类玩家。这一结果表明,现有的强化学习算法已经能够学习出一个表现良好的策略来解决完整的单人游戏问题,这为探索多智能体场景提供了良好的起点。作为补充实验,我们在完全相同的算法实现下,还测试了在该设定中加入硬编码必杀技招式表的情况。尽管这可能使智能体更容易学到更具进攻性的策略,但我们并未观察到整体胜率的显著提升。这表明,没有编码必杀技的智能体也能有效地学习出对抗CPU的策略。持续使用必杀技会使智能体陷入易受攻击的境地,而防御策略在游戏中同样至关重要。此外,鉴于经验丰富的人类玩家也能轻松施放必杀技(几乎瞬时执行动作序列),我们认为硬编码必杀技招式表不会成为训练智能体相对于人类玩家的优势所在。

4.7.2 双人基线算法的性能

为回答问题(b),我们在场景sf_ryu_vs_ryu中评估了第4.6节提到的五种SOTA算法:独立PPO(IPPO)、双时间尺度IPPO(2Timescale)、虚拟自我博弈(FSP)、策略空间响应神谕(PSRO)以及联盟训练(League)。IPPO和2Timescale可归类为独立学习范式,而FSP、PSRO和League可归类为基于种群的学习范式。对于每种算法,我们都使用在sf_ryu_vs_ryu(cpu)场景中针对最难的CPU训练得到的预训练策略来初始化智能体种群。我们使用我们不在sf_ryu_full_game中进行预训练,因为sf_ryu_vs_ryu并不需要与除Ryu以外角色对抗的技能。

```

1.0 LE0_left 0.94 0.66 0.87 0.97 0.93 0.97
LE0_left_h_10M 0.09 0.10 0.08
LE0_left_h_20M 0.29 0.08 0.21 0.8
LE1_left 0.86 0.47 0.84 0.90 0.82 0.86
LE1_left_h_10M 0.21 0.03 0.04 0.6 LE1_left_h_20M 0.35 0.41 0.38
MA0_left 0.93 0.91 0.77 0.95 0.95 0.45 0.88 0.90
0.4 MA0_left_h_0M 0.03 0.02 0.00 0.02
MA0_left_h_10M 0.14 0.25 0.22 0.17
MA0_left_h_20M 0.25 0.26 0.25 0.29 0.2
ME0_left 0.43 0.98 1.00 0.06
ME0_left_h_10M 0.26 0.31 0.31 0.0 _10 M t i g h t _h M t _h M t _h_0M t i g h t _h M g h t _h M t g h t _h M E _20 _10 0 _r _10 1 _r i _10 0 _r i
i g h 0 _r i g h i g h i g h M A g h L E g h L E M E 0 _r 0 _r 0 _r 1 _r i 0 _r i 0 _r M A M A M A L E L E

```

图4.5:联盟训练某一阶段中每对智能体的收益矩阵。在联盟训练中,每一方(左侧或右侧)分别设有一个主智能体(MA)、两个联盟利用者(LE0、LE1)以及一个主利用者(ME)。每一行的名称表示智能体信息,格式为角色_方位_检查点。检查点=h_xM表示在x百万步时保存的智能体历史版本。数值表示左侧(行)玩家对右侧(列)玩家的胜率。例如,ME0_right以高概率击败了所有MA0_left_h_xM,表明联盟中的主利用者能够充分利用先前的主智能体。同样,MA0_left对所有右侧智能体(除MA0_right外)都保持高胜率,这表明当前步数下的主智能体优于联盟中的其他智能体。

带有硬编码必杀技的转换动作Atrans,以充分释放智能体的潜力。为公平比较,我们使用相同的代码库(FightLadder-Baselines)并固定骨干PPO算法的超参数。我们对IPPO和2Timescale训练约5000万步,直至三个随机种子下的Elo值均趋于饱和;对FSP和

左侧 5 IPPO 4 League 2Timescale 3 PSRO 数量 FSP CPU 00 250 500 750 1000 1250 1500 1750 右侧
数量

00 250 500 750 1000 1250 1500 1750 Elo 图4.6:各基线方法排名前十的智能体的Elo评分分布。PSRO训练约2.5亿步,对League则由于种群规模更大而训练约7亿步。图4.5可视化展示了联盟训练过程中某一阶段的联盟片段。对每种算法,我们分别报告了训练过程中种群内智能体的训练Elo值。结果见4.10节,该结果表明所有基线算法在训练初期均在不断提升其策略。随后,IPPO和2Timescale逐渐收敛,并在Elo峰值附近振荡,而FSP、PSRO和League则持续提升其评分。这表明IPPO和2Timescale在训练过程中可能存在优化方面的问题,而基于种群的方法可能更适合用于格斗游戏中的策略学习。为比较不同的基线算法,我们从每种算法中选取排名前十的智能体(左右两侧各五个)组成一个新的种群,并计算该组智能体与CPU策略的测试Elo值。我们在表4.2中报告了每种算法的最高Elo值,并在图4.6中展示了这些智能体Elo值的分布情况。由此我们发现,League和PSRO显著优于其他基线方法,且基于种群的方法比独立学习方法取得了更好的结果,这与我们此前逐一考察各基线Elo值时的观察结果一致。另一方面,我们注意到CPU策略能够击败该组中的大多数智能体

除了少数表现最优的智能体之外,这说明现有的SOTA算法要在这类格斗游戏中达到高级甚至超人类水平的表现仍然极具挑战性。我们还注意到,无论是在各算法的单独评估中(4.10节,图4.10-4.14),还是在跨算法的整体评估中(表4.2),双方智能体在Elo值上都表现出不对称的强弱差异。这种不平衡可能源于多种因素,例如优化的不稳定性、种群或Elo计算带来的方差等,这可作为未来工作中一个有趣的研究问题。

表4.2:各基线方法的训练步数与最佳Elo评分比较,以CPU的Elo值作为参照。

方法	训练步数(左/右)	Elo(左/右)
IPPO	46M / 46M	1082 / 1164
League	647M / 630M	1682 / 1503
2Timescale	51M / 46M	1080 / 919
PSRO	176M / 161M	1262 / 1517
FSP	262M / 244M	1079 / 1150
CPU	N/A	1395 / 1541

4.7.3 训练智能体的不可利用性

为回答问题(c),我们依据第4.5节提出的评估方法,衡量各基线算法的不可利用性。具体而言,我们分别从每种双人基线算法中选取Elo值最高的模型,并将其可利用性与用于初始化第4.7.2节中基于种群方法的单人预训练模型进行比较。实际可利用性的计算方法是:将训练好的策略固定在一侧,并在另一侧部署一个PPO智能体作为利用者(exploiter)。该PPO利用者将被训练至收敛,根据定义33,该利用者的成功率即为原策略的可利用性估计值。

单智能体强化学习利用者。鉴于PPO在此前实验中于单人和双人场景下均表现出不错的性能,我们选用PPO作为训练利用者的算法。表4.3展示了各比较方法评估得到的可利用性

表4.3:各方法可利用性的比较。数值越低,表示所评估的策略对利用行为的鲁棒性越强。

方法 可利用性(左/右)

IPPO	0.96 $\sim$ 0.03 / 0.91 $\sim$ 0.03
League	0.94 $\sim$ 0.05 / 0.94 $\sim$ 0.00
2Timescale	0.96 $\sim$ 0.02 / 0.90 $\sim$ 0.05
PSRO	0.97 $\sim$ 0.02 / 0.88 $\sim$ 0.05
FSP	1.00 $\sim$ 0.00 / 0.95 $\sim$ 0.01
PPO	0.99 $\sim$ 0.02 / 0.99 $\sim$ 0.01

在三个随机种子下评估,由此我们观察到,通过PPO训练得到的单人预训练策略更容易被利用,其可利用性高于几乎所有选定的双人基线策略。因此,这一结果表明,像League和PSRO这样的双人学习算法有助于提升所学策略的鲁棒性。另一方面,PPO利用者最终学会了击败所有基线方法的策略(胜率大于0.5),这意味着这些算法都未能得到精确的纳什均衡策略,甚至与之相差甚远。因此,弥合这一差距是未来研究中一个具有挑战性的方向。

人类玩家作为利用者。除了使用强化学习算法来利用所学模型外,我们还尝试通过人力来利用其策略。在人类评估过程中,被评估的模型对人类玩家展现出了一定的鲁棒性(例如,在人类玩家进攻时进行防御),但一些简单的策略(例如在适当时机结合防御姿态与低踢)仍能相当稳定地击败它们。因此,基于这两项利用实验,我们观察到,现有的竞争性MARL算法在像《街霸》这样的竞技格斗游戏中,很难学习到不可利用的策略,这为研究界提出了一项新的挑战。

4.8 FightLadder细节

4.8.1 密集奖励

第*i*个智能体在第*t*步的塑形密集奖励定义如下:

$$r_i,t = \alpha \lambda pHP_i,t - \lambda HP_i,t - q \cdot pHP_i,t - \lambda HP_i,t - q \cdot r_i,bonus_s, \quad (4.1)$$

其中 α 为缩放因子, HP_i,t 表示智能体*i*在第*t*步的生命值(hit-point), λ 控制所学智能体的进攻性, i 表示对手智能体。在游戏结束时,智能体*i*将获得一个奖励 $r_i,bonus_s$ 。

4.8.2 游戏设置

表4.4展示了FightLadder所支持的全部游戏----《街头霸王II》(SF2)、《饿狼传说2》(FF2)、《真人快打》(MK)、《街头霸王III》(SF3)以及《拳皇97》(KOF97)----的观测、动作、奖励及环境中的其他要素。表4.4:FightLadder所支持游戏的规格说明。

SF2 FF2 MK SF3 KOF97

观测(像素) 100^128^3 112^128^3 112^160^3 112^192^3 112^192^3

支持人类动作 是 是 是 是 是 支持转换动作 是 是 是 否 否 塑形密集奖励 是 是 是 是 是 默认每步帧数 8 8 8 3 3 默认堆叠帧数 3 12 12 12 9 9 生命值、倒计时、 生命值、倒计时 生命值、倒计时、 生命值 生命值、倒计时、 可用附加信息 计分板、位置 计分板 位置、能量状态

4.8.3 MARL游戏平台比较

表4.5将我们的FightLadder与几个主要面向竞争性设定的流行MARL游戏平台进行了比较,比较维度包括观测空间、动作空间、是否 为提高计算效率,我们对堆叠帧进行均匀采样作为观测。

包含基线方法以及游戏中的智能体数量。在观测空间方面,"连续(Continuous)"表示以连续数值形式呈现的游戏向量形式潜在状态信息,"图像(Image)"表示视觉RGB信息作为观测。PommerMan [Resnick et al., 2018]使用网格环境,因此其观测仅包含离散值。在动作空间方面,除Arena [Song et al., 2020b]之外,大多数游戏仅涉及离散动作值。在这些平台的智能体数量方面,MPE提供了多种竞争性设定,如1v1、1vN、1v1v1等。MAgent包含100万个相互对抗的智能体,而在Neural MMO [Suarez et al., 2021]中,智能体数量为256或1024。我们的FightLadder和Arena中的团队模式支持两队之间的竞争性设定,其中每队包含多个角色,可由一个团队策略或多个独立的智能体策略进行控制。

表4.5:主流MARL游戏平台比较。

环境	观测空间	动作空间	基线	智能体数量	MPE [Mordatch and Abbeel, 2018]	连续	离散	有 1v1、1vN和1v1v1.....	MAgent [Zheng et al., 2018]	连续+图像	离散	有 100万	Arena [Song et al., 2020b]	连续+图像	连续/离散	有 1v1、N vN和团队模式	Neural MMO [Suarez et al., 2021]	连续	离散	有 256和1024	PettingZoo Atari [Terry et al., 2021a]	连续+图像	离散	无 1v1	PommerMan [Resnick et al., 2018]	离散	离散	无 2v2	FightLadder(本文)	连续+图像	离散	有 1v1和团队模式
----	------	------	----	-------	---------------------------------	----	----	----------------------	-----------------------------	-------	----	--------	----------------------------	-------	-------	-----------------	----------------------------------	----	----	------------	--	-------	----	-------	----------------------------------	----	----	-------	-----------------	-------	----	------------

4.9 FightLadder-Baselines的基线算法

独立学习(IPPO)。独立学习是将单智能体强化学习直接扩展到MARL中的一种简单方法。它将联合优化问题分解为每个智能体各自的优化问题,同时将所有其他智能体视为环境的一部分。它可以通过为每个玩家同时运行单智能体强化学习算法来轻松实现。理论上,这种独立学习范式存在次优性问题[Tan, 1993; Foerster et al., 2018],因为当其他智能体不断更新其策略时,环境会变得非平稳。然而,近期的研究[de Witt et al., 2020; Yu et al., 2022]发现,通过适度的超参数调优,IPPO在某些合作性MARL任务中相较于其他最先进算法可以成为一个强有力的基线。

双时间尺度学习(2Timescale)。双时间尺度学习遵循独立学习范式,但要求两个玩家依照双时间尺度规则更新梯度,即一个玩家使用比另一个玩家小得多的步长。由于这一改动,双时间尺度学习具备一些良好的理论性质----已被证明,在一些温和的假设下,满足双时间尺度条件的独立策略梯度算法在双人零和随机博弈中会收敛到纳什均衡[Daskalakis et al., 2020]。

算法14 用于马尔可夫博弈(MGs)的基于种群方法

```

1: 初始化策略  $\mu_0 \leftarrow \text{rand}(\mathcal{U}, \mathcal{V})$ ,  $\nu_0 \leftarrow \text{rand}(\mathcal{H}, \mathcal{P})$ 
2: 初始化策略集合:  $\mu \leftarrow \mu_0$ ,  $\nu \leftarrow \nu_0$ 
3: 初始化元策略:  $\rho_\mu \leftarrow \text{rand}(\mathcal{S})$ ,  $\rho_\nu \leftarrow \text{rand}(\mathcal{S})$ 
4: for  $t \leftarrow 1, \dots, T$  do
5:   if  $t \% 2 \neq 0$  then
6:      $\mu \leftarrow \text{Best\_Response}(\rho_\nu)$ 
7:      $\nu \leftarrow \text{rand}(\mathcal{H}, \mathcal{P})$ 
8:   依据具体算法更新  $\rho_\nu$ 
9:   else
10:     $\nu \leftarrow \text{Best\_Response}(\rho_\mu)$ 
11:     $\mu \leftarrow \text{rand}(\mathcal{U}, \mathcal{V})$ 
12:   依据具体算法更新  $\rho_\mu$ 
13: 返回  $\mu, \rho_\mu, \nu, \rho_\nu$ 

```

基于种群的方法。独立学习框架仅针对对手当前版本进行训练,由于缺乏多样性,可能出现失败或收敛缓慢的情况[Dresher et al., 2016]。基于种群的方法旨

* 虚拟自我博弈(Fictitious Self-Play, FSP)[Heinrich et al., 2015b],其中 ρ_ν 为均匀分布 $\text{Uniform}(\nu_0, \nu_1, \dots, \nu_{t-1})$ 。

* 策略空间响应神谕(Policy-Space Response Oracles, PSRO),其中 $\tilde{\mu}, \tilde{v}$ 通过求解 $\mu_0, \mu_1, \dots, \mu_{t-1}$ 与 $v_0, v_1, \dots, v_{t-1}$ 之间收益矩阵博弈的纳什均衡,

* 联盟训练(League Training, League),其中种群中引入了三种类型的智能体---主智能体、联盟利用者以及主利用者。主智能体既与自身对弈,也与种群中所有先前版本的智能体对弈;联盟利用者与所有先前的智能体对弈;主利用者则优化针对主智能体的最佳应对策略。每种类型的智能体采用不同的采样分布,该分布是自我博弈与优先虚拟自我博弈的混合。更多实现细节请读者参阅[Vinyals et al., 2019]。

4.9.1 训练细节

图4.7、4.8和4.9分别报告了FSP、PSRO和League中种群内策略的收益矩阵,其数值代表左侧玩家对右侧玩家的胜率。我们在一台配备192个CPU和8块A6000 GPU的服务器上训练了所有智能体。

4.10 各算法的Elo结果

图4.10从三个互补的角度---匹配胜率、训练进度以及策略索引---展示了IPPO种群的Elo分布情况。图4.11报告了2Timescale方法相应的Elo演化情况。图4.12展示了FSP产生的智能体的Elo分布情况。图4.13展示了PSRO的Elo结果。图4.14总结了经League训练得到种群的Elo表现情况。

1.0 1.0

FSP0_left 0.91 0.99 0.8 0.8 FSP0_left 0.62

0.6 0.6

FSP0_left_h_0M 0.00

0.4 0.4

FSP0_left_h_0M 0.52 0.2 0.2 FSP0_left_h_10M 0.11

0.0 0.0 _0M ght _10 _0M ght _ri M t_h _ri t_h FS t_h gh FSP0 _ri gh P0 _ri gh _ri FS FS FSP0 P0 P0

1.0 1.0

FSP0_left 0.76 0.84 0.95 0.99 1.00 FSP0_left 0.99 0.99 0.99

0.8 FSP0_left_h_0M 0.00 0.8

FSP0_left_h_0M 0.00 FSP0_left_h_10M 0.03 0.6 0.6

FSP0_left_h_10M 0.05 FSP0_left_h_20M 0.07

0.4 0.4 FSP0_left_h_30M 0.06 FSP0_left_h_20M 0.08

FSP0_left_h_40M 0.22 0.2 0.2

FSP0_left_h_30M 0.48 FSP0_left_h_50M 0.40

0.0 0.0 _20 _10 _0M gh t _40 _30 _20 _10 _0M t gh M M t_h _ri M M M M t_h _ri t_h t_h gh FSP0 t_h t_h t_h t_h gh FSP0 gh gh _ri gh
gh gh gh _ri _ri _ri FS _ri _ri _ri _ri FS FSP0 FSP0 P0 FSP0 FSP0 FSP0 FSP0 P0

1.0 1.0

FSP0_left 0.64 0.88 0.94 0.96 0.96 0.97 1.00 FSP0_left 0.96 0.97 0.94 0.97 1.00 1.00 1.00 1.00

FSP0_left_h_0M 0.00 FSP0_left_h_0M 0.01 0.8 0.8 FSP0_left_h_10M 0.02

FSP0_left_h_10M 0.02 FSP0_left_h_20M 0.03 0.6 0.6 FSP0_left_h_20M 0.04 FSP0_left_h_30M 0.02

FSP0_left_h_30M 0.06 FSP0_left_h_40M 0.11 0.4 0.4 FSP0_left_h_50M 0.19 FSP0_left_h_40M 0.16

FSP0_left_h_60M 0.09

FSP0_left_h_50M 0.21 0.2 0.2 FSP0_left_h_70M 0.04

[illegible]

图4.7:FSP细节(训练顺序从左上到右下):在FSP中,每一方(左方或右方)都对应一个智能体。每一行的名称表示智能体信息,格式为Character_Side_Checkpoint。Checkpoint=h_xM表示在第x百万步保存的智能体历史版本。数值表示左方(行)玩家对右方(列)玩家的胜率。

1.0 1.0

PSRO0_left 1.00 0.96 0.8 0.8 PSRO0_left 0.63

0.6 0.6

PSRO0_left_h_0M 0.02 0.65 0.02

0.4 0.4

PSRO0_left_h_0M 0.65 0.47 0.2 0.2 PSRO0_left_h_10M 0.31 0.98 0.03

0.0 0.0 _0M t_igh _10 _0M t_igh M t_h RO t_h RO0_r t_h igh 0_r igh igh PS RO PS RO RO0_r 0_r 0_r PS PS PS

1.0 1.0

PSRO0_left 0.95 0.92 0.89 0.99 1.00 0.96 PSRO0_left 0.93 0.98 1.00 0.96

0.8 PSRO0_left_h_0M 0.02 0.01 0.02 0.07 0.02 0.65 0.04 0.8

PSRO0_left_h_0M 0.02 0.07 0.02 0.65 0.04 PSRO0_left_h_10M 0.06 0.01 0.03 0.02 0.31 0.98 0.04 0.6 0.6

PSRO0_left_h_10M 0.03 0.02 0.31 0.98 0.04 PSRO0_left_h_20M 0.08 0.00 0.01 0.63 0.99 0.93 0.01

0.4 0.4 PSRO0_left_h_30M 0.04 0.01 0.07 0.93 1.00 0.96 0.00 PSRO0_left_h_20M 0.01 0.63 0.99 0.93 0.01

PSRO0_left_h_40M 0.16 0.39 0.87 0.94 0.94 0.99 0.07 0.2 0.2

PSRO0_left_h_30M 0.07 0.93 1.00 0.96 0.00 PSRO0_left_h_50M 0.08 0.97 0.90 0.82 0.91 0.98 0.06

0.0 0.0 _30 _20 _10 _0M t_igh _50 _40 _30 _20 _10 _0M t_igh M M M t_h RO M M M M M t_h RO t_h t_h t_h igh 0_r t_h t_h t_h t_h
t_h igh 0_r igh igh igh PS igh igh igh igh igh PS RO RO RO RO0_r RO RO RO RO RO RO0_r 0_r 0_r 0_r PS 0_r 0_r 0_r 0_r PS PS
PS PS PS PS PS PS PS

1.0 1.0 PSRO0_left 0.95 0.98 0.92 0.88 1.00 1.00 0.96 PSRO0_left 0.92 0.91 0.97 0.98 0.92 0.94 1.00 1.00 0.96

PSRO0_left_h_0M 0.00 0.00 0.09 0.02 0.01 0.02 0.07 0.02 0.65 0.04 PSRO0_left_h_0M 0.09 0.02 0.01 0.02 0.07 0.02 0.65 0.04 0.8 0.8

PSRO0_left_h_10M 0.02 0.00 0.35 0.06 0.01 0.03 0.02 0.31 0.98 0.00 PSRO0_left_h_10M 0.35 0.06 0.01 0.03 0.02 0.31 0.98 0.00

PSRO0_left_h_20M 0.04 0.06 0.22 0.08 0.00 0.01 0.63 0.99 0.93 0.01 PSRO0_left_h_20M 0.22 0.08 0.00 0.01 0.63 0.99 0.93 0.01 0.6 0.6

PSRO0_left_h_30M 0.05 0.05 0.18 0.04 0.01 0.07 0.93 1.00 0.96 0.00 PSRO0_left_h_30M 0.18 0.04 0.01 0.07 0.93 1.00 0.96 0.00

PSRO0_left_h_40M 0.04 0.04 0.02 0.16 0.39 0.87 0.94 0.94 0.99 0.01

PSRO0_left_h_40M 0.02 0.16 0.39 0.87 0.94 0.94 0.99 0.01 0.4 0.4 PSRO0_left_h_50M 0.02 0.06 0.23 0.08 0.97 0.90 0.82 0.91 0.98 0.03

PSRO0_left_h_50M 0.23 0.08 0.97 0.90 0.82 0.91 0.98 0.07 PSRO0_left_h_60M 0.03 0.09 0.12 1.00 0.91 0.65 0.53 0.79 0.94 0.01 0.2 0.2

PSRO0_left_h_60M 0.12 1.00 0.91 0.65 0.53 0.79 0.94 0.10 PSRO0_left_h_70M 0.19 0.08 0.96 0.99 0.92 0.86 1.00 0.98 0.98 0.18

PSRO0_left_h_70M 0.96 0.99 0.92 0.86 1.00 0.98 0.98 0.17 PSRO0_left_h_80M 0.23 0.93 0.84 0.91 0.93 0.87 0.98 0.99 1.00 0.04 0.0 0.0

_60 M M _50 _40 M M _30 M _20 M _10 _0M t_igh _80 M _70 M _60 M _50 M M _40 M _30 M _20 M _10 _0M t_igh t_h t_h t_h t_h
t_h t_h t_h 0_r t_h t_h t_h t_h t_h t_h t_h t_h t_h 0_r igh igh igh igh igh igh PSRO igh igh igh igh igh igh igh igh PSRO 0_r 0_r
0_r 0_r 0_r 0_r 0_r 0_r 0_r 0_r 0_r 0_r 0_r 0_r PS PS PS PS PS PSRO PS PS PS PS PS PS PS PS PSRO RO RO RO RO RO RO
RO RO RO RO RO RO RO RO

图4.8:PSRO细节(训练顺序从左上到右下):在PSRO中,每一方(左方或右方)都对应一个智能体。每一行的名称表示智能体信息,格式为Character_Side_Checkpoint。Checkpoint=h_xM表示在第x百万步保存的智能体历史版本。数值表示左方(行)玩家对右方(列)玩家的胜率。

1.0 1.0 LE0_left 0.53 0.93 LE0_left 0.61
 LE0_left_h_10M 0.15 0.19 0.40 0.8 0.8
 LE1_left 0.25 0.73 LE1_left 0.54
 LE1_left_h_10M 0.16 0.27 0.30 0.6 0.6
 MA0_left 0.58 0.56 0.57 MA0_left 0.85 0.91 0.97 0.80
 0.4 MA0_left_h_0M 0.08 0.03 0.00 0.02 0.4
 MA0_left_h_0M 0.48 0.52 0.53 MA0_left_h_10M 0.61 0.34 0.50 0.58
 0.2 0.2 ME0_left 0.97 0.06
 ME0_left 0.48 ME0_left_h_10M 0.44 0.71 0.70
 0.0 0.0 gh t_0M 0_r t t igh t igh 10 M gh t_0M 0_r t t igh t igh 0_r t_h igh 1_r 0_r t_h 0_r i t_h igh 1_r 0_r ME i MA LE LE _ ME 0_r MA
 LE LE 0_r igh igh igh 0_r MA MA ME
 1.0 1.0 LE0_left 0.94 0.66 0.87 0.97 0.93 0.97 LE0_left 0.89 0.86 0.34 0.89 0.94 0.97 0.78 0.93 0.68 0.72 0.94
 LE0_left_h_10M 0.09 0.06 0.01 LE0_left_h_10M 0.09 0.10 0.08 LE0_left_h_20M 0.07 0.09 0.70 LE0_left_h_20M 0.29 0.08 0.21 0.8 0.8
 LE0_left_h_30M 0.06 0.22 0.79
 LE1_left 0.86 0.47 0.84 0.90 0.82 0.86 LE1_left 0.94 0.91 0.76 0.92 0.94 0.98 0.91 0.98 0.81 0.88 0.90
 LE1_left_h_10M 0.10 0.02 0.72 LE1_left_h_10M 0.21 0.03 0.04 0.6 LE1_left_h_20M 0.09 0.38 0.06 0.6 LE1_left_h_20M 0.35 0.41 0.38
 LE1_left_h_30M 0.28 0.54 0.83
 MA0_left 0.93 0.91 0.77 0.95 0.95 0.45 0.88 0.90 MA0_left 0.84 0.94 0.91 0.80 0.85 0.90 0.98 0.43 0.90 0.92 0.84 0.88 0.94
 0.4 MA0_left_h_0M 0.16 0.02 0.05 0.02 0.4 MA0_left_h_0M 0.03 0.02 0.00 0.02 MA0_left_h_10M 0.30 0.14 0.13 0.76 MA0_left_h_10M
 0.14 0.25 0.22 0.17 MA0_left_h_20M 0.28 0.11 0.12 0.76
 MA0_left_h_20M 0.25 0.26 0.25 0.29 MA0_left_h_30M 0.44 0.16 0.28 0.87 0.2 0.2 ME0_left 0.31 0.67 0.28 0.82 0.06 ME0_left 0.43 0.98
 1.00 0.06 ME0_left_h_10M 0.12 0.17 0.76 ME0_left_h_10M 0.26 0.31 0.31 ME0_left_h_30M 0.06 0.05 0.70 igh 0.0 10 0.0 10 M MA 0_r
 0_r igh igh t_h t_h _20 _10 M t 30 M 0_r 0_r igh ht 0_r _30 ig MA ht_h 0_r _20 t_h M M t_h _0_r MA 0_r igh t_h M _0M t_h _t_h
 igh M MA t_h _0_r 10M igh M M t_h _0 igh ME LE 1_r MA igh 0_r t_h igh _10 t igh igh ME LE A0_1_r rig igh ht LE t_h _1_r igh 0M
 t_h 2 _10 M 0_r LE 0_r LE 1_r M igh t 0_r 0_r LE E1_ 0_r rigL igh ht LE t_h _0_r 3 igh t_h _10 igh 0M LE t_h _0_r igh 0M 2 ME MA LE
 0_r M igh t ME ME MA MA t_h LE M _10 0_r igh t
 1.0 1.0 LE0_left 0.96 0.91 0.61 0.56 0.78 0.91 0.99 0.66 0.87 0.93 0.84 0.87 0.98 LE0_left 0.620.870.88 0.310.310.310.610.880.89
 0.570.800.90 0.410.750.730.83 LE0_left_h_10M 0.05 0.09 0.36 LE0_left_h_10M 0.05 0.10 0.19 LE0_left_h_20M 0.06 0.05 0.14
 LE0_left_h_20M 0.06 0.05 0.15 LE0_left_h_30M 0.09 0.05 0.14 LE0_left_h_30M 0.06 0.11 0.14 0.8 LE0_left_h_50M 0.07 0.26 0.38 0.8
 LE1_left 0.920.960.92 0.630.830.900.940.971.00 0.850.990.95 0.860.920.970.95 LE1_left 0.94 0.96 0.50 0.91 0.96 0.94 0.99 0.61 0.95 0.92
 0.92 0.90 0.92 LE1_left_h_10M 0.04 0.11 0.29 LE1_left_h_10M 0.05 0.11 0.23 LE1_left_h_20M 0.08 0.14 0.44 LE1_left_h_20M 0.09 0.27
 0.24 LE1_left_h_30M 0.11 0.12 0.40 0.6 LE1_left_h_40M 0.10 0.17 0.59 0.6 LE1_left_h_30M 0.14 0.26 0.31 LE1_left_h_50M 0.11 0.31
 0.61 LE1_left_h_40M 0.17 0.31 0.37 LE1_left_h_60M 0.26 0.36 0.83 MA0_left 0.95 0.94 0.91 0.81 0.87 0.93 0.97 0.98 0.51 0.75 0.93 0.94
 0.88 0.84 0.94 MA0_left 0.930.950.940.910.790.870.860.940.950.970.480.920.970.97 0.910.960.960.98 MA0_left_h_0M 0.11 0.01 0.03
 0.20 MA0_left_h_0M 0.13 0.02 0.03 0.14 0.4 MA0_left_h_10M 0.44 0.11 0.11 0.44 0.4 MA0_left_h_10M 0.17 0.13 0.15 0.31
 MA0_left_h_20M 0.22 0.05 0.15 0.40 MA0_left_h_20M 0.12 0.09 0.15 0.25 MA0_left_h_30M 0.29 0.07 0.28 0.55 MA0_left_h_40M 0.37
 0.29 0.38 0.66 MA0_left_h_30M 0.27 0.17 0.34 0.36 MA0_left_h_50M 0.52 0.19 0.55 0.74 MA0_left_h_40M 0.32 0.29 0.63 0.43 0.2
 MA0_left_h_60M 0.75 0.40 0.69 0.85 0.2
 ME0_left 0.53 0.65 0.87 0.91 0.99 0.06 ME0_left 0.640.690.630.810.770.950.06 ME0_left_h_10M 0.11 0.11 0.52 ME0_left_h_10M 0.10
 0.26 0.34 ME0_left_h_30M 0.07 0.04 0.18 ME0_left_h_30M 0.13 0.06 0.22 ME0_left_h_50M 0.11 0.23 0.55 0_r i ME ght_ 0.0 ME 0 0.0
 0_r h_3 igh 0M MA ME0 M 0_r _ri t_h _10 ME _righ 0 t_ ME _righ h_50 0_r t_h M igh _30 t_ M MA t_igh ght 0_r h_4 i MA ght_ M
 0_r h_3 i 0 0 MA M h_10 0_r E0 M MA igh _rig 0_r t_h ht MA igh _50 0 t_ M MA ght_ M 0_r _2 ig 0M MA ht_h 0_r _10 h igh M t_h

MA_right_h_40_0_t_M MA_right_h_30_0_r_t_h M MA_ight_20M_0_r_h_ LE MA0 M_1_r_ri ig g LE ht_h ht 1_r_4 ig 0_0 igh 10M
 LEMA_h_0_1_r_0 M t LE ight_ri gh 1_r_h t LE ht_h M_1_r_2 igh 0M LE LE1 M t_h_10 LE ight_40_1_r_h M igh_20 LE LE_10_0_1
 Mt_h M_0_r_ri igh ght LE t_h 0_r_3 ig 0 LE ht_h M_0_r_2 LE right_ri gh 0_r_h t LE ight_50_0_h M LE right_30_0_r_h M ME igh
 0Mt_h LE 0M_1_0_r igh t igh_20 t_h M_ LE 10M_0_r igh t

图4.9:联盟训练细节(训练顺序从左上到右下):在联盟训练中,每一方对应一个主智能体(MA)、两个联盟剥削者(LE0、LE1)和一个主剥削者(ME)。每一行的名称表示智能体信息,格式为Character_Side_Checkpoint。Checkpoint=h_xM表示在第x百万步保存的智能体历史版本。数值表示左方(行)玩家对右方(列)玩家的胜率。

1.0 IPPO: Left IPPO: Left IPPO: Left 5M 11 M 17 M 8 种子1 0.8 23 M 29 M 35 M 1500 种子2 41 M 47 M 6 种子3 胜率 0.6 0.4 1000 Elo
数量 0.2 500 2 0.0 600 800 1000 1200 1400 1600 1800 0 0 1 2 3 4 5 0 400 600 800 1000 1200 1400 1600 Elo 步数 1e7 Elo 1.0 IPPO: Right
IPPO: Right IPPO: Right 5M 11 M 17 M 8 种子1 0.8 23 M 29 M 35 M 1500 种子2 41 M 47 M 6 种子3 胜率 0.6 0.4 1000 Elo 数量
0.2 500 2 0.0 400 600 800 1000 1200 1400 1600 0 0 1 2 3 4 5 0 400 600 800 1000 1200 1400 1600 Elo 步数 1e7 Elo

图4.10:使用IPPO算法训练的智能体种群的Elo评分。上方三幅图为左方玩家,下方三幅图为右方玩家。Elo评分分别对应匹配策略的胜率(左图)、训练步数(中图)以及策略数量(右图)绘制。

1.0 2Timescale: Left 2Timescale: Left 2Timescale: Left 5M 13 M 21 M 1500 15 种子1 0.8 29 M 37 M 45 M 种子2 53 M 61 M 种子3 胜率
0.6 1000 10 0.4 Elo 数量 500 5 0.2 0.0 400 600 800 1000 1200 1400 1600 0 0 1 2 3 4 5 0 400 600 800 1000 1200 1400 1600 Elo 步数 1e7 Elo
1.0 2Timescale: Right 2Timescale: Right 2Timescale: Right 5M 13 M 21 M 10 种子1 1500 0.8 29 M 37 M 45 M 8 种子2 53 M 61 M 种子
3 胜率 0.6 1000 6 0.4 Elo 数量 4 0.2 2 0.0200 400 600 800 1000 1200 1400 0 0 1 2 3 4 5 0 400 600 800 1000 1200 1400 1600 Elo 步数 1e7
Elo

图4.11:使用2Timescale算法训练的智能体种群的Elo评分。

1.0 FSP: Left FSP: Left FSP: Left 5M 38 M 71 M 1500 3 FSP 0.8 104 M 137 M 170 M 203 M 236 M 2 胜率 0.6 1000 0.4 Elo 数量 500 1 0.2
0.0 400 600 800 1000 1200 1400 0 0.0 0.5 1.0 1.5 2.0 2.5 0 400 600 800 1000 1200 1400 1600 Elo 步数 1e8 Elo 1.0 FSP: Right FSP: Right
FSP: Right 5M 38 M 71 M 1500 3 FSP 0.8 104 M 137 M 170 M 203 M 236 M 2 胜率 0.6 1000 0.4 Elo 数量 500 1 0.2 0.0 400 600 800 1000
1200 1400 0 0.0 0.5 1.0 1.5 2.0 2.5 0 400 600 800 1000 1200 1400 1600 Elo 步数 1e8 Elo

图4.12:使用FSP算法训练的智能体种群的Elo评分。

1.0 PSRO: Left PSRO: Left PSRO: Left 5M 36 M 67 M 1500 3 PSRO 0.8 98 M 129 M 160 M 191 M 222 M 1000 2 胜率 0.6 0.4 Elo 数量
500 1 0.2 0.0400 600 800 1000 1200 0 0.0 0.5 1.0 1.5 2.0 0 400 600 800 1000 1200 1400 1600 Elo 步数 1e8 Elo 1.0 PSRO: Right PSRO:
Right PSRO: Right 5M 36 M 67 M 1500 8 PSRO 0.8 98 M 129 M 160 M 191 M 222 M 6 1000 胜率 0.6 0.4 Elo 数量 4 0.2 2 0.0 400 600
800 1000 1200 1400 0 0.0 0.5 1.0 1.5 2.0 2.5 0 400 600 800 1000 1200 1400 1600 Elo 步数 1e8 Elo

图4.13:使用PSRO算法训练的智能体种群的Elo评分。

1.0 League: Left League: Left 25 League: Left 5M 116 M 1500 MA0 0.8 227 M 338 M 20 ME0 449 M 560 M LE0 胜率 0.6 1000 15 数量
Elo LE1 0.4 10 0.2 5 0.0 600 800 1000 1200 1400 0 0 1 2 3 4 5 6 7 0 400 600 800 1000 1200 1400 1600 Elo 步数 1e8 Elo 1.0 League: Right
League: Right League: Right 5M 116 M 1500 30 MA0 0.8 227 M 338 M ME0 449 M 560 M LE0 胜率 0.6 1000 20 数量 Elo LE1 0.4 500
10 0.2 0.0600 800 1000 1200 0 0 1 2 3 4 5 6 7 0 400 600 800 1000 1200 1400 1600 Elo 步数 1e8 Elo

图4.14:使用联盟训练方法训练的智能体种群的Elo评分。

第5章

多人一般和博弈

本节内容基于论文《Learning Distributed and Fair Policies for Network Load Balancing as Markov Potential Game》[Yao and Ding, 2022], 该论文与Zhiyuan Yao合作完成,曾发表于NeurIPS 2022。

5.1 引言

在云数据中心(DC)和分布式网络系统中,服务器部署在具有多处理器的基础设施上以提供可扩展的服务[Dragoni et al., 2017]。为了优化工作负载分配并减少额外的排队延迟,负载均衡器(LB)在此类系统中扮演着重要角色。最先进的网络负载均衡器依赖于启发式机制[Eisenbud et al., 2016; Desmouceaux et al., 2018; Aghdai et al., 2018b],以满足数据平面对低延迟和高吞吐量的约束。然而,这些启发式方法无法适应动态环境,并且需要人工干预,这可能导致云端最令人头疼的错误之一----错误配置。强化学习方法已经在分布式系统和网络问题中展现出性能提升[Chen et al., 2018; Mao et al., 2018; Xu et al., 2019; Sivakumar et al., 2019],但将强化学习应用于网络负载均衡问题仍具有挑战性。首先,与传统的工作负载分配或任务调度问题不同[Chen et al., 2018; Mao et al., 2018],网络负载均衡器对系统的观测是有限的,包括

[图形/流程图:原 PDF 中含嵌入式矢量图,未在文本抽取中展开。]

任务大小与实际服务器负载状态。由于服务器只知道自己已分配的任务数量,它们可能因大象任务(elephant task)的冲突而超载,从而导致服务质量(QoS)下降。

其二,为了保证云环境中的高服务可用性,数据中心(DC)中会部署多个负载均衡器(LB)。网络流量被分摊到所有 LB 上。这种多智能体设置使得 LB 只能获得系统的部分观测信息。

其三,现代数据中心基于异构硬件和弹性基础设施 [Kumar et al., 2020a],服务器容量各不相同。根据服务器的实际处理能力为其分配正确的权重是一项挑战,而这一过程通常需要人工干预----这可能导致易出错的配置 [Eisenbud et al., 2016; Aghdai et al., 2018b]。

客户端 算法 15 LB 系统转移协议 w_i 工作负载

1: 初始化服务器负载, $X_{jpt} \leftarrow 0, \forall j \in \mathcal{N}$ 服务器负载 i

均衡器 2: 对每个时间步 t 执行

3: 对每个 LB 智能体 $i \in \mathcal{M}$ 执行

4: 为到来的任务 w_{ipt} 选择动作 a_{ijpt} 服务器

5: 对每个服务器 j 执行 M

$X_{i=1}^M$ 6: 更新工作负载: 下一时间步:更新服务器负载状态 rM

$X_{jpt} \leftarrow X_{jpt} + \frac{1}{M} \sum_{i=1}^M w_{ipt} a_{ijpt} - v_{jpt} + \frac{1}{M} \sum_{i=1}^M w_{ipt} a_{ijpt}$

图 5.1: 网络负载均衡。

最后,考虑到分布式网络环境下低延迟、高吞吐量的约束条件,强化学习(RL)模型的交互式训练过程以及集中训练-分散执行(CTDE)方案 [Foerster et al., 2018] 可能带来额外的通信与管理开销。

在本文中,我们通过博弈论中的多智能体方法研究网络负载均衡问题,具体做法是通过指定合适的奖励函数----即基于方差的公平性(variance-based fairness)----将其形式化为马尔可夫势博弈(Markov potential game)。我们提出了一种分布式多智能体强化学习(MARL)网络负载均衡机制,该机制能够仅基于局部观测和推断来利用异步动作。负载均衡性能

提升情况通过基于事件的仿真和真实世界实验1进行评估。

5.2 相关工作

网络负载均衡算法。网络 LB 的主要目标是在服务器间公平地分配工作负载。网络负载均衡系统的系统转移协议在算法 15 中描述,并在图 5.1 中展示。现有的负载均衡算法对部分观测和不准确的服务器权重较为敏感。等价代价多路径(ECMP)LB 随机将新请求分配给服务器 [Theo Julienne, 2018; Araújo et al., 2018; Miao et al., 2017],这使得它们对服务器负载状态差异一无所知。加权代价多路径(WCMP)LB 根据服务器所配置资源(例如 CPU 算力)按比例分配权重 [Eisenbud et al., 2016; Shi et al., 2020; Cohen et al., 2021]。然而,静态分配的权重可能并不对应服务器实际的处理能力。如图 5.2a 所示,当应用程序的资源需求不同时,具有相同 IO 速度但不同 CPU 容量的服务器,其实际处理速度也会不同。主动 WCMP(AWCMP)是 WCMP 的一种变体,它周期性地探测服务器利用率信息(CPU/内存/IO 使用情况) [Aghdai et al., 2018a,b]。然而,主动探测会导致观测延迟,并产生额外的控制消息,这会降低分布式网络系统的性能。本地最短队列(LSQ)将新请求分配给本地观测到的当前网络连接数最少的服务器 [Goren et al., 2020; Barbette et al., 2020]。它并不考虑服务器处理能力的差异。最短期望延迟(SED)将"期望延迟"定义为本地观测到的服务器队列长度除以静态配置的服务器处理速度。然而,LSQ 和 SED 对部分观测和错误配置较为敏感。如图 5.2b 所示,当网络

在数据和仿真代码方面,仿真与真实世界实验的源代码和数据均已开源,见 <https://github.com/ZhiyuanYaoJ/MARLLB>。

CPU 差异比例 ECMP WCMP LSQ SED 3 IO 差异比例 4

QoS(延迟) q2CPU 2 q1CPU 2 IO 密集型 CPU-IO CPU 密集型 纯 CPU 0 理想 2 智能体 3 智能体 权重错误 2 智能体 3 智能体 均衡型 权重 权重错误 权重错误

(a) 由于实际服务器处理速度既取决于所配置的 (b) 当观测因多智能体而变得部分化,且服务器权重资源,也取决于应用程序特征,因此很难准确估计。配置错误时,现有网络负载均衡算法的性能会下降。

图 5.2: 现有网络负载均衡算法在真实场景下并非最优。

流量被分摊到多个 LB 之间,或服务器权重配置错误时,每种负载均衡算法的 QoS 性能相较于理想设置(全局观测且服务器权重配置准确)都会下降,而这种情况在真实世界的云数据中心中很普遍。

在本文中,我们提出了一种基于分布式 MARL 的负载均衡算法,该算法考虑了动态变化的队列长度(例如现代数据中心网络中的亚毫秒级 [Guo et al., 2015]),并能自主适应实际的服务器处理能力,而无需 LB 智能体或服务器之间的额外通信。

马尔可夫势博弈。势博弈(PG) [Monderer and Shapley, 1996; Sandholm, 2001; Marden et al., 2009; Candogan et al., 2011] 具有一种特殊的函数,称为势函数(potential function),它刻画了这样一种性质:任何一名参与者对其自身动作的单独偏离,都会以相同的方式改变其自身收益和势函数的值。PG 的一个理想性质是,纯纳什均衡(NE)在标准形式设置下总是存在,并且与势函数的最大值相一致。自我博弈(self-play) [Fudenberg et al., 1998] 已被证明对 PG 是收敛的。马尔可夫博弈(MG)是标准形式博弈向多步序贯设置的扩展。PG 与 MG 的结合产生了马尔可夫势博弈(MPG) [Fox et al., 2022; Leonardos et al., 2021],其中纯 NE 同样被证明存在。一些位于博弈论与强化学习交叉领域的算法 [Macua et al., 2018; Mguni et al., 2021; Fox et al., 2022] 已被提出用于求解 MPG。例如,独立自然策略梯度(independent nature policy gradient)已被证明对 MPG 收敛到纳什均衡(NE) [Fox et al., 2022]。

仿真的随机马尔可夫模型详见 Sec. 5.5

表 5.1: 探测频率、测量质量与通信开销之间的权衡。

探测频率(/s)	2.22	2.86	4.00	6.67	20.00	CPU(%)	48.33	44.56	39.84	32.65	21.97	RMSE	任务数	2.07	1.85	1.61	1.31	0.91	CPU(%)	0.28	0.40	0.52	0.68	0.85	斯皮尔曼相关系数	任务数	0.47	0.56	0.66	0.77	0.89	通信	2LB-7服务器	2.15	2.76	3.86	6.44	9.32	开销(kbps)	6LB-20服务器	18.40	23.66	33.12	55.20	165.60
----------	------	------	------	------	-------	--------	-------	-------	-------	-------	-------	------	-----	------	------	------	------	------	--------	------	------	------	------	------	----------	-----	------	------	------	------	------	----	----------	------	------	------	------	------	----------	-----------	-------	-------	-------	-------	--------

多智能体强化学习。近年来,MARL [Yang and Wang, 2020] 已被视为求解不同类型博弈的一条重要途径。在合作场景下,一系列基于联合值分解(joint-value factorisation)的工作已被提出,包括 VDN [Sunehag et al., 2017]、COMA [Foerster et al., 2018]、MADDPG [Lowe et al., 2017] 以及 QMIX [Rashid et al., 2018]。在这些工作中,团队内的玩家会被赋予一个全局奖励,但各自的策略经过优化以执行各自的动作,这被称为 CTDE 设置。MPG 满足值分解方法的假设条件,其中定义明确的势函数即为联合奖励。然而,在真实世界的分布式系统中部署 CTDE 强化学习模型,会因为智能体的同步以及轨迹的聚合而产生额外的通信延迟与管理开销。这些额外的管理与通信开销可能导致显著的性能下降----受限的吞吐量和增加的延迟----尤其是在数据中心网络中。如表 5.1 所列,当我们使用主动探测来测量服务器利用率信息时,更高的探测频率能带来更好的测量质量----就 CPU 使用率和服务器上正在运行的任务数量而言。然而,更高的探测频率同样会带来更大的通信开销,尤其是在大规模数据中心网络中。关于通信开销影响的详细实验设置以及定性和定量分析,详见 Sec. ??。通过利用 MPG 的特殊结构,独立学习方法可以由于联合状态与动作空间的分解而变得更加高效,这一点在所提出的方法中得到了利用。类似 MATRPO [Li and He, 2020]、IPPO [de Witt et al., 2020] 的方法遵循完全分散式设置,但适用于一般的合作博弈。

在分配公平性方面,FEN [Jiang and Lu, 2019] 被提出作为多智能体系统中公平奖励分配的一种分散式方法。他们将公平性定义为变异系数(coefficient of variation),并针对每个智能体进行分解。另一项工作 [Zimmer et al., 2021] 提出了一种用于合作博弈中公平策略的分散式学习方法。然而,这些方法中的分散式学习方式并未得到充分论证,而本文中负载均衡问题被正式刻画为一个 MPG,并验证了分布式训练的有效性。

5.3 方法

5.3.1 问题描述

我们将负载均衡问题形式化为一个具有强分布式和并发特性的离散时间动态博弈,其中智能体之间不存在集中式控制机制。我们令 M 表示 LB 智能体的数量(rM 表示 LB 智能体的集合 $t1, \dots, M u$), N 表示服务器的数量(rN 表示服务器的集合 $t1, \dots, N u$)。在博弈的时间跨度 H 内的每个时间步(或轮次) $t \in [1, H]$,每个 LB 智能体 i 接收一个工作负载 $w_i(t) \in \mathcal{W}$,其中 $\mathcal{W}$ 是工作负载分布, LB 智能体使用其负载均衡策略 $\pi_i \in \Pi$ 将任务分配给服务器,其中 Π 是负载均衡策略集合。在每个时间步 t , LB 智能体 i 采取动作 $a_i(t) \in \{1, \dots, N\}$,根据该动作,任务 $w_i(t)$

按分布 $\alpha_i(t)$ 被分配。 $\alpha_{ij}(t)$ 是将任务分配给服务器 j 的概率质量, $\sum_{j=1}^N \alpha_{ij}(t) = 1$ 。因此,在每个时间步,第 i 个 LB 分配给服务器 j 的工作负载为 $w_i(t) \alpha_{ij}(t)$ 。在每个时间间隔内,每个服务器 j 能够根据自身属性(例如所配置的资源,包括 CPU、内存等)处理一定的工作负载 v_j 。我们得到服务器负载状态(待处理的剩余工作负载) $X_j(t) \in [0, \max\{0, M r r_i - 1 w_i(t) \alpha_{ij}(t) v_j\}]$ 。令 l_j 表示服务器 j 处理完所有剩余工作负载所需的时间,这也是新到达任务的潜在排队时间

$X_j(t) \in [0, \max\{0, w_i(t) \alpha_{ij}(t) v_j\}]$

$X_j(t)$

$r(t) = \sum_{i \in \mathcal{I}} w_i(t) \alpha_{ij}(t) X_j(t)$ 其中 $r(t)$ 是奖励函数, $\alpha_{ij}(t)$ 是

从时间步 t 到时间步 $t+1$ 的转移在算法 15 中给出。奖励: $r(t) = R(t)$, 其中 R 是奖励函数, $\alpha_{ij}(t)$ 是

表示每台服务器上估计的剩余处理时间, $\delta_i(t)$ 是一个随机变量, 用以使该过程具有随机性。

定义 34.(完工时间, Makespan) 在自利型负载均衡问题中, 完工时间定义为:

$$T = \max_{j \in \mathcal{J}} \{T_j\} \quad (5.1)$$

网络负载均衡问题具有多商品流问题(multi-commodity flow problem)的特征, 是 NP-难的, 这使得在微秒级别内用简单的算法方案求解变得困难 [Sen et al., 2013]。该问题可以被形式化为一个约束优化问题, 目标是在时间跨度 T 内最小化完工时间:

$$\min_{\mathbf{x}} \max_{j \in \mathcal{J}} \{T_j\} \quad (5.2) \quad \text{s.t. } \sum_{i \in \mathcal{I}} w_i(t) \alpha_{ij}(t) x_{ij}(t) \leq C_j, \quad \forall j \in \mathcal{J}, \quad \forall t \in \{0, 1, \dots, T-1\} \quad (5.3)$$

在现代真实的网络负载均衡系统中, 网络请求的到达通常在到达速率和预期工作负载两方面都不可预测, 这为问题引入了很大的随机性。此外, 由于存在噪声测量和部分观测, 完工时间的估计可能不准确, 这表明实际的服务器负载状态或处理能力未被正确捕捉。大象工作负载的瞬时冲突或鼠标工作负载的突发, 常常发生, 而这些并不能反映服务器的处理能力, 从而误导观测。为解决这一问题, 我们引入公平性作为原始目标完工时间的替代方案。具体而言, 完工时间是在每台服务器的层面上估计的, 而公平性的估计可以分解到 LB 层面, 这使得能够在不失一般性的情况下评估各个 LB 的

性能。由于 LB 存在部分可观测性,这在负载均衡系统中更为自然。

5.3.2 分配公平性

我们主要介绍两种负载均衡分配公平性:(1) 基于方差的公平性(VBF)和 (2) 基于乘积的公平性(PBF)。将证明,针对任一种公平性进行优化,对于最小化完工时间是充分但非必要的。

定义 35.(基于方差的公平性) 对于每台服务器 $j \in \{1, \dots, N\}$ 上处理完所有剩余任务所需时间的向量 $\mathbf{l} = (l_1, \dots, l_N)$, 令 $\mathbf{l}_{ptq} = (l_{1j}, \dots, l_{Nj})$, 则工作负载分配的基于方差的公平性即为任务时间的负样本方差, 定义为:

$$VBF(\mathbf{l}) = \frac{1}{N} \sum_{j=1}^N l_{j,ptq}^2 - \left(\frac{1}{N} \sum_{j=1}^N l_{j,ptq} \right)^2 \quad (5.5)$$

按 LB 定义的 VBF 为: $F(\mathbf{l}) = \frac{1}{N} \sum_{j=1}^N l_{j,ptq}^2 - \left(\frac{1}{N} \sum_{j=1}^N l_{j,ptq} \right)^2$, 其中 $l_{j,ptq} = \frac{1}{N} \sum_{i=1}^N l_{ij,ptq}$ 。

引理 36. 负载均衡系统的 VBF 满足以下性质:

$$F(\mathbf{l}) = \frac{1}{N} \sum_{j=1}^N l_{j,ptq}^2 - \left(\frac{1}{N} \sum_{j=1}^N l_{j,ptq} \right)^2 \quad (5.6)$$

这一性质使 VBF 成为负载均衡任务中奖励函数的一个良好选择。我们将在后续章节中进一步讨论。该引理的证明见 Sec. 5.6。

命题 37. 在负载均衡问题约束条件(式 (5.3) 和 (5.4))下,最大化 VBF 对于最小化完工时间是充分的: $\max_{\mathbf{l}} F(\mathbf{l}) \iff \min_{\mathbf{l}} \max_{j \in \{1, \dots, N\}} l_{j,ptq}$ 。这对于按 LB 定义的 VBF 同样成立,即 $\max_{\mathbf{l}} F(\mathbf{l}) \iff \min_{\mathbf{l}} \max_{j \in \{1, \dots, N\}} l_{j,ptq}$ 。

定义 38.(基于乘积的公平性 [Yao et al., 2022]) 对于每台服务器 $j \in \{1, \dots, N\}$ 上处理完所有剩余任务所需时间的向量 $\mathbf{l} = (l_1, \dots, l_N)$, 工作负载分配的基于乘积的公平性定义为: $F(\mathbf{l}) = \frac{1}{N} \sum_{j=1}^N l_{j,ptq} \prod_{i=1}^N l_{ij,ptq}$ 。按 LB 定义的 PBF 为: $F(\mathbf{l}) = \frac{1}{N} \sum_{j=1}^N l_{j,ptq} \prod_{i=1}^N l_{ij,ptq}$ 。

命题 39. 在负载均衡问题约束条件(式 (5.3) 和 (5.4))下,最大化基于乘积的公平性对于最小化完工时间是充分的: $\max_{\mathbf{p}} \prod_{i=1}^n p_i \leq \frac{1}{n} \min_{\mathbf{p}} \max_{i=1, \dots, n} p_i$ 。

命题 37 和 39 的证明见 Sec. 5.6。由命题 37 和 39 可知,这两种公平性都可以作为优化完工时间的有效替代目标,这将在我们提出的 MARL 方法中作为有效的奖励函数加以利用。

5.3.3 博弈论框架

马尔可夫博弈定义为 $\text{MG}(\mathcal{H}, M, S, A^M, P, r^M, \gamma)$, 其中 H 是博弈的时间跨度, M 是博弈中玩家的数量, S 是状态空间, A^M 是所有玩家的联合动作空间, A_i 是第 i 个玩家的行动空间, $r^M: S \times A^M \rightarrow \mathbb{R}$ 是在给定联合动作下第 i 个玩家的奖励函数。

MG 中第 i 个玩家的随机策略空间定义为 $\Pi_i: S \rightarrow \text{Pr}(A_i)$, $\Pi = \prod_{i=1}^M \Pi_i$ 。

π

对于马尔可夫博弈 MG , 在策略 $\pi \in \Pi^M$ 下, 第 i 个玩家在第 h 步的状态值函数 $V_i, h: S \rightarrow \mathbb{R}$ 和状态-动作值函数 $Q_{\pi_i, h}: S \times A_i \rightarrow \mathbb{R}$ 定义为:

" H 步

v_j 是 H 步

$\tilde{v}$

v_j

π

v_{π}

v

$V_{i,h}(\pi) := \mathbb{E}_{\pi} [r_i(s_h, a_h) + \gamma V_{i,h+1}(\pi) | s_h = s, a_h = a]$

$v_{i,h}(\pi) := \mathbb{E}_{\pi} [r_i(s_h, a_h) + \gamma V_{i,h+1}(\pi) | s_h = s, a_h = a]$

v

$h=1$ 到 $h=H$ (5.7)

定义 40. (ϵ -近似纳什均衡) 给定一个具有元组 $(\mathcal{H}, M, S, A^M, P, r^M, \gamma)$ 的马尔可夫博弈 MG , 令 π_{-i} 表示除第 i 个玩家外其他玩家的策略, 若 $\pi \in \Pi^M$ 是一个 ϵ -纳什均衡:

π 是一个 ϵ -纳什均衡:

π^*, π_{-i}^*

。

π_i, π_{-i}

$V_i(\pi) \leq V_i(\pi_{-i}^*, \pi_i) + \epsilon, \forall i \in \{1, \dots, M\}$ 。 (5.8)

若 $\epsilon = 0$, 则为精确纳什均衡。

定义41.(马尔可夫势博弈) 马尔可夫博弈 $M = \langle \mathcal{H}, \mathcal{M}, S, A^M, P, \Pi^M, r^M, q \rangle$ 是一个马尔可夫势博弈(MPG),如果对 $\forall i \in \mathcal{P}, r^M_i, \pi_i, \pi_i^P, \pi_i^S, s \in \mathcal{S}$
 $\pi_i, \pi_i^P, \pi_i^S, \pi_i^S$

$$\forall i \in \mathcal{P}, \pi_i^S = \phi(\pi_i, \pi_i^P, \pi_i^S) + \gamma \sum_{s' \in \mathcal{S}} P_{ii'}^s \pi_i^S, \quad (5.9)$$

其中 $\phi(\pi)$ 是与玩家索引无关的势函数。

引理42. 纯纳什均衡(PNE)在势博弈(PG)中总是存在,势函数的局部极大值点即为PNE。PNE在MPG中同样存在。[Monderer and Shapley, 1996]

定理43. 多智能体负载均衡问题是一个MPG,其中每个LB智能体 $i \in \mathcal{P}$ 的奖励 r_i 采用VBF $F_i(\pi)$,那么假设对 $\forall s \in \mathcal{S}$ 在步骤 $h \in \mathcal{H}$ 处,势函数为时间累积
 $r_i(\pi_i, \pi_i^P, \pi_i^S)$
 $t \in \mathcal{H}$ 的 $F_i(\pi)$ 。

该定理的证明基于引理36,详见第5.6节。这一定理对我们方法的建立至关重要,因为它证明了多智能体负载均衡问题可以被表述为一个以时间累积VBF作为其势函数的MPG。同时,选择逐LB的VBF作为个体智能体的奖励函数,是使其成为MPG的关键所在,容易验证PBF无法保证这一性质。由引理42可知势函数的极大值点即为MPG的NE,而由命题37可知最大化VBF为最小化makespan提供了充分条件。因此,针对上述定理所指定的个体奖励函数进行有效的独立优化,将导向负载均衡任务makespan的最小化点。这里所说的有效独立优化,是指达到了MPG的NE。

5.3.4 分布式方法

基于以上分析,负载均衡问题可以被表述为多玩家部分可观测马尔可夫博弈的episodic版本,记为 $POMGP = \langle \mathcal{H}, \mathcal{M}, S, O^M, O^M, A^M, P, r^M, q \rangle$,其中 $\mathcal{M}, \mathcal{H}, S$

—[图形/流程图:原PDF中含嵌入式矢量图,未在文本抽取中展开。]—

LB Agent 1

Mini-batch LB Agent 0

Generated Transitions

Data Plane

图5.3: 所提出的用于网络LB的分布式MARL框架概览。

$O_i, h : S^{\tilde{N}} \times \mathcal{P}(\mathcal{O}_i) \times \mathcal{R}^M \rightarrow \mathbb{R}$ 是第 i 个 LB 智能体在给定联合动作下的奖励函数。POMG 中第 i 个智能体的随机策略空间定义为 Π_i : 在 MPG 中, 独立策略梯度可以找到势函数的最大值, 即该博弈的 PNE。这启发我们以分解的方式利用策略优化, 即对每个 LB 智能体分别进行分布式 RL 策略学习。然而, 由于系统的部分可观测性以及直接估计 makespan (式(5.1)) 的困难, 每个智能体无法直接获取全局势函数。为解决这一问题, 可以将前述公平性 (第 5.3.2 节) 部署为每个智能体的奖励函数, 这使得价值函数可以作为势函数目标的有效替代。这也将

算法16 用于MPG的分布式LB 1: 初始化:

```

2: LB策略  $\pi_{\theta_i}$  和评论家  $Q_{\phi_i}$  网络,重放缓冲区  $B_i$ ,  $@i \in \mathcal{P} \cup \mathcal{M}$ ;
3: 服务器处理速度函数  $v_j$ ,  $@j \in \mathcal{P} \cup \mathcal{N}$ ; 4: 由第i个LB初始观测到的服务器j上的即时队列长度:  $q_{ij} \leftarrow 0$ ,  $@i \in \mathcal{P} \cup \mathcal{M}$ ,  $j \in \mathcal{P} \cup \mathcal{N}$ ; 5:
while 未收敛 do 6: 重置服务器负载状态  $X_{j \in \mathcal{P} \cup \mathcal{N}} \leftarrow 0$ ,  $@j \in \mathcal{P} \cup \mathcal{N}$ ; 7: 每个LB智能体  $i$  ( $i \in \mathcal{P} \cup \mathcal{M}$ ) 接收各自的观测  $o_i \in \mathcal{O}$ ; 8: for  $t \leftarrow 1, \dots, H$  do 9: 初始化分布式工作负载  $m_{ij}$ ,  $w_i \in \mathcal{W}$ ,  $i \in \mathcal{P} \cup \mathcal{M}$ ,  $j \in \mathcal{P} \cup \mathcal{N}$ ; 10: 获取动作  $a_i \in \mathcal{A}$ ; 11:  $j^* \leftarrow \arg \min_{j \in \mathcal{P} \cup \mathcal{N}} \{q_{ij} + v_j(w_i)\}$ ; 12:  $w_i$  分配给服务器  $j^*$ ; 13:  $m_{ij^*} \leftarrow m_{ij^*} + w_i$ ; 14:  $\alpha_{ij^*} \leftarrow \alpha_{ij^*} + 1$ ; 15: 对每个服务器  $j$  do 16: 更新工作负载:  $X_{j \in \mathcal{P} \cup \mathcal{N}} \leftarrow \max(X_{j \in \mathcal{P} \cup \mathcal{N}}, m_{ij^*} / \alpha_{ij^*})$ ; 17:  $X_{j \in \mathcal{P} \cup \mathcal{N}} \leftarrow X_{j \in \mathcal{P} \cup \mathcal{N}} - 1$ ; 18: 每个智能体接收各自的奖励  $r_i$ ; 19: 每个智能体  $i$  收集观测  $\tilde{o}_i \in \mathcal{O}$ ; 20: 更新重放缓冲区:  $B_i \leftarrow B_i \cup (\tilde{o}_i, a_i, r_i, \tilde{o}_i)$ ; 21: 用梯度更新评论家:  $\nabla_{\phi_i} E_{\pi_{\theta_i}} [r_i + \gamma V_{\phi_i}(\tilde{o}_i) | \tilde{o}_i, a_i]$ ; 22: 其中  $V_{\phi_i}(\tilde{o}_i) \leftarrow E_{\pi_{\theta_i}} [r_i + \gamma V_{\phi_i}(\tilde{o}_i) | \tilde{o}_i, a_i]$ ; 23: 用梯度更新策略:  $-\nabla_{\theta_i} E_{\pi_{\theta_i}} [r_i + \gamma V_{\phi_i}(\tilde{o}_i) | \tilde{o}_i, a_i]$ ; 24: return 学习智能体的最终模型

```

联合目标(makespan或势函数)转化为每个智能体的个体目标(逐LB公平性)。命题37和39验证了朝这些公平性指标优化对于最小化makespan而言是充分的。 算法16展示了所提出的用于负载均衡问题的分布式LB方法,该问题是一个部分可观测的MPG。分布式策略优化基于Soft Actor-Critic(SAC)[Haarnoja et al., 2018]算法,这是一种最大熵RL方法。 它优化目标 $E_{\pi_{\theta_i}} [r_i + \gamma V_{\phi_i}(\tilde{o}_i) | \tilde{o}_i, a_i]$, 其中 $H_{\pi_{\theta_i}}$ 是策略 π_{θ_i} 的熵。 具体而言,评论家Q网络通过梯度 $\nabla_{\phi_i} E_{\pi_{\theta_i}} [r_i + \gamma V_{\phi_i}(\tilde{o}_i) | \tilde{o}_i, a_i]$ 进行更新,其中 $V_{\phi_i}(\tilde{o}_i) \leftarrow E_{\pi_{\theta_i}} [r_i + \gamma V_{\phi_i}(\tilde{o}_i) | \tilde{o}_i, a_i]$, Q_{ϕ_i} 是目标Q网络;演员策略 π_{θ_i} 通过梯度 $-\nabla_{\theta_i} E_{\pi_{\theta_i}} [r_i + \gamma V_{\phi_i}(\tilde{o}_i) | \tilde{o}_i, a_i]$ 进行更新。RL方法的其他关键要素涉及观测、动作和奖励函数,详述如下。 观测。每个LB智能体只能部分观测流经自身的流量,包括服务器级和LB级的测量。对于每个LB,服务器级

观测包括----针对每台服务器----正在进行的任务数量,以及采样的任务时长和任务完成时间(TCT)。具体而言,在算法16第12-14行中, w_i 是由第*i*个LB分配到服务器上的到来工作负载,LB本身对其不可观测。 q_{ik}^{t-1} 是由第*i*个LB局部观测到的第*k*台服务器上的任务数量,这是由于传输层存在有限可观测性的现实约束所致。“+1”是为了考虑新到达的任务。任务时长和TCT样本的观测,以及采样任务到达间隔时间(作为整体系统负载状态指示)的LB级测量,被简化为5个标量----即平均值、90分位数、标准差、折扣平均值以及加权折扣平均值4----作为LB智能体的输入。动作。为在控制平面和数据平面之间不同的时序约束进行衔接,每个LB智能体使用两个因子的比值, $\arg \min_k \frac{p_{ik}^{t-1}}{a_{ik}^{t-1}}$,将新到达的任务分配给第*j*台服务器,其中正在进行的任务数量 q_{ik}^{t-1} 有助于在每连接级别追踪动态系统服务器占用情况----从而能够以微秒级速度做出负载均衡决策----而 a_{ik}^{t-1} 是周期性更新的、由RL推断出的服务器处理速度。如算法16第14行所示, α_{ij}^{t-1} 是时间区间 $t_{i,j}^{t-1}$ 内工作负载分配分布的统计估计。奖励。分布式MPG LB的个体奖励选定为每个LB智能体上采样任务时长折扣平均值的VBF(如定义35),从而使LB组联合优化朝向式(43)中定义的势函数。任务时长信息作为连接初始化结束时刻(例如TCP流量的三次握手)与首个数据包确认时刻(例如TCP流量的首个ACK包)之间的时间间隔而被收集。鉴于LB智能体的观测有限且部分,任务时长信息通过测量每台服务器上新到来任务的排队和处理延迟,来近似估计剩余工作负载 l 。这种基于PBF和MS的奖励也被用于CTDE MARL算法中作为对比实现。模型。所提出RL框架的架构如图5.3所示。每个LB智能体包含一个重放缓冲区,以及一对演员-评论家网络,其架构

标注4 折扣平均权重计算为 $0.9t^{-t}$,其中 t 是样本时间戳, t_1 是计算简化标量的时刻。

QMix|PBF Distr-LB|VBF+logVBF WCMP QMix|Makespan Distr-LB|VBF QMix|VBF Centr-LB|VBF LSQ QMix|VBF
Distr-LB|VBF+logVBF Distr-LB|VBF Centr-LB|VBF+logVBF SED QMix|PBF Centralised SAC|VBF 0.4 1.0

QoS - TCT (s) 0.2 0.1 CDF 0.5 0.05 wiki页面 静态页面 0.0 10 2 10 1 100 101 10 3 10 2 10 1 100 101 0 200 400 600 800 1000 迭代次数
QoS - TCT (s) QoS - TCT (s) (a) 学习曲线。 (b) TCT的CDF。

图5.4: 实验结果表明,所提出的使用VBF作为奖励的分布式RL框架能够收敛,并比现有LB算法和CTDE RL算法有效地实现更优的负载均衡性能(更低的TCT和更佳的QoS)。

其架构如右上方所示。此外还有一对引导用的演员-评论家网络,其网络架构相同,但以延迟且软更新的方式更新。每个LB智能体接收从数据平面提取的观测 o_i^{ptq} (例如正在进行的任务数 t_{qij}^u 、任务时长、TCT)以及上一时间步的动作 $a_i^{pt'lq}$ 作为输入,并周期性地生成新动作 a_i^{ptq} ,该动作用于更新数据平面中的服务器分配函数 $\arg \min_j \Pr N s_{ij} a_{ij}^{lq}$ 。门控循环单元 (GRU)[Chung et al., 2014]被应用于所有智能体,以利用序列历史信息来处理部分可观测性。

5.4 评估

我们开发了(i)一个基于事件的模拟器,用于研究所提出算法所达到的NE与理论最优负载均衡策略(具有完美观测)所达到的NE之间的差距,以及(ii)一个在DC网络中物理服务器上搭建的真实测试平台,提供Apache网络服务,使用真实世界的网络流量[Urdaneta et al., 2009],以评估所提出算法的真实性能,并与生产环境中最先进的LB[Eisenbud et al., 2016]进行比较。中等规模真实世界测试平台:如图5.4a所示,在一个具有2个LB智能体和7台服务器的中等规模真实世界DC网络设置中,经过120个训练 episode后,所提出的分布式LB(Distr-LB)算法能够基于VBF作为奖励从环境中学习,并收敛到比QMix更优的QoS。

表5.2: 中等规模真实网络设置下平均QoS(秒)的比较。

第III时期(758.787次查询/秒) 第IV时期(784.522次查询/秒) 方法 Wiki 静态 Wiki 静态

WCMP 0.412~ 0.101 0.134~ 0.059 0.834~ 0.323 0.492~ 0.276
 LSQ 0.620~ 0.442 0.339~ 0.316 0.357~ 0.373 0.173~ 0.299
 SED 0.215~ 0.210 0.051~ 0.081 0.346~ 0.496 0.169~ 0.330
 Jain 0.193~ 0.073 0.026~ 0.022 0.204~ 0.084 0.039~ 0.047

RLB-SAC [Yao et al., 2022]

G 0.149~ 0.049 0.015~ 0.011 0.155~ 0.052 0.011~ 0.011
 MS 0.217~ 0.157 0.048~ 0.069 0.263~ 0.202 0.073~ 0.092
 QMix-LB VBF 0.141~ 0.025 0.008~ 0.004 0.286~ 0.162 0.068~ 0.066
 PBF 0.211~ 0.153 0.047~ 0.078 0.181~ 0.042 0.018~ 0.009
 Distr-LB VBF 0.159~ 0.054 0.017~ 0.009 0.196~ 0.091 0.032~ 0.033
 (本文) VBF+logVBF 0.108~ 0.022 0.004~ 0.001 0.104~ 0.013 0.006~ 0.003
 VBF 1.068~ 0.386 0.570~ 0.378 1.378~ 0.377 0.867~ 0.350

Centr-LB

VBF+logVBF 0.759~ 0.254 0.306~ 0.222 1.013~ 0.168 0.520~ 0.167

表5.3: 中等规模模拟器中不同类型应用平均QoS(秒)的比较。

50%-CPU+50%-IO 75%-CPU+25%-IO 100%-CPU

Oracle 6.437~ 1.006 1.469~ 0.102 1.291~ 0.075
 PBF 10.230~ 0.108 1.828~ 0.054 2.200~ 0.288

QMix-LB

VBF 10.936~ 0.470 2.023~ 0.255 2.125~ 0.074
 Distr-LB VBF 10.335~ 0.362 1.695~ 0.104 1.643~ 0.016
 (本文) VBF+logVBF 8.797~ 0.459 1.873~ 0.328 2.004~ 0.042

集中式RL智能体(Centr-LB)由于状态和动作空间的增大,在120个episode内难以学习。一个实验发现是,通过在基于VBF的奖励中加入对数项,可以帮助Distr-LB学习。基于模拟的NE差距评估: 为评估Distr-LB性能与理论最优策略之间的差距,我们在模拟器中实现了一个Oracle LB,它对系统具有完美观测(现实中不可获得),并针对每次负载均衡决策最小化makespan。表5.3表明,对于不同类型的应用,Distr-LB相比

表5.4: 大规模真实网络设置下平均QoS(秒)的比较。

第I时期(2022.855次查询/秒) 第II时期(2071.129次查询/秒) 方法 Wiki 静态 Wiki 静态

WCMP 0.473~ 0.102 0.194~ 0.090 0.460~ 0.241 0.239~ 0.212
 LSQ 0.266~ 0.127 0.063~ 0.065 0.218~ 0.246 0.082~ 0.152
 SED 0.169~ 0.062 0.020~ 0.025 0.166~ 0.141 0.050~ 0.070
 RLB-SAC-G[Yao et al., 2022] 0.182~ 0.049 0.013~ 0.009 0.111~ 0.029 0.010~ 0.009
 VBF 0.181~ 0.062 0.019~ 0.020 0.188~ 0.147 0.052~ 0.075

QMix-LB

PBF 0.210~ 0.041 0.013~ 0.006 0.104~ 0.009 0.005~ 0.003
 Distr-LB VBF 0.228~ 0.055 0.019~ 0.011 0.174~ 0.102 0.035~ 0.039
 (本文) VBF+logVBF 0.161~ 0.033 0.008~ 0.003 0.094~ 0.015 0.004~ 0.001

表5.5: 使用大规模真实世界设置,不同流量速率下Wiki页面99百分位QoS(秒)的比较。

流量速率(次查询/秒) 方法 731.534 1097.3 1463.067 1828.834 2194.601 2377.484 2560.368 2743.251 2926.135 0.175 0.212 0.249 0.342
0.827 2.103 10.662 17.656 17.999 LSQ

~0.015 ~0.025 ~0.043~0.121~0.572~0.654~2.557~0.714~0.253
 0.201 0.261 0.322 0.360 0.618 2.175 11.444 22.086 22.727 SED
 ~0.022 ~0.079 ~0.099~0.088~0.268~1.328~3.861~4.892~5.632
 0.160 0.205 0.248 0.284 0.567 1.276 7.005 10.560 15.745 VBF
 Distr-LB ~0.010 ~0.036 ~0.086~0.113~0.306~0.647~1.147~1.042~0.254
 (本文) 0.161 0.216 0.249 0.348 0.439 1.533 4.427 9.391 15.347 VBF+logVBF
 ~0.008 ~0.052 ~0.068~0.122~0.121~0.670~0.443~0.329~0.572

QMix能够取得更接近最优的性能。由于该模拟器是基于本文所提出的负载均衡模型实现的,我们的理论分析可以直接应用,而VBF---作为一个势函数---有助于独立协作的LB智能体取得良好性能。额外的对数项在真实世界系统中显示出经验上的性能提升,但在这些模拟结果中并非必然如此。一方面,模拟中生成的任务流量具有更高的期望工作负载(均值和标准差均 ≈ 1 s),而对数项对接近0的方差更为敏感,这正是真实世界实验设置中的情形。另一方面,尽管模拟器对所表述的LB问题进行了建模,但它未能捕捉到真实世界系统中的复杂性---例如Apache的backlog、多进程优化、上下文切换、多级缓存、网络队列等。例如,批处理[The Fast Data Project (fd.io)]有助于减少缓存和指令缺失,却使不同任务产生相近的处理时间,从而降低了任务处理延迟的方差,使其在真实世界系统中更接近0。额外的对数项放大了这种低方差差异,从而更好地评估负载均衡决策。

表5.6:在大规模真实环境测试平台下,不同流量速率下静态页面第99百分位QoS(秒)的比较

流量速率(queries/s) 方法 731.534 1097.3 1463.067 1828.834 2194.601 2377.484 2560.368 2743.251 2926.135 0.014 0.015 0.015 0.018
0.217 0.856 11.066 16.874 17.155 LSQ

~0.001 ~0.000 ~0.000~0.003~0.305~0.554~3.095~0.391~0.217

0.014 0.015 0.016 0.018 0.071 1.252 11.272 21.941 20.708 SED

~0.000 ~0.000 ~0.001~0.001~0.066~1.489~3.975~5.970~5.423

0.014 0.015 0.016 0.017 0.041 0.338 6.670 9.743 15.506 VBF

Distr-LB ~0.000 ~0.000 ~0.001~0.000~0.025~0.364~1.152~0.863~0.056

(本文方法) 0.014 0.015 0.016 0.018 0.072 0.465 3.970 8.782 15.095 VBF+logVBF

~0.000 ~0.001 ~0.000~0.002~0.087~0.403~0.545~0.187~0.497

大规模真实环境测试平台:为了在真实场景下评估Distr-LB在大规模数据中心网络中的性能,我们将真实环境测试平台扩展至包含6个LB代理和20台服务器,并施加更重的网络流量(超过2000 queries/s),以评估在中等规模场景下表现最优的LB算法的性能,并与生产环境中使用的LB算法进行比较。经过200轮训练后的测试结果如表5.4所示,其中Distr-LB在所有情形下都取得了最佳性能。QMix的表现也优于生产环境中的LB算法。但作为一种CTDE算法,与Centr-LB类似,它要求代理之间相互通信各自的轨迹,经过200轮训练后,这在每个episode结束时(按episode训练)产生了221MiB的通信开销,而每个目标机架的流量速率的95百分位却小于1MiB/s [Roy et al., 2015]。扩展性实验:使用同样的包含6个LB代理和20台服务器的大规模真实环境测试平台,我们通过施加不同流量速率的网络流量轨迹,比较表现最优的4种LB方法,来进行扩展性实验。表5.5和表5.6分别展示了Wiki页面和静态页面的第99百分位QoS。如表5.5和表5.6所列,在低流量速率下,由于服务器均未被充分利用,我们所提出的Distr-LB的优势并不明显,因为所有资源都处于过度配置状态。随着流量速率的增加(直至服务器达到100%饱和),我们的方法优于最好的经典LB方法。

5.5 随机马尔可夫模型

图5.2b的仿真结果基于一个基本的负载均衡设置,该设置包含2台处理能力不同的服务器 w_1, w_2 (即服务器1的速度是服务器2的2倍)。每台服务器都有一个

统即为具有状态 p_1, p_2 、离开速率 μ_1, μ_2 以及到达速率 $\lambda_1, \lambda_2, \gamma_1, \gamma_2$ 的马尔可夫过程。为简化并保证稳定性,系统在名义容量下运行(即 $\lambda < \mu$)。

图5.2b中每种负载均衡算法的QoS性能,是在不同配置下,以连接的加权服务时长($\int_0^T (p_1(t)w_1 + p_2(t)w_2) dt$)来衡量的。当LB具有准确的观测和配置(观测100%的流量)---即

$\gamma = 0$ ---并根据实际处理速度分配服务器权重 w_1, w_2

“ $w_1 = 2w_2$ ”时,WCMP和SED表现最佳。当LB只能观测到部分网络流量(50% γ 和 33% γ 分别对应于 $\gamma = \lambda, \gamma = \lambda/2$),而其余的网络流量在两台服务器之

μ_1
 μ_2

“ $\gamma = \lambda/2$ ”,即便LB代理能够看到全部流量($\gamma = 0$),WCMP和SED的性能仍会下降。SED同时考虑了服务器队列长度和处理速度,因而能做出更为明智的负载均衡决策,但其性能仍有可能因

对服务器队列长度的部分观测以及不准确的服务器权重而下降。

5.6 分布公平性分析

基于方差的公平性分析

引理44. 用于负载均衡系统的VBF满足以下性质:

$$F_{\pi_i}, \pi_i p_{li} q = F_{\pi_i}, \pi_i p_{li} q = F_{\pi_i}, \pi_i p_{li} q = F_{\pi_i}, \pi_i p_{li} q \quad (5.10)$$

证明. 根据基于方差的公平性的定义(定义35),对 $\pi_i p_{li} q, j p_{li} q$,我们有以下关系:

$$N_j \pi_i p_{li} q = N_j \pi_i p_{li} q$$

$$F_{\pi_i}, \pi_i p_{li} q = F_{\pi_i}, \pi_i p_{li} q \quad (5.11)$$

$$N_j \pi_i p_{li} q = N_j \pi_i p_{li} q$$

$$F_{\pi_i}, \pi_i p_{li} q = F_{\pi_i}, \pi_i p_{li} q \quad (5.12)$$

$$N_j \pi_i p_{li} q = N_j \pi_i p_{li} q$$

通过将代理 i 标记为改变其策略的代理,并稍微简化记号,记 r

$$l_j = l_j \pi_i p_{li} q, \text{其中 } l_j = k \pi_i p_{li} q.$$

$$N_j \pi_i p_{li} q = N_j \pi_i p_{li} q$$

$$F_{\pi_i}, \pi_i p_{li} q = F_{\pi_i}, \pi_i p_{li} q \quad (5.13)$$

$$N_j \pi_i p_{li} q = N_j \pi_i p_{li} q$$

$$(5.13) N_j \pi_i p_{li} q = N_j \pi_i p_{li} q$$

$$F_{\pi_i}, \pi_i p_{li} q = F_{\pi_i}, \pi_i p_{li} q \quad (5.14)$$

$$N_j \pi_i p_{li} q = N_j \pi_i p_{li} q$$

$$F_{\pi_i}, \pi_i p_{li} q = F_{\pi_i}, \pi_i p_{li} q \quad (5.15)$$

$$N_j \pi_i p_{li} q = N_j \pi_i p_{li} q$$

$$F_{\pi_i}, \pi_i p_{li} q = F_{\pi_i}, \pi_i p_{li} q \quad (5.16)$$

$$N_j \pi_i p_{li} q = N_j \pi_i p_{li} q$$

$$F_{\pi_i}, \pi_i p_{li} q = F_{\pi_i}, \pi_i p_{li} q \quad (5.17)$$

$$N_j \pi_i p_{li} q = N_j \pi_i p_{li} q$$

其中第二项是一个不依赖于变化策略 π_i 的公共项。因此,该项将在 $F \pi_i, ' \pi_i \text{ plq } ' F \pi_i, ' \pi_i \text{ plq } " F \pi_i, ' \pi_i \text{ pli q } ' F \pi_i, ' \pi_i \text{ pli q } " F \pi_i$ 中被抵消,从而完成证明。

命题45. 在满足负载均衡问题约束(式(5.3)和(5.4))的前提下,最大化VBF足以使makespan最小化:

$$\max_{\mathbf{F}} \min_{\tilde{\mathbf{n}}} \max_{\mathbf{q}} \text{plj } q(5.18) j$$

这一结论对于逐LB的VBF也成立,即 $\max_i F_i \leq \min_j p_j$ 。

证明。给定式(5.3)中的稳定性约束 $\| \mathbf{M} \mathbf{r} \mathbf{n} \mathbf{i} \|_1 \leq \mathbf{w} \mathbf{j}$, 我们记系统中的总工作负载为 $\mathbf{C} = \mathbf{N} \mathbf{r} \mathbf{j}$, 并记 $\mathbf{l} \mathbf{k} = \max_j \mathbf{p} \mathbf{r} \mathbf{n} \mathbf{s} \mathbf{l} \mathbf{j}$ 。根据式(5.4)中的约束, 我们有 $\mathbf{C} \geq 0, \mathbf{l} \mathbf{j} \mathbf{p} \mathbf{q} \mathbf{j} \geq 0$ 。

$$\max_{\mathbf{F}} \mathbf{F}^T \mathbf{p} \mathbf{q} \hat{=} \min_{\mathbf{F}} \|\mathbf{F} \mathbf{p} \mathbf{q}\| \quad (5.19)$$

$$F_{plq} = p_{plj} q_{lq} \quad (5.20)$$

$$N_j \ll N \ll N_j^2$$

$$p_{lj} = q_2 \quad (5.21)$$

$$N_j \left(\frac{1}{N} \sum_{i=1}^N N_i \right) \approx \frac{1}{N} \sum_{i=1}^N N_i^2$$

$$|l_j| \leq 2 \quad (5.22)$$

$$N_j \leftarrow 1, N_{2j} \leftarrow 1, N_{N1} \leftarrow 2, C_2$$

$$| \cdot | \quad (5.23)$$

由均值不等式 (5.24) 得

当 $l_j \mid l_k, @j \% k, j \in \text{PrNs}$ 成立时取等。因此,

$$C2 \max F_{plq} \tilde{n} \min_{plk} q_2' \quad (5.25) \quad N2 \hat{o} \min_{lk} \quad (5.26)$$

$$\hat{\sigma} = \min \max_{j \in \{1, \dots, n\}} \left| \frac{1}{n} \sum_{i=1}^n \frac{1}{\sigma_i} \right| \quad (5.27)$$

C 2

该条件是充分而非必要的,因为 $\min_{\mathbf{p}} \|\mathbf{q}\|_2 \leq \sqrt{N} \|\mathbf{p}\|_1$ 本质上是在最小化 $\|\mathbf{F}\mathbf{p}\|_q$ 的上界。

基于乘积的公平性分析

命题46。在满足负载均衡问题约束(式(5.3)和(5.4))的前提下,最大化基于乘积的公平性足以使makespan最小化:

$$\max_{\mathbf{p}} \|\mathbf{F}\mathbf{p}\|_q \hat{=} \min_{\mathbf{p}} \max_{\mathbf{l}} \|\mathbf{l}\|_q \quad (5.28)$$

证明。对于每台服务器 $j \in \{1, \dots, N\}$ 上的工作负载向量 $\mathbf{l} = (l_1, \dots, l_N)$, 根据公平性的定义,

$$\|\mathbf{l}\|_q = \left(\sum_{j=1}^N l_j^q \right)^{1/q} \leq \max_{j \in \{1, \dots, N\}} l_j \quad (5.29)$$

不失一般性,设 $\mathbf{l} = \max_{j \in \{1, \dots, N\}} l_j \mathbf{1}$, 则,

$$\|\mathbf{l}\|_q = \max_{j \in \{1, \dots, N\}} l_j \quad (5.30)$$

与命题45的证明类似,给定式(5.3)中的稳定性约束 $\mathbf{r} \leq \mathbf{M} \mathbf{r}$, 我们记系统中的总工作负载为 $C = \sum_{j=1}^N l_j$ 。

根据式(5.4)中的约束,我们有 $C \leq 0$, $\mathbf{l} \geq 0$ 。由均值不等式,

$$\begin{aligned} \sum_{j=1}^N l_j &\leq N \|\mathbf{l}\|_1 \\ \sum_{j=1}^N l_j &\leq N \|\mathbf{l}\|_q \quad (5.31) \end{aligned}$$

$$\sum_{j=1}^N l_j \leq N \|\mathbf{l}\|_q$$

$$\sum_{j=1}^N l_j \leq N \|\mathbf{l}\|_q$$

当 $\mathbf{l} = \mathbf{1}$, $\mathbf{l} = \mathbf{1}$ 成立时取等。因此,

$$\begin{aligned} C &\leq N \|\mathbf{l}\|_q \\ \max_{\mathbf{p}} \|\mathbf{F}\mathbf{p}\|_q &\hat{=} \max_{\mathbf{p}} \|\mathbf{p}\|_q \quad (5.32) \end{aligned}$$

$$\begin{aligned} \hat{=} \min_{\mathbf{l}} \|\mathbf{l}\|_q \quad (5.33) \end{aligned}$$

$$\hat{=} \min_{\mathbf{l}} \max_{j \in \{1, \dots, N\}} l_j \quad (5.34)$$

反之则不一定成立,因为 $\max_{i \in N} C_i^k$

$N \setminus i$

并不意味着 $\max_{i \in N} F_i(p)$,所以最大化基于乘积的线性公平性对于最小化makespan是充分而非必要的。证明完毕。

用于MPG的基于方差的公平性

定理47. 若以VBF $F_i(p)$ 作为每个LB代理 $i \in N$ 的奖励 r_i ,则多代理负载均衡问题是一个MPG,进一步假设对于每一步 $h \in \{1, \dots, H\}$ 的任意 $\pi \in \Pi$, 势函数为

$r_i(\pi_i, \pi_{-i})$

则 π^h 是 F 的局部极大值。

证明。

设

$\pi^h = (\pi_i^h, \pi_{-i}^h)$

$\pi^h = (\pi_i^h, \pi_{-i}^h)$

$\pi_i, \pi_{-i}, \pi_{-i}^h, \pi_i^h$

π_i, π_{-i}

$V_i(\pi_i, \pi_{-i}) = F_i(\pi_i, \pi_{-i})$ 则 π^h 是 F 的局部极大值, 即 $\pi^h \geq \pi_i, \pi_{-i}^h$ 且 $\pi_i^h \geq \pi_i, \pi_{-i}^h \geq \pi_{-i}$

则 π^h 是 F 的局部极大值

(5.35) $H \setminus \{i\} \times H \setminus \{i\}$

$\pi_i^h, \pi_{-i}^h \geq F_i(\pi_i^h, \pi_{-i}^h) \geq F_i(\pi_i, \pi_{-i})$ (5.36)

则 π^h 是 F 的局部极大值

$H \setminus \{i\}$

π_i^h

$\pi_i, \pi_{-i}, \pi_i^h, \pi_{-i}^h$

则 $F(\pi_i^h, \pi_{-i}^h) \geq F(\pi_i, \pi_{-i})$ (5.37) 则 π^h

$\pi_i^h, \pi_{-i}^h \geq F_i(\pi_i^h, \pi_{-i}^h) \geq F_i(\pi_i, \pi_{-i})$ (5.38)

注意, s 是环境的真实状态,因此涉及到完成剩余任务所需的期望时间 l 。

引理48. ϵ -近似MPG的NE是 ϵ -近似NE。[Ali et al., 2019]

证明。我们知道MPG存在NE π^* , π^*

。

则,

π^*, π^*

。

$\pi_i, \pi_{-i}, \pi_i^*, \pi_{-i}^*$

$V_i(\pi_i, \pi_{-i}) = F_i(\pi_i, \pi_{-i}) \geq F_i(\pi_i^*, \pi_{-i}^*) \geq 0$ (5.39)

对于另一个具有不同价值函数 V_p 但相同势函数的博弈,这些策略可以是 ϵ -近似NE,

。

$\pi_i, \pi_{-i}, \pi_i^*, \pi_{-i}^*$

$V_p(\pi_i, \pi_{-i}) = V_p(\pi_i^*, \pi_{-i}^*) \leq \epsilon$, $\pi_i^* \in \Pi_i, \pi_{-i}^* \in \Pi_{-i}$ (5.40)

原文第 129 页

第三部分

基础模型中的强化学习

第6章

扩散世界模型

本节内容基于论文《扩散世界模型:超越逐步展开的未来建模用于离线强化学习》("Diffusion World Model: Future Modeling Beyond Step-by-Step Rollout for Offline Reinforcement Learning") [Ding et al., 2024], 该论文与 Amy Zhang、Yuandong Tian 和 Qinqing Zheng 合作完成, 此前已发表于 ICLR 2024 GenAI4DM 研讨会。

6.1 引言

世界模型是 AI 系统执行规划与推理的基础模块 [Ha and Schmidhuber, 2018b]。回报 v.s. 仿真时域 (walker2d-medium-v2) 它们充当真实环境的模拟器, 用以预测某些动作将产生的未来结果, 并可据此推导出策略。基于模型的强化学习 (MBRL) 中这类方法的代表性用例包括动作搜索 [Schrittwieser et al., 2020; Ye et al., 2021]、在此类模拟器中进行策略优化 [Dean et al., 2020; Feinberg et al., 2018; Hafner et al., 2019a; Sutton, 1991], 或是两者的结合 [Chitnis et al., 2023; Hansen et al., 2023, 2022b]。

图6.1: 使用扩散世界模型和一步动态模型训练的 TD3+BC 的回报。

世界模型的预测精度对基于模型的强化学习方法的最终性能至关重要。传统的基于模型的方法构建了一个一步动态模型 $p(s_{t+1}, r_t | s_t, a_t)$, 该模型根据当前状态 s_t 和当前动作 a_t 预测奖励 r_t 和下一状态 s_{t+1} [Hafner et al., 2019a, 2020, 2023; Hansen et al., 2022a; Janner et al., 2019; Kaiser et al., 2019; Kidambi et al., 2020; Yu et al., 2020]。当针对未来多步进行规划时, p 会被递归调用, 从而导致误差迅速累积, 长时域展开的预测变得不可靠。图6.1绘制了使用一步动态模型的MB方法的性能。随着展开长度的增加, 回报迅速崩溃, 凸显了此类模型的误差累积问题 [Asadi et al., 2019; Lambert et al., 2022; Xiao et al., 2019]。近来, 人们对利用序列建模技术来解决决策问题的兴趣日益增长, 如多项研究所示 [Ajay et al., 2022; Chen et al., 2021; Janner et al., 2022, 2021; Micheli et al., 2022; Robine et al., 2023; Zheng et al., 2023b, 2022]。在这一主题下, 多项工作提出了基于Transformer [Chen et al., 2022; Micheli et al., 2022; Robine et al., 2023] 或基于扩散模型 [Alonso et al., 2024; Janner et al., 2022; Lu et al., 2023b; Rigter et al., 2023; Yang et al., 2023; Zhang et al., 2023] 的动态模型, 或与此密切相关的方法。正如我们将在本节末尾以及第6.2节中所回顾的那样, 尽管现有的大多数方法都将这些序列模型用作规划所用的动态模型, 但它们只利用当前及以往步骤的信息来建模一步未来结果 s_{t+1} 和 r_t 。在规划时, 它们仍是逐步进行规划的。这就引出了本文所要回答的一个引人深思的问题:

序列建模工具能否通过联合预测未来多个步骤, 有效降低长时域预测中的误差累积?

在本文中, 我们提出了扩散世界模型(Diffusion World Model, DWM)。在给定当前状态 s_t 、动作 a_t 以及期望回报 g_t 的条件下, DWM同时预测多步未来的状态

则需要被调用 H 次。这大大减少了误差累积的来源。如图6.1所示,扩散世界模型对长时域仿真具有鲁棒性,即便仿真时域达到31步,其性能也不会出现下降。为了验证所提出的DWM,我们考虑离线RL的设置,其目标是在没有在线交互的情况下,从静态数据集中学习策略。脱离在线训练避免了探索所带来的副作用,使我们能够彻底研究世界模型的质量。我们提出了一种通用的Dyna型 [Sutton, 1991] 基于模型的框架。简而言之,我们首先使用离线数据集训练一个扩散世界模型,然后以演员-评论家(actor-critic)的方式,使用该扩散世界模型生成的想象数据来训练策略。特别地,为了生成训练评论家(critic)所需的目标价值,我们引入了基于扩散模型的价值扩展(Diffusion Model Based Value Expansion, Diffusion-MVE),该方法使用扩散世界模型生成的未来轨迹,来模拟直至选定时域为止的回报。正如我们稍后将详细阐述的那样,Diffusion-MVE可以被解读为通过生成式建模对离线RL进行的一种价值正则化,或者换个角度看,是一种利用合成数据进行离线Q学习的方式。我们的框架可以灵活地搭载任何可供选择的MF演员-评论家RL方法,并且所输出的策略在推理时是高效的,因为世界模型不会介入动作生成过程。在实证方面,我们在来自D4RL数据集 [Fu et al., 2020] 的9个运动任务上对基于扩散的世界模型和传统的一步世界模型进行了基准测试,所有任务都处于连续的动作和观测空间中。主要结果如下:

1. 我们的结果证实了DWM优于一步模型,基于DWM的算法取得了44%的性能提升。
2. 我们进一步考虑了本方法的一种变体,即用Transformer架构 [Vaswani et al., 2017] 替代扩散模型。尽管Transformer是一种序列模型,但其固有的自回归结构更容易产生误差累积。我们证实,基于DWM的算法相比基于Transformer的算法取得了37.5%的性能提升。

方法 强化学习设定 扩散模型 模型用途 动作预测 SynthER [Lu et al., 2023b] 离线/在线 ppst ,at ,st`1 ,rt q 转移级数据增强 MF 方法

DWMs [Alonso et al., 2024] 在线 ppst`1 |st ,at ,...,st`T`1 ,at`T`1 q 逐步规划 REINFORCE [Williams, 1992]
PolyGrad [Rigter et al., 2023] 在线 ppst ,st`1 ,...,st`T`1 ,rt`T`1 |st ,at ,...,at`T`1 q 生成同策略轨迹用于策略优化 随机朗之万动力学
PGD [Jackson et al., 2024] 离线 ppst ,at ,rt ,...,st`T`1 ,at`T`1 ,rt`T`1 ,st`T`1 q 通过策略梯度引导生成同策略轨迹用于数据增强 MF 离线方法
UniSim^{*} [Yang et al., 2023] 离线 ppst`1 |st ,at q 逐步规划 REINFORCE [Williams, 1992]
Diffuser [Janner et al., 2022] 离线 ppat ,...,st`T`1 ,at`T`1 |st q 从样本中提取 at 从样本中提取
DD [Ajay et al., 2022] 离线 ppst`1 ,...,st`T`1 |st ,gt q 从样本中提取 st`1 逆动力学模型
DWM(本文方法) 离线 ppst ,st`1 ,rt`1 ,...,st`T`1 ,rt`T`1 |st ,at ,gt q 多步规划 MF 离线方法
*UniSim 的观测可能包含多帧,但其扩散模型本质上仍是一步模型。

表 6.1:具有代表性的基于扩散模型的 MBRL 方法比较。

3. 我们还将本文算法与 Decision Diffuser [Ajay et al., 2022] 进行了比较,后者是一种密切相关的基于模型的离线强化学习方法,该方法模拟仅包含状态的轨迹,同时使用逆动力学模型来预测动作。两种方法的性能相当。

4. 同时,由于建模误差不可避免,基于模型(MB)的方法相较于直接通过与真实环境交互来学习策略的无模型(MF)方法,通常在最终性能上表现更差。我们的结果表明,基于 DWM 的 MB 算法与其 MF 对应方法相当,甚至略有超越。我们认为这促使我们进一步开展基于模型的强化学习方法的研究,因为这类方法具有样本效率方面的优势 [Dean et al., 2020; Deisenroth et al., 2013],因而更有可能适用于实际的现实世界问题。

与其他基于扩散模型的离线强化学习方法的关键区别 最近,多种形式的扩散模型 [Ajay et al., 2022; Alonso et al., 2024;

Jackson et al., 2024; Janner et al., 2022; Lu et al., 2023b; Rigter et al., 2023; Yang et al., 2023; Zhang et al., 2023] 已被引入用于世界建模及相关工作。这些工作针对不同的数据设定(离线或在线强化学习),并利用扩散模型对不同类型的数据分布进行建模。在应用于下游强化学习任务时,它们也采用了不同的方式来导出策略。第 6.2 节将对这些工作进行详细综述,我们在表 6.1 中总结了本文工作与这些工作的关键区别。

6.2 相关工作

基于模型的强化学习 一种常用的 MB 技术是动作搜索。利用世界模型,可以模拟从提议分布或策略先验 [Nagabandi et al., 2018; Williams et al., 2015] 中采样得到的候选动作的结果,并搜索最优动作。这类方法已成功应用于 Atari 和围棋等游戏 [Schrittwieser et al., 2020; Ye et al., 2021],以及具有像素观测的连续控制问题 [Hafner et al., 2019b]。另一种方式是通过与世界模型交互来优化策略。这一思想最初来源于 Dyna 算法 [Sutton, 1991]。该领域各工作之间的主要区别在于其对模型生成数据的使用方式。例如,Dyna-Q [Sutton, 1990] 和 MBPO [Janner et al., 2019] 使用世界模型生成的转移数据来增强真实环境数据,然后在增强数据集或生成数据集上执行 MF 算法。Feinberg et al. [2018] 提出通过在世界模型中将策略展开至一定视野范围来改进价值估计。Dreamer 系列工作 [Hafner et al., 2019a, 2020, 2023] 将展开数据同时用于价值估计和策略学习。最近,Hansen et al. [2023, 2022b];Chitnis et al. [2023] 将这两种技术结合起来解决连续控制问题。由于无法穷举所有 MB 方法,我们建议读者参阅 Wang et al. [2019];Amos et al. [2021] 以获得更全面的综述和基准评测。上述大多数方法依赖于简单的一步世界模型 $p(\text{next } s, \text{rt} | \text{st}, \text{at } q)$ 。Dreamer 系列工作 [Hafner et al., 2019a, 2020, 2023] 使用循环神经网络(RNN)来利用过去信息以预测下一

状态。近来,Robine et al. [2023];Micheli et al. [2022];Chen et al. [2022] 分别独立提出了基于 Transformer 的世界模型作为 RNN 的替代方案。Janner et al. [2020] 使用生成模型来学习未来状态的占用度量,从而可以通过单次前向传播实现长视野展开。

离线强化学习 直接将在线强化学习方法应用于离线强化学习通常会导致较差的性能。这种失败通常归因于外推误差 [Fujimoto et al., 2019]。为解决这一问题,研究者引入了若干保守性概念,以促使策略保持接近离线数据。对于无模型方法,这些概念被应用于价值函数 [Kumar et al., 2020b; Kostrikov et al., 2021; Garg et al., 2023] 或策略本身 [Wu et al., 2019; Jaques et al., 2019; Kumar et al., 2019; Fujimoto and Gu, 2021]。保守性也已通过改进的 MDP 被引入到 MB 技术中。例如,MOPO [Yu et al., 2020] 在 MBPO 的基础上,在生成转移时对预测奖励进行重新标记。它从预测奖励中减去世界模型预测的不确定性,从而以软性方式促进具有低不确定性结果的状态-动作对。与此类似,MOReL [Kidambi et al., 2020] 使用一个构造出的带有终止状态的悲观 MDP 来训练策略。当世界模型预测的不确定性较高时,智能体将被转移至终止状态,并获得一个负奖励作为惩罚。

面向强化学习的序列建模 近来,将序列建模工具应用于强化学习问题的研究兴趣大量涌现。[Chen et al., 2021; Janner et al., 2021] 首次将离线轨迹视为自回归序列,并使用 Transformer 架构 [Vaswani et al., 2017] 对其建模。这启发了一系列后续研究,包括 [Meng et al., 2021; Lee et al., 2022a]。诸如扩散模型 [Ho et al., 2020; Sohl-Dickstein et al., 2015; Song et al., 2020a]、流匹配 [Lipman et al., 2022] 和一致性模型 [Song et al., 2023] 等归一化流已被引入到各种强化学习算法中,参见例如 [Chi et al., 2023; Ding and Jin, 2023; Du et al.,

2023; Hansen-Estruch et al., 2023; Jia et al., 2023; Mishra and Chen, 2023; Wang et al., 2022; Xu et al., 2023]。近来若干工作以多种方式利用扩散模型

一个逆动力学模型,根据当前状态和预测的下一状态来预测动作,因此在推理时必须调用扩散模型。而本文的方法则可以与任何执行速度较快的 MF 离线强化学习方法相结合。

6.3 预备知识

离线强化学习。我们考虑一个由 p, S, A, R, p_0, γ 定义的无限视野马尔可夫决策过程(MDP),其中 S 为状态空间, A 为动作空间。设 Δp_{S^q} 为状态空间上

给定一条轨迹

$\sqcup(\tau \leftarrow s_0, a_0, r_0, \dots, s|\tau|, a|\tau|, r|\tau|)$, 其中 $|\tau|$ 为总时间步数, 时刻 t 的回报-剩余(return-to-go, RTG) 为 $r|\tau|$

gt“t1“tγt'trt1。在离线强化学习中,我们被限制为仅能从由某些未知策略生成的静态数据集中学习策略。在本文中,我们用 Doffline 表示离线数据分布,用

扩散模型。扩散概率模型 [Ho et al., 2020; Sohl-Dickstein et al., 2015; Song et al., 2020a] 是一类通过迭代去噪过程从噪声生成样本的生成模型。它定义了一个固定的马尔可夫链,称为前向过程或扩散过程,该过程从数据点

‘?’

xp0q 开始,迭代地向 xpkq 添加高斯噪声:xpk'1q |xp0q " N 1 'βk I ,0 d' k d' K ' 1。当扩散步数 K Ñ 8 时,xpKq 本质上变为随机噪声。我们学习相

$N(\mu, \Sigma)$ 表示随机变量 $z \sim N(\mu, \Sigma)$ 。从扩散模型中采样, 相当于先采样一个随机噪声 $x_0 \sim N(0, I_q)$, 然后运行逆过程。设 μ_t, Σ_t 表示随机变量 $z_t \sim N(\mu_t, \Sigma_t)$ 。

$p\theta$ $pxpk$ '1q $|xpkq$ q “ φ xpk '1q ; $\mu\theta$ $pxpkq$ q, $\Sigma\theta$ $pxpkq$ q 参数化, 1 d' k d' K, 并

优化边缘似然 $p_{\theta}(\mathbf{x}_{0:K}|\mathbf{y})$ 的变分下界。存在多种等价的方式来优化该下界 [Kingma et al., 2021], 本文采用如下的噪声预测方式。可以将 $\mathbf{x}_{pk}$ 重写为 $\mathbf{x}_{pk} = \mathbf{A}_k \mathbf{x}_{p0} + \mathbf{b}_k + \boldsymbol{\varepsilon}_k$, 其中 $\mathbf{A}_k \in \mathbb{R}^{d \times d}$

$\mathbf{b}_k \in \mathbb{R}^d$, $\boldsymbol{\varepsilon}_k \in \mathbb{R}^d$ 为注入到 $\mathbf{x}_{pk}$ 中的噪声(在缩放之前)。随后我们参数化一个神经网络 $\boldsymbol{\varepsilon}_{pk}(\mathbf{x}_{p0}, \mathbf{y}, \mathbf{A}_k, \mathbf{b}_k)$ 来预测注入到 $\mathbf{x}_{pk}$ 中的噪声 $\boldsymbol{\varepsilon}_k$ 。此外, 可

$$\mathbb{E}_{p(\mathbf{x}_{p0}|\mathbf{y})} \|\boldsymbol{\varepsilon}_{pk}(\mathbf{x}_{p0}, \mathbf{y}, \mathbf{A}_k, \mathbf{b}_k) - \boldsymbol{\varepsilon}_k\|^2, \quad (6.1)$$

其中 $\mathbf{x}_{p0}$ 和 $\mathbf{y}$ 分别为从数据分布中采样得到的真实数据点和条件信息, $\boldsymbol{\varepsilon}_k \in \mathbb{R}^d$, $\mathbf{A}_k$ 为注入噪声, k 为在 1 到 K 之间均匀采样得到的扩散步数, $b_k \sim \text{Bernoulli}(p_k)$ 用于指示是否使用空条件, 最后 $\mathbf{x}_{pk} = \mathbf{A}_k \mathbf{x}_{p0} + \mathbf{b}_k + \boldsymbol{\varepsilon}_k$ 。算法 19 详细描述了如何从引导扩散模型中采样。在第 8.3 节中, 我们将在离线强化学习的背景下介绍 $\mathbf{x}_{p0}$ 和 $\mathbf{y}$ 的具体形式, 并讨论如何利用扩散模型来简化规划过程。

6.4 扩散世界模型

在本节中, 我们介绍一种基于扩散世界模型进行基于模型的离线强化学习的通用方法。本文的框架包含两个训练阶段, 将分别在第 6.4.1 节和第 6.4.2 节中详细介绍。在第一阶段, 我们训练一个扩散模型, 在给定当前状态、动作和目标回报的条件下, 预测一系列未来的状态和奖励。接下来, 我们使用一种演员-评论家(actor-critic)方法训练离线策略, 其中我们利用预训练的扩散模型进行基于模型的价值估计。算法 17-18 给出了该框架结合一个具有延迟更新的简单演员-评论家算法的实现, 其中我们假设采用确定性的离线策略。本文的框架可以很容易地在多种

算法 17 扩散世界模型训练 1: 超参数: 扩散步数 K , 空条件概率 p_{uncond} , 噪声参数 $\bar{\alpha}_k, k \in [1, K]$ 。 2: while 未收敛 do 3: 从 Doffline 中采样一条长度为 T 的子轨迹: $x_{p0q} \in \mathcal{D}_{p_{\text{st}}, a_{\text{t}}, r_{\text{t}}, s_{\text{t}}^1, r_{\text{t}}^1, \dots, r_{\text{t}}^T}$

..., $s_{\text{t}}^{T-1}, r_{\text{t}}^{T-1} \in \mathcal{Q}$ 。

4: 计算回报-剩余 $g_{\text{t}} \in \mathcal{D}_{\text{Th}}[0, 1]$

γr_{t}^h 。

5: % 通过公式 (6.1) 优化 DWM 6: 采样 $\varepsilon \sim \mathcal{N}(p_0, \frac{1}{K})$ 和 $k \in [1, K]$ (均匀分布)。 7: 计算 $x_{pkq} \in \bar{\alpha}_k x_{p0q} + (1 - \bar{\alpha}_k) \varepsilon$ 。 8: 以 $p_{\text{uncond}} + \frac{1}{K}$ 的概率设置 $y \in \mathcal{Q}$, 否则 $y \in g_{\text{t}}$ 。

9: 沿 $\nabla_{\theta} \varepsilon_{\theta} p_{x, k, y} \in \mathcal{E}^2$ 方向进行梯度更新。

> p_0q

10: 返回: 扩散世界模型 p_{θ} 。

算法 18 基于 DWM 的离线模型化强化学习通用演员-评论家框架

1: 输入: 预训练的扩散世界模型 p_{θ} 。

2: 超参数: 展开长度 H , 条件回报-剩余 g_{eval} , 引导参数 ω , 目标网络更新频率 n 。

3: 初始化演员和评论家网络 π_{ψ}, Q_{ϕ} 。

4: 初始化目标网络权重: $\psi^- \leftarrow \psi, \phi^- \leftarrow \phi$ 。 5: while 未收敛 do 6: 从 Doffline 中采样状态-动作对 $p_{\text{st}}, a_{\text{t}} \in \mathcal{Q}$ 。 7: % 扩散模型价值扩展

8: 在引导参数 ω 下, 从 $p_{\theta}(\cdot | s_{\text{t}}, a_{\text{t}}, g_{\text{eval}})$ 中采样 $\hat{r}_{\text{t}}, \hat{s}_{\text{t}}^1, \hat{r}_{\text{t}}^1, \dots, \hat{s}_{\text{t}}^{T-1}, \hat{r}_{\text{t}}^{T-1}$ 。

9: 计算目标 Q 值:

$H-1$

$\ddot{y}$

$y \in \gamma \hat{r}_{\text{t}}^h + \gamma H Q_{\phi} p_{\text{st}}^H, \pi_{\psi} p_{\text{st}}^H \in \mathcal{Q}$

$h \leftarrow 0$

10: % 更新评论家

11: 更新评论家: $\phi \leftarrow \phi + \eta \nabla_{\phi} \{Q_{\phi} p_{\text{st}}, a_{\text{t}} \in \mathcal{Q}\}$

12: % 更新演员

13: 更新演员: $\psi \leftarrow \psi + \eta \nabla_{\psi} Q_{\phi} p_{\text{st}}, \pi_{\psi} p_{\text{st}} \in \mathcal{Q}$

14: % 更新目标网络 15: if 迭代次数 mod $n \neq 0$ then

16: $\bar{\phi} \leftarrow \bar{\phi} + \omega p_{\phi} - \bar{\phi} q$

17: $\bar{\psi} \leftarrow \bar{\psi} + \omega p_{\psi} - \bar{\psi} q$

方式上进行推广。首先, 我们可以将其推广至考虑随机策略的情形。此外, 本文所提出的演员-评论家算法是最简单的形式, 它可以扩展为与各种现有的离线学习算法相结合。在第 6.5 节中, 我们讨论了算法 18 的三种具体实现, 分别嵌入了 TD3+BC [Fujimoto and Gu, 2021]、IQL [Kostrikov et al., 2021] 以及带悲观奖励的 Q 学习 [Yu et al., 2020]。

6.4.1 条件扩散模型

我们在长度为 T 的子轨迹上训练一个回报条件扩散模型 $p_{\theta}(x_{0:T} | s_t, a_t, y_t)$, 其中条件变量是该子轨迹的 RTG。也就是说, $y = \sum_{t=0}^{T-1} r_t + V(s_T) - V(s_0)$ 。

6.4.2 基于扩散世界模型的模型化强化学习

如算法 18 所总结, 我们提出了一种演员-评论家(actor-critic)算法, 其中评论家在扩散模型生成的合成数据上训练, 而演员则基于该评论家进行策略评估训练。简而言之, 我们通过 DWM 模拟得到的短期回报之和, 加上由代理状态-动作价值函数 Q (通过时序差分(TD)学习得到)估计的长期回报值, 来估计 Q 值。值得注意的是, 在我们的框架中, DWM 只介入评论家的训练, 算法 18 具有通用性, 可以与任何基于价值的模型无关(MF)算法相连接。我们将在第 6.5 节介绍它的 3 种不同实例化方式。

定义 49(H 步扩散模型价值扩展)。设 (s_t, a_t) 为一个状态-动作对。从扩散模型 $p_{\theta}(x_{0:T} | s_t, a_t, y_t)$ 中采样得到 $\hat{s}_t, \hat{s}_{t+1}, \hat{s}_{t+2}, \dots, \hat{s}_{t+H-1}, \hat{s}_{t+H}$ 。设

$$Q_t = H \cdot \text{diff}(p_{s_t, a_t} | p_{\theta}) + \sum_{h=0}^{H-1} \gamma^h r_{t+h} + \gamma^H Q_{t+H}, \quad (6.2)$$

其中 $\hat{v}_t^H = \pi(p \hat{s}_t^H | q, Q^* | p \hat{s}_t^H, \hat{a}_t^H | q)$ 是最终状态-动作对的代理价值。

我们利用这一扩展来计算 TD 学习中的目标值,见算法 18。这一机制是我们算法成功的关键,并具有若干吸引人的特性。

1. 在部署标准的模型化价值扩展(MVE, Feinberg et al. [2018])时,想象轨迹是通过递归查询一步动态模型 $p(s_{t+1}, r_t | s_t, a_t | q)$ 得到的,这正是误差累积的根本原因。相较于 MVE,我们的 DWM 一次性整体生成想象轨迹(不含动作)是其优势所在。

2. 更有趣的是,MVE 在查询 $p(s_{t+1}, r_t | s_t, a_t | q)$ 时使用策略预测的动作 $\hat{a}_t = \pi(p \hat{s}_t | q)$ 。这可以看作是在模拟环境中对策略 π 的一种同策略(on-policy)价值估计。相比之下, (a)我们的方法可以看作是一种策略迭代算法,在策略评估(算法 18 第 7-9 行)和策略提升(第 10 行)步骤之间交替进行。这里, $Q^* | H$

diff 是策略价值函数 Q_π 的估计量,具有可调的前瞻视野

H ,并可通过改变 geval 来实现悲观或乐观的估计。参数化的 Q_ϕ 通过均方误差朝目标 $Q^* | H$ diff 进行优化。在离线

强化学习的背景下,TD 学习常常导致 Q_π 的高估 [Thrun and Schwartz, 2014; Kumar et al., 2020b]。这是因为 π 可能产生分布外(out-of-distribution)的动作,导致 Q^* 的取值出现误差,而策略又被定义为使 Q^* 最大化。这种高估会对最终部署到线上环境时策略的泛化能力产生负面影响。为缓解这一问题,大量离线强化学习方法对价值函数施加各种形式的正则化 [Garg et al., 2023; Kostrikov et al., 2021; Kumar et al., 2020b],以确保所得策略保持接近数据分布。由于 DWM 完全基于离线数据训练,它可以被视为对生成该离线数据集的行为策略的一种综合。

换言之,diffusion-MVE 通过生成式建模的方式为离线强化学习引入了一种价值正则化。此外,我们的方法与现有的价值悲观化概念有显著不同。离线强化学习的一个难点在于,生成离线数据集的行为策略往往质量中等偏低,因此所得数据集可能仅包含中低回报的轨迹。因此,许多为离线强化学习引入的正则化技术往往过于悲观 [Ghasemipour et al., 2022; Nakamoto et al., 2023]。为解决这一问题,我们在从 DWM 中采样时通常以较大的分布外(OOD)geval 值为条件。换句话说,我们要求 DWM 在一个乐观目标下输出想象轨迹。

(b)另一种视角是,我们也可以将该方法视为一种离线 Q 学习算法 [Watkins and Dayan, 1992],其中 Q 利用离策略数据估计最优价值函数 Q^* 。同样地,这里的离策略数据是由扩散模型生成的,并以分布外的 RTG 值为条件。本质上,我们的方法可以刻画为在合成数据上进行的离线 Q 学习。

与基于 Transformer 的世界模型比较。细心的读者可能会想知道 DWM 与现有基于 Transformer 的世界模型 [Chen et al., 2022; Micheli et al., 2022; Robine et al., 2023] 之间的关键区别。这些模型在给定当前状态 s_t 和动作 a_t 的情况下,利用 Transformer 的自回归结构来整合过去的信息以预测 s_{t+1} 。为了预测未来的多个时间步,它们必须进行迭代预测。相比之下,DWM 通过单次查询即可完成长视野的预测。值得注意的是,完全可以用 Transformer 替代本工作中的扩散模型,我们将在第 6.5.4 节中说明我们做出这一设计选择的理由。

6.5 实验

我们的实验旨在回答以下问题。(1)与一步动态模型相比,DWM 是否能有效降低复合误差,并在

模型化强化学习(MBRL)中取得更好的性能?(2)所提出的算法 18 与其他基于扩散模型的模型化方法相比表现如何,以及(3)与它们的模型无关(model-free)对应方法相比又如何? 为回答这些问题,我们考虑了算法 18 的三种实例化方式,在第二阶段中分别集成 TD3+BC [Fujimoto and Gu, 2021]、IQL [Kostrikov et al., 2021] 以及带有悲观奖励的 Q 学习(我们称之为 PQL)作为离线强化学习算法。这些算法分别在动作(TD3+BC)、价值函数(IQL)和奖励(PQL)上采用了不同的保守性概念。具体而言,PQL 算法受 MOPO 算法 [Yu et al., 2020] 启发,我们通过世界模型预测奖励的不确定性对其进行惩罚。尽管如此,它在评论家学习上与 MOPO 有所不同。MOPO 在模型生成的转移数据上使用标准的 TD 学习,而我们则使用 MVE 或 Diff-MVE 进行价值估计。下文中,我们将这些算法分别称为 DWM-TD3BC、DWM-IQL 和 DWM-PQL。对于 DWM-IQL,我们观察到使用基于 λ -return 技术 [Schulman et al., 2015b] 的 Diff-MVE 变体可以提升性能,

因此我们将其作为默认特性纳入其中。这些算法的详细描述见第 6.7 节。我们在第 6.5.1-6.5.3 节中给出比较结果,并在第 6.5.4 节中对我们为 DWM 所做的设计选择进行消融实验。在第 6.9.4 节中,我们与额外的基线方法进行了实验比较,包括数据增广 [Lu et al., 2023b] 和自回归扩散 [Rigter et al., 2023] 方法。

基准与超参数。我们在来自 D4RL [Fu et al., 2020] 基准的 9 个运动任务数据集上进行实验,并报告归一化后的回报(0-1,1 表示专家水平的表现)。在全文中,我们对每个算法使用不同的随机种子训练 5 个实例,并各评估 10 个回合。所有报告的数值均为在 5 个随机种子上聚合得到的均值和标准差。我们将 DWM 的序列长度设为 $T \times 8$ (在第 6.5.1 节中讨论)。训练时的扩散步数为 $K \times 5$ 。对于 DWM 推理,我们采用了一种加速推理技术,将扩散步数减少至 $N \times 3$,详见第 6.5.4 节。DWM 的训练与采样细节参见第 6.6.1 节,而

hopper-m hopper-mr hopper-me

回报(均值 $\pm$ 标准差) 0.8 0.8 1.00 0.6 0.75 0.6 0.4 0.50 0.4 0.25 0.2 100 walker2d-m 100 101 100 10 1 1 walker2d-mr walker2d-me

回报(均值 $\pm$ 标准差) 0.8 1.0 0.4 0.6 0.4 0.2 0.5 0.2 0.0 0.0 100 10 halfcheetah-m 1 100 10 halfcheetah-mr 1 100 10 halfcheetah-me

0.5 0.8

回报(均值 $\pm$ 标准差) 0.4 0.3 0.6 0.4 0.2 0.4 0.3 0.1 0.2 0.0 100 101 100 101 100 101 模拟视野 模拟视野 模拟视野 OneStep-TD3BC
OneStep-IQL DWM-TD3BC DWM-IQL

图 6.2:使用 DWM 和一步模型、在不同模拟视野下算法 18 的性能表现。横轴范围为 $r1, 3l_s$,采用对数坐标。

各离线算法的训练细节参见第 6.8 节。我们进一步在稀疏奖励任务上进行了大量实验,结果详见第 6.9.5 节。

6.5.1 DWM 与一步动态模型的比较

我们首先研究 DWM 在降低 MBRL 复合误差方面的有效性,并将其与使用一步动态模型的对应方法进行比较。接下来,我们评估所提出算法 18 与一步动态模型对应方法的性能,即用一步动态模型替代 DWM 并使用标准 MVE。我们分别将这些基线称为 OneStep-TD3BC、OneStep-IQL 和 OneStep-PQL。

长视野规划与复合误差比较。为探究不同世界模型对长模拟视野的响应,我们比较了 DWM 方法(DWM-TD3BC 和 DWM-IQL)与其一步对应方法(OneStep-TD3BC 和 OneStep-IQL)在策略训练所用模拟视野 H 变化时的性能。为探索 DWM 模型的极限,我们额外训练了一组

序列长度更长($T=32$)的 DWM,并研究下游强化学习算法在 H P t1, 3, 7, 15, 31u 下的表现。使用一步动态模型的算法其模拟视野范围为 1 到 5。图 6.2 展示了在 9 个任务上的结果。OneStep-IQL 和 OneStep-TD3BC 随着模拟视野的增加表现出明显的性能下降。在大多数任务中,它们的性能在相对较短的模拟视野(如一或两步)时达到峰值。这表明使用一步动态模型进行更长的模型化推演会遭受严重的复合误差。相反,DWM-TD3BC 和 DWM-IQL 即使在视野长度为 31 的情况下,也能保持相对较高的回报,而没有出现显著的性能退化。需要注意的是,在最终结果表 6.2 中,我们报告的是使用序列长度 $T=8$ 的 DWM 的结果,因为使用 $T=32$ 带来的性能提升较为有限。详见第 6.9.7 节。我们还额外进行了分析 DWM 与一步模型预测的复合误差的实验。第 6.9.2 节中的结果表明,DWM 在降低复合误差方面表现更优,这验证了我们的假设。

离线强化学习性能。表 6.2 报告了算法 18 在 D4RL 数据集上使用 DWM 和一步动态模型时的性能表现。我们在模拟视野 H P t1, 3, 5, 7u 以及一组评估用 RTG 值范围内进行了扫描。我们所搜索的 RTG 值在不同环境间有所不同,见表 6.7。我们发现的主要趋势是:所提出的 DWM 显著优于一步动态模型对应方法,取得了 44% 的显著性能提升。这归因于扩散模型强大的表达能力,以及一次性预测整段序列这一特点,从而规避了多步推演中的复合误差问题。这一点将在下一段关于模拟视野的研究中进一步讨论。

6.5.2 DWM 与 Decision Diffuser 的比较

我们进一步将 DWM-TD3BC(表现最好的基于 DWM 的算法)与另一种密切相关的方法 Decision Diffuser (DD) [Ajay et al., 2022] 进行了比较,后者同样使用扩散模型对离线数据集中的轨迹进行建模。如第 8.1 节所述,

环境 OneStep-TD3BC OneStep-IQL OneStep-PQL DWM-TD3BC DWM-IQL DWM-PQL

hopper-m 0.39~ 0.04 0.45~ 0.05 0.63~ 0.12 0.65~ 0.10 0.54~ 0.11 0.50~ 0.09
 walker2d-m 0.39~ 0.15 0.52~ 0.24 0.74~ 0.14 0.70~ 0.15 0.76~ 0.05 0.79~ 0.08
 halfcheetah-m 0.44~ 0.05 0.44~ 0.03 0.45~ 0.01 0.46~ 0.01 0.44~ 0.01 0.44~ 0.01
 hopper-mr 0.26~ 0.05 0.25~ 0.03 0.32~ 0.03 0.53~ 0.09 0.61~ 0.13 0.39~ 0.03
 walker2d-mr 0.23~ 0.13 0.24~ 0.07 0.62~ 0.22 0.46~ 0.19 0.35~ 0.14 0.35~ 0.13
 halfcheetah-mr 0.43~ 0.01 0.42~ 0.02 0.42~ 0.01 0.43~ 0.01 0.41~ 0.01 0.43~ 0.01
 hopper-me 0.31~ 0.18 0.39~ 0.19 0.43~ 0.18 1.03~ 0.14 0.90~ 0.25 0.80~ 0.18
 walker2d-me 0.60~ 0.25 0.57~ 0.18 0.61~ 0.22 1.10~ 0.00 1.04~ 0.10 1.10~ 0.01
 halfcheetah-me 0.27~ 0.12 0.61~ 0.22 0.61~ 0.22 0.75~ 0.16 0.71~ 0.14 0.69~ 0.13
 0.368~ 0.105 0.432~ 0.115 0.537~ 0.128 0.679~ 0.098 0.641~ 0.117 0.610~ 0.080

均值

0.446~ 0.116 0.643~ 0.07

表 6.2:在 D4RL 数据集上,采用一步模型与 DWM 的模型化(MB)方法的比较。

环境 DD DWM-TD3BC 归一化回报 推理时间(秒) 归一化回报 推理时间(秒)

hopper-m 0.49~ 0.07 4.11~ 4.64 0.65~ 0.10 1.18~ 0.51
 walker2d-m 0.67~ 0.16 8.09~ 1.24 0.70~ 0.15 2.00~ 0.63
 halfcheetah-m 0.49~ 0.01 8.18~ 3.77 0.46~ 0.01 1.81~ 0.54
 hopper-mr 0.66~ 0.15 6.21~ 4.21 0.53~ 0.09 0.64~ 0.52
 walker2d-mr 0.44~ 0.26 5.94~ 4.32 0.46~ 0.19 0.83~ 0.45
 halfcheetah-mr 0.38~ 0.06 7.46~ 9.72 0.43~ 0.01 0.60~ 0.17
 hopper-me 1.06~ 0.11 8.82~ 2.96 1.03~ 0.14 2.62~ 1.22
 walker2d-me 0.99~ 0.15 9.26~ 1.30 1.10~ 0.00 3.60~ 3.90
 halfcheetah-me 0.91~ 0.01 9.53~ 2.50 0.75~ 0.16 3.77~ 2.43
 均值 0.677~ 0.109 7.531~ 3.651 0.679~ 0.098 1.620~ 1.379

表 6.3:DWM-TD3BC 与 Decision Diffuser (DD) 的性能相当,而 DWM-TD3BC 的速度比 DD 快 4.6 倍。

我们的方法与它们的方法有显著不同。DWM 同时以状态 s_t 和动作 a_t 为条件,而 DD 仅以 s_t 为条件。更重要的是,我们使用想象出的推演轨迹训练下游的模型无关策略,而 DD 则通过一个逆动态模型,利用当前状态 s_t 和预测的下一状态 s_{t+1} 来预测动作。这意味着,在推理阶段,DD 需要生成整段轨迹,这在计算上效率较低。相反,基于 DWM 的方法效率更高,因为我们在此阶段不再需要从训练好的 DWM 中采样。表 6.3 报告了 DWM-TD3BC 和 DD 的性能与推理时间。推理时间是在 600 个评估回合上取平均得到的。DWM-TD3BC 的性能与 Decision Diffuser 相当,且推理速度快 4.6 倍。我们预计,对于更高维度的问题,这一速度差异会进一步扩大。

环境 TD3+BC DWM-TD3BC IQL DWM-IQL

hopper-m	0.58~0.11	0.65~0.10	0.48~0.08	0.54~0.11
walker2d-m	0.77~0.09	0.70~0.15	0.75~0.15	0.76~0.05
halfcheetah-m	0.47~0.01	0.46~0.01	0.46~0.07	0.44~0.01
hopper-mr	0.53~0.19	0.53~0.09	0.25~0.02	0.61~0.13
walker2d-mr	0.75~0.19	0.46~0.19	0.48~0.23	0.35~0.14
halfcheetah-mr	0.43~0.01	0.43~0.01	0.44~0.01	0.41~0.01
hopper-me	0.90~0.28	1.03~0.14	0.86~0.22	0.90~0.25
walker2d-me	1.08~0.01	1.10~0.00	1.09~0.00	1.04~0.10
halfcheetah-me	0.73~0.16	0.75~0.16	0.60~0.23	0.71~0.14
平均	0.693~0.116	0.679~0.098	0.601~0.112	0.641~0.117

表 6.4:DWM 方法与其无模型(MF)对照方法的性能表现。

6.5.3 DWM 与无模型对照方法的比较

最后,我们将基于 DWM 的算法与其无模型(MF)对照方法进行比较,即 TD3+BC 对比 DWM-TD3BC,IQL 对比 DWM-IQL。表 6.4 报告了相应结果。对于每一组对比,我们将获胜的性能结果加粗标出。可以看到,DWM-IQL 的表现优于 IQL,而 DWM-TD3BC 与 TD3+BC 相当。与使用真实(ground-truth)样本的 MF 算法不同,基于模型(MB)的算法在近似动力学过程中难免会产生额外的建模误差。值得注意的是,使用传统单步动力学模型的 MB 算法(结果见表 6.2)明显逊于其 MF 对照方法,而 DWM 通过降低复合误差(compounding errors)缓解了动力学建模带来的这一劣势。

6.5.4 消融实验

扩散模型与 Transformer 的比较。算法 18 能够兼容多种类型的序列模型,包括 Transformer [Vaswani et al., 2017]----目前最成功的序列模型之一。然而,与单步动力学模型存在的复合误差问题类似,Transformer 由于其自回归(autoregressive)结构,本质上也会存在误差累积问题。因此,我们假设 Transformer 的表现会逊色,并选择了扩散模型。为验证这一假设,我们在所提出的算法中将扩散模型替换为 Transformer,并将所得性能与 DWM 方法进行比较。我们特别考虑了与 TD3+BC 和 IQL 的结合,将所得算法分别称为 T-TD3BC 和 T-IQL。我们对 T-TD3BC 和 T-IQL 进行测试,采用

环境 T-TD3BC T-IQL DWM-TD3BC DWM-IQL

hopper-m	0.58~0.08	0.55~0.08	0.65~0.10	0.54~0.11
walker2d-m	0.60~0.16	0.72~0.12	0.70~0.15	0.76~0.05
halfcheetah-m	0.42~0.03	0.43~0.01	0.46~0.01	0.44~0.01
hopper-mr	0.25~0.06	0.26~0.09	0.53~0.09	0.61~0.13
walker2d-mr	0.13~0.06	0.23~0.12	0.46~0.19	0.35~0.14
halfcheetah-mr	0.40~0.01	0.39~0.01	0.43~0.01	0.41~0.01
hopper-me	0.66~0.25	0.62~0.16	1.03~0.14	0.90~0.25
walker2d-me	0.58~0.15	1.03~0.09	1.10~0.00	1.04~0.10
halfcheetah-me	0.36~0.17	0.44~0.08	0.75~0.16	0.71~0.14
平均	0.442~0.101	0.519~0.084	0.679~0.098	0.641~0.117

表 6.5: 算法 18 分别使用 DWM 与基于 Transformer 的世界模型时的性能表现。

对模拟时域 H P t1, 3, 5, 7u 进行参数扫描, 与 DWM 方法所用的方式相同。对于评估用的 RTG, 我们采用 Decision Transformer [Chen et al., 2021] 中所使用的数值, 并应用与 DWM 相同的归一化方式。如表 6.5 所示, 在各种离线强化学习算法实例和环境中, DWM 始终优于基于 Transformer 的世界模型。实验细节参见第 6.9.3 节。

其他消融实验。第 6.9.6 节讨论了我们在训练 DWM 和轨迹采样时所使用的扩散步数。第 6.9.8 节讨论了我们从 DWM 中采样时所使用的评估 RTG 数值。

6.6 实现细节

6.6.1 扩散世界模型

我们在此总结实验中所使用的架构和超参数设置。在所有实验中, 我们使用了自行实现的 PyTorch 版本, 该实现在很大程度上受到以下代码库的影响:

Decision Diffuser [Ajay et al., 2022] <https://github.com/anuragajay/decision-diffuser> Diffuser [Janner et al., 2022] <https://github.com/janner/diffuser> SSORL [Zheng et al., 2023a] <https://github.com/facebookresearch/ssorl/>

架构。如第 6.4.1 节所述,本文所使用的扩散世界模型 p_θ 被设计用于建模长度为 T 的子轨迹 $p_{st}, at, rt, st^1, rt^1, \dots, st^T, rt^T$ 。在推断阶段,给定

$$\{st^1, rt^1, \dots, st^T, rt^T\} \sim p_\theta(p_{st}, at, y \mid q). \quad (6.3)$$

我们选择不对序列中未来的动作进行建模,原因有两点。首先,我们所提出的扩散模型价值扩展(定义 49)并不需要未来时间步的动作信息。其次,已有研究发现,通过扩散模型建模连续动作的准确性较低 [Ajay et al., 2022]。在本文中,我们统一训练长度为 $T = 8$ 的状态-奖励序列引导扩散模型。扩散步数为 $K = 5$ 。无条件(null conditioning)的概率 p_{uncond} 设为 0.25,批大小为 64。我们采用 Nichol 和 Dhariwal [2021] 提出的余弦噪声调度方案。折扣因子为 $\gamma = 0.99$,并且我们使用任务特定的奖励缩放系数对折扣 RTG 进行归一化:Hopper 任务为 400, Walker 任务为 550, Halfcheetah 任务为 1200。按照 Ajay et al. [2022] 的做法,我们的噪声预测器 ϵ_θ 采用时序 U-net [Janner et al., 2022; Ronneberger et al., 2015a],由 6 个重复的残差块组成,每个残差块包含 2 个时序卷积、随后的组归一化(group norm) [Wu and He, 2018] 以及最后的 Mish 非线性激活函数 [Mish, 2019]。扩散步数 k 首先被转换为其正弦位置编码,并通过一个 2 层 MLP 投影到潜在空间;RTG 数值则通过一个 3 层 MLP 转换为其潜在嵌入。在我们的扩散世界模型中,作为附加条件的初始动作 at 同样通过一个 3 层 MLP 转换为潜在嵌入,并进一步与扩散步数和 RTG 的嵌入进行拼接。

优化。我们使用 Adam 优化器对模型进行优化,所有数据集的学习率均为 1×10^{-4} 。我们最终考虑的模型参数 θ 是训练过程中所得参数的指数滑动平均(EMA)带引导的采样。要从扩散模型中采样,我们首先需要采样一个随机噪声 $x_{p0} \sim \mathcal{N}(0, I_q)$, 然后运行逆向过程。算法 19 给出了在无分类器引导(classifier-free guidance)下训练的扩散模型进行采样的一般流程。在离线强化学习的场景中,扩散世界模型根据当前状态 s_t 、当前动作 a_t 以及目标回报 g_{eval} 生成未来的状态和奖励,参见第 8.3 节。因此,采样过程与算法 19 略有不同,因为我们需要将初始状态和初始动作分别约束为 s_t 和 a_t 。改进后的算法在算法 20 中给出。

按照 Ajay et al. [2022] 的做法,我们对扩散模型采用低温采样(low temperature sampling)技术。在每个扩散步从高斯分布 $\mathcal{N}(\mu_{\theta, \alpha_2 \Sigma_{\theta, q}}, \Sigma_{\theta, q})$ 中采样时,温度设定为 β_k 。算法 19 和算法 20 运行完整的逆向过程。在此基础上,我们进一步应用 Nichol 和 Dhariwal [2021] 提出的步幅采样(stride sampling)技术以加速采样过程。具体地,在完整的逆向过程中,我们从 $k = K$ 直到 $k = 1$, 逐个通过 x_{pk} 生成 x_{pk-1} :

??

$$x_{pk-1} \sim \mathcal{N}(\bar{\alpha}_{k-1} \beta_{k-1} p_{0q} + \alpha_k p_1 - \bar{\alpha}_k \beta_k q, \Sigma_{k-1})$$

$$x_{pk-1} \sim \mathcal{N}(\bar{\alpha}_{k-1} \beta_{k-1} p_{0q} + \alpha_k p_1 - \bar{\alpha}_k \beta_k q, \Sigma_{k-1}), \quad (6.4)$$

$$\bar{\alpha}_{k-1} = \bar{\alpha}_k + \beta_k$$

其中

$$\beta_k p_1 - \bar{\alpha}_k \beta_k q$$

其中 $\hat{x}_{p0q}$ 是算法 19 中对真实数据点的预测, σ_k

$$\sigma_k = \beta_k$$

是步 k 处噪声的标准差(算法 19 中的第 8 行)。我们注意到 $\bar{\alpha}_k = \sum_{i=k}^K \beta_i$, 其中噪声调度 β_k 是预先定义好的,参见第 6.3 节和

第 6.6.1 节。

算法 19 从引导扩散模型中采样

```

1: 输入:训练好的噪声预测模型  $\epsilon_\theta$ , 条件参数  $y$ , 引导参数  $\omega$ , 扩散步数  $K$ 
2: 采样  $x_{pK} \sim N(p_0, I)$  3: for  $k \leftarrow K, \dots, 1$  do
    4:  $\hat{\epsilon} \sim \epsilon_\theta(p_{xpk}, k, y, \omega)$  5:  $\omega_q \sim \epsilon_\theta(p_{xpk}, k, \emptyset)$ 
5: % 估计真实数据点  $x_{p0}$ 
    6:  $\hat{x}_{p0} \leftarrow x_{p1} - \alpha_k \hat{\epsilon}$ 
    7: % 从后验分布  $q(x_{p1} | x_{pk}, x_{p0})$  中采样 8: % 参见 Ho et al. [2020] 的公式 (6) 和 (7)
    9:  $\hat{\mu} \leftarrow x_{p1} - \alpha_k \hat{\epsilon}$ 
    10:  $\hat{\Sigma} \leftarrow \Sigma_{p1} - \alpha_k \hat{\epsilon} \hat{\epsilon}^\top$ 
    11: 采样  $x_{pk} \sim N(\hat{\mu}, \hat{\Sigma})$ 
12: 输出:  $x_{p0}$ 

```

运行一次完整的逆向过程相当于对公式 (6.4) 求值 K 次, 这十分耗时。为加速采样, 我们在 1 到 K 之间等间距选取 N 个扩散步, 即 $\tau_1, \dots, \tau_N$, 其中 $\tau_N = K$ 。随后我们仅对所选的步 $\tau_1, \dots, \tau_N$ 求值公式 (6.4)。这样可将推断时间有效地缩短为原来的约 N/K 。在我们的实验中, 我们训练扩散模型时使用 $K = 5$ 个扩散步, 采样时使用 $N = 3$ 个推断步, 相关数值选取的依据见第 6.5.4 节。

6.6.2 单步动力学模型

传统的单步动力学模型 $f_\theta(p_{st}^1, r_t | s_t, a_t)$ 通常表示为状态与奖励空间上的一个参数化概率分布, 并通过对单步转移进行对数似然最大化来优化:

$$\max_{\theta} \mathbb{E}_{p_{st}^1, r_t, s_t^1} [\log f_\theta(p_{st}^1, r_t | s_t, a_t)] \quad (6.5)$$

θ

其中 p_{st}^1, r_t, s_t^1 是从离线数据分布 D_{offline} 中采样得到的。与 Kidambi et al. [2020] 一样, 我们将 f_θ 建模为高斯分布 $N(\mu_\theta, \Sigma_\theta)$, 其中均值 μ_θ 和方

算法 20 扩散世界模型采样 1: 输入: 训练好的噪声预测模型 ε_θ , 初始状态 s_t , 初始动作 a_t , 目标回报 $geval$, 引导参数 ω , 扩散步数 K 2: 采样 $x_{pK} \sim N(p_0, I_q)$ 3: % 施加对 s_t 和 a_t 的条件约束 4: $x_{pK} r_0 : \text{dimpst } q \backslash \text{dimpat } q_s \in \text{concatenate } p_s, a_t$ 5: for $k = 1, \dots, K$ do

6: $\hat{\varepsilon} \in \varepsilon_\theta \text{pxpk}_q, k, \text{geval } q \backslash p_1 \backslash \omega_q \sim \varepsilon_\theta \text{pxpk}_q, k, \emptyset_q$

7: % 估计真实数据点 x_{p0q}

1 $\backslash \text{pk}_q ? \sim$

8: $\hat{x}_{p0q} \in ? x \backslash 1 \backslash \hat{\alpha}_k \hat{\varepsilon}$

$\hat{\alpha}_k$ 9: % 从后验分布 $q_{\text{pxpk}} \backslash I_q | x_{\text{pk}_q}, x_{p0q}$ 中采样 10: % 参见 Ho et al. [2020] 的公式 (6) 和 (7)

$\hat{\alpha}_k \backslash 1 \beta_k p_{0q} \alpha_k p_1 \backslash \hat{\alpha}_k \backslash 1 q \text{pk}_q$

11: $\hat{\mu} \in \hat{x} \backslash x$

1 $\backslash \hat{\alpha}_k 1 \backslash \hat{\alpha}_k$

$\beta_k p_1 \backslash \hat{\alpha}_k \backslash 1 q$

12: $\Sigma \in I$

1 $\backslash \hat{\alpha}_k$

13: 采样 $x_{pk} \backslash 1 q \sim N(p\hat{\mu}, \Sigma_q)$

14: % 施加对 s_t 和 a_t 的条件约束 15: $x_{pk} \backslash 1 q r_0 : \text{dimpst } q \backslash \text{dimpat } q_s \in \text{concatenate } p_s, a_t$ 16: 输出: x_{p0q}

隐藏层采用 ReLU 激活函数, 每层含 256 个隐藏单元。 Σ_θ 的最终层通过 SoftPlus 函数激活以确保其有效性。我们使用 Adam 优化器, 以 1×10^{-4} 的学习率

6.7 基于扩散世界模型的离线强化学习方法

在第 6.5 节中, 我们考虑了算法 18 的 3 种实例, 分别将 TD3+BC、IQL、带悲观奖励的 Q 学习(PQL)整合进我们的框架中。这些算法均是专为离线强化学习设计的, 其保守性(conservatism)概念分别定义在动作(TD3+BC)、价值函数(IQL)和奖励(PQL)之上。下文中, 我们将这些实例分别称为 DWM-TD3BC、DWM-IQL 和 DWM-PQL。它们的具体实现将在下文介绍。

6.7.1 DWM-TD3BC:结合扩散世界模型的 TD3+BC

TD3+BC [Fujimoto and Gu, 2021] 在 TD3 算法 [Fujimoto et al., 2018] 的基础上,采用显式的行为克隆(behavior cloning)正则化来学习确定性策略。该算法的流程如下。critic 网络的训练完全遵循 TD3 算法。我们通过 TD 学习,以双 Q 学习(double Q-learning)[Fujimoto et al., 2018] 的方式学习两个 critic 网络 Q_{ϕ^1} 和 Q_{ϕ^2} 。对于转移 (s, a, r, s') , TD 学习的目标值为:

$$y = r + \gamma \min_i Q_{\phi^i}(s', a) - \text{Clip}(\pi_{\psi}(s, a) - \pi_{\psi}^{\text{old}}(s, a), -\epsilon, \epsilon), \quad (6.6)$$

其中 $\epsilon \sim \mathcal{N}(0, \sigma^2)$ 是随机噪声, π_{ψ} 是目标策略, $\text{Clip}(\cdot)$ 是一个将动作向量各维度的值限制在 $[-\epsilon, \epsilon]$ 范围内的操作符。 Q_{ϕ^1} 和 Q_{ϕ^2} 均通过最小化均方误差

$$\sum_{(s,a,r,s') \sim \text{Doffline}} \left(Q_{\phi^i}(s, a) - y \right)^2, \quad i \in \{1, 2\}. \quad (6.7)$$

在训练策略时, TD3+BC 优化以下带正则化的目标:

$$\mathbb{E}_{(s,a) \sim \text{Doffline}} \left[\lambda Q_{\phi^1}(s, \pi_{\psi}(s, a)) - \frac{1}{2} \left\| \pi_{\psi}(s, a) - \pi_{\psi}^{\text{old}}(s, a) \right\|^2 \right], \quad (6.8)$$

其中 Doffline 为离线数据分布。若去掉行为克隆正则项 $\frac{1}{2} \left\| \pi_{\psi}(s, a) - \pi_{\psi}^{\text{old}}(s, a) \right\|^2$, 上述问题即退化为对应于确定性策略梯度定理 [Silver et al., 2014] 的目标函数。注意在 TD3+BC 中, 目标策略 π_{ψ} 和目标 critic 网络 Q_{ϕ^i} 的更新都是延迟进行的。整个算法总结在算法 21 中。

算法 21 DWM-TD3BC

1: 输入: 离线数据集 $\mathcal{D}_{\text{offline}}$, 预训练好的扩散世界模型 p_θ , 模拟时域 H , 条件 RTG geval , 策略/目标网络更新频率 n , 系数 λ , 动作扰动/裁剪参数: σ, C

2: 初始化 actor 和 critic 网络: $\pi, \psi, Q, \phi_1, Q, \phi_2$ 3: 初始化目标网络权重: $\psi^- \leftarrow \psi, \phi_1^- \leftarrow \phi_1, \phi_2^- \leftarrow \phi_2$ 4: for $i = 1, 2, \dots$ 直至收敛 do

5: 从 $\mathcal{D}_{\text{offline}}$ 中采样状态-动作对 (s, a) 6: % 扩散模型价值扩展

7: 采样 $\hat{r}, \hat{s}_1, \hat{s}_2, \dots, \hat{s}_T, \hat{r}_T, \hat{s}_T$ $p_\theta(p|s, a, \text{geval})$

8: 采样 $\epsilon \sim \mathcal{N}(0, \sigma^2 I_q)$

9: $\hat{a} \leftarrow H \circ \text{Clip}(\pi(\psi(p\hat{s}_T^H, q^-, \epsilon), C), C)$

计算目标 Q 值: $y \leftarrow H^T$

$r \leftarrow H^T \epsilon$

10: $h \leftarrow \gamma \hat{r} + \gamma \min_{p \in \Pi} p\hat{s}_T^H, 2u \leftarrow Q\phi_1(p\hat{s}_T^H, \hat{a}^T, H, q)$

11: % 更新 critic 网络

12: $\phi_1 \leftarrow \phi_1 - \eta \nabla \phi_1 \|Q\phi_1(p\hat{s}_T^H, \hat{a}^T, H, q) - y\|_2^2$

13: $\phi_2 \leftarrow \phi_2 - \eta \nabla \phi_2 \|Q\phi_2(p\hat{s}_T^H, \hat{a}^T, H, q) - y\|_2^2$

14: % 更新 actor 和目标网络(延迟更新) 15: if $i \bmod n = 0$ then

16: $\psi \leftarrow \psi - \eta \nabla \psi \rho \lambda Q\phi_1(p\hat{s}_T^H, \hat{a}^T, H, q) - \pi(p\hat{s}_T^H, q)\|_2^2$ % 更新 actor

17: $\phi_1^- \leftarrow \phi_1 - \eta \nabla \phi_1^- \|Q\phi_1^-(p\hat{s}_T^H, \hat{a}^T, H, q) - y\|_2^2$

18: $\phi_2^- \leftarrow \phi_2 - \eta \nabla \phi_2^- \|Q\phi_2^-(p\hat{s}_T^H, \hat{a}^T, H, q) - y\|_2^2$

19: $\psi^- \leftarrow \psi - \eta \nabla \psi^- \rho \lambda Q\phi_1^-(p\hat{s}_T^H, \hat{a}^T, H, q) - \pi(p\hat{s}_T^H, q)\|_2^2$

20: 输出: π, ψ

6.7.2 DWM-IQL: 结合扩散世界模型的 IQL

IQL [Kostrikov et al., 2021] 在离线数据集上应用悲观价值估计。除了 TD3+BC 中所使用的双 Q 函数外, IQL 还引入了一个额外的状态价值函数 $V \xi_{psq}$, 其通过期望回归(expectile regression)估计得到:

$$\tau \min_{\pi} \mathbb{E}_{p \sim \pi} [L(\pi, \psi, \phi_1, \phi_2, V \xi_{psq})], \quad (6.9)$$

其中 $L(\pi, \psi, \phi_1, \phi_2, V \xi_{psq}) = \mathbb{E}_{p \sim \pi} [\tau \sum_{t=0}^{\infty} \gamma^t (V \xi_{psq}(s_t) - Q_{\psi, \phi_1}(s_t, a_t))^2]$, 超参数 $\tau \in [0, 1]$, $q \in [0, 1]$ 。当 $\tau \rightarrow 1$ 时, $V \xi_{psq}$ 本质上是在估计 Q_{ψ, ϕ_1} 的最大值。这可以视为隐式地执行了策略提升(policy improvement)。

算法 22 DWM-IQL 1: 输入: 离线数据集 $\mathcal{D}_{\text{offline}}$, 预训练好的扩散世界模型 p_{θ} , 模拟时域 H , 条件 RTG g_{eval} , 目标网络更新频率 n , 期望回归损失参数 τ 2: 初始化 actor、critic 和价值网络 $\pi, \psi, Q\phi_1, Q\phi_2, V\xi$ 3: 初始化目标网络: $\phi_1^- \leftarrow \phi_1, \phi_2^- \leftarrow \phi_2$ 4: for 迭代 $i = 1, 2, \dots$ 直至收敛 do 5: 从 $\mathcal{D}_{\text{offline}}$ 中采样状态-动作对 (s, a) 6: % 扩散模型价值扩展

```

7: 采样  $\tau_t, \hat{s}_t^1, \hat{s}_t^2, \dots, \hat{s}_t^T$ 
 $\tau_t = \tau \cdot p_{\theta}(\tau | s_t, a_t, g_{\text{eval}})$ 
8: 计算目标 Q 值:  $y = H^T \hat{s}_t^1$ 
 $h = 0, \gamma \hat{\tau}_t^h \cdot \gamma V\xi p\hat{s}_t^h$ 
9: % 更新价值网络
10:  $\xi \leftarrow \xi - \eta \nabla_{\xi} L_{\text{pt1}, 2u} Q\phi_i(p, a, V\xi p)$ 
11: % 更新 critic(Q 网络)
12:  $\phi_1^- \leftarrow \phi_1 - \eta \nabla_{\phi_1} Q\phi_1(p, a, y)$ 
13:  $\phi_2^- \leftarrow \phi_2 - \eta \nabla_{\phi_2} Q\phi_2(p, a, y)$ 
14: if  $i \bmod n = 0$  then
15: % 更新 actor
16:  $\psi \leftarrow \psi - \eta \nabla_{\psi} \exp \beta \min_{\text{pt1}, 2u} Q\phi_i(p, a, V\xi p) \log \pi \psi(p, a) | s$ 
17: % 更新目标网络
18:  $\phi_1^- \leftarrow \phi_1^- + \omega \phi_1$ 
19:  $\phi_2^- \leftarrow \phi_2^- + \omega \phi_2$ 
20: 输出:  $\pi, \psi$ 
```

通过优势加权回归 (Advantage Weighted Regression) [Peng et al., 2019] 提取, 即求解

$$\max_{\psi} \mathbb{E}_{p(a|\psi)} [\mathcal{D}_{\text{offline}} \exp \beta p Q\phi(p, a, V\xi p) \log \pi \psi(p, a)] \quad (6.10)$$

ψ

在 IQL 中, 目标 critic 网络 $Q\phi^-$ 的更新是延迟进行的。整个算法总结在算法 22 中。

6.7.3 DWM-PQL: 结合扩散世界模型的悲观 Q 学习

此前的离线强化学习算法, 如 MOPO [Yu et al., 2020], 曾直接将保守性概念应用于奖励函数, 本文将此类方法称为悲观 Q 学习 (PQL)。具体而言, Yu et al. [2020] 最初提出的算法

学习一个由 m 个单步动力学模型组成的集成 $\{\theta_i\}_{i=1}^m$, 并使用修正后的奖励

$$\tilde{r}_t = r_t + \alpha \sum_{i=1}^m \text{ups}_i(\theta_i, \mathbf{s}_t) \quad (6.11)$$

来学习 Q 函数, 其中 $\tilde{r}$ 是集成的均值预测, $\text{ups}_i, \alpha, \theta_1, \dots, \theta_m$ 是使用该集成对预测不确定性的一种度量。Yu et al. [2020] 将每个动力学模型参数化为

给定 $\mathbf{s}_t, \mathbf{a}_t$ 和 $\mathbf{g}_t$, 我们从 DWM 中随机采样 m 条序列, 即

$$\begin{aligned} & \{\mathbf{s}_t^1, \mathbf{a}_t^1, \mathbf{g}_t^1, \mathbf{s}_{t+1}^1, \mathbf{a}_{t+1}^1, \mathbf{g}_{t+1}^1, \dots, \mathbf{s}_{T-1}^1, \mathbf{a}_{T-1}^1, \mathbf{g}_{T-1}^1\} \\ & \dots \\ & \{\mathbf{s}_t^m, \mathbf{a}_t^m, \mathbf{g}_t^m, \mathbf{s}_{t+1}^m, \mathbf{a}_{t+1}^m, \mathbf{g}_{t+1}^m, \dots, \mathbf{s}_{T-1}^m, \mathbf{a}_{T-1}^m, \mathbf{g}_{T-1}^m\} \end{aligned} \quad (6.12)$$

然后, 我们取第 1 个样本作为 DWM 的输出, 并对其奖励进行修正:

$$\tilde{r}_t = r_t + \alpha \sum_{i=1}^m \text{ups}_i(\theta_i, \mathbf{s}_t) \quad (6.13)$$

$$\tilde{r}_t = r_t + \alpha \sum_{i=1}^m \text{ups}_i(\theta_i, \mathbf{s}_t)$$

这提供了一种高效的方法, 可以沿着扩散模型预测的轨迹为每个时间步构造带不确定性惩罚的奖励。请注意, 这一方法并不适用于最后一个时间步所预测的奖励。算法的其余部分遵循 IQL, 但在更新价值网络时使用 MSE 损失而非期望分位数损失 (expectile loss)。

DWM-PQL 算法总结于算法 23 中。

对于采用单步动力学模型的基线方法, 从采样 $\mathbf{s}_t, \mathbf{a}_t$ 出发的想象轨迹, 是通过从单步动力学模型 $f(\mathbf{s}_t, \mathbf{a}_t)$ 与策略 $\pi(\mathbf{s}_t)$ 递归采样得到的。该

算法 23 DWM-PQL

```

1: 输入:离线数据集 Doffline,预训练的扩散世界模型  $p_\theta$ ,模拟时域 H,条件回报目标(RTG)geval,目标网络更新频率 n,悲观系数  $\lambda$ ,用于不确定性估计的采样数
2: 初始化 actor、critic 与 value 网络  $\pi, \psi, Q\phi_1, Q\phi_2, V\xi$  3: 初始化目标网络: $\phi_1^- \in \phi_1, \phi_2^- \in \phi_2$  4: for 迭代 i “ 1, 2, .
.. 直至收敛 do 5: 从 Doffline 中采样状态-动作对  $pst, at$  q 6: % 扩散模型价值扩展 7: 采样 m 条子轨迹  $t^j, s^j, r^j, t^j$ 
, ...,  $s^j, t^j, r^j, t^j$ 
j m
8: 按公式 (6.13) 修正第一条子轨迹的奖励:
 $\tilde{r}, \tilde{s}^1, \tilde{r}^1, \dots, \tilde{s}^T, \tilde{r}^T, \tilde{r}^T$ 
9: 计算目标 Q 值: $y = H^T h$ 
 $h = \gamma \tilde{r} + \gamma V\xi p^T H q$ 
10: % 更新价值网络  $\phi_1, \phi_2$ 
11:  $\xi \leftarrow \eta \nabla_{\xi} \text{miniPt}_{1,2u} Q\phi_i ps, aq \leftarrow V\xi psq$ 
12: % 更新 critic(Q 网络)
13:  $\phi_1 \leftarrow \eta \nabla_{\phi_1} Q\phi_1 pst, at q \leftarrow y$ 
14:  $\phi_2 \leftarrow \eta \nabla_{\phi_2} Q\phi_2 pst, at q \leftarrow y$ 
15: if i mod n “ 0 then
16: % 更新 actor  $\pi, \psi$ 
17:  $\psi \leftarrow \eta \nabla_{\psi} \exp \beta \text{miniPt}_{1,2u} Q\phi_i ps, aq \leftarrow V\xi psq \log \pi \psi pa \leftarrow sq$ 
18: % 更新目标网络
19:  $\phi_1^- \leftarrow \eta \nabla_{\phi_1^-} Q\phi_1^- pst, at q \leftarrow y$ 
20:  $\phi_2^- \leftarrow \eta \nabla_{\phi_2^-} Q\phi_2^- pst, at q \leftarrow y$ 
21: 输出: $\pi, \psi$ 

```

6.8 离线强化学习算法的训练与评估细节

6.8.1 通用设置

我们在 TD3+BC 和 IQL 上进行了初步测试,以选择数据归一化方面的最佳实践。基于这些结果,TD3+BC、O-TD3BC 和 DWM-TD3BC 采用观测归一化,而其他算法(O-PQL、DWM-PQL、IQL、O-IQL 和 DWM-IQL)则同时采用观测归一化和奖励归一化。所有模型均在 NVIDIA Tesla V100 PCIe GPU 设备上训练。对于训练 200 个 epoch(每个 epoch 1000 次迭代),无模型算法(如 TD3+BC 和 IQL)通常耗时约 2000 秒,基于 DWM 的模型型算法(如 DWM-TD3BC 和 DWM-IQL)通常耗时约 18000 秒,基于 Transformer 模型的算法

(如 T-TD3BC 和 T-IQL)通常耗时约 8000 秒,基于单步模型的算法(如 O-TD3BC 和 O-IQL)通常耗时约 2300 秒。具体计算时间会因任务而异。

所有算法均使用固定批大小 128 进行训练,并使用一组固定的预训练动力学模型(单步模型和扩散模型)。所有数据的折扣因子均设置为 $\gamma = 0.99$ 。

6.8.2 无模型算法

TD3+BC 和 IQL 训练 1×10^6 次迭代,actor、critic 和 value 网络的学习率均为 3×10^{-4} 。actor、critic 和 value 网络均由 3 层 MLP 参数化,每层 256 个隐藏单元。TD3+BC 和 IQL 的超参数见表 6.6。

TD3+BC IQL 策略噪声 0.2 期望分位数(expectile) 0.7 噪声裁剪 0.5 β 3.0 策略更新频率 2 最大权重 100.0 目标更新频率 2 策略更新频率 1 α 2.5 优势归一化 否 EMA w 0.005 EMA w 0.005

表 6.6:训练 TD3+BC 和 IQL 的超参数。

基线 DD [Ajay et al., 2022] 算法使用序列长度 $T = 32$ 、扩散步数 $K = 5$ 训练的扩散模型。它还需要额外训练一个用于动作预测的逆动力学模型(IDM),该模型由一个 3 层 MLP 参数化,每个隐藏层 1024 个隐藏单元,并使用 ReLU 激活函数。该 MLP 的 dropout 率为 0.1。IDM 在每个环境上均训练 2×10^6 次迭代。为了与其他 DWM 方法进行公平比较,DD 采用与 DWM 相同的 $N = 3$ 次内部采样步数。我们对 DD 与其他 DWM 方法在相同范围内搜索评估用的 RTG 进行几何平均。

6.8.3 基于模型的算法

DWM-TD3BC、DWM-IQL 和 DWM-PQL 训练 5×10^5 次迭代。表 6.7 总结了我们为每个实验搜索的超参数。其余超参数与网络结构与上述章节中原始的采用单步动力学模型的对应方法由于根据我们的经验观察收敛相对较快,训练 2×10^5 次迭代。大多数超参数同样遵循 TD3+BC 和 IQL。PQL 类算法(O-PQL 和 DWM-PQL)进一步在 $t=0.01, 0.1, 1.0$ 中搜索悲观系数 κ (定义见公式 (6.13))。

环境 评估用 RTG H

hopper-medium-v2 [0.6, 0.7, 0.8] [1,3,5,7] walker2d-medium-v2 [0.6, 0.7, 0.8] [1,3,5,7] halfcheetah-medium-v2 [0.4, 0.5, 0.6] [1,3,5,7]
 hopper-medium-replay-v2 [0.6, 0.7, 0.8] [1,3,5,7] walker2d-medium-replay-v2 [0.6, 0.7, 0.8] [1,3,5,7] halfcheetah-medium-replay-v2
 [0.4, 0.5, 0.6] [1,3,5,7] hopper-medium-expert-v2 [0.7, 0.8, 0.9] [1,3,5,7] walker2d-medium-expert-v2 [0.8, 0.9, 1.0] [1,3,5,7]
 halfcheetah-medium-expert-v2 [0.6, 0.7, 0.8] [1,3,5,7]

表 6.7:我们为 DWM 方法搜索的超参数列表。

6.9 补充实验 6.9.1 DWM 长时域规划的详细结果

本节提供第 6.5.1 节中关于 DWM 长时域规划实验的详细结果。表 6.8 总结了 DWM-IQL 和 DWM-TD3BC 在不同模拟时域 $t=1, 3, 7, 15, 31$ 下的归一化回报(均值与标准差)。

回报(均值 标准差)

环境 模拟时域 DWM-IQL DWM-TD3BC

1 0.54 $\pm$ 0.11 0.68 $\pm$ 0.12 3 0.55 $\pm$ 0.10 0.63 $\pm$ 0.11 hopper-medium-v2 7 0.56 $\pm$ 0.09 0.66 $\pm$ 0.13 15 0.58 $\pm$ 0.12 0.77 $\pm$ 0.15 31 0.61 $\pm$ 0.11 0.79 $\pm$ 0.15

1 0.65 $\pm$ 0.23 0.56 $\pm$ 0.13 3 0.74 $\pm$ 0.11 0.74 $\pm$ 0.13 walker2d-medium-v2 7 0.71 $\pm$ 0.13 0.74 $\pm$ 0.11 15 0.66 $\pm$ 0.15 0.73 $\pm$ 0.13 31 0.67 $\pm$ 0.20 0.75 $\pm$ 0.12

1 0.44 $\pm$ 0.01 0.35 $\pm$ 0.03 3 0.44 $\pm$ 0.01 0.39 $\pm$ 0.01 halfcheetah-medium-v2 7 0.44 $\pm$ 0.01 0.40 $\pm$ 0.01 15 0.44 $\pm$ 0.02 0.40 $\pm$ 0.01 31 0.44 $\pm$ 0.01 0.40 $\pm$ 0.01

1 0.18 $\pm$ 0.06 0.52 $\pm$ 0.21 3 0.37 $\pm$ 0.18 0.44 $\pm$ 0.23 hopper-medium-replay-v2 7 0.39 $\pm$ 0.14 0.52 $\pm$ 0.28 15 0.37 $\pm$ 0.18 0.67 $\pm$ 0.25 31 0.37 $\pm$ 0.15 0.59 $\pm$ 0.22

1 0.32 $\pm$ 0.15 0.13 $\pm$ 0.02 3 0.27 $\pm$ 0.24 0.19 $\pm$ 0.10 walker2d-medium-replay-v2 7 0.25 $\pm$ 0.20 0.22 $\pm$ 0.14 15 0.26 $\pm$ 0.19 0.22 $\pm$ 0.10 31 0.27 $\pm$ 0.19 0.17 $\pm$ 0.12

1 0.38 $\pm$ 0.05 0.02 $\pm$ 0.00 3 0.39 $\pm$ 0.02 0.17 $\pm$ 0.05 halfcheetah-medium-replay-v2 7 0.39 $\pm$ 0.02 0.22 $\pm$ 0.03 15 0.38 $\pm$ 0.03 0.26 $\pm$ 0.03 31 0.37 $\pm$ 0.03 0.26 $\pm$ 0.05

1 0.86 $\pm$ 0.25 0.88 $\pm$ 0.17 3 0.90 $\pm$ 0.19 0.94 $\pm$ 0.22 hopper-medium-expert-v2 7 0.88 $\pm$ 0.28 0.93 $\pm$ 0.24 15 0.85 $\pm$ 0.20 0.91 $\pm$ 0.19 31 0.84 $\pm$ 0.23 0.93 $\pm$ 0.23

1 0.80 $\pm$ 0.22 0.74 $\pm$ 0.21 3 1.02 $\pm$ 0.09 0.89 $\pm$ 0.13 walker2d-medium-expert-v2 7 0.98 $\pm$ 0.2 0.82 $\pm$ 0.19 15 1.06 $\pm$ 0.05 0.84 $\pm$ 0.14 31 1.05 $\pm$ 0.06 0.87 $\pm$ 0.03

1 0.60 $\pm$ 0.18 0.39 $\pm$ 0.01 3 0.52 $\pm$ 0.14 0.43 $\pm$ 0.07 halfcheetah-medium-expert-v2 7 0.63 $\pm$ 0.13 0.44 $\pm$ 0.03 15 0.66 $\pm$ 0.14 0.50 $\pm$ 0.08 31 0.65 $\pm$ 0.17 0.49 $\pm$ 0.09

表 6.8:DWM-TD3BC 与 DWM-IQL 在不同模拟时域下归一化回报的比较。报告的数值为在不同 RTG 值(列于表 6.7)下的最佳表现。

6.9.2 世界建模:预测误差分析

我们额外进行了一项实验,以评估 DWM 在模拟时域 $H = 8$ 下对观测和奖励的预测误差,以 walker2d-medium-expert-v2 任务为例。我们从离线数据集中随机采样一段子序列 $p_{st}, \dots, st^7 q$, 并让 DWM 在给定第一个状态 st 和真实动作 at 的条件下预测后续的状态和奖励。对于单步模型,该模型在给定当前状态 st (在前一步中预测得到) 和真实动作 at 的条件下,迭代地预测奖励 rt 和下一状态 st^1 。我们在表 6.9 中报告了每个 rollout 时间步预测样本与真实值之间的均方误差(MSE)。每种方法均使用五个模型进行评估,每个模型 100 条序列,报告均值与标准差。最后一行还计算了整条序列的平均预测误差。结果表明,使用 DWM 相较于传统的单步模型,在序列建模中显著降低了预测误差,尤其是在预测时间步较大时更为明显。

单步模型 DWM 步数 观测 奖励 观测 奖励

1	0.0000	~ 0.0000	8.05e '05	~ 0.00011	0.0000	~ 0.0000	8.89e '05	~ 0.00018
2	0.0363	~ 0.0455	2.03e '04	~ 0.00031	0.1050	~ 0.1668	1.50e '04	~ 0.00020
3	0.1576	~ 0.2308	4.73e '04	~ 0.00084	0.4173	~ 0.3902	2.34e '04	~ 0.00021
4	0.3503	~ 0.3547	7.68e '04	~ 0.00148	0.4525	~ 0.5021	2.33e '04	~ 0.00018
5	0.6173	~ 0.4945	1.13e '03	~ 0.00222	0.4796	~ 0.5035	2.71e '04	~ 0.00039
6	0.9185	~ 0.8678	2.00e '03	~ 0.00417	0.4854	~ 0.4759	4.01e '04	~ 0.00102
7	1.2394	~ 0.9788	3.40e '03	~ 0.00757	0.5353	~ 0.5076	3.89e '04	~ 0.00094
8	1.4890	~ 0.9612	4.47e '03	~ 0.00844	0.5146	~ 0.5814	3.77e '04	~ 0.00064
平均	0.6010	~ 0.4916	0.0015	~ 0.0031	0.3737	~ 0.3984	0.0003	~ 0.0004

表 6.9:单步模型与 DWM 在 walker2d-medium-expert-v2 上,不同 rollout 时间步($H = 8$)下观测与预测误差(MSE)的比较。

6.9.3 基于 Transformer 的世界模型

遵循与 DWM 算法 18 相同的流程,Transformer 模型被训练用于在给定初始状态-动作对的条件下预测未来的状态-奖励序列。我们使用一个具有 4 个注意力头的 4 层 Transformer 架构,与 Zheng et al. [2022] 中的架构类似。具体而言,在状态-动作-奖励序列中,除第一个动作外,其余所有动作均被掩码置零。与原始 DT [Chen et al., 2021] 不同(其损失函数仅包含动作预测误差),此处 Transformer 使用状态和奖励预测损失进行训练。Transformer 的优化器与超参数遵循 ODT [Zheng et al., 2022]。对于 hopper、walker2d 和 halfcheetah 环境,Transformer 评估所用的 RTG 分别取值 3600{400 “ 9.0、5000{550 < 9.1、6000{1200 “ 5.0。T-TD3BC 和 T-IQL 的完整结果分别见表 6.10 和表 6.11。

模拟时域

环境 1357

hopper-m	0.50 ^{0.05}	0.57 ^{0.08}	0.58 ^{0.08}	0.57 ^{0.08}
walker2d-m	0.36 ^{0.15}	0.40 ^{0.20}	0.60 ^{0.16}	0.53 ^{0.17}
halfcheetah-m	0.18 ^{0.07}	0.41 ^{0.03}	0.38 ^{0.08}	0.42 ^{0.03}
hopper-mr	0.24 ^{0.01}	0.23 ^{0.05}	0.25 ^{0.06}	0.22 ^{0.08}
walker2d-mr	0.12 ^{0.04}	0.09 ^{0.05}	0.13 ^{0.06}	0.12 ^{0.05}
halfcheetah-mr	0.40 ^{0.01}	0.39 ^{0.02}	0.39 ^{0.03}	0.39 ^{0.02}
hopper-me	0.41 ^{0.13}	0.57 ^{0.19}	0.66 ^{0.25}	0.52 ^{0.15}
walker2d-me	0.34 ^{0.22}	0.58 ^{0.15}	0.58 ^{0.26}	0.46 ^{0.37}
halfcheetah-me	0.14 ^{0.06}	0.31 ^{0.09}	0.36 ^{0.17}	0.29 ^{0.12}

表 6.10:T-TD3BC 的归一化回报。

模拟时域

环境 1357

hopper-m	0.48	0.08	0.54	0.10	0.55	0.08	0.51	0.09
walker2d-m	0.54	0.18	0.62	0.19	0.72	0.12	0.72	0.14
halfcheetah-m	0.42	0.03	0.42	0.02	0.43	0.01	0.43	0.01
hopper-mr	0.17	0.05	0.24	0.09	0.26	0.09	0.20	0.07
walker2d-mr	0.17	0.12	0.17	0.14	0.23	0.12	0.16	0.11
halfcheetah-mr	0.38	0.04	0.39	0.01	0.38	0.04	0.39	0.03
hopper-me	0.62	0.16	0.59	0.21	0.47	0.21	0.47	0.21
walker2d-me	0.67	0.23	0.87	0.21	1.03	0.09	0.71	0.22
halfcheetah-me	0.39	0.19	0.43	0.13	0.44	0.08	0.43	0.09

表 6.11:T-IQL 的归一化回报。

6.9.4 补充基线

数据增强

作为一种数据增强(DA)方法,SynTHER [Lu et al., 2023b] 在转移(transition)层面学习一个无条件扩散模型,并使用生成的数据来增强训练分布。本节中,我们对 DWM 和 SynTHER 类型的数据增强方法进行了补充实验。基于 TD3+BC 和 IQL 的两种数据增强方法分别称为 DA-TD3BC 和 DA-IQL。为了公平比较,DA-TD3BC 和 DA-IQL 的模型训练方式与 DWM 相同,并采用相同的 RTG 值和时域参数搜索方式。结果表明,在 TD3+BC 和 IQL 两种情况下,DWM 在所有任务上的表现均持续优于 DA 算法。为了与 DWM 进行公平比较,本实验中我们采用与 DWM 相同的数据预处理方式,这与原始 SynTHER 论文中的状态-奖励归一化方式不同。在我们的实验中,我们使用 CDF 归一化器,将状态向量的每个维度独立地变换到 $r \in [1, 1s]$ 区间,即

通过用边际 CDF 进行变换,使数据在每个维度上呈均匀分布。具体而言,我们将原始奖励 r_{raw} 变换为 $r = \frac{r_{raw} - r_{min}}{r_{max} - r_{min}}$,其中 r_{min} 和 r_{max} 是奖励的最小值和最大值。环境 DA-TD3BC DWM-TD3BC DA-IQL DWM-IQL

hopper-m	0.65 $\pm$ 0.10	0.65 $\pm$ 0.10	0.51 $\pm$ 0.10	0.54 $\pm$ 0.11
walker2d-m	0.63 $\pm$ 0.18	0.70 $\pm$ 0.15	0.74 $\pm$ 0.09	0.76 $\pm$ 0.05
halfcheetah-m	0.44 $\pm$ 0.01	0.46 $\pm$ 0.01	0.44 $\pm$ 0.01	0.44 $\pm$ 0.01
hopper-mr	0.53 $\pm$ 0.09	0.53 $\pm$ 0.09	0.25 $\pm$ 0.04	0.61 $\pm$ 0.13
walker2d-mr	0.37 $\pm$ 0.22	0.46 $\pm$ 0.19	0.42 $\pm$ 0.24	0.35 $\pm$ 0.14
halfcheetah-mr	0.43 $\pm$ 0.01	0.43 $\pm$ 0.01	0.42 $\pm$ 0.04	0.41 $\pm$ 0.01
hopper-me	1.03 $\pm$ 0.14	1.03 $\pm$ 0.14	0.55 $\pm$ 0.19	0.90 $\pm$ 0.25
walker2d-me	1.09 $\pm$ 0.04	1.10 $\pm$ 0.00	0.76 $\pm$ 0.13	1.04 $\pm$ 0.10
halfcheetah-me	0.72 $\pm$ 0.14	0.75 $\pm$ 0.16	0.62 $\pm$ 0.14	0.71 $\pm$ 0.14
平均	0.654 $\pm$ 0.103	0.679 $\pm$ 0.098	0.523 $\pm$ 0.109	0.641 $\pm$ 0.117

表 6.12:我们的 DWM 方法与数据增强(DA)方法在 D4RL 数据集上的比较。结果为在 5 个随机种子上的汇总。

自回归扩散

我们对前人工作 [Rigter et al., 2023] 中提到的自回归扩散(Autoregressive Diffusion,AD)方法进行了实验,作为一种补充基线。AD 本质上是一种使用扩散模型而非 MLP 的单步模型,该模型可以被自回归地展开,并用于基于模型的价值扩展(Model Based Value Expansion,MVE)。我们发现,AD 在实际集成到 MVE 框架中时计算效率低下。我们检验了在不同时域取值下,针对 walker2d-medium-v2 环境采样一批 128 条未来轨迹所需的实际运行时间(以秒计)。

我们将轨迹数量设为128,因为这是我们训练RL智能体所使用的批大小。结果在100次试验上取平均。对于两种方法,我们都使用相同模型规模的扩散模型,采样扩散步数为3(以实现快速推理)。本实验在A6000 GPU上进行,时间单位为秒。

结果展示在表6.13中。DWM的采样时间是恒定的,因为它是一个序列模型,实际操作中我们对整个序列进行扩散,然后根据H取其中一部分;而AD的采样时间随H的增加呈线性增长。当H=7时,采样时间大约是DWM的6.67倍。我们在论文中报告的MB方法训练了 5×10^5 次迭代(见D.3节)。这意味着,如果使用AD,仅生成轨迹就需要27.5小时(而DWM大约只需4小时)。这表明AD的计算成本过高,难以融入MBRL框架。

方法 H=1 H=3 H=5 H=7 DWM(以T=8训练) 0.031 0.031 0.031 0.031 自回归扩散模型 0.030 0.087 0.145 0.198

表6.13:在不同视野H值下,AD与DWM采样时间(秒)的比较

6.9.5 附加环境:稀疏奖励任务

为进一步在各类任务上验证本方法,我们在具有稀疏奖励的迷宫型任务上进行了实验。所比较的方法包括TD3+BC、IQL、对应的DWM变体以及DD。训练和评估流程与第6.8节中的主要实验完全一致。结果汇总于表6.14,显示了基于DWM的算法在稀疏奖励设定下的优越性能。

环境	DWM-TD3BC	DWM-IQL	TD3+BC	IQL	DD	maze2d-umaze	0.36 ± 0.23	0.39 ± 0.29	0.05 ± 0.15	0.08 ± 0.16	0.40 ± 0.52
maze2d-medium	0.57 ± 0.50	0.41 ± 0.09	0.00 ± 0.006	0.10 ± 0.10	0.20 ± 0.19	maze2d-large	0.28 ± 0.13	0.11 ± 0.13	-0.01 ± 0.03	0.01 ± 0.07	0.02 ± 0.07
antmaze-umaze	0.86 ± 0.29	0.66 ± 0.47	0.58 ± 0.49	0.64 ± 0.48	0.31 ± 0.45						

表6.14:不同方法在稀疏奖励任务上的比较:三个maze2d任务和一个antmaze任务。

6.9.6 消融实验:训练与推理时扩散步数的数量

训练扩散步数 K 会强烈影响建模质量, K 值越大通常带来越好的性能。同时,从扩散模型中采样被认为是一个缓慢的过程,因为它涉及 K 个内部去噪步骤。我们采用步幅采样技术[Nichol and Dhariwal, 2021]来通过减少内部步数 N 加速采样过程,详见第6.6.1节。然而,采样速度的提升是以质量为代价的。在推理速度与预测精度之间取得平衡十分重要。我们研究了如何选择 K 和 N 的数量,以在不牺牲模型性能的前提下显著加速采样。

我们以不同的扩散步数 $K \in \{5, 10, 20, 30, 50, 100\}$ 训练DWM,序列长度为 $T=8$ 。我们设置四个推理步数比率 $r_{\text{infer}} \in \{0.2, 0.3, 0.5, 1.0\}$,并在步幅采样中使用 $N = \lceil r_{\text{infer}} \cdot K \rceil$ 。

图6.3:以 $T=8$ 训练、扩散步数 K 不同的DWM,随推理步数比率 r_ratio 变化时,平均观测和奖励预测误差(在9个任务和模拟视野 $H \in [7]$ 上取平均)。

图6.4:以 $T=32$ 训练、扩散步数 K 不同的DWM,随推理步数比率 r_ratio 变化时,平均观测和奖励预测误差(在9个任务和模拟视野 $H \in [31]$ 上取平均)。

也针对更长序列长度 $T=32$ 的DWM重复了上述实验。结果同样支持选择 $r_infer=0.5$,但更倾向于 $K=10$,见图6.4。

详细结果

设 τ 表示一个长度为 T 的子轨迹($s_t, a_t, r_t, s_{t+1}, \dots, s_{t+T-1}, r_{t+T-1}$)。DWM p_θ 在状态和奖励序列上的平均预测误差定义为:

$$\begin{aligned} \bar{\epsilon}_s &= E_{\tau \sim D_{\text{offline}}, \hat{s}_t \sim p_\theta(-|s_1, a_1, g_{\text{eval}}))} \left[\frac{1}{T} \sum_{t'=t}^{t+T-1} \|\hat{s}_{t'} - s_{t'}\|^2 \right], \text{以及} \\ \bar{\epsilon}_r &= E_{\tau \sim D_{\text{offline}}, \hat{s}_t \sim p_\theta(-|s_1, a_1, g_{\text{eval}}))} \left[\frac{1}{T} \sum_{t'=t}^{t+T-1} \|\hat{r}_{t'} - r_{t'}\|^2 \right]. \end{aligned} \quad (6.14)$$

我们首先注意到,所有预测误差都取决于评估RTG g_{eval} 。为了呈现清晰,我们在表6.7所列的多个 g_{eval} 值上进行搜索,并报告最佳结果。

除图6.3之外,图6.4展示了 $T=32$ (更长序列)、扩散步数 $K \in \{5, 10, 20\}$ 的扩散模型的平均预测误差。基于这些结果,选择 $K=10$ 和 $r_{infer}=0.5$ 以在预测精度和推理速度之间取得良好平衡。相应的数值结果列于表6.15和表6.16。

扩散步数K 推理步数比率r_ratio 推理步数N 观测误差 奖励误差

0.2	1	2.815	0.009		
0.3	2	1.028	0.001	0.5	3
1.0	5	0.851	0.001		
0.2	1	3.114	0.011		
0.3	2	1.601	0.002	0.5	3
1.0	5	0.943	0.001		
0.2	1	3.052	0.014		
0.3	2	1.595	0.002	0.5	3
1.0	5	0.890	0.001		
0.2	1	3.112	0.018		
0.3	2	1.623	0.003	0.5	3
1.0	5	0.896	0.001		
0.2	1	3.275	0.022		
0.3	2	1.726	0.003	0.5	3
1.0	5	0.944	0.001		
0.2	1	3.239	0.023		
0.3	2	1.732	0.003	0.5	3
1.0	5	0.923	0.001		

表6.15:以 $T=8$ 训练、不同推理步数 $N = r_infer - K$ 的DWM的平均(在各任务和模拟视野 $H \in [7]$ 上取平均)观测和奖励预测误差。

扩散步数K 推理步数比率r_infer 推理步数N 观测误差 奖励误差

0.2	1	7.390	0.030	
0.3	2	6.003	0.015	0.5
3	5.065	0.010		
1.0	5	4.853	0.010	
0.2	1	6.650	0.029	
0.3	2	5.799	0.016	0.5
3	4.811	0.010		
1.0	5	5.157	0.011	
0.2	1	6.273	0.031	
0.3	2	5.254	0.015	0.5
3	4.794	0.011		
1.0	5	5.088	0.012	

表6.16:以T=32训练、不同推理步数N= r_infer - K 的DWM的平均(在各任务和模拟视野H∈[31]上取平均)观测和奖励预测误差。

6.9.7 消融实验:扩散世界模型的序列长度

我们进一步比较了以序列长度T=8和T=32训练的DWM算法的平均性能。表6.17展示了9个任务上(在RTG值和模拟视野H上搜索得到)的平均最佳回报。尽管DWM对长视野模拟具有稳健性,并且在某些情况下我们发现最优H大于8,但我们发现使用T=32能提升DWM-IQL的性能,却略微损害了DWM-TD3BC的性能。

DWM-TD3BC	DWM-IQL(无λ)
T=8	T=32
T=8	T=32
0.68±0.10	0.60±0.12
0.57±0.09	0.61±0.10

表6.17:使用不同序列长度的DWM算法在9个任务上的平均性能。

因此,我们在主要实验中选择 $T=8$ 。

6.9.8 消融实验:分布外(OOD)评估RTG值

我们发现评估RTG值在决定算法性能方面起着关键作用。我们关于轨迹预测的初步实验表明,分布内评估RTG的表现不如分布外(OOD)RTG,见第6.9.8节。图6.5报告了DWM-IQL和DWM-TD3BC在3个任务上、不同 g_{eval} 值下的回报。我们报告了在模拟视野1、3、5和7上取平均的结果。所比较的RTG值在每个任务上各不相同,但均为分布外。第6.9.8节展示了每个任务训练RTG的分布。结果表明,实际回报并不总是与指定的 g_{eval} 相匹配。这是回报条件化RL方法的一个众所周知的问题 [Emmons et al., 2021; Zheng et al., 2022; Nguyen et al., 2022]。尽管如此,分布外评估RTG总体上表现良好。图6.5显示DWM-TD3BC和DWM-IQL对分布外评估RTG均具有稳健性。我们强调,所报告的回报是在具有不同模拟视野的训练实例上取平均得到的,而表6.2中报告的峰值性能更高。我们的直觉是鼓励扩散模型对当前状态的未来回报采取乐观的看法。另一方面,评估RTG也不能过高。如halfcheetah-mr任务所示,当 $RTG_{g_{eval}} > 0.4$ 时,进一步增加会使两种方法的实际性能进一步下降。最优RTG值因任务而异,完整实验结果如下所示。

图6.5:使用不同评估RTG值(括号中标注)的DWM方法比较。

我们注意到,回报和RTG的归一化方式不同:D4RL基准计算的回报是未折扣的,并以某个SAC策略的表现进行归一化,而我们在训练中使用的RTG是折扣的,并以手动选定的常数进行归一化。

离线数据分布

图6.6分析了整个D4RL数据集在九个任务上的归一化折扣回报(作为训练中的RTG标签)。与我们实验中的RTG值(表6.7)相比,数据的最大值通常小于能带来更高性能的评估RTG值,这与我们的实证实验观察一致。

hopper-medium-v2 hopper-medium-replay-v2 hopper-medium-expert-v2 最大回报=0.61 最大回报=0.64 最大回报=0.68 密度 密度 密度 归一化折扣回报 归一化折扣回报 归一化折扣回报

walker2d-medium-v2 walker2d-medium-replay-v2 walker2d-medium-expert-v2 最大回报=0.54 最大回报=0.54 最大回报=0.65 密度 密度 密度 归一化折扣回报 归一化折扣回报 归一化折扣回报

halfcheetah-medium-v2 halfcheetah-medium-replay-v2 halfcheetah-medium-expert-v2 最大回报=0.37 最大回报=0.35 最大回报=0.63 密度 密度 密度 归一化折扣回报 归一化折扣回报 归一化折扣回报

图6.6:各环境的归一化折扣回报。

分布内 与 分布外 RTG

与公式(6.14)类似,我们可以为每个时间步 $t, t \leq t' \leq t+T-1$ 定义细分预测误差。图6.7和图6.8绘制了使用不同 g_eval 值的结果。分布外RTG总体表现更好。值得注意的是,预测误差随视野增大而自然增长。直观地说,在固定环境下,D4RL数据集的初始状态非常相似,而经过多个时间步之后的后续状态则变得相当不同。

图6.7:不同RTG下,DWM在每个预测时间步的细分预测误差。该DWM以 $T=8$ 和 $K=5$ 训练。

图6.8:不同RTG下,DWM在每个预测时间步的细分预测误差。该DWM以 $T=32$ 和 $K=10$ 训练。

6.9.9 消融实验: λ 回报值估计

Dreamer系列工作[Hafner et al., 2019a, 2020, 2023]将 λ 回报技术[Schulman et al., 2015b]应用于基于想象轨迹的值估计。该技术可以无缝地嵌入我们的框架中,作为对标准Diff-MVE的一种改进。更确切地说,给定从离线

数据集中采样的状态-动作对 (s_t, a_t) ,我们对 $h=H, \dots, 0$ 递归地计算 λ 目标值:

$$Q_{t+h}^{\lambda} = \hat{r}_{t+h} + \gamma - \{ (1-\lambda)Q_{t+h}(\hat{s}_{t+h+1}, \pi_{t+h}(\hat{s}_{t+h+1})) \} + \lambda Q_{t+h+1}^{\lambda}, \text{ 若 } h < H$$

$$Q_{t+H}^{\lambda} = Q_{t+H}(\hat{s}_{t+H}, \pi_{t+H}(\hat{s}_{t+H})), \text{ 若 } h=H \quad (6.15)$$

其中使用DWM预测的状态 $\{\hat{s}_{t+h}\}_{h=0}^H$ 和奖励 $\{\hat{r}_{t+h}\}_{h=0}^H$ 。我们可以将 Q_t^{λ} 用作TD学习的目标Q值,作为算法18的一种修改。对于同时学习纯

我们进行实验,比较了DWM-TD3BC和DWM-IQL中原始diff-MVE与 λ 回报变体,使用 $\lambda=0.95$ 。我们在RTG值(见表6.7)和模拟视野1、3、5、7上进行搜索。结

DWM-TD3BC DWM-IQL

环境 无 λ 有 λ 无 λ 有 λ

hopper-m 0.65 ± 0.10 0.68 ± 0.13 0.50 ± 0.08 0.54 ± 0.11 walker2d-m 0.70 ± 0.15 0.74 ± 0.08 0.62 ± 0.19 0.76 ± 0.05 halfcheetah-m
 0.46 ± 0.01 0.40 ± 0.01 0.46 ± 0.01 0.44 ± 0.01 hopper-mr 0.53 ± 0.09 0.50 ± 0.23 0.29 ± 0.04 0.61 ± 0.13 walker2d-mr 0.46 ± 0.19
 0.23 ± 0.10 0.27 ± 0.09 0.35 ± 0.14 halfcheetah-mr 0.43 ± 0.01 0.39 ± 0.02 0.43 ± 0.01 0.41 ± 0.01 hopper-me 1.03 ± 0.14 1.05 ± 0.16
 0.78 ± 0.24 0.90 ± 0.25 walker2d-me 1.10 ± 0.00 0.89 ± 0.13 1.08 ± 0.03 1.04 ± 0.10 halfcheetah-me 0.75 ± 0.16 0.71 ± 0.22 0.73 ± 0.14
 0.74 ± 0.16 平均 0.68 0.62 0.57 0.64

表6.18:使用原始Diff-MVE和 λ 回报变体的DWM方法性能比较。

6.9.10 消融实验:RTG重标记与模型微调

与传统RL中的动态规划不同,像扩散模型和DT这样的序列建模方法被怀疑无法拼接次优轨迹。RTG重标记被提出用以缓解DT[Yamagata et al., 2023]的这一问题,方法是迭代地将训练数据集中的RTG g 重标记为:

$$\tilde{g}_t = r_t + \gamma \cdot \max(g_{t+1}, V(s_{t+1})) = \max(g_t, r_t + \gamma \cdot V(s_{t+1})), \quad (6.16)$$

其中 V 函数是使用CQL[Kumar et al., 2020b]单独从离线数据集中学习的,使用 $\max$ 算子是为了防止由于 V 的悲观性而导致低估。Yamagata et al.[2023]的原

我们在原始DWM-IQL算法策略学习阶段的DWM微调中应用了RTG重标记,不使用 λ 回报技术。值函数 V 来自IQL算法。我们将整个策略学习的前10%步作为

环境 DWM-IQL(无 λ) DWM-IQL(有RTG重标记)

hopper-m 0.50 ± 0.08 0.59 ± 0.13

walker2d-m 0.62 ± 0.19 0.65 ± 0.17

halfcheetah-m 0.46 ± 0.01 0.47 ± 0.01

hopper-mr 0.29 ± 0.04 0.27 ± 0.02

walker2d-mr 0.27 ± 0.09 0.32 ± 0.15

halfcheetah-mr 0.43 ± 0.01 0.43 ± 0.02

hopper-me 0.78 ± 0.24 0.88 ± 0.26

walker2d-me 1.08 ± 0.03 1.1 ± 0.0

halfcheetah-me 0.73 ± 0.14 0.79 ± 0.10

平均 0.57 0.61

表6.19:通过在策略训练阶段进行RTG重标记来微调DWM的结果:归一化回报(均值±标准差)

第7章

一致性模型作为强化学习策略

本节基于与Chi Jin合作撰写的论文《Consistency Models as a Rich and Efficient Policy Class for Reinforcement Learning》[Ding and Jin, 2023],此前发表于ICLR 2024。

7.1 引言

参数化策略表示是基于策略的深度强化学习(DRL)的重要组成部分[Sutton and Barto, 2018; Arulkumaran et al., 2017; Dong et al., 2020]。以往的工作已经开发出多种策略参数化方法。对于离散动作空间,现有的策略参数化包括Softmax动作偏好[Sutton and Barto, 2018]、用于类别分布的Gumbel-Softmax[Jang et al., 2016]、决策树[Frosst and Hinton, 2017; Ding et al., 2020]等。对于连续动作空间,最典型的选择是单峰高斯分布。然而,在实践中,示范数据集往往包含来自多种混合行为策略的样本。为了捕捉数据分布中的多模态性,高斯混合模型(GMM)[Jacobs et al., 1991; Ren et al., 2021]、变分自编码器(VAE)[Kingma, 2013; Kumar et al., 2019]、去噪扩散概率模型(DDPM)[Ho et al.,

2020;Song et al., 2020a;Wang et al., 2022;Chi et al., 2023;Hansen-Estruch et al., 2023;Venkatraman et al., 2023]被用作策略表征方法。

深度强化学习中策略表征的必备条件包括:1. 函数类的强表达能力被认为是对离线强化学习[Wang et al., 2022]或模仿学习(IL)[Chi et al., 2023]中的多模态数据分布进行建模的关键;2. 模型的可微性通常是使用随机梯度下降进行优化时所必需的;3. 采样的计算效率和时间效率对于依赖与环境交互进行学习的强化学习智能体而言至关重要。以往使用动作扩散模型(即扩散策略)的工作证明了扩散模型对多模态动作分布的表达能力[Wang et al., 2022;Chi et al., 2023;Hansen-Estruch et al., 2023;Janner et al., 2022;Ajay et al., 2022]。尽管GMM和VAE同样能够捕捉多模态性,但研究发现,在模仿学习和离线强化学习场景中,采用大采样步数的扩散模型具有更强的表达能力[Wang et al., 2022;Chi et al., 2023]。然而,众所周知,采用大量步数进行渐进式去噪的扩散模型会导致采样速度缓慢。对于严重依赖从环境中采样的在线强化学习而言,动作推断可能成为关键瓶颈。若直接将扩散策略用于采用策略梯度进行优化的在线场景,则需要针对采样步数对扩散网络进行反向传播,这由于耗时和内存占用较大而难以扩展。一致性模型[Song et al., 2023]基于概率流常微分方程(ODE)被提出作为一种补救方案,其性能与扩散模型相当,但计算时间大幅减少,从而支持少步生成过程,显著降低了推理阶段的时间消耗。

本文首次尝试将一致性模型----一种既具表达能力又高效的生成模型----用作深度强化学习的策略表征。该一致性策略被嵌入到行为克隆(BC)方法和一种演员-评论家(AC)算法中,分别命名为Consistency-BC和Consistency-AC。实验评估涵盖了三种典型的强化学习场景:离线、离线到在线以及在线场景。在这三种场景下,我们在D4RL数据集[Fu et al., 2020]上对两种生成模型----扩散模型和一致性模型----所构建的策略进行了全面比较。

对于离线强化学习,我们提出了一种新的损失缩放方法,用于稳定带策略正则化的一致性策略的训练过程,并展示了两种生成式策略模型的表达能力。这一点通过如下方式得以体现:采用扩散模型或一致性模型这类富有表达力的模型进行行为克隆,能够得到相当不错的策略,其性能优于一些以往的离线强化学习方法。通过利用演员-评论家风格的算法并配合必要的策略正则化以避免生成分布外(out-of-distribution)动作,性能进一步得到提升。一致性策略的快速采样过程不仅有助于缩短训练时间(例如,离线BC训练时间缩短43%),更重要的是,通过加速动作推断提高了在环境中进行在线交互的时间效率。在采用离线数据集训练的初始化模型进行离线到在线设置,以及从零开始学习的在线设置中,一致性策略在部分任务上展现出与扩散策略相当甚至更高的性能,同时训练和推理所需的实际运行时间大幅缩短。源代码已公开¹。

7.2 相关工作

离线强化学习与离线到在线强化学习。离线强化学习是在固定数据集上进行策略优化的问题。众所周知,该问题存在针对数据集之外的状态和动作产生价值高估的困扰。现有解决该方法的方法可分为以下几类:(1)通过批约束、行为克隆(BC)或散度约束(例如,Kullback-Leibler散度、最大均值差异)显式地将学习策略约束在离线数据附近,包括批约束深度Q学习(BCQ)[Fujimoto et al., 2019]、TD3+BC[Fujimoto and Gu, 2021]、Onestep RL[Brandfonbrener et al., 2021]、优势加权演员-评论家(AWAC)[Nair et al., 2020]、自举误差累积削减(BEAR)[Kumar et al., 2019]、BRAC[Wu et al., 2019]、扩散Q学习(Diffusion QL)[Wang et al., 2022]、极端Q学习(X-QL)[Garg et al., 2023]以及演员受限Q学习(ARQ)[Goo and Niekum, 2022];或(2)通过悲观的价值估计进行隐式正则化, https://github.com/quantumiracle/Consistency_Model_For_Reinforcement_Learning

例如保守Q学习(CQL)[Kumar et al., 2020b]、随机集成混合(REM)[Agarwal et al., 2020]、隐式Q学习(IQL)[Kostrikov et al., 2021]、隐式扩散Q学习(IDQL)[Hansen-Estruch et al., 2023]、基于模型的离线策略优化(MOPO)[Yu et al., 2020]等。MoRel[Kidambi et al., 2020]是一种基于模型的离线强化学习算法,通过构建悲观MDP来学习保守策略,该方法并不明确归属于上述两类之一。

离线到在线强化学习通常在线训练初期阶段会因训练样本的分布偏移而出现性能灾难性下降的问题。以往研究已探讨了利用离线数据或预训练策略进行在线微调的方法,包括Hybrid Q learning[Song et al., 2022]、RLPD[Ball et al., 2023]、Cal-QL[Nakamoto et al., 2023]、Action-free Guide[Zhu et al., 2023]、演员-评论家对齐(ACA)[Yu and Zhang, 2023]以及[Lee et al., 2022b]。

基于分数的生成模型在强化学习中的应用。在强化学习的策略表征方面,近期工作也使用去噪扩散概率模型(DDPM)[Ho et al., 2020; Song et al., 2020a](本文中我们泛称其为扩散模型,原始扩散模型可追溯至[Sohl-Dickstein et al., 2015])来捕捉离线数据集集中的多模态分布。Diffusion QL[Wang et al., 2022]在Q学习+BC方法中使用扩散模型进行策略表征。隐式扩散Q学习(IDQL)[Hansen-Estruch et al., 2023]是IQL的一种采用扩散策略的变体。Diffusion policies[Chi et al., 2023]将扩散模型应用于机器人领域模仿学习场景下的策略表征。Diffuser[Janner et al., 2022]和Decision Diffuser[Ajay et al., 2022]将决策Transformer架构与扩散模型相结合,用于基于离线数据集的模型化强化学习。扩散策略也被用于目标条件模仿学习[Reuss et al., 2023]和人类行为模仿[Pearce et al., 2023]。Q引导策略优化(QGPO)[Lu et al., 2023a]提出了一种针对扩散采样过程中间引导的新表述形式。潜在扩散约束Q学习(LDCQ)[Venkatraman et al., 2023]提出

将潜在扩散模型与批约束Q值相结合,以处理离线数据集的拼接(stitching)问题和外推误差。

7.3 预备知识

7.3.1 离线与在线强化学习

对于强化学习,我们定义一个马尔可夫决策过程 $(\mathcal{S}, \mathcal{A}, R, T, \rho_0, \gamma)$, 其中 $\mathcal{S}$ 是状态空间, $\mathcal{A}$ 是动作空间, $R: \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ 是奖励函数, $T: \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$ 是状态转移函数, ρ_0 是初始状态分布, γ 是折扣因子。

7.3.2 一致性模型

扩散模型[Ho et al., 2020; Song et al., 2020a]通过随机微分方程(SDE)求解多模态分布匹配问题,而一致性模型[Song et al., 2023]则求解一个等价的概率流。

而言,它近似一个参数化的一致性函数 $f_{\theta} : \mathcal{X}^T, \tau \mapsto \mathcal{X}$,其定义为从步骤 τ 处的含噪样本 x_{τ} 映射回原始样本 x_{ϵ} 的映射,而不是像扩散模型中的反向扩散过程。

7.4 一致性模型作为强化学习策略

一致性模型作为强化学习中的策略表征可以按如下方式表述。为了将一致性模型映射为MDP中的策略,我们设定:

$$c \sim p_{\theta}(c|x, a), p_D(p(a|s, q)) \sim \text{Consistency_Inference}(p_{\theta}, q) \quad (7.1)$$

其中 $p_D(p(a|s, q))$ 是离线数据集 D 中的动作-状态条件分布。

一致性动作推断。通过将条件变量 c 设为状态 s 、生成变量 x 设为动作 a ,一致性函数 f_{θ} 可用于根据数据集的条件分布从状态生成动作,即一个行为强化学

算法24 一致性动作推断 算法26 离线一致性演员-评论家 1: 输入: $s, f, \theta, N, t, \tau, n, \text{unPrN}$ s1: 输入: 离线数据集 D

2: 初始化 $a \in \Theta$, $\hat{a} \in \Psi$, $\tau \in \Gamma$, $\hat{\tau} \in \hat{\Gamma}$, $\pi \in \Pi$, $\hat{\pi} \in \hat{\Pi}$, $Q \in \mathcal{Q}$, $\hat{Q} \in \hat{\mathcal{Q}}$
3: for $n = 1$ to 2 do 3: 初始化目标网络参数: $\theta^1 \in \Theta$, $\phi^1 \in \Phi$, $\varphi^1 \in \Phi$
4: $\hat{\pi} \in \hat{\Pi}$, $\hat{a} \in \hat{\mathcal{A}}$, $\hat{\tau} \in \hat{\Gamma}$, $\hat{z} \in \mathcal{Z}$, $\hat{Q} \in \hat{\mathcal{Q}}$ 4: for 策略训练迭代 $k = 1, \dots, K$ do
5: $a \in \mathcal{A}$, $\tau \in \Gamma$, $z \in \mathcal{Z}$ 5: 采样小批量 $B = \{s, a, r, s'\}$ 5: 返回: a 6: % Q 值更新 7: 使用公式7.3更新 $Q \in \mathcal{Q}$, $\hat{Q} \in \hat{\mathcal{Q}}$; 算法
25 一致性行为克隆 8: % 策略更新

- 1: 输入: 离线数据集 D 9: 使用公式7.4更新策略 π_θ (借助模型 f_θ);
- 2: 初始化一致性策略 π_θ , 目标 $\theta^* \in \Theta$ 10: % 目标更新
- 3: for 迭代 $k = 1, \dots, K$ do 11: 更新目标: $\theta^* \leftarrow \alpha \theta + (1 - \alpha) \theta^*$, $\phi^* \leftarrow \alpha \phi + (1 - \alpha) \phi^*$, $i \leftarrow i + 1$, $2u$;
- 4: 使用公式7.2所示损失 $L_{c, p, q}$ 更新策略 π_θ (借助模型 f_θ); 12: 返回: $\pi_\theta, Q\phi, Q\psi$
- 5: 更新目标: $\theta^* \leftarrow \alpha \theta^* + (1 - \alpha) \theta$

一致性模型中去噪过程的时间跨度,而非样本轨迹的episode长度。

一致性行为克隆。借助离线数据集 D , 可以通过改编原始损失 [Song et al., 2023] 来训练作为策略的条件一致性模型:

" 3 i

$$Lc p\theta q \text{ " En"U p1,N '1q,ps,aq"D,z"N p0,lq } \lambda p\tau n \text{ qd f}\theta \text{ ps, atn`1 , } \tau n`1 \text{ q, f}\theta^T \text{ ps, atn , } \tau n \text{ q (7.2)}$$

其中 λp^q 是与步骤相关的权重函数, $\text{atan}^a \text{tan} z, dp^q$ 是距离度量。 f_{θ}^T 是 f_{θ} 的指数移动平均值, 用于在训练中稳定目标估计。在经典的演员-评论家算

一致性演员-评论家。作为对当前策略的状态-动作价值的估计,参数化的 $Q_{\phi}(s, a)$ 函数可以借助批数据 $B \tilde{D}$, 采用双Q学习损失[Fujimoto et al., 2018]进行学习:

$$L_{\phi} = \mathbb{E}_{(s, a) \sim \pi} [B(s, a) - Q_{\phi}(s, a)]^2 \quad (7.3)$$

其中 Q_{ϕ}^{π} 是 $Q_{\phi}(s, a)$ 的延迟更新版本,用于稳定训练。

在离线数据集上学习经过正则化的策略 π_{θ} 时,采用策略梯度(通过最大化期望的 $Q_{\phi}(s, a)$ 函数)与带一致性损失 L_{ϕ} 的行为克隆正则化相结合的方式:

$$L_{\theta} = L_{\phi} + \eta L_{\text{BC}} \quad (7.4)$$

“ %0

$$L_{\phi} = \mathbb{E}_{(s, a) \sim \pi_{\theta}} [B(s, a) - Q_{\phi}(s, a)]^2 \quad (7.5)$$

其中 $a \sim \pi_{\theta}$ 是按照算法24所示的一致性策略进行的动作推断。可以注意到,通过 N 个去噪步骤生成的动作会在上述公式中通过 $Q_{\phi}(s, a)$ 产生策略梯度损失缩放。如公式7.2所示的一致性损失匹配来自两个连续时间步 t_n 和 t_{n-1} 的去噪预测。由于采用了 N -step 调度方案(详见第7.6节),它们之间的差值 $|t_n - t_{n-1}|$

系数 η 是一个与 k 无关的常数超参数,因此无法缓解这一问题。尽管原始一致性模型在图像生成中采用 $\lambda \tau_n q^{-1}$,但我们通过实验发现,在离线强化学习中,

ξ

$$\lambda \tau_n, \tau_n^{-1}; k q^{-1} \tau_n^{-1} p k q^{-1} \tau_n p k q|,$$

其中 ξ 根据任务设定(或吸收进 η 中)。分母便捷地刻画了迭代 k 时的损失尺度。

7.5 实验评估

为评估所提出的一致性策略及相应算法的表达能力和计算效率,我们在D4RL基准测试的四个任务套件(Gym、AntMaze、Adroit、Kitchen)上,在三种典型的强化学习场景下进行了实验:离线(第7.5.1节、第7.5.2节)、离线到在线以及在线场景(第7.5.3节)。众所周知,D4RL离线数据集可能表现出多模态性,因为其样本可能是通过多种策略混合采集或沿着各种次优轨迹采集得到的,这使得策略表征的表达能力至关重要[Fu et al., 2020; Wang et al., 2022]。对于离线强化学习,分别采用行为克隆型算法(Consistency-BC、Diffusion-BC)和演员-评论家型算法(Consistency-AC、Diffusion-QL)对作为策略的生成模型进行评估,评估内容涵盖性能和计算时间两方面。Diffusion-QL尽管名称中带有QL,但也是一种演员-评论家算法。作为消融研究,比较了Consistency-AC的多个变体,并报告其中最佳性能。对于离线到在线及在线强化学习场景,比较了不同方法的学习曲线和最终结果。在评估时,按照以往工作[Wang et al., 2022]的设置,每个模型在Gym任务上评估10个episode,在其他任务上评估100个episode。默认情况下,一致性策略采用去噪步数 $N = 2$,配合一个

在大多数 D4RL 任务上性能已经饱和,而扩散策略使用 $N = 5$ [Wang et al., 2022]。不同 N 取值的影响将在第 7.5.2 节中讨论。

7.5.1 离线强化学习:行为克隆

实证发现 1:仅通过行为克隆(不含任何强化学习成分),使用具有多模态特性的表达性策略表示(如一致性模型或扩散模型)即可达到与许多现有主流离线强化学习方法相当的性能。学习一致性策略所需的计算量远少于学习扩散策略。

所提出的方法 Consistency-BC 遵循算法 25,使用一致性策略进行行为克隆,而 Diffusion-BC 则是将策略表示替换为扩散模型,并将策略损失替换为扩散模型训练损失 $L_{p\theta q}$

表 7.1:展示了原始 BC(使用高斯分布)、Consistency-BC、Diffusion-BC 以及若干离线强化学习基线方法在 D4RL Gym、AntMaze、Adroit 和 Kitchen 任务上的平均得分。对于 Consistency-BC 和 Diffusion-BC,每个单元格包含两个数值:一个是离线模型选择的结果,另一个(括号中)是在线模型选择的结果。每个结果均为五个随机种子的平均值,并报告了标准差。每行中的粗体值为该行的最高值。

Gym 任务 BC Consistency-BC Diffusion-BC AWAC Diffuser MoRel Onestep RL TD3+BC DT

halfcheetah-m	42.6	31.0	0.4 (46.2	0.4)	45.4	1.8 (46.3	0.2)	43.5	44.2	42.1	48.4	48.3	42.6
hopper-m	52.9	71.7	8.0 (78.3	2.6)	65.3	5.8 (71.1	5.5)	57.0	58.5	95.4	59.6	59.3	67.6
walker2d-m	75.3	83.1	0.3 (84.1	0.3)	81.2	1.7 (84.3	0.5)	72.4	79.7	77.8	81.8	83.7	74.0
halfcheetah-mr	36.6	34.4	5.3 (45.4	0.7)	41.7	0.4 (44.1	0.3)	40.5	42.2	40.2	38.1	44.6	36.6
hopper-mr	18.1	99.7	0.5 (100.4	0.6)	67.9	28.1 (99.1	2.3)	37.2	96.8	93.6	97.5	60.9	82.7
walker2d-mr	26.0	73.3	5.7 (80.8	2.4)	77.5	4.7 (80.8	4.5)	27.0	61.2	49.8	49.5	81.8	66.6
halfcheetah-me	55.2	32.7	1.2 (39.6	3.4)	90.8	1.1 (93.5	0.4)	42.8	79.8	53.3	93.4	90.7	86.8
hopper-me	52.5	90.6	9.3 (96.8	4.6)	107.6	4.3 (111.7	0.3)	55.8	107.2	108.7	103.3	98.0	107.6
walker2d-me	107.5	110.4	0.7 (111.6	0.7)	108.9	0.6 (110.5	0.5)	74.5	108.4	95.6	113.0	110.1	108.1

平均 51.9 69.7 (75.9) 76.3 (82.4) 50.1 75.3 72.9 76.1 75.3 74.7

AntMaze 任务 BC Consistency-BC Diffusion-BC AWAC BCQ BEAR Onestep RL TD3+BC DT

antmaze-u	54.6	75.8	4.0 (87.0	4.5)	71.8	8.2 (76.8	3.9)	56.7	78.9	73.0	64.3	78.6	59.2
antmaze-ud	45.6	77.6	6.3 (82.4	3.4)	61.2	9.4 (78.8	7.0)	49.3	55.0	61.0	60.7	71.4	53.0
antmaze-mp	0.0	56.8	30.1 (71.6	14.5)	43.4	37.8 (56.8	34.5)	0.0	0.0	0.0	0.3	10.6	0.0
antmaze-md	0.0	31.6	22.4 (66.0	6.5)	29.8	36.3 (69.4	12.3)	0.7	0.0	8.0	0.0	3.0	0.0
antmaze-lp	0.0	10.2	4.6 (15.0	3.8)	14.6	11.2 (22.4	5.8)	0.0	6.7	0.0	0.0	0.2	0.0
antmaze-lid	0.0	12.8	8.2 (19.8	4.0)	26.6	10.7 (33.0	8.2)	1.0	2.2	0.0	0.0	0.0	0.0

平均 16.7 44.1 (57.0) 41.2 (53.3) 18.0 23.8 23.7 20.9 27.3 18.7

Adroit 任务 BC Consistency-BC Diffusion-BC SAC BCQ BEAR BRAC-p BRAC-v REM

pen-human-v1	25.8	52.4	13.7 (63.7	7.4)	61.1	5.9 (66.7	4.9)	4.3	68.9	-1.0	8.1	0.6	5.4
pen-cloned-v1	38.3	33.4	6.0 (51.9	6.6)	57.6	9.5 (62.7	6.1)	-0.8	44.0	26.5	1.6	-2.5	-1.0

平均 32.1 42.9 (57.8) 59.4 (64.7) 1.8 56.5 12.8 4.9 -1.0 2.2

Kitchen 任务 BC Consistency-BC Diffusion-BC SAC BCQ BEAR BRAC-p BRAC-v AWR

kitchen-c	33.8	45.2	5.0 (50.9	3.6)	76.5	8.9 (87.3	6.8)	15.0	8.1	0.0	0.0	0.0	0.0
kitchen-p	33.8	22.6	3.8 (23.8	2.8)	50.3	3.0 (52.9	1.6)	0.0	18.9	13.1	0.0	0.0	15.4
kitchen-m	47.5	23.5	1.8 (24.3	1.3)	56.5	6.6 (64.7	4.6)	2.5	8.1	47.2	0.0	0.0	10.6

平均 38.4 30.4 (33.0) 61.1 5.8 11.7 20.1 0.0 0.0 8.7

如论文 [Wang et al., 2022] 所述。经典高斯策略 BC 及以往离线强化学习基线方法的结果,包括 AWAC [Nair et al., 2020]、Diffuser [Janner et al., 2022]、MoRel [Kidambi et al., 2020]、Onestep RL [Brandfonbrener et al., 2021]、TD3+BC [Fujimoto and Gu, 2021]、Decision Transformer(DT)[Chen et al., 2021]、BCQ [Fujimoto et al., 2019]、BEAR [Kumar et al., 2019]、BRAC [Wu et al., 2019] 以及 REM [Agarwal et al., 2020],均取自先前论文 [Wang et al., 2022]。SAC [Haarnoja et al., 2018] 是用于收集 D4RL Gym 任务数据的算法。

表 7.1 的结果表明,即使仅使用 BC 方法,采用多模态策略表示对离线强化学习也具有优势。作为参考,括号中的数值允许通过在线评估从训练模型集合中挑选出最佳模型,这一数值代表了在不利用在线评估的标准离线选择方式下所能达到的最大可能值。图 7.1:不同任务下 Consistency-BC 与 Diffusion-BC 的平均训练时间(每轮训练所耗秒数)。与使用高斯分布策略的原始 BC 相比,具有多模态特性的 Consistency-BC 在 14/20 个任务上表现更优,Diffusion-BC 则在全部 20/20 个任务上表现优于或等同于 BC。通过在 BC 中引入多模态表示,各任务上平均归一化得分的提升是显著的,这主要是由于离线数据集中由多种策略混合所带来的多模态性。此外,与并非仅使用 BC 的以往离线强化学习基线方法相比,Consistency-BC 和 Diffusion-BC 表现相当,甚至在某些任务上表现更优,例如 Gym 任务中的 walker2d-medium-v2、hopper-medium-replay-v2、walker2d-medium-replay-v2、walker2d-medium-expert-v2,以及大多数 AntMaze、Adroit 和 Kitchen 任务。

一致性策略的表达能力略逊于扩散策略,这在我们的预期之内,因为它大幅减少了采样步数。然而,与扩散策略相比,一致性策略表现出更高的计算效率,如

图 7.1 所示,在 20 个任务中平均计算时间减少了 42.97%。每个任务的详细计算时间见第 7.7.1 节表 7.5。

7.5.2 离线强化学习:一致性 Actor-Critic

表 7.2:Consistency-AC 与 SOTA 基线方法在 D4RL Gym、AntMaze、Adroit 和 Kitchen 任务上离线强化学习设置下的性能。对于 Consistency-AC 和 Diffusion-QL,每个单元格包含两个数值:一个为离线模型选择的结果,另一个(括号中)为在线模型选择的结果。每行中的粗体值为该行的最高值。

任务 CQL IQL X-QL ARQ IDQL-A Diffusion-QL Consistency-AC

halfcheetah-m 44.0 47.4 48.3 45~ 0.3 51.0 51.1~ 0.5 (51.5~ 0.3) 69.1~ 0.7 (71.9~ 0.8)
 hopper-m 58.5 66.3 74.2 61~ 0.4 65.4 90.5~ 4.6 (96.6~ 3.4) 80.7~ 10.5 (99.7~ 2.3)
 walker2d-m 72.5 78.3 84.2 81~ 0.7 82.5 87.0~ 0.9 (87.3~ 0.5) 83.1~ 0.3 (84.1~ 0.3)
 halfcheetah-mr 45.5 44.2 45.2 42~ 0.3 45.9 47.8~ 0.3 (48.3~ 0.2) 58.7~ 3.9 (62.7~ 0.6)
 hopper-mr 95.0 94.7 100.7 81~ 24.2 92.1 101.3~ 0.6 (102.0~ 0.4) 99.7~ 0.5 (100.4~ 0.6)
 walker2d-mr 77.2 73.9 82.2 66~ 7.0 85.1 95.5~ 1.5 (98.0~ 0.5) 79.5~ 3.6 (83.0~ 1.5)
 halfcheetah-me 91.6 86.7 94.2 91~ 0.7 95.9 96.8~ 0.3 (97.2~ 0.4) 84.3~ 4.1 (89.2~ 3.3)
 hopper-me 105.4 91.5 111.2 110~ 0.9 108.6 111.1~ 1.3 (112.3~ 0.8) 100.4~ 3.5 (106.0~ 1.3)
 walker2d-me 108.8 109.6 112.7 109~ 0.5 112.7 110.1~ 0.3 (111.2~ 0.9) 110.4~ 0.7 (111.6~ 0.7)

平均 77.6 77.0 83.7 76.2 82.1 87.9 (89.3) 85.1 (89.8)

antmaze-u 74.0 87.5 93.8 97~ 0.8 94.0 93.4~ 3.4 (96.0~ 3.3) 75.8~ 1.6 (85.6~ 3.9)
 antmaze-ud 84.0 62.2 82.0 62~ 12.1 80.2 66.2~ 8.6 (84.0~ 10.1) 77.6~ 6.3 (82.4~ 3.4)
 antmaze-mp 61.2 71.2 76.0 80~ 8.3 84.5 76.6~ 10.8 (79.8~ 8.7) 56.8~ 30.1 (71.6~ 14.5)

平均 73.1 73.6 83.9 79.7 86.2 78.7 (86.6) 70.1 (79.9)

pen-human-v1 35.2 71.5 - 45~ 5.2 (v0) - 72.8~ 9.6 (75.7~ 9.0) 63.4~ 7.7 (67.9~ 5.3)
 pen-cloned-v1 27.2 37.3 - 50~ 7.1 (v0) - 57.3~ 11.9 (60.8~ 11.8) 50.1~ 2.2 (53.7~ 3.4)

平均 31.2 54.4 - 47.5 - 65.1 (68.3) 56.8 (60.8)

kitchen-c 43.8 62.5 82.4 37~ 14.2 - 84.0~ 7.4 (84.5~ 6.1) 51.9~ 6.0 (67.6~ 2.7)
 kitchen-p 49.8 46.3 73.7 50~ 5.0 - 60.5~ 6.9 (63.7~ 5.2) 38.2~ 1.8 (39.8~ 1.6)
 kitchen-m 51.0 51.0 62.5 39~ 9.4 - 62.6~ 5.1 (66.6~ 3.3) 45.8~ 1.5 (46.7~ 0.9)

平均 48.2 53.3 72.9 42.0 - 69.0 (71.6) 45.3 (51.4) 总平均 66.2 69.6 - 67.4 - 80.3 (83.2) 72.1 (77.9)

实证发现 2:在 TD3-BC 类型的算法中,将扩散模型替换为一致性模型用于离线强化学习,会带来模型训练和推理速度的提升,性能略有下降,但仍优于其他一些基线方法。

对于离线强化学习,所提出的方法 Consistency-AC 遵循算法 26,使用一致性模型进行策略表示,并将一致性策略嵌入到带有 BC 策略正则化的 actor-critic 算法中,以避免生成分布外的动作。以往基线方法的结果,包括 CQL [Kumar et al., 2020b]、IQL [Kostrikov

et al., 2021]、X-QL [Garg et al., 2023]、ARQ [Goo and Niekum, 2022]、IDQL-A [Hansen-Estruch et al., 2023] 以及 Diffusion-QL [Wang et al., 2022],均取自相应论文所报告的结果。

表 7.2 的结果展示了不同方法在五个随机种子下的平均归一化得分,并为 Diffusion-QL 和 Consistency-AC 报告了标准差。表 7.2 中的结果可与表 7.1 中的结果直接比较,因为它们遵循相同的离线强化学习设置。

表 7.2 显示,尽管 Consistency-AC 的平均得分(72.1)略低于 Diffusion-QL(80.3),但在大多数任务上(如 Gym 和 Adroit)其表现优于其他基线方法。AntMaze 任务对 Consistency-AC 方法而言较为困难,我们推测这可能是由于数据集中稀疏的奖励信号所致,这使得 Q 学习的难度成为比通过行为克隆建模多模态分布更为突出的瓶颈。Q 值估计的保守性可能很重要,但与所提出的一致性策略是正交的。考虑到 Consistency-AC 在训练和推理阶段去噪步数的减少,可将其视为计算效率与多模态分布近似精度之间的一种权衡,这将在下文进行讨论。

方法 N 训练时间(秒/轮) 推理时间(毫秒/样本) 平均归一化得分

50 206.44[±]16.70 30.65[±]2.10 -
 20 108.65[±]2.85 13.04[±]0.90 109.2[±]1.1 (111.1[±]1.9)
 10 76.54[±]10.74 6.87[±]0.55 108.6[±]0.6 (112.5[±]0.2)

Diffusion-QL

5 57.06[±]19.16 3.76[±]0.29 108.2[±]5.6 (112.3[±]0.2)
 2 31.59[±]10.32 1.96[±]0.10 53.6[±]16.6 (103.5[±]10.0)
 1 30.23[±]8.75 1.37[±]0.09 2.8[±]1.5 (13.1[±]12.5)
 50 150.84[±]31.02 26.50[±]1.92 -
 20 76.22[±]9.92 11.12[±]0.77 101.3[±]6.3 (107.3[±]0.2)
 10 54.04[±]4.43 5.95[±]0.44 98.4[±]4.3 (107.1[±]4.0)

Consistency-AC

5 40.79[±]2.79 3.39[±]0.29 101.4[±]4.7 (110.1[±]1.6)
 2 31.94[±]1.55 1.84[±]0.21 102.4[±]3.0 (106.2[±]1.6)
 1 28.51[±]1.78 1.23[±]0.11 6.2[±]5.4 (19.1[±]9.3)

表 7.3:两种方法在任务 hopper-medium-expert-v2 上,随不同去噪步数 N 变化的计算时间对比。灰色行为两种模型所采用的默认 N 值。

计算时间。为评估 Consistency-AC 与 Diffusion-QL 在不同去噪步数 N 下的计算效率,我们在 hopper-medium-expert-

时间差更大

在更小的 $N(=2)$ 时饱和

图 7.2: 两种模型在 hopper-medium-expert 任务上的平均归一化得分与训练时间随 N 变化的曲线。

v2 环境下针对 $N \in \{1, 2, 5, 10, 20, 50\}$ 进行了训练和推理时间的实验评估。作为基于概率流的生成模型,一致性模型和扩散模型的计算时间都直接取决于去噪步数 N ,而一致性模型 [Song et al., 2023] 从设计上就要求以更少的步数达到与扩散模型相近的生成性能。结果汇总于表 7.3,包括不同去噪步数 N 下模型训练的训练时间(每轮所耗秒数)和推理时间(每样本所耗毫秒数),以及每个 N 下训练 2000 轮后模型的平均归一化得分。每个单元格包含五个随机种子下的均值和标准差。Consistency-AC 仅在 $N = 2$ 时性能即饱和,而 Diffusion-QL 在 $N = 5$ 时饱和,后者所耗训练时间约为前者的 1.786 倍,而性能仅略有提升(1.057 倍)。表中 $N = 50$ 处的“-”表示由于超出限定的作业时间(72 小时)而缺失的平均得分。此外,如图 7.2 所示,随着 N 的增加,Consistency-AC 在训练和推理的时间消耗方面均表现出比 Diffusion-QL 更好的扩展规律,这一点通过第 7.7.1 节图 7.5 中的线性拟合结果进一步得到了验证。

消融研究。[Hansen-Estruch et al., 2023] 提出在扩散策略的网络参数化中使用带层归一化的残差网络,即 LN-

Resnet,我们的实验中也对一致性策略进行了测试。作为消融研究,我们比较了 Consistency-AC 在离线强化学习设置下的不同变体,包括:(1)通过设置 $\eta = 0$ 我们发现,对于一致性模型,LN-Resnet 在各任务上并不能持续改善 MLP 的表现,其主要益处体现在 Adroit 任务上。在不使用损失缩放的情况下,大多数任务的性能显著下降(平均下降 37.8%),尽管在某些特定任务(如 AntMaze)上不使用损失缩放反而可能略微提升性能。对于除 AntMaze 之外的大多数任务,Consistency-BC 无法达到最佳性能,而带有适当缩放的 Q 学习损失 $L_{\text{QL}}(\theta, q)$ (如式 7.5)有助于进一步提高得分。

图 7.3:Consistency-AC 各变体在离线强化学习设置下跨任务的对比。

7.5.3 离线到在线及在线强化学习

实证发现 3:在离线到在线强化学习中,一致性策略的性能与扩散策略接近但略逊一筹,但计算效率有显著提升。

对于在线强化学习,我们同时考虑了从零开始的在线学习以及离线到在线的设置,即以在离线数据集上训练的模型作为初始化,进行在线微调。

如前文第 7.5.2 节所述,离线模型的选择可以通过在线或离线方式进行,分别对应没有或没有在线经验的模型评估。这两类模型都被用于在线微调开始时初始化

表7.4:离线到在线和在线RL中各方法的归一化分数(最后一个epoch)对比。 离线到在线 在线 Gym任务 SAC AWAC ACA Diffusion-QL Consistency-AC Diffusion-QL Consistency-AC

halfcheetah-m	75.2	50.5	66.6	99.6~2.3 (99.8~1.6)	98.7~1.8 (97.3~2.9)	47.3~2.9	55.1~7.0
hopper-m	73.4	97.5	96.5	77.2~25.6 (60.0~11.8)	60.5~8.6 (61.8~26.6)	82.8~30.6	86.3~28.4
walker2d-m	79.6	1.9	74.7	118.3~5.8 (117.5~5.9)	108.9~3.0 (107.9~10.5)	77.0~25.7	69.4~38.9
halfcheetah-mr	68.9	46.8	59.0	96.3~3.9 (97.6~1.2)	80.7~10.5 (82.3~9.4)	43.5~5.7	56.5~8.0
hopper-mr	74.0	96.0	85.5	68.4~20.3 (90.6~24.0)	74.6~25.1 (63.4~16.7)	94.0~12.2	75.8~26.8
walker2d-mr	85.4	80.8	85.2	95.7~18.8 (105.5~13.7)	102.0~11.6 (96.5~17.9)	87.8~29.0	69.0~42.3
halfcheetah-me	82.2	68.8	93.7	103.9~2.2 (102.9~1.8)	99.6~4.1 (95.1~9.7)	39.7~3.6	56.7~5.8
hopper-me	65.4	73.1	98.0	71.7~31.1 (67.9~18.6)	65.4~5.7 (54.7~28.4)	62.5~22.2	78.6~14.6
walker2d-me	87.2	45.2	110.5	117.0~6.3 (111.2~10.6)	101.8~13.3 (89.2~16.2)	74.6~39.0	86.2~27.8

平均 76.8 62.3 85.5 94.2 88.0 67.7 70.4

表7.4总结了Consistency-AC和Diffusion-QL在9个Gym任务上、两种设定下、五个随机种子取得的平均分数的定量结果,同时也给出了离线到在线基线方法SAC、AWAC和ACA [Yu and Zhang, 2023]的结果。Consistency-AC和Diffusion-QL均在离线数据集上预训练2000个epoch。每个模型在线微调或从零学习均训练一百万步。SAC、AWAC和ACA的结果取自论文[Yu and Zhang, 2023],其中每个模型在线微调100k步。每个单元格有两个值:一个是使用离线模型选择作为初始化,另一个(括号中)是使用在线模型选择作为初始化。在离线到在线设定下,Consistency-AC的归一化分数略低于Diffusion-QL,但在从零开始的在线RL设定下则更高。平均而言,两种方法在在线设定下取得的数值均低于离线到在线设定,这证明了使用预训练生成式策略模型初始化能够提升学习效率。然而,由于训练设置为固定的总时间步数并使用相同的学习率

1×10^{-5} , 纯在线模型尚未收敛到其最优性能。该学习率是针对微调设定选取的,其目的并非展示在线RL经过充分训练后能取得高于100的分数,而是为了与实证发现4:在在线RL设定下,一致性策略在计算效率上大多优于扩散策略,在某些情况下(尤其是难度较高的任务上)也能在样本效率上更优。

图7.4:Diffusion-QL和Consistency-AC在在线RL以及使用离线模型选择的离线到在线RL设定下(均训练一百万个环境步)在时间轴上的学习曲线。每条曲线均经过平滑处理并在五个随机种子上取平均,阴影区域表示95%置信区间。

图7.4展示了Consistency-AC和Diffusion-QL在离线到在线和在线RL两种设定下、在三个示例任务上、经过一百万在线训练步的学习曲线。不同方法完成整个训练所耗费的时间不同。图中x轴为墙钟时间,因此各曲线呈现出不同的长度。在大多数任务上,一致性策略的性能与扩散策略相当,但完成整个在线训练所需的时间明显更短。除了三个相对容易学到良好策略的hopper任务外,离线到在线方法通常比在线方法更具样本效率。在在线设定下,一致性策略表现出明显比扩散策略更高效的学习,尤其是在像halfcheetah这样更复杂的任务上。在9个任务中的8个上,一致性策略呈现出比扩散策略更陡峭的分数上升斜率。我们推测,模型的表达能力在离线设定下比在在线设定下更为关键。对于MDP中的确定性最优策略,过于表

达能力强的策略模型(如扩散模型)可能因过度探索而在在线设定下阻碍收敛。对于离线到在线设定,这一优势则不那么明显,这可能是由于一致性策略从离线预训练中获得的初始性能较低所致。关于两种方法训练时间的进一步分析,请参见第7.8.2节、表7.7和图7.6。

7.6 一致性模型的训练与推理细节

训练。用于建模数据分布 p_{data} 的一致性模型 f_θ 的损失函数为[Song et al., 2023]:

$$\begin{aligned} \mathcal{L}_{\text{data}} = \mathbb{E}_{\mathbf{p}, \mathbf{z} \sim p_{data}(\mathbf{p}, \mathbf{z})} \left[\frac{1}{2} \|\mathbf{f}_\theta(\mathbf{p}, \mathbf{z}) - \mathbf{z}\|_2^2 \right] \quad (7.6) \end{aligned}$$

其中 $d(\mathbf{p}, \mathbf{q})$ 为距离度量,我们采用l2距离 $d(\mathbf{p}, \mathbf{q}) = \|\mathbf{p} - \mathbf{q}\|_2^2$ 。训练时,子序列 $\mathbf{t}_{rn} \in \mathbb{R}^N$ 与推理时不同,遵循Karras边界[Karras et al., 2022]的调度方式: $\mathbf{t}_{rn} = \mathbf{p} \odot \mathbf{t}_{rn}$

$\mathbf{p} \in \mathbb{R}^N$ 为调度函数 Q_b 的输入

$k \in \mathbb{N}$ 为当前训练迭代次数

推理。训练完成后,给定初始噪声输入 $\mathbf{x}_T \sim N(\mathbf{0}, \mathbf{I})$,一致性模型 f_θ 可用于生成样本,可采用单步采样 $\mathbf{x}_0 = f_\theta(\mathbf{x}_T, \mathbf{t}_0)$ 或按照给定的时间序列 $\mathbf{t}_{rn} \in \mathbb{R}^N$ 采样

$$\mathbf{x}_0 = f_\theta(\mathbf{x}_T, \mathbf{t}_{rn})$$

我们的实验遵循Song et al. [2023],使用常数 $\epsilon = 0.002, T = 80; \rho = 7; s_0 = 2, s_1 = 150$ 。

7.7 离线RL实验细节

7.7.1 计算时间

总体训练时间。表7.5展示了Diffusion-BC和Consistency-BC在D4RL Gym、AntMaze、Adroit和Kitchen任务上训练过程中计算时间的对比。每个结果均在五个随机种子上取平均并报告标准差。由于不同环境训练的总epoch数不同,比较基于每个epoch的时间消耗。两种方法在每个epoch内使用相同的批量大小和迭代次数,以及相同的网络架构。

表7.5:两种BC方法在D4RL Gym、AntMaze、Adroit和Kitchen任务上的训练时间(每epoch的秒数)。

任务 Diffusion-BC Consistency-BC

halfcheetah-m	67.93~2.00	43.07~0.61
hopper-m	61.00~1.58	38.00~0.49
walker2d-m	68.17~1.51	43.58~0.90
halfcheetah-mr	67.07~2.07	42.75~0.83
hopper-mr	64.89~3.29	38.03~0.47
walker2d-mr	66.11~1.69	42.70~0.54
halfcheetah-me	67.73~1.78	43.60~0.86
hopper-me	63.04~4.25	38.56~0.56
walker2d-me	69.10~2.83	43.88~0.72
平均	66.12~2.33	41.57~0.66
antmaze-u	97.13~4.21	47.88~2.59
antmaze-ud	104.83~4.50	47.20~2.23
antmaze-mp	109.66~3.82	57.92~4.46
antmaze-md	112.25~2.41	51.80~2.59
antmaze-lp	113.15~1.01	56.52~3.17
antmaze-lid	118.62~3.42	53.89~2.74
平均	109.27~3.44	52.54~3.05
pen-human-v1	92.20~3.17	46.94~2.92
pen-cloned-v1	94.64~6.16	50.33~2.23
平均	93.42~4.67	48.64~2.58
kitchen-c	96.77~2.74	66.71~4.64
kitchen-p	94.25~2.27	61.85~2.74
kitchen-m	93.02~3.49	66.60~2.59
平均	94.68~2.83	65.05~3.32
总平均	86.08~3.16	49.09~2.34

扩展规律。图7.5基于表7.3中在hopper-medium-expert-v2环境上的结果,进一步展示了在离线RL设定下,Diffusion-QL和Consistency-AC的训练时间与推理时间随N增加的扩展规律。注意,Consistency-AC的系数在训练(2.47对比3.54)和推理(0.515对比0.598)方面均小于Diffusion-QL,这表明随着N增加,其时间消耗增长更小。

图7.5:在hopper-medium-expert-v2环境上评估的Diffusion-QL和Consistency-AC在离线RL设定下,训练时间(左)和推理时间(右)随去噪步数N的变化。

7.7.2 消融实验

针对离线RL设定,比较了Consistency-AC的四种变体,包括(1)Consistency-BC,即设置 $\eta = 0$ 且不使用损失缩放(Eq. 7.2中 $\lambda \rho \pi \eta q = 1$);(2)仅去除损失缩放;(3)采

表7.6:Consistency-AC各变体在D4RL Gym、AntMaze、Adroit和Kitchen任务上离线RL设定下的性能。每个单元格有两个值:一个是离线模型选择,另一个(括号中)是在线模型选择。每个结果均在五个随机种子上取平均并报告标准差。Gym任务 Consistency-BC ($\eta = 0$) Consistency-AC (无损失缩放) Consistency-AC (MLP) Consistency-AC (LN-Resnet)

halfcheetah-m	31.0 $\sim$ 0.4 (46.2 $\sim$ 0.4)	69.1 $\sim$ 0.7 (71.9 $\sim$ 0.8)	50.1 $\sim$ 0.4 (50.4 $\sim$ 0.2)	50.6 $\sim$ 0.3 (50.9 $\sim$ 0.2)
hopper-m	71.7 $\sim$ 8.0 (78.3 $\sim$ 2.6)	80.7 $\sim$ 10.5 (99.7 $\sim$ 2.3)	78.0 $\sim$ 3.9 (86.4 $\sim$ 4.0q)	74.1 $\sim$ 6.7 (83.7 $\sim$ 8.5)
walker2d-m	83.1 $\sim$ 0.3 (84.1 $\sim$ 0.3)	5.5 $\sim$ 1.7 (21.2 $\sim$ 1.6)	63.0 $\sim$ 5.2 (75.0 $\sim$ 1.8q)	66.2 $\sim$ 5.2 (75.4 $\sim$ 1.9)
halfcheetah-mr	34.4 $\sim$ 5.3 (45.4 $\sim$ 0.7)	58.7 $\sim$ 3.9 (62.7 $\sim$ 0.6)	47.3 $\sim$ 0.2 (47.8 $\sim$ 0.3q)	47.8 $\sim$ 0.3 (48.4 $\sim$ 0.1)
hopper-mr	99.7 $\sim$ 0.5 (100.4 $\sim$ 0.6)	80.2 $\sim$ 9.0 (103.4 $\sim$ 1.2)	94.5 $\sim$ 6.4 (100.9 $\sim$ 0.2q)	98.7 $\sim$ 2.9 (100.6 $\sim$ 0.3)
walker2d-mr	73.3 $\sim$ 5.7 (80.8 $\sim$ 2.4)	72.3 $\sim$ 15.4 (105.1 $\sim$ 1.6)	76.8 $\sim$ 5.5 (86.1 $\sim$ 1.2q)	79.5 $\sim$ 3.6 (83.0 $\sim$ 1.5)
halfcheetah-me	32.7 $\sim$ 1.2 (39.6 $\sim$ 3.4)	22.6 $\sim$ 10.4 (55.2 $\sim$ 11.6)	84.3 $\sim$ 4.1 (89.2 $\sim$ 3.3q)	61.7 $\sim$ 13.6 (68.4 $\sim$ 6.7)
hopper-me	90.6 $\sim$ 9.3 (96.8 $\sim$ 4.6)	10.1 $\sim$ 16.2 (10.8 $\sim$ 15.6)	100.4 $\sim$ 3.5 (106.0 $\sim$ 1.3q)	43.1 $\sim$ 5.7 (54.5 $\sim$ 11.2)
walker2d-me	110.4 $\sim$ 0.7 (111.6 $\sim$ 0.7)	2.7 $\sim$ 4.3 (14.9 $\sim$ 6.9)	91.1 $\sim$ 3.4 (97.7 $\sim$ 3.2q)	84.1 $\sim$ 5.1 (97.5 $\sim$ 1.6)
平均	69.7 (75.9)	44.7 (60.5)	76.7 (82.2)	67.3 (73.6)
AntMaze任务	Consistency-BC ($\eta = 0$)	Consistency-AC (无损失缩放)	Consistency-AC (MLP)	Consistency-AC (LN-Resnet)
antmaze-u	75.8 $\sim$ 4.0 (87.0 $\sim$ 4.5)	75.4 $\sim$ 5.8 (82.6 $\sim$ 3.8)	68.8 $\sim$ 2.3 (82.2 $\sim$ 4.7)	75.8 $\sim$ 1.6 (85.6 $\sim$ 3.9)
antmaze-ud	77.6 $\sim$ 6.3 (82.4 $\sim$ 3.4)	75.2 $\sim$ 6.6 (80.2 $\sim$ 2.8)	68.6 $\sim$ 4.4 (78.4 $\sim$ 1.1)	72.4 $\sim$ 3.5 (81.2 $\sim$ 1.9)
antmaze-mp	56.8 $\sim$ 30.1 (71.6 $\sim$ 14.5)	45.2 $\sim$ 26.9 (73.2 $\sim$ 8.4)	52.2 $\sim$ 29.8 (70.4 $\sim$ 7.1)	10.0 $\sim$ 22.4 (59.4 $\sim$ 12.8)
平均	70.1 (80.3)	65.3 (78.7)	63.2 (77.0)	52.7 (75.4)
Adroit任务	Consistency-BC ($\eta = 0$)	Consistency-AC (无损失缩放)	Consistency-AC (MLP)	Consistency-AC (LN-Resnet)
pen-human-v1	52.4 $\sim$ 13.7 (63.7 $\sim$ 7.4)	8.4 $\sim$ 24.0 (22.1 $\sim$ 20.5)	60.6 $\sim$ 10.2 (66.6 $\sim$ 7.5)	63.4 $\sim$ 7.7 (67.9 $\sim$ 5.3)
pen-cloned-v1	33.4 $\sim$ 6.0 (51.9 $\sim$ 6.6)	48.2 $\sim$ 10.8 (58.2 $\sim$ 12.6)	35.8 $\sim$ 3.9 (40.5 $\sim$ 2.6)	50.1 $\sim$ 2.2 (53.7 $\sim$ 3.4)
平均	42.9 (57.8)	28.3 (40.2)	48.2 (53.6)	56.8 (60.8)
Kitchen任务	Consistency-BC ($\eta = 0$)	Consistency-AC (无损失缩放)	Consistency-AC (MLP)	Consistency-AC (LN-Resnet)
kitchen-c	45.2 $\sim$ 5.0 (50.9 $\sim$ 3.6)	10.0 $\sim$ 20.1 (25.5 $\sim$ 24.6)	51.9 $\sim$ 6.0 (67.6 $\sim$ 2.7)	36.9 $\sim$ 3.2 (38.0 $\sim$ 2.5)
kitchen-p	22.6 $\sim$ 3.8 (23.8 $\sim$ 2.8)	7.7 $\sim$ 16.9 (17.0 $\sim$ 14.5)	38.2 $\sim$ 1.8 (39.8 $\sim$ 1.6)	25.8 $\sim$ 5.5 (28.6 $\sim$ 2.7)
kitchen-m	23.5 $\sim$ 1.8 (24.3 $\sim$ 1.3)	9.7 $\sim$ 21.3 (15.8 $\sim$ 20.2)	45.8 $\sim$ 1.5 (46.7 $\sim$ 0.9)	26.0 $\sim$ 3.0 (28.8 $\sim$ 2.1)
平均	30.4 (33.0)	9.1 (19.4)	45.3 (51.4)	29.6 (31.8)
总平均	59.7 (67.0)	40.1 (54.1)	64.5 (72.5)	56.8 (65.0)

7.8 离线到在线以及在线RL细节

7.8.1 算法

算法27 离线到在线一致性Actor-Critic

```
1: 输入:离线预训练策略 $\pi_\theta$ 和评论家网络 $Q_{\phi_1}, Q_{\phi_2}$   
 $\theta^T \leftarrow \theta, \phi_1 \leftarrow \phi_1, \phi_2 \leftarrow$   
 $\phi_2$   
2: 初始化在线数据集 $D \leftarrow \emptyset$ , 目标网络参数: $\theta^T \leftarrow \theta, \phi_1 \leftarrow \phi_1, \phi_2 \leftarrow$   
 $\phi_2$   
3: for episode  $j = 1, \dots, M$  do  
4: 重置环境并观察 $s_1$ 。 5: for  $t = 1, \dots, H$  do 6: % 采集样本  
7: 基于 $s_t$ ,使用一致性策略 $\pi_\theta$ 按照算法24推断动作 $a_t$ 。  
8: 执行动作 $a_t$ ,观察奖励 $r_t$ 、下一状态 $s_{t+1}$ 。 9: 将数据样本 $(s_t, a_t, r_t, s_{t+1})$ 存入 $D$ 。 10: 采样小批量 $B \leftarrow \{(s_t, a_t, r_t, s_{t+1})\}_{t \in \text{batch}}$   
 $D$ ; 11: %  $Q$ 值更新 12: 使用Eq. 7.3更新 $Q_{\phi_1}, Q_{\phi_2}$ ; 13: % 策略更新  
14: 通过Eq. 7.5的损失 $L_q(\theta)$ 更新策略 $\pi_\theta$ (采用模型 $f_\theta$ );  
15: % 目标更新  
16: 更新目标: $\theta^T \leftarrow \alpha \theta^T + (1 - \alpha) q_\theta, \phi_1^T \leftarrow \alpha \phi_1^T + (1 - \alpha) \phi_1, \phi_2^T \leftarrow \alpha \phi_2^T + (1 - \alpha) \phi_2$ 
```

算法28 在线一致性Actor-Critic

```

1: 初始化策略 $\pi_\theta$ 和评论家网络 $Q_{\phi_1}, Q_{\phi_2}$ 
 $\theta^T \leftarrow \theta, \phi_1^T \leftarrow \phi_1, \phi_2^T \leftarrow \phi_2$ 
2: 初始化在线数据集 $D \leftarrow \emptyset$ , 目标网络参数: $\theta^T \leftarrow \theta, \phi_1^T \leftarrow \phi_1, \phi_2^T \leftarrow \phi_2$ 
3: for episode  $j \leftarrow 1, \dots, M$  do
4: 重置环境并观察 $s_1$ 。 5: for  $t \leftarrow 1, \dots, H$  do 6: % 采集样本
7: 基于 $s_t$ ,使用一致性策略 $\pi_\theta$ 按照算法24推断动作 $a_t$ 。
8: 执行动作 $a_t$ ,观察奖励 $r_t$ 、下一状态 $s_{t+1}$ 。 9: 将数据样本 $p(s_t, a_t, r_t, s_{t+1})$ 存入 $\tilde{D}$ 。 10: 采样小批量 $B \leftarrow \{p(s, a, r, s') \mid (s, a, r, s') \in \tilde{D}\}$ ;
11: % Q值更新 12: 使用Eq. 7.3更新 $Q_{\phi_1}, Q_{\phi_2}$ ; 13: % 策略更新
14: 通过Eq. 7.5的损失 $L_{\phi_1, \phi_2}$ 更新策略 $\pi_\theta$ (采用模型 $f_\theta$ );
15: % 目标更新
16: 更新目标: $\theta^T \leftarrow \alpha \theta^T + (1 - \alpha) \theta, \phi_1^T \leftarrow \alpha \phi_1^T + (1 - \alpha) \phi_1, \phi_2^T \leftarrow \alpha \phi_2^T + (1 - \alpha) \phi_2$ ;

```

7.8.2 计算时间

表7.7展示了在离线到在线和在线RL设定下,使用Diffusion-QL和Consistency-AC完成一百万环境步的总体训练时间,图7.6汇总了各设定下的平均训练时间。在线设定下计算时间的减少幅度不如离线设定(如图7.1)那样显著,因为智能体动作推断之后的环境仿真步骤消耗了很大一部分时间。模型推理和更新的改进不会影响环境仿真时间。

表7.7:Gym任务上离线到在线和在线设定下的总体训练时间(小时)。

离线到在线 在线

Gym任务 Diffusion-QL Consistency-AC Diffusion-QL Consistency-AC

halfcheetah-m	11.55~	4.08	9.60~	2.33	9.09~	0.88	8.77~	0.96
hopper-m	8.97~	1.48	6.90~	0.79	8.06~	0.88	6.99~	0.95
walker2d-m	9.17~	0.29	8.19~	1.91	8.23~	1.04	6.98~	0.89
halfcheetah-mr	9.18~	0.22	7.72~	0.89	8.72~	0.88	7.76~	1.02
hopper-mr	8.22~	0.20	7.26~	1.69	8.12~	0.81	6.92~	0.78
walker2d-mr	8.85~	0.22	7.32~	0.88	8.07~	1.01	7.05~	1.05
halfcheetah-me	9.24~	0.21	8.46~	1.96	8.54~	0.74	7.65~	1.02
hopper-me	8.28~	0.20	7.47~	0.57	8.02~	1.07	7.01~	0.99
walker2d-me	9.93~	1.27	9.41~	2.26	8.35~	0.73	8.29~	0.86
平均	9.27~	0.91	8.04~	1.48	8.36~	0.89	7.49~	0.95

图7.6:Diffusion-QL和Consistency-AC在9个Gym任务上离线到在线和在线训练的平均训练时间(小时)。

第8章

少步视频生成中的强化学习

本节基于论文"DOLLAR: Few-Step Video Generation via Distillation and Latent Reward Optimization"[Ding and Jin, 2024],该论文与 Chi Jin、Difan Liu、Haitian Zheng、Krishna Kumar Singh、Qiang Zhang、Yan Kang、Zhe Lin和Yuchen Liu合作完成,此前发表于ICCV 2025。

8.1 引言

扩散概率模型[Sohl-Dickstein et al., 2015; Song and Ermon, 2019; Ho et al., 2020; Song et al., 2021b]近年来革新了连续域中的生成建模。凭借在多种数据格式和模态上出色的表达能力和灵活性,扩散模型在文本生成图像和文本生成视频(T2V)等任务上取得了重大突破。然而,尽管生成质量有了大幅提升,由于其固有的大量迭代采样步骤,扩散模型的效率在实际应用中仍是一个制约因素。这一效率挑战在视频生成建模中更为严峻,因为更高维度的空间需要更大的模型规模、更丰富的训练数据、更大的输入输出张量,以及更多的采样迭代次数。此外,实际应用

往往需要与训练分布不同的生成质量----例如更高的美学标准或多样化的风格选择----这就需要进行高效的后续训练调整或微调,以在满足特定需求的同时管理预训练的巨大成本。

图 8.1:通过结合变分分数蒸馏、一致性蒸馏和潜在奖励微调,我们的 DOLLAR 方法在 4 步推理下,相比 DMD [Yin et al., 2024b] 提升了样本多样性和保真度,生成的视频质量高于 LCM [Luo et al., 2023; Wang et al., 2023c],并相比教师模型实现了约 15.6 倍的加速。

为解决扩散模型中的效率挑战,模型蒸馏 [Polino et al., 2018; Luhman and Luhman, 2021; Salimans and Ho, 2022] 已在各种模型和领域中被广泛研究。具体而言,分数蒸馏旨在通过对齐教师模型和学生扩散模型之间的分布,来提升 3D 生成 [Wang et al., 2023a; Poole et al., 2022; Wang et al., 2024c] 和图像合成 [Yin et al., 2024b; Luo et al., 2024; Xie et al., 2024b; Salimans et al., 2024] 的效率。然而,尽管该方法能实现较高的样本保真度,却常常遭遇模型崩溃问题 [Yin et al., 2024b; Lu and Song, 2024]。另一种方法----一致性蒸馏(CD)[Song et al., 2023]----旨在确保沿扩散轨迹的样本预测保持一致。尽管 CD 能提升样本多样性,但也存在局限:它往往会降低样本保真度,并在大规模文本到视频(T2V)应用中产生过度平滑的输出。

蒸馏方法的另一个挑战是,学生模型的性能通常受限于教师模型的性能上限。此前解决这一局限的努力包括将变分分数蒸馏(VSD)[Wang et al., 2024c; Yin et al., 2024a] 或一致性蒸馏 [Kim et al., 2023] 与 GAN [Goodfellow et al., 2014] 损失相结合,这在一定程度上提升了训练分布内的样本保真度,但受限于对抗训练判别信号的稀疏性,提升幅度有限。因此,生成样本在细致的视觉质量细节和文本到图像对齐方面仍可能存在不足,而这两方面都需要更密集的反馈信号。近期开发的图像或视频奖励模型为弥补这一差距提供了有前景的潜力,能够为生成质量的细粒度改进提供更丰富的信号。

在本工作中,我们通过引入更多的教师去噪步数,并将一致性蒸馏(CD)与变分分数蒸馏(VSD)相结合,来解决一致性蒸馏(CD)的局限性,从而在蒸馏后用少步模型生成高质量、多样化的样本。然而,仅凭这一点尚不足以超越教师模型,或可靠地满足下游应用的特定偏好,因为生成样本在视觉质量和文本到视频对齐方面仍可能面临挑战。虽然使用高质量数据集进行模型后训练是一种潜在的解决方案,但其实施成本往往较高。为克服这些局限,我们进一步引入了一种高效的奖励模型微调方法,该方法能使学生模型的性能超越教师模型,并通过定制的奖励指标使其与任意预定义要求保持一致。性能提升情况如图 9.1 所示。

我们提出在潜在空间中学习一个双重奖励模型,该模型由像素空间的奖励模型引导,并利用这个潜在奖励模型(LRM)的梯度直接微调扩散模型。这种方法结合了像素空间中奖励梯度方法与随机策略梯度方法的优势,具备以下几个优点:(1)它利用了潜在奖励模型丰富的梯度信息,实现了高效有效的调优;(2)它不要求原始奖励模型可微,从而拓宽了对多种奖励模型的适用性;(3)通过消除对大型像素空间奖励模型和解码器进行反向传播的需要,它显著降低了微调过程中的计算和内存开销。

LRM 方法具有通用性,可适应多种奖励类型---包括图像、视频、文本-图像和文本-视频奖励---从而增强了其实用性。

总而言之,我们的贡献有三方面:(1)我们提出了一种结合 VSD 和 CD 损失的扩散模型蒸馏方法,使得高效的少步 T2V 模型成为可能;(2)我们通过引入多个教师去噪步数的通用化方法来增强 CD 的有效性;(3)我们提出使用紧凑的潜在空间奖励模型进行基于奖励的微调,该方法不要求原始奖励指标可微,且在内存和计算方面更高效。所有评估均在大规模 T2V 设置下进行。将这些创新结合起来,我们提出了 DOLLAR 方法(基于蒸馏与潜在奖励优化),显著提升了视频生成的质量和效率,并为实时应用铺平了道路。

8.2 相关工作

视频生成。近期的进展已将扩散模型从图像合成扩展到视频生成,以应对时空数据的复杂性。Video Diffusion Models [Ho et al., 2022] 等开创性工作使扩散过程能够处理时间动态,从而实现连贯且高保真的视频片段生成。为提升计算效率,潜在扩散模型(LDM)[Rombach et al., 2022b] 在压缩的潜在空间中进行扩散建模,这一策略后来在视频领域被进一步改进,包括 [Blattmann et al., 2023b]、[Harvey et al., 2022]、Stable Video Diffusion [Blattmann et al., 2023a] 和 VideoCrafter2 [Chen et al., 2024b]。文本到视频生成方面的进展体现在 Imagen Video [Ho et al., 2022]、Make-A-Video [Singer et al., 2022b]、Phenaki [Villegas et al., 2022]、CogVideo [Hong et al., 2022]、CogVideoX [Yang et al., 2024]、Text2Video-Zero [Khachatryan et al., 2023] 以及 ModelScopeT2V [Wang et al., 2023b] 等模型中,这些模型根据文本描述生成视频,即所谓的文本到视频(T2V)模型。混合方法,例如 Dual

Diffusion Models [Xiao et al., 2023], 将扩散模型与其他生成框架相结合, 以提升时间连贯性和分辨率。此外还有一些近期的先进方法, 如 Lumiere [Bar-Tal et al., 2024]、SF-V [Zhang et al., 2024]、LaVie [Wang et al., 2023d]、Pyramidal Flow Matching [Jin et al., 2024]。扩散变换器(DiT)[Peebles and Xie, 2023] 通过引入变换器架构进一步提升了扩散模型的可扩展性, 使其能够适应各种分辨率和时长的训练视频 [Polyak et al., 2024]。尽管取得了这些进展, 计算成本、时间一致性以及合适的评估指标等挑战依然存在, 这也指引着基于扩散模型的视频生成未来的研究方向。

扩散模型的效率。扩散模型在生成任务中取得了最先进的效果, 但计算量巨大, 需要数百步采样。DDIM [Song et al., 2021a] 减少了推理时的步数, 但当步数进入极少的范围时, 性能会显著下降。为解决这一问题, 有研究提出了针对生成模型的知识蒸馏, 将预训练教师模型的知识迁移到学生模型 [Luhman and Luhman, 2021]。渐进式蒸馏(Progressive Distillation)[Salimans and Ho, 2022] 将多次迭代压缩为单次前向传播。借助教师模型和学生模型之间的分布匹配目标, 分数蒸馏最初被提出用于基于扩散模型的 3D 生成 [Poole et al., 2022]。变分分数蒸馏(VSD)随后被应用于 3D [Wang et al., 2024c] 和图像生成 [Yin et al., 2024b; Luo et al., 2024; Xie et al., 2024b; Salimans et al., 2024]。将对抗训练与扩散模型相结合的方法被提出用于少步图像生成 [Xiao et al., 2021; Yin et al., 2024a]。另一分支的方法对扩散轨迹施加了其他限制。一致性模型(Consistency models)[Song et al., 2023] 通过训练模型在不同噪声水平下输出一致的结果, 实现了单步生成。潜在一致性模型(LCM)[Luo et al., 2023] 将图像扩散模型蒸馏为一致性模型。VideoLCM [Wang et al., 2023c] 和 AnimateLCM [Wang et al., 2024a] 将一致性蒸馏应用于视频扩散模型。DPM-Solver [Lu et al., 2022] 引入了一种快速的 ODE 求解器, 将扩散采样步数减少至约 10 步。矫正流(Rectified flow)[Liu et al., 2022; Lipman et al., 2022] 是扩散模型的一种

特例,它在训练过程中强制去噪轨迹保持直线性,以实现高质量的少步采样。Instaflow [Liu et al., 2023] 采用该方法实现了图像生成的单步采样。现有方法均存在局限性:基于 VSD 的方法在蒸馏后会因生成样本多样性下降而遭受模型崩溃问题;一致性模型方法则往往保真度较低,生成样本在质量上通常不如教师模型。

基于奖励的微调。为进一步提升图像和视频生成在美学质量和文本-图像对齐等方面的质量,研究人员近期提出了多种针对扩散模型的基于奖励的微调方法 [Xu et al., 2024; Chung et al., 2022; Clark et al., 2023; Prabhudesai et al., 2024; Li et al., 2024b; Black et al., 2023; Clark et al., 2023; Domingo-Enrich et al., 2024]。最常见的方法是来自可微奖励模型的直接奖励梯度。ReFL [Xu et al., 2024] 通过 DM 中单步预测的 x_0 反向传播奖励梯度,与扩散后验采样(diffusion posterior sampling)[Chung et al., 2022] 类似。DRaFT-K [Clark et al., 2023] 将扩散过程中的奖励梯度反向传播截断至最后 K 步。VADER [Prabhudesai et al., 2024] 将此方法应用于扩散视频模型。T2V-Turbo [Li et al., 2024b] 将奖励梯度应用于视频扩散模型,梯度通过奖励模型和解码器共同反向传播。该梯度被应用于经过蒸馏的一致性模型以实现单步生成,从而避免通过 DM 进行多步反向传播。与这些方法不同,去噪扩散策略优化(DDPO)[Black et al., 2023] 将去噪过程视为决策过程,并应用诸如 REINFORCE 和 PPO 之类的随机策略梯度算法对其进行优化,而不需要可微的奖励函数。然而,由于缺乏梯度信息,DDPO 被发现其样本效率低于奖励梯度方法 [Clark et al., 2023]。为了利用丰富的奖励梯度信息,同时绕过通过大型奖励模型和解码器的反向传播,我们提出使用潜在奖励模型对扩散模型进行基于梯度的微调。Adjoint Matching [Domingo-Enrich et al.,

2024] 将奖励微调建模为随机最优控制问题,并提出了一种无记忆的流匹配方法,以确保微调后的模型收敛到倾斜分布(tilted distribution)。

8.3 方法论

图 8.2:方法概述:少步生成器 G_θ 被训练从潜在空间中的随机噪声生成高质量样本,其训练由变分分数蒸馏(VSD)、一致性蒸馏(CD)和潜在奖励模型(LRM)微调目标共同引导。VSD 损失能提升样本质量,但存在模式崩溃的风险;而 CD 损失能在不损害生成质量的前提下提升样本多样性。LRM 通过绕过庞大的像素空间奖励模型和解码器,实现了基于奖励的优化以进一步提升样本质量,从而降低内存占用,并消除了对可微奖励模型的需求。

我们方法的概览如图 9.2 所示。

8.3.1 扩散模型

假设数据分布为 $x_0 \sim q_{p \times 0, q}$, 扩散模型通过沿马尔可夫链逐步去噪来近似该分布。前向扩散过程遵循 $x_t: \mathbb{R}^{p \times 0, t, q} \sim \mathcal{N}(x_0, \epsilon_t)$, $\epsilon_t \sim \mathcal{N}(0, I_q)$ 。对于 DDPM,

??

$$a_t = \alpha_t, b_t = 1 - \alpha_t \quad (8.1)$$

其中 $\alpha_t = \prod_{i=1}^t \alpha_i$ 遵循预先指定的噪声调度 $\alpha_t, t \in \{1, \dots, T\}$ 。对于通用的方差保持(variance-preserving)调度 [Song et al., 2021b], 它满足 $a_t \leq b_t \leq 1$, 因此可

等价地写作 $x_t = \cos(\omega t) x_0 + \sin(\omega t) v_t$, $t \in [0, \pi/2]$, 只需对时间序列进行简单的映射。采用速度预测 v_t 的标准扩散模型优化遵循以下损失函数:

$$\mathcal{L}_v = \mathbb{E}_{x_0} \left[\sum_{t \sim \mathcal{T}} \left\| v_t - \hat{v}_t \right\|^2 \right] \quad (8.2)$$

$$v_t = \sin(\omega t) x_0 + \cos(\omega t) v_0 \quad (8.3)$$

对于矫正流(RF)[Liu et al., 2022] 或流匹配(flow matching)[Lipman et al., 2022], 在 $t \in [0, 1]$, 其速度目标为常数 $v_t = v_0$, $t \in [0, 1]$ 。训练完成后, 反向扩散过程通过迭代去噪遵循 $x_t = \text{Denoise}(x_{t+\Delta t}, \theta_t)$ 。我们模型的训练和推理细节见第 8.6.2 节。

8.3.2 一致性蒸馏

一致性模型 [Song et al., 2023] 将一致性损失作为从预训练教师模型 v_{θ_1} 进行蒸馏的方法, 采用离散的子采样时间调度 $t_1 = \epsilon, t_2 = 2\epsilon, \dots, t_N = T$:

$$\mathcal{L}_{\text{CD}} = \mathbb{E}_{x_0} \left[\sum_{t \sim \mathcal{T}} \left\| p_{\theta_1}(x_t) - p_{\theta_2}(x_t) \right\|^2 \right] \quad (8.4)$$

$$\hat{x}_t = \text{Denoise}(p_{\theta_1}(x_{t+\Delta t}), \theta_t) \quad (8.5)$$

其中 $\lambda_{p_{\theta_1}}(x_t)$ 是一个随时间变化的系数, 在实践中通常被设为常数; $d_p(\cdot, \cdot)$ 是一个距离度量, 如 MSE 或 Huber 损失; θ_t 是 θ 的指数滑动平均。我们并未采用传统

为增强针对条件生成(带条件变量 $c \in \mathcal{C}$, 例如文本提示)的蒸馏效果,我们对教师去噪函数应用了无分类器引导(CFG)[Ho and Salimans, 2022] 增强,做法与 VideoLCM [Wang et al., 2023c] 类似,但在本工作中是针对 v_θ 预测:

$$v_{\theta w} p_{x|t_n, t_n, c_q} - v_{\theta} p_{x|t_n, t_n, c_q} - w v_{\theta} p_{x|t_n, t_n, c_q} - v_{\theta} p_{x|t_n, t_n, \emptyset} q \quad (8.6)$$

、~

在条件生成中,该式将替换公式 (8.15) 中的 v_θ 。

8.3.3 变分分数蒸馏

变分分数蒸馏(VSD)[Wang et al., 2024c] 的提出目标是通过用经过适当训练的扩散模型来近似分数,实现教师模型与学生模型之间的分布匹配。具体而言,最小化真实(教师)样本分布 p_{real} 与虚假(学生)样本分布 p_{fake} 之间 Kullback-Leibler(KL)散度的损失形式为:

$$p_{\text{fake}} p_{x|q} \text{ LVSD} : - \mathbb{E}_{x \sim p_{\text{fake}}} [\text{DKL}(p_{\text{fake}} || p_{\text{real}} | q)] = - \mathbb{E}_{x \sim p_{\text{fake}}} [\log \frac{p_{\text{fake}}(x|q)}{p_{\text{real}}(x|q)}] \quad (8.7)$$

$$p_{\text{real}} p_{x|q} \text{ LVSD} : - \mathbb{E}_{x \sim p_{\text{real}}} [\text{DKL}(p_{\text{real}} || p_{\text{fake}} | q)] = - \mathbb{E}_{x \sim p_{\text{real}}} [\log \frac{p_{\text{real}}(x|q)}{p_{\text{fake}}(x|q)}] \quad (8.8)$$

其导数为:

$$\nabla_{\theta} \text{DKL} = - \mathbb{E}_{x \sim p_{\text{real}}} [\log \frac{p_{\text{real}}(x|q)}{p_{\text{fake}}(x|q)}] \nabla_{\theta} p_{\text{real}} p_{x|q} - \mathbb{E}_{x \sim p_{\text{fake}}} [\log \frac{p_{\text{fake}}(x|q)}{p_{\text{real}}(x|q)}] \nabla_{\theta} p_{\text{fake}} p_{x|q} \quad (8.9)$$

其中分数函数 $s_{\text{real}} p_{x|q} = \nabla_x \log p_{\text{real}} p_{x|q}$ 和 $s_{\text{fake}} p_{x|q} = \nabla_x \log p_{\text{fake}} p_{x|q}$ 分别对应两个分布。 $G_{\theta} p_{x|q}$ 是学生网络通过对含噪训练样本迭代去噪所进行的,为准确估计虚假分数 s_{fake} ,虚假扩散模型 $v_{\theta} \text{fake}$ 由教师模型初始化,并根据学生样本分布动态调整。

图 8.3:不同奖励微调方法的比较:(1)直接奖励梯度方法局限于较小的奖励模型或视频模型,或较短的输入序列,并且同样需要可微的奖励模型;(2)潜在奖励模型结构紧凑,绕了解码器进行基于梯度的优化,因此在大型奖励模型和解码器超出可用显存(VRAM)时依然适用;(3)DDPO 在处理大型视频模型并跟踪多步样本的对数概率时,同样受到显存限制的制约。

在处理学生生成样本时,虚假分数模型依照公式 (8.13) 中的扩散损失 $LCV_{p \theta \text{ fake } q}$ 进行更新,以实现分数估计的目的。

最先进的蒸馏方法----分布匹配蒸馏(DMD)[Yin et al., 2024b,a]----主要采用 VSD 损失进行学生模型蒸馏。我们放弃了 [Yin et al., 2024a] 中的对抗损失,因为 GAN 训练通常不稳定,且带来的提升可能十分有限。我们用一致性蒸馏损失和奖励模型微调取代了对抗训练,正如我们的实验所证明的,这能产生更丰富的梯度信号。

8.3.4 潜在奖励微调

奖励微调是在后训练阶段将样本分布与指定的偏好指标对齐的一种有效方法,如图 8.6 所示。

此前基于奖励的优化方法要么(1)需要来自像素空间奖励模型的直接梯度 [Xu et al., 2024; Clark et al., 2023],要么(2)依赖于对多个扩散步数样本的对数概率估计,如 DDPO [Black et al., 2023],二者的比较见图 8.3。方法(1)的一个主要缺点是它仅适用于可微的奖励函数,而对于像 JPEG

可压缩性 [Black et al., 2023] 等。除此之外,奖励模型通常作用于原始 RGB 像素空间,这就要求奖励梯度不仅要反向传播通过庞大的奖励模型,还要反向传播通过解码器,因为实际框架通常遵循 LDM [Rombach et al., 2022b] 进行潜空间建模。方法 (2) 由于缺乏丰富的奖励梯度信息 [Clark et al., 2023],被发现在奖励优化方面效率较低,并且由于需要在多个扩散步骤上进行梯度估计而占用更多内存。一项同期工作 [Li et al., 2024a] 探索了在潜空间中的奖励,但仅限于图像生成。视频模型迫切需要紧凑的奖励模型,这是一个尚未被充分探索的领域,其中利用基于图像的奖励来指导视频生成仍然并非易事。

我们提出学习一个双重潜在奖励模型(LRM),用于在潜空间中直接优化视频扩散模型,该模型支持第 8.7.3 节中详述的任何类型的奖励度量。这里我们以图像奖励为例。给定一个提供的图像奖励模型 $R: \mathbb{R}^3 \rightarrow \mathbb{R}$, 其输入为 RGB 图像 $i \in \mathbb{R}^3$, 我们用如下损失来近似 $\tilde{r} = \text{LRM}(R)(\phi: \mathbb{R}^3 \rightarrow \mathbb{R})$:

$$\mathcal{L}_{\text{LRM}}(\phi) = \mathbb{E}_{\mathbf{x} \sim \mathcal{D}} \left[\left(\tilde{r}(\mathbf{x}) - \text{Decp}(\mathbf{x}) \right)^2 \right] \quad (8.10)$$

其中 $\mathbf{x} \sim \mathcal{D}$ 是生成器生成的样本集合, Decp 是解码器。我们同时使用真实图像和生成图像来提高所学习的 LRM 在生成样本上的鲁棒性。为了减轻训练 LRM 时的计算负担,我们应用公式 (8.15) 对生成样本进行单步预测,而不是沿整条轨迹进行迭代去噪。这种方法通过避免在迭代采样过程中进行梯度反向传播,显著降低了内存使用量。尽管我们的蒸馏学生模型最多以 4 个采样步骤运行,但如果采用完整去噪过程生成样本,内存占用仍可能非常大。表 8.1 比较了在批大小为 1 的视频样本上,像素空间奖励模型与 LRM 的参数数量和内存开销。HPSv2 和 PickScore 是我们实验中使用的两个像素空间奖励模型(如第 8.5.3 节所述)。

表 8.1:像素空间的 HPSv2、PickScore 奖励模型与 LRM 在参数数量和 GPU 显存(VRAM)开销方面的比较。

模型	参数数量	前向显存	反向显存	图像 LRM	文本-图像 LRM
HPSv2/PickScore	6.32 亿	5.926 GB	90 GB	189,441 8.998 MB	763,009 15.500 MB
				17.772 MB	26.277 MB

借助潜空间上紧凑且可微的 LRM,我们应用直接奖励梯度优化来微调扩散模型:

$$\mathcal{L}_{\text{FT}}(\theta; \phi) = \mathbb{E}_{\mathbf{p}(\mathbf{I}, \mathbf{c})} \left[\mathbb{E}_{\mathbf{p}(\mathbf{I}|\phi)} \left[\mathbb{E}_{\mathbf{p}(\mathbf{c}|\phi)} \left[\mathcal{L}_{\text{FT}}(\theta; \phi) \right] \right] \right] \quad (8.11)$$

在实践中,我们可以先预训练 LRM,然后在固定 LRM 的情况下微调扩散模型,也可以交替训练 LRM 和微调扩散模型。为简单起见,我们采用第二种方式。如果原始奖励模型以额外的文本输入为条件,即 $\mathbf{p}(\mathbf{I}, \mathbf{c})$,则 LRM 也以条件方式运作,记为 $\mathbf{p}(\mathbf{I}|\phi, \mathbf{c})$ 。LRM 方法可以适配任何类型的奖励模型,包括图像、视频、文本-图像和文本-视频奖励。对于仅图像的 LRM,我们使用卷积神经网络;而对于文本-图像 LRM,我们在卷积特征提取器之后应用一个交叉注意力模块,以将文本嵌入与图像特征相结合。对于基于视频的 LRM,2D 卷积被替换为 3D 卷积神经网络。更多细节见第 8.7.3 节。

8.4 多目标蒸馏

单独使用 VSD(如 DMD 方法中)进行蒸馏可能导致严重的模式坍塌,而 CD(如 LCM 方法中)由于在样本分布上进行平均,往往会产生质量较低的样本,图 8.4 中给出了相关证据。通过整合一致性蒸馏、变分分数蒸馏和潜在奖励微调目标,我们的方法能够在蒸馏后实现高质量、多样化样本的少步生成,

其优化目标为如下损失函数:

$$\mathcal{L}_{\theta,q} = \mathcal{L}_{VSD}(\theta,q) + \beta \mathcal{L}_{CD}(\theta,q) + \beta \mathcal{L}_{FT}(\theta,q) + \varphi(q) \quad (8.12)$$

在蒸馏过程中,虚假分数网络通过 $\mathcal{L}_{CV}(\theta, \text{fake } q)$ 进行更新,LRM $\mathcal{L}_{\phi}$ 则通过 $\mathcal{L}_{LRM}(\theta, \phi, q)$ 进行更新。我们方法的伪代码见第 8.6.1 节。

8.5 实验

8.5.1 实现细节

学生模型与教师模型。我们实验中的视频扩散模型基于扩散变换器(DiT)架构 [Peebles and Xie, 2023]。尽管我们的方法与架构无关,也可以应用于诸如 U-Net [Rombach et al., 2022b] 这样的模型,但我们选择变换器架构是因为其可扩展性。教师和学生 T2V 扩散模型与 Open-Sora [Zheng et al., 2024] 的一个修改版本相同,并遵循 LDM 框架 [Rombach et al., 2022b; Blattmann et al., 2023b],利用在潜空间中的 DiT 建模,该潜空间由一个预训练的 3D 变分自编码器(VAE) [Kingma, 2013; Zheng et al., 2024] 编码。该 3D VAE 逐块编码和解码视频,以减轻计算负担,将 16 帧视频的块编码为 5 个潜在嵌入。这些嵌入随后被切分为序列,作为 DiT 的输入。借助 DiT 架构,模型可以适配任意的视频时长和分辨率;但在我们的实验中,视频生成设置被限定为 128 帧、分辨率为 192×320 ,得到长度为 9600 的切分序列。

模型训练与推理。教师模型采用具有 1000 个采样步骤的 DDPM,但在推理时使用 50 步的 DDIM,而学生模型则被蒸馏为类似先前工作 [Li et al., 2024c] 的 4 步采样,CFG 权重为 7.5 用于

CD。每个学生模型的蒸馏实验均在 8 块 A100 GPU 上进行。更多实现细节见第 8.6.3 节。

奖励度量。我们使用人类偏好分数 v2(HPSv2) [Wu et al., 2023a] 和 PickScore [Kirstain et al., 2023] 作为文本-图像奖励模型。二者都是在大规模、带有人类偏好标注的文本到图像数据集上微调的 CLIP 类模型。尽管我们的方法与直接使用 VBench 奖励度量进行优化是兼容的,但我们有意避免这样做,因为 VBench 分数是最终评估标准之一。直接针对特定的 VBench 指标或简单的图像奖励(如 JPEG 可压缩性)进行优化,可以显著提升奖励分数,但也可能导致对特定指标的过度优化,从而降低整体生成质量。因此,我们默认采用更通用的、基于偏好的奖励模型 HPSv2 和 PickScore 进行奖励微调。不过,我们的方法仍与其他奖励模型兼容。

评估。为了忠实反映模型性能,我们对结果同时应用自动评估基准 VBench [Huang et al., 2024] 和人工评估。VBench 从视频视觉质量和语义对齐两个方面评估 T2V 模型的 16 个维度,各项指标得分越高表示表现越好。按照标准的 VBench 评估协议,我们使用一组 946 个长提示词,每个提示词生成五个视频。每个维度的最终得分是在该指标下所有生成视频上的平均值。此外,我们还通过在 VBench 评估中比较长描述性提示词和短提示词的表现,来考察提示词长度的影响。为了进一步评估文本-视频对齐能力,我们对蒸馏后的学生模型进行了各种风格和运动的采样。

8.5.2 与 SOTA 方法和模型的比较

与 SOTA 蒸馏方法的比较。在图 8.4 中,我们比较了我们蒸馏模型的样本与 SOTA 蒸馏方法的样本,包括 (1).基于 VSD 的

方法,用于 DMD [Yin et al., 2024b,a];(2).基于 CD 的方法,通常用于 LCM [Luo et al., 2023] 和 VideoLCM [Wang et al., 2023c],也被 T2V-Turbo [Li et al., 2024c] 和 FastVideo [Zhang et al., 2025b] 所采用。这些蒸馏方法均采用 4 步推理。我们还展示了教师模型以 50 步 DDIM 生成的样本。由于训练过程中的高波动性以及超参数的敏感性,LCM 生成的样本模糊,质量低于其他方法,尽管通过全面的超参数搜索可能达到更好的性能。DMD 相比我们的方法在保真度和样本多样性上均较低。在蒸馏后的少步模型上应用 LRM 进一步提升了质量和对齐效果,超越了教师模型的性能上限。由于奖励梯度具有梯度反向传播的性质,将其应用于采样步骤较多的教师模型颇具挑战,而对蒸馏后的学生模型则十分适用。

表 8.2:不同模型的 VBench 分数比较。

模型	Pika	Gen-2	Gen-3	Kling	T2V-Turbo	教师	我们的方法(PickScore)	我们的方法(HPSv2)	主体一致性	96.76	97.61	97.10	98.33
	96.28	83.99	93.77	92.57	背景一致性	98.95	97.61	96.62	97.60	97.02	93.78	96.80	96.14
	97.48	运动平滑度	99.51	99.58	99.23	99.40	97.34	98.09	97.76	98.59	动态程度	37.22	18.89
	63.15	66.96	63.34	46.94	63.04	61.21	63.80	63.14	成像质量	62.33	67.42	66.82	65.62
	87.24	93.96	85.79	91.63	93.84	多物体	46.69	55.47	53.64	68.05	54.65	52.59	69.71
	99.00	99.00	颜色	85.31	89.49	80.90	89.90	89.90	77.00	77.95	74.78	空间关系	65.65
	48.91	54.57	50.86	55.58	49.99	55.06	52.72	时间风格	24.44	24.12	24.71	24.17	25.51
	24.42	24.83	24.45	23.50	整体一致性	25.47	26.17	26.69	26.42	28.16	27.89	26.93	26.85
	83.83	语义分数	71.26	73.03	75.17	75.68	72.57	73.71	77.90	77.51	总分	80.40	80.58
												82.32	81.85
												81.01	80.25
												82.37	82.57

与 SOTA 模型在 VBench 上的比较。VBench 的评估结果总结在表 8.2 中,最高分以粗体标出,第 2、3 位以下划线标出。我们采用 VSD+CD+LRM 蒸馏方法取得了优于基线的性能,这些基线包括 Pika [Pika Labs]、Gen-2、Gen-3、Kling [Kuaishou, 2024] 以及 T2V-Turbo [Li

图 8.4:使用我们的方法、SOTA 蒸馏方法以及顶尖视频模型(未蒸馏)在 VBench 上合成的视频样本。

图 8.5:我们的 DOLLAR 方法(配合 HPSv2)与各基线相比的人类偏好结果。百分比表示偏好率。

et al., 2024b](采用完整推理步骤),以及我们的教师模型。可视化结果显示在图 8.4 中。我们模型最高的语义分数表明其在文本-视频对齐方面相较基线有显著提升。质量分数反映的是视觉质量,该指标受主体一致性、背景一致性、时间闪烁和运动平滑度等帧一致性指标的严重影响,而这些指标通常在视频缺乏运动时较高。如表格和图 8.4 所示,我们的模型在运动的动态程度上明显更高。总分是所有指标的加权和,反映了视频的总体偏好,我们的方法达到 82.37 和 82.57,超过了表中的所有模型,也超越了教师模型。学生模型在 16 个指标中的 9-10 个上得分高于教师模型。这表明我们方法的性能并非受限于教师模型的上限,这一点超越了用于学生和教师分布匹配的 VSD 损失所能达到的效果。额外的 CD 损失强制模型预测在带噪真实图像上的自一致性,为学生模型在质量和语义表现上超越教师模型提供了信号来源,而这又通过 LRM 微调得到进一步提升。

人工评估。我们进一步进行了人工评估,从视觉质量、文本-视频对齐、运动和总体偏好四个独立指标出发,对不同模型生成的视频进行可视化比较。主要评估结果显示在图 8.5 中。评估结果表明,我们采用 HPSv2 奖励的方法在总体偏好方面比 DDPO 方法更受青睐(57.3%),也比教师模型更受青睐(51.1%),与 Gen-3 模型的表现相近(45.6%)。我们蒸馏后的学生模型的视觉质量明显高于教师模型(58.4%)和 Gen-3(55.9%)。此外,我们发现 PickScore 能提升视觉质量,但可能导致运动表现变差。HPSv2 奖励调优不仅能提升视觉质量,在运动和文本-视频对齐方面也表现更好。

8.5.3 与像素空间奖励的比较

我们将 LRM 与像素空间奖励微调方法 DDPO [Black et al., 2023] 进行比较,该方法将扩散过程视为一个 MDP,采用 REINFORCE 算法来优化扩散模型。更多细节见第 8.7.6 节。像 ReFL [Xu et al., 2024] 和 DRaFT [Clark et al., 2023] 这样的直接奖励梯度方法,在我们的场景下超出了单 GPU 的显存容量,因此未将其纳入基线比较。

图 8.6:比较两个样本在有(第一行)和没有(第二行)奖励微调情况下生成的样本。第一个样本:沿时间序列从每种方法采样的一个视频中提取 4 帧。第二个样本:从一个视频中提取一帧,同一提示词下采样了 4 个视频。

表 8.5 比较了 LRM 和 DDPO 的 VBench 分数与最终奖励值。最后一行「奖励」表示微调后对应的奖励值,例如,如果模型是用 PickScore 奖励模型微调的,则报告 PickScore 值,HPSv2 情况类似。均值和标准差是在 VBench 提示词下生成的 500 个视频上报告的。如图 8.6 所示,奖励微调有助于通过更明确地展现「浮现」效果,来提升第一个提示词的文本-图像对齐效果,并提升第二个提示词中文字在画面中显示的准确性。照明风格也通过微调得到了改善。DDPO

和 LRM 方法的更多细节,包括奖励优化的学习曲线,见第 8.7 节。

8.5.4 蒸馏方法的消融实验

表 8.3 展示了我们蒸馏方法的消融实验,将其与 VSD 以及 VSD+CD 进行比较。VSD 与教师模型的表现相当,加入额外的 CD 损失后,样本多样性得以提升。我们的 VSD+CD+LRM 方法在整体上实现了高样本质量和多样性。

表 8.3:教师模型与采用不同蒸馏方法的学生模型的比较,学生模型均采用 4 步采样。

模型	教师	学生	方法	DDIM	50 步	VSD	VSD+CD	VSD+CD+LRM	质量分数	81.89	80.95	82.16	83.83	语义	分数	73.71	76.61	74.58
Vendi																		
(像素)																		
Vendi																		
(Inception)																		

多样性度量。模型生成的多样性并未被 VBench 捕捉到。我们对不同蒸馏方法的生成多样性进行了定性和定量的比较。我们使用 Vendi 分数 [Friedman and Dieng, 2022] 定量测量所采样视频的多样性,该分数基于样本集合的相似度矩阵。表 8.3 中报告了跨提示词视频样本的均值和标准差。我们发现,基于 Inception 的 Vendi 分数比基于像素的替代方案与视觉观察结果更为一致。虽然 VSD 能产生高质量样本,但它往往导致模式坍缩。我们的方法通过引入 CD 损失来解决这一多样性限制,并通过 LRM 微调进一步提升生成质量。

推理时间。表8.4展示了教师模型使用50步DDIM推理,以及学生模型使用1、2、4步推理时,每个样本的推理时间消耗。这里,"diffusion time"(扩散时间)仅指

表8.4:教师模型与蒸馏学生模型在不同函数评估次数(NFE)下的时间消耗(占教师模型总推理时间的百分比)

模型 教师 学生 步数(NFE) 50 4 2 1

扩散时间(%)	91.94~0.32	5.88~0.03	2.16~0.01	0.33~0.02
推理时间(%)	100.00~0.66	13.06~0.17	9.30~0.11	7.45~0.12

表8.5:LRM与DDPO在VBench及训练奖励指标上的比较。 DOLLAR为我们最终采用VSD+CD+LRM的方法。

奖励模型	PickScore	HPsv2	方法	VSD+DDPO	VSD+LRM	DOLLAR	VSD+DDPO	VSD+LRM	DOLLAR	质量	82.99	84.01	83.49	
	82.97	83.53	83.83	语义	77.26	72.51	77.90	74.56	75.67	77.51	总分	81.84	81.71	82.37
							81.29	81.96	82.57	奖励	0.207	0.207	0.210	
											0.271	0.276	0.277	

8.5.5 消融研究

蒸馏时间步。我们提出的方法支持对教师采样使用任意时间步子集。默认情况下,如8.5.1节所述,我们使用4步采样作为学生模型,以平衡质量与效率。在此,我们研究了蒸馏过程中采样步数变化所带来的影响,具体测试了

1步(时间步[999])、2步(时间步[499, 999])以及等间距的4步(时间步[249, 499, 749, 999])。尽管我们的方法并不要求等间距,但为保证本实验的一致性,采用了该配置。评估得到的VBench分数见表8.6。三个使用VSD损失蒸馏得到的学生模型,相较于采用50步推理的教师模型,均表现出相当甚至更优的性能。这些微小差异可归因于检查点选择和评估方差。通过视觉检查和人工评估,我们发现推理步数更多的模型往往表现更好,而这一点可能并未在VBench分数的微小差异中充分体现。

模型	教师	学生(VSD)	模型	教师	学生(VSD+CD)	推理步数	50	1	2	4	CD配合Denoisem	- m	“ 1	m	“ 5	质量	81.89	81.61	82.71	80.95	
质量	81.89	80.75	82.16	语义	73.71	76.66	73.86	76.61	语义	73.71	71.57	74.58	总分	80.25	80.62	80.94	80.08	总分	80.25	78.92	80.65

表8.6:采用VSD蒸馏的学生模型 表8.7:一致性蒸馏(CD)中教师 在不同推理步数下的比较(VBench, 去噪步数m对VBench(长提示) 长提示)。 的影响。

一致性蒸馏去噪步数。在8.3.2节中,我们介绍了采用多步教师去噪函数Denoisem p” q的一致性蒸馏方法。我们在实验中对m的取值进行了消融,发现较大的取值如m “ 5能提升蒸馏性能,详见表8.7。学生模型遵循4步采样方案,CD损失则按前述方式应用于步长为20的50步DDIM采样方案。

8.6 DOLLAR方法细节

8.6.1 伪代码

我们DOLLAR方法的伪代码如算法29所示

算法29 DOLLAR的训练流程

```

1: 输入:通过公式(8.13)的LCV预训练得到的教师模型 $v_{\theta 1}$ ,预训练的编码器和解码器,
数据集 $D = \{t_{pc}, i_{q}\}$ 
2: 输出:蒸馏得到的学生少步生成器 $G_{\theta}$ 
3: //从教师模型初始化学生模型和虚假分数模型
4:  $\theta \leftarrow \theta 1, \theta_{fake} \leftarrow \theta 1$ 
5: while 训练 do
6: 采样批次  $p_c, i_q \sim D$ , 编码  $x \sim \text{Encoder}(p_{i_q})$ 
7: //通过蒸馏更新生成器
8:  $\hat{x} \leftarrow G_{\theta}(p_c, \varepsilon_q, \varepsilon \sim \mathcal{N}(p_0, I_q))$ 
9: 均匀采样 $t_n$ ,前向扩散  $x_{t_n} \sim F(p_x, t_n, m_q)$ 
10:  $LG \leftarrow \text{LVSD}(p_{\theta}; \theta 1, \theta_{fake}, \hat{x}, c_q \sim \eta 1 \text{ LCD}(p_{\theta}; \theta 1, x_{t_n}, m_q) // \text{VSD对应公式(8.8)}, CD \text{对应公式(8.4)})$ 
11:  $G_{\theta} \leftarrow \text{GradientDescent}(p_{\theta}, LG, q)$ 
12: //更新虚假分数模型
13: 均匀采样 $t$ ,前向扩散  $x_t \sim F(p_{\hat{x}}, t, q)$ 
14:  $\theta_{fake} \leftarrow \text{GradientDescent}(\theta_{fake}, \text{LCV}(p_{x_t}, q) // \text{公式(8.13)})$ 
15: //训练潜空间奖励模型 合并批次  $\tilde{x} = \{x, \hat{x}\}$ 
16:  $R_l \leftarrow \text{GradientDescent}(p_{\phi}, \text{LLRM}(p_{\phi}; \tilde{x}), R_{qq} // \text{公式(8.10)})$ 
17: //通过潜空间奖励微调更新生成器
18:  $G_{\theta} \leftarrow \text{GradientDescent}(p_{\theta}, \text{LFT}(p_{\theta}; \hat{x}, R_l), q // \text{公式(8.11)})$ 

```

8.6.2 扩散模型训练与推理

共轭预测目标。与以往工作[Ho et al., 2020; Rombach et al., 2022b]中采用的噪声预测,以及Instaflow[Liu et al., 2023]中标准的速度预测目标不同,我们采用了一种共轭速度预测目标:

“ %0

$$\text{LCV}(p_{\theta}, q) = \mathbb{E}_{x_0 \sim q, \varepsilon \sim \mathcal{N}(p_0, I_q, t)} \|v_{\theta}(p_{x_t}, t, q) - p_{x_0}\|_2^2 \quad (8.13)$$

其中样本 x_t 沿着由公式(8.1)所定义的调度方式在扩散轨迹上进行扩散。该模型参数化为在每个时间步 t 上预测RF轨迹上的速度 v_t ,目标为恒定值 $p_{x_0} - \varepsilon_q$ (此外

图8.7:共轭速度预测的示意:扩散与矫正流之间 v 预测的关系。

为何采用共轭预测目标?矫正流损失是StableDiffusion3[Esser et al., 2024]、MovieGen[Polyak and Amit Zohar, 2025]、Hunyuan Video[Kong and Qi Tian, 2024]等模型中常用的生成式建模目标。尽管以往的视频生成模型采用DDPM噪声调度,但按照公式(8.2)沿扩散轨迹进行直接速度预测,其目标是随时间变化的,如公式(8.3)所示,相较于公式(8.13)中的恒定速度目标,在实践中更难学习。然而,标准矫正流[29]的目标并不沿扩散轨迹,这要求以不同于DDPM的噪声调度来建模该轨迹。通过扩散轨迹与矫正流之间的共轭关系,共轭目标在沿扩散轨迹生成保方差噪声样本的同时预测恒定速度,因此更易于学习。

推理。训练完成后,反向扩散过程遵循如下形式:

$$\begin{aligned}
 & \mathbf{x}_t \stackrel{1}{:} \text{Denoise}(\mathbf{x}_t, t, \theta_q) \\
 & ?? \\
 & \tilde{\mathbf{a}}_t \stackrel{1}{:} \tilde{\mathbf{a}}_t \stackrel{1}{:} \\
 & b? \\
 & \text{"p} \tilde{\mathbf{a}}_t \stackrel{1}{:} \stackrel{1}{:} \tilde{\mathbf{a}}_t \stackrel{1}{:} \sigma_t^2 ? q \tilde{\mathbf{x}}_0 \stackrel{?}{:} \mathbf{x}_t \stackrel{\sigma}{:} t \varepsilon \text{ (8.14)} \\
 & \stackrel{1}{:} \tilde{\mathbf{a}}_t \stackrel{1}{:} \tilde{\mathbf{a}}_t \\
 & ? \\
 & \text{其中 } \tilde{\mathbf{x}}_0 \text{ " } \mathbf{x}_t \stackrel{?}{:} \tilde{\mathbf{a}}_1 \stackrel{1}{:} \tilde{\mathbf{a}}_t \stackrel{v\theta}{:} p_{\mathbf{x}_t}, t, q \\
 & ? \\
 & t \stackrel{1}{:} \tilde{\mathbf{a}}_t
 \end{aligned}$$

为预测的原始样本。证明见8.6.5节。

8.6.3 实现细节

教师模型和学生模型均在内部的图像与视频数据集(带文本标注)上训练,数据规模约为 10^8 张图像和 10^4 个视频。

教师模型采用标准DDPM设置,共1000个采样步: $t \in \{1, \dots, 1000\}$ 。在推理阶段,教师模型使用DDIM采样以50步生成高质量样本, $t \in \{19, 39, \dots, 999\}$ 。为降低GPU上的显存占用,我们采用梯度检查点(gradient checkpointing)以及全共享数据并行(FSDP)[Zhao et al., 2023],以数据并行方式在各GPU间分片模型权重和梯度。此外,我们采用Bfloat16数据类型进行混合精度训练。对于采用LRM的微调,由于所用批大小较小($=1$),我们对7个步骤进行梯度累积以稳定训练。

8.6.4 学生-教师参数化

学生-教师参数化有两种不同方式:同构和异构。

在同构学生-教师参数化中,学生和教师网络遵循相同的变量预测形式,即我们设定中的 v 预测,通过如下变换:

?

$$\begin{aligned} x_t &= \mathbf{1}^T \tilde{\mathbf{a}}_t v_{\theta} p_{x_t}, t_q \\ x_{\theta} p_{x_t}, t_q &= ? \quad ? \quad (8.15) \\ \tilde{\mathbf{a}}_t &= \mathbf{1}^T \tilde{\mathbf{a}}_t \end{aligned}$$

该式的证明见8.6.5节。学生模型 v_{θ} 在蒸馏开始时将从教师模型 v_{θ_1} 初始化。

在异构学生-教师参数化中,学生网络可以直接预测 x_{θ} ,而不利用公式(8.15)。为了在学生蒸馏中最充分地利用教师模型,我们默认采用同构参数化。

8.6.5 推导

公式(8.14)的证明

我们从DDPM[Ho et al., 2020]的前向扩散过程出发。单步扩散过程的分布?

$q_{p_{x_t} | x_{t-1}, q} \sim N(p_{x_t}; \alpha_t x_{t-1}, p_{1-t} \beta_t q)$ 可等价地写为:

??

$$x_t = \alpha_t x_{t-1} + \sqrt{1 - \alpha_t} \epsilon, \quad \epsilon \sim N(0, I_q) \quad (8.16)$$

其中 $t \in [1, T]$ 。根据链式法则,我们有

??

$$x_t = \tilde{\alpha}_t x_0 + \sqrt{1 - \tilde{\alpha}_t} \epsilon \quad (8.17)$$

其中 $\tilde{\alpha}_t = \prod_{i=1}^t \alpha_i$ 。等价地,我们有 $x_t | x_0 \sim N(p_{x_t}; \tilde{\alpha}_t x_0, p_{1-t} \tilde{\beta}_t q)$ 。该式也可用于预测:

?

$$\begin{aligned} \mathbf{1}^T \tilde{\mathbf{a}}_t \\ \hat{x}_0 = ? \quad x_t = ? \quad \epsilon_{\theta} \quad (8.18) \end{aligned}$$

$\tilde{\alpha}_t \tilde{\alpha}_t$

该式称为Tweedie公式。 ε_θ 为由参数 θ 参数化的模型对 ε 的近似预测。

反向扩散过程中去噪函数(公式8.14)的证明如下:

?

$$x_t = \alpha_t x_0 + \sqrt{1 - \alpha_t} \varepsilon_t \quad (8.14)$$

?

$$\alpha_t = \prod_{s=0}^{t-1} \beta_s \quad (8.15)$$

$$\beta_s = \frac{\sigma_{s+1}^2}{\sigma_s^2}$$

?

$$\sigma_s^2 = \sigma^2 \sum_{i=0}^{s-1} \beta_i$$

$$\sigma_t^2 = \sigma^2 \sum_{i=0}^{t-1} \beta_i$$

?

$$x_t = \alpha_t x_0 + \sqrt{1 - \alpha_t} \varepsilon_t \quad (8.16)$$

$$x_t = \alpha_t x_0 + \sqrt{1 - \alpha_t} \varepsilon_t$$

?

$$x_t = \alpha_t x_0 + \sqrt{1 - \alpha_t} \varepsilon_t$$

$$x_t = \alpha_t x_0 + \sqrt{1 - \alpha_t} \varepsilon_t$$

$$x_t = \alpha_t x_0 + \sqrt{1 - \alpha_t} \varepsilon_t$$

其中第一个等式遵循DDIM论文[Song et al., 2021a]中的后验采样。第二个等式代入了Tweedie公式。方差项为

$$\sigma_t^2 = \sigma^2 \sum_{i=0}^{t-1} \beta_i$$

$$\sigma_t^2 = \sigma^2 \sum_{i=0}^{t-1} \beta_i$$

。

公式(8.15)的证明

按照公式(8.13)的InstafLOW目标,网络直接预测 v_θ ,以逼近沿矫正流(RF)轨迹的目标速度 $\tilde{v}_y$,即干净样本与高斯噪声之差:

$$v_\theta \approx \tilde{v}_y - x_0 \varepsilon \quad (8.19)$$

由于RF样本 y_t 是扩散样本 x_t 的一种缩放形式,如下所示:

x_t

$$y_t = \alpha_t x_t = \alpha_t (\alpha_t x_0 + \sqrt{1 - \alpha_t} \varepsilon_t) \quad (8.20)$$

$$\alpha_t = \prod_{s=0}^{t-1} \beta_s$$

?

$$y_t = \alpha_t x_t = \alpha_t (\alpha_t x_0 + \sqrt{1 - \alpha_t} \varepsilon_t) \quad (8.21)$$

$$\alpha_t = \prod_{s=0}^{t-1} \beta_s$$

满足 $y_0 = x_0$ 。

给定速度预测 v_θ ,我们可以通过在公式(8.19)和(8.20)中将 x_0 替换为预测值 x_θ ,推导出对原始样本 x_θ 的预测,如下所示:

$$y_t x_\theta = y_t \cdot p_1 \cdot y_t q p x_\theta \cdot v_\theta y_q \quad (8.22)$$

?

$$1 \cdot \tilde{\alpha}_t$$

$$x_\theta = y_t \cdot p_1 \cdot y_t q v_\theta = y_t \cdot ? \cdot v_\theta \quad (8.23)$$

$$\tilde{\alpha}_t \cdot 1 \cdot \tilde{\alpha}_t$$

由此完成证明。

8.6.6 推理时间分析

在此,我们对主论文8.5.4节表8.4中所展示的推理时间实验结果进行详细分析。由于绝对时间成本受硬件相关因素以及批大小、所用GPU数量等推理配置的影响,故未予报告。相反,我们更强调相对时间消耗,将其作为跨配置比较更可靠的指标。值得注意的是,扩散采样时间与采样步数之间的关系并非严格线性。例如,第一个扩散采样步仅占总推理时间的0.33%,比后续步骤快约6.2倍。这种差异可能是由于初始高斯噪声输入的推理过程更快,或早期推理阶段硬件缓存占用相对较低所致。此外,总推理时间与扩散采样时间之间的差异,包含了文本预处理与编码,以及从潜空间解码回原始像素空间的额外成本。这些过程总计约占总推理时间的7%。

8.7 奖励模型微调

8.7.1 微调效果证据

如图8.8所示,经奖励模型调优后生成的样本,在美学质量、光照条件、色彩等方面,与数据集中的原始训练样本(图中左侧)相比可能存在显著差异。

图8.8:训练数据集中样本(左)与使用HPSv2奖励进行奖励调优后生成样本(右)的可视化对比。

8.7.2 直接奖励梯度

在本节中,我们详细讨论为何像ReFL[Xu et al., 2024]和DRaFT[Clark et al., 2023]这类直接奖励梯度方法无法高效地适配显存。以HPSv2[Wu et al., 2023a]模型为例。它采用了CLIP模型的微调版ViT-H/14变体,包含32个图像Transformer层和24个文本Transformer层,每层有16个注意力头。这总计构成6.33亿个参数。即便采用FP16数据类型,模型权重也将占用1.25 GB显存。

即使批大小为1,仅前向推理时,尺寸为p128, 3, 192, 320q的输入视频张量也将占用约6 GB显存。通过该模型进行反向传播,由于需存储梯度,将大幅增加显存开销。此外,显存占用大致随批大小线性增长,难以扩展。实践中,采用CLIP-H模型的PickScore[Kirstain et al., 2023]具有相似的显存成本。奖励模型与LRM的参数数量及显存成本比较见表8.1。若我们通过在视频中子采样来提取帧以进行奖励优化,不同帧数H下的反向显存(VRAM)成本如表8.8所示。这表明,即便进行帧子采样,显存成本仍可能过高,难以在视频模型训练中承受。

表8.8:HPSv2、PickScore奖励模型在不同数量(H)的图像(192 × 320)帧下的反向显存(VRAM)成本。模型 H “ 12 H “ 24 H “ 64 H “ 128 HPSv2/PickScore 12.373 GB 20.577 GB 48.413 GB 90 GB

鉴于扩散建模在潜空间中进行,直接奖励梯度方法还需要将梯度从奖励模型经由庞大的预训练解码器进行反向传播,这进一步增加了显存使用的负担。

8.7.3 面向不同奖励类型的潜空间奖励模型

所提出的潜空间奖励模型方法兼容前文所介绍的任意类型的奖励指标,无论其是否可微或输入格式如何。此处我们考虑几种常用的奖励指标类型:图像奖励、文本-图像奖励、视频奖励以及文本-视频奖励。针对每一类别,我们提供示例并说明LRM如何以其多样化的架构支持这些指标。该兼容性的总结见表8.9,详情如下:

* 图像奖励: $I \rightarrow R$ 。

LRM为 $R(\phi_{pq}: X \rightarrow R, x \leftarrow \text{Encode}_{pq}, i \in \mathcal{I})$ 。其图像骨干网络为二维卷积神经网络(CNN)。

表8.9:不同像素空间奖励度量对应的潜在奖励模型总结。奖励类型 LRM函数 架构 示例 图像奖励 $R(\phi_{pq}: X \tilde{N} R$ 2D CNN 骨干网络 LAION 美学质量 [Schuhmann et al., 2022], JPEG 压缩率 [Black et al., 2023] 文本-图像奖励 $R(\phi_{px, cq}: X \wedge C \tilde{N} R$ 2D CNN + 文本嵌入, 交叉注意力 HPS [Wu et al., 2023b,a], ImageReward [Xu et al., 2024], PickScore [Kirstain et al., 2023] 视频奖励 $R(\phi_{pq}: X H \tilde{N} R$ 带帧平均奖励的2D CNN,或3D CNN骨干网络 VBench 质量分数(主体一致性、运动平滑度等) 文本-视频奖励 $R(\phi_{px, cq}: X H \wedge C \tilde{N} R$ 带帧平均奖励的2D CNN,或3D CNN骨干网络,加文本嵌入、交叉注意力 ViCLIP [Wang et al., 2023e], VideoScore [He et al., 2024], InternVideo2 [Wang et al., 2024b], VBench 语义分数(物体类别、人物动作、颜色等)

示例包括 LAION 美学质量 [Schuhmann et al., 2022]、JPEG 压缩率 [Black et al., 2023]。

* 文本-图像奖励: $C \wedge I \tilde{N} R$ 。

该 LRM 为 $R(\phi_{px, cq}: X \wedge C \tilde{N} R, x \text{ “ Encode } p_i, q_i \text{ P I. 其图像骨干网络为2D CNN,输入包括文本嵌入 } ec, \text{ 并通过交叉注意力模块混合图像特征 } ex \text{ 与文本特征 } ec: \text{Softmax } p_{ex} q \text{ ” } K_{pec} q \text{ J } q \text{ ” } V_{pec} q \text{。} \text{”}$

示例包括人类偏好分数(HPS)[Wu et al., 2023b,a]、ImageReward [Xu et al., 2024]、PickScore [Kirstain et al., 2023]。

* 视频奖励: $I H \tilde{N} R$, 其中 H 为每段视频的帧数。

该 LRM 可以是:(1) $R(\phi_{pq}: X \tilde{N} R, x \text{ “ Encode } p_i, q_i \text{ P I, 使用2D CNN图像骨干网络,以帧平均奖励 } H1 H r l k \text{ “ } 1 R \phi_{pxk} q \text{ 作为视频奖励;或者(2) } R(\phi_{px1, \dots, xH} q: X H \tilde{N} R, \text{使用3D CNN作为视频骨干网络。} \text{”}$

示例包括 VBench 中的7项质量分数(主体一致性、背景一致性、运动平滑度等)。

* 文本-视频奖励: $C \wedge I \wedge H \wedge \tilde{N} R$ 。

该 LRM 可以是:(1) $Rl\phi_{px, cq}: X \wedge C \wedge \tilde{N} R$, 使用 2D CNN 图像骨干网络, 以帧平均奖励 $H1 H r l k " 1 R\phi_{pxk, cq}$ 作为视频奖励; 或者 (2) $Rl\phi_{px1, \dots, xH, cq}: X H \wedge C \wedge \tilde{N} R$, 使用 3D CNN 作为视频骨干网络, 额外输入文本嵌入 ec , 并通过交叉注意力混合图像特征 ex 与文本特征 ec : $Softmax p Q p e x q " K p e c q J q " V p e c q$ 。

示例包括 ViCLIP [Wang et al., 2023e]、VideoScore [He et al., 2024]、InternVideo2 [Wang et al., 2024b], 以及 VBench 中的 9 项语义分数度量(物体类别、人物动作、颜色等)。

架构细节。仅图像的 LRM $Rl\phi_{pxq}$ 的架构详见表 8.10。文本-图像 LRM $Rl\phi_{px, cq}$ 的架构详见表 8.11。对于视频 LRM 和文本-视频 LRM, 我们在实验中采用相同的架构, 并进行帧平均处理。

讨论。潜在奖励模型可以通过两种方式使用: 一种是预先训练, 另一种是在微调过程中与学生模型同步训练, 正如我们的实验所展示的那样。此外, 该方法也可以扩展用于对教师模型进行微调。另一种可选方式是完全绕开像素空间中的奖励模型, 从一开始就直接采用潜在空间中的奖励模型。然而, 我们认为这种做法可能仅局限于特定的固定潜在空间, 在不同模型间可能缺乏泛化能力。这是因为预训练的编码器-解码器模型可能存在很大差异, 通常并不共享统一的潜在空间, 尤其是在现有的图像和视频模型中。表 8.10: 图像潜在奖励模型的架构

层	输入形状	输出形状	卷积核大小	步幅	填充	参数数量
输入	(batch, C, H, W)					
Conv2d + GroupNorm + SiLU	(batch, C, H, W)	(batch, 128, 6, 10)	4x4	4	1	24,704
Conv2d + GroupNorm + SiLU	(batch, 128, 6, 10)	(batch, 128, 3, 5)	3x3	2	1	147,584
AdaptiveAvgPool2d	(batch, 128, 3, 5)	(batch, 128, 1, 1)				0
Conv2d	(batch, 128, 1, 1)	(batch, 128, 1, 1)				0
Flatten	(batch, 128, 1, 1)	(batch, 128)				0
Linear	(batch, 128)	(batch, 1)				129
总参数量						189,441

表8.11:文本-图像潜在奖励模型的架构

层	输入形状	输出形状	卷积核大小 / 投影 步幅 填充	参数数量
输入图像	(batch, C, H, W)			
Conv2d + GroupNorm + SiLU	(batch, C, H, W)	(batch, 128, 6, 10)	4x4 4 1	24,704
Conv2d + GroupNorm + SiLU	(batch, 128, 6, 10)	(batch, 128, 3, 5)	3x3 2 1	147,584
AdaptiveAvgPool2d	(batch, 128, 3, 5)	(batch, 128, 1, 1)	- - - 0	0
Conv2d	(batch, 128, 1, 1)	(batch, 128, 1, 1)	1x1 1 0	16,512
Flatten(图像特征)	(batch, 128, 1, 1)	(batch, 128)	- - - 0	0
输入文本	(batch, L, D)			
文本 MLP	(batch, L, D)	(batch, 256, 128)	- - - 524,544	平均池化(文本特征)
平均池化(文本特征)	(batch, 256, 128)	(batch, 128)	- - - 0	0
查询投影(线性层)	(batch, 128)	(batch, 128)	- - - 16,512	键投影(线性层)
键投影(线性层)	(batch, 128)	(batch, 128)	- - - 16,512	值投影(线性层)
值投影(线性层)	(batch, 128)	(batch, 128)	- - - 16,512	注意力机制
注意力机制	(batch, 128)	(batch, 128)	- - - 16,512	Softmax
Softmax	(batch, 1, 1)	(batch, 1, 1)	- - - 0	0
最终线性层(输出层)	(batch, 128)	(batch, 1)	- - - 129	总参数量 763,009

8.7.4 潜在奖励模型训练

图8.9和图8.10分别展示了在蒸馏过程中,采用两种原始像素空间奖励 HPSv2 和 PickScore 时,潜在奖励模型(LRM)的学习曲线。训练所用损失为 VSD+LRM。左图展示了 LRM 预测值相对于真实像素空间奖励值的 MSE 损失。右图展示了训练数据集样本 x_0^X 上,LRM 预测的奖励值 $R(\phi_{px0}, cq)$ 与真实奖励值 R_{px0}, cq 的对比。这表明,即使在维度显著更低的潜在空间中运行,LRM 也能在2000至3000次训练迭代内实现快速收敛。较小的近似误差保证了使用学习到的 LRM 进行微调的有效性。

图8.9:采用 HPSv2 奖励的 LRM 学习过程。

图8.10:采用 PickScore 奖励的 LRM 学习过程。

8.7.5 潜在奖励模型微调

图8.11展示了在采用 VSD+LRM 损失进行蒸馏的过程中,LRM 对生成样本($x_0 \in \mathcal{X}$,由式(8.15)给出)预测的奖励值 $R(\phi_{\theta}^{\pi}(x_0))$,分别针对 HPSv2 和 PickScore 两种奖励度量。图中水平虚线为训练数据集中样本的平均奖励值。对于 HPSv2,生成样本的奖励值随着 LRM 微调迅速超过训练数据。对于 PickScore,生成样本的奖励值也逐渐提升,接近训练数据的水平。

图8.11:在 HPSv2 和 PickScore 奖励度量下的潜在奖励模型微调过程。

8.7.6 去噪扩散策略优化

去噪扩散策略优化(Denoising Diffusion Policy Optimization,DDPO)[Black et al., 2023] 作为与我们所提出的 LRM 方法进行对比的基线方法。DDPO 将扩散过程视为一个马尔可夫决策过程(MDP),并应用 REINFORCE 算法对扩散模型进行优化。该方法需要在所有扩散步骤上估计样本的对数概率,随后对其求和,并以最终奖励作为权重构成优化目标。考虑到显存限制,我们的方法更适用于少步采样模型或沿扩散轨迹采用梯度截断的配置。在我们的实验中,显存限制使得无法对超过2个步骤进行对数概率估计。因此,我们对学生模型采用截断步数为2(即在时间步 [249, 499] 处进行对数概率估计)。该截断方法已在此前的工作中得到验证 [Clark et al., 2023; Ren et al., 2024]。在实验中,我们采用 DDPOSF 进行在线策略梯度优化。

通过在扩散模型的去噪过程上应用 REINFORCE 算法,DDPOSF 算法遵循如下的评分函数策略梯度:

T

$$\nabla_{\theta} J = \mathbb{E}_r \nabla_{\theta} \log p_{\theta}(x_{t+1} | x_t, c_q R_{x_0}, c_q s) \quad (8.24)$$

t “ 1

这是梯度估计的在线版本,需要在采样过程的同时采样 x_{t+1} 并计算概率 $p_{\theta}(x_{t+1} | x_t, c_q)$,从而保证模型参数 θ 在采样和概率估计过程中保持一致。为保持在

$$\sigma^2$$

DDPOSF 的实际操作流程详见算法30。由于显存限制,我们采用带截断的 REINFORCE 策略梯度,训练时最多允许对 $N \times 2$ 步进行梯度追踪。具体而言,对于采样时间序列 $t_r, \dots, t_{min}$ s “r999, 749, 499, 249s 的学生模型,梯度更新步骤仅取最后两步 $t_{n-1} \times 499, 249u$,而非全部时间步。该截断用于估计样本在 t_{n-1} 处的对数概率。这里, $Decp^q$ 表示预训练的视频解码器,而奖励模型 R 在原始像素空间中运行。我们使用 $.detachpq$ 表示停止梯度函数。

算法30 DDPO 实际操作流程 1: 输入:蒸馏得到的学生模型 G_θ , 数据集 D “tpc, iqu

```

2: 输出:微调后的学生少步生成器 G $\theta$ 
3: while 训练中 do
4:   //沿整条扩散轨迹从随机噪声开始采样
   x $\leftarrow$   $\mathcal{D}(\epsilon, I, q)$ 
   for t $\in$  {1, 2, ..., T} do
       7: 用  $v_{\theta}(x_t)$ ,  $\sigma_t$  得到后验高斯分布  $p_{\theta}(\epsilon_t | x_t)$ ,  $\sigma_t$  //式(8.25)
       8: 采样  $\epsilon_t \sim p_{\theta}(\epsilon_t | x_t)$ 
       9: 估计  $\log p_{\theta}(x_{t-1} | x_t, \epsilon_t)$  //式(8.26)
10: 得到奖励 R “  $R_t = -\log p_{\theta}(x_{t-1} | x_t, \epsilon_t)$  ” //带截断的 REINFORCE 策略梯度
12: LDDPOSF “  $\nabla_{\theta} \sum_{t=1}^T \log p_{\theta}(x_{t-1} | x_t, \epsilon_t) R_t$  ”
13:  $G_{\theta} \leftarrow \text{GradientDescent}_{\theta}$ , LDDPOSF  $q$ 

```

学习曲线。图8.12展示了针对两种奖励度量,VSD+DDPO 的训练过程。该学习曲线展示了生成样本 $\hat{x}_0$ 通过沿完整扩散轨迹迭代去噪所得到的奖励值 R_{px0} , cq ,

在微调过程中的变化情况。

图8.12:在 HPSv2 和 PickScore 奖励模型下,采用 VSD+DDPO 进行奖励模型微调的过程。

DDPO 的学习曲线与图8.11所示的 LRM 方法的学习曲线不能直接比较。这一差异源于 DDPO 需要沿整条扩散轨迹采样以获得用于奖励评估的预测值 $\hat{x}_0$,而 LRM 则采用一步预测,即?

$$x_t \sim \mathcal{N}(\mu_t, \sigma_t^2)$$

$$x_0 \sim \mathcal{N}(\mu_0, \sigma_0^2)$$

$$\bar{x}_t \sim \mathcal{N}(\mu_t, \sigma_t^2)$$

,如式(8.15)所定义。因此,LRM 的样本在微调过程中往往噪声更大,所得到的奖励值也更低。若要进行公平比较,则应在模型微调完成后,对最终生成样本的奖励值进行评估,如本文正文表8.5所示。

第9章

视频世界模型

本节内容基于论文《学习面向交互式视频生成的世界模型》("Learning World Models for Interactive Video Generation")[Chen et al., 2025b],该论文与 Taiye Chen、Xun Hu 和 Chi Jin 合作完成,此前已发表于 NeurIPS 2025。

9.1 引言

能够基于不同动作模拟未来结果的基础世界模型,对于有效的规划与决策至关重要 [Watter et al., 2015; Ha and Schmidhuber, 2018a; Hafner et al., 2020]。为实现这一目标,这类模型必须同时具备交互性(即支持动作条件化)以及长时程的时空一致性。尽管近期视频生成方面的进展,特别是扩散模型 [Sohl-Dickstein 图9.1:世界模型具备记忆能力,通过对环境保持感知并基于当前状态和动作生成预测,从而实现可靠的长期未来预测。图中示例来自 Minecraft 游戏。 et al., 2015; Song and Ermon, 2019; Ho et al., 2020; Song et al., 2021b],已展现出良好前景,但将其扩展到生成长时程、交互式且一致的视频仍是一项重大挑战 [Brooks et al., 2024;

Bruce et al., 2024; Valevski et al., 2024]。自回归方法 [Weissenborn et al., 2019; Harvey et al., 2022; Li et al., 2024e; Xie et al., 2024a] 基于先前的输出逐帧或逐块生成视频,天然适合建模长时程依赖关系并融入交互性。然而,这类方法面临着源自两个基本且往往相互耦合的局限性的重大挑战:误差累积(compounding errors)以及记忆机制不足。误差累积产生的原因在于,早期预测中的微小误差会随时间不断累积,导致与合理的未来状态发生显著偏离。我们的分析表明,这可能是当前自回归范式的固有问题。记忆机制不足则会妨碍模型在较长时间跨度内维持一致的物体身份、空间布局和世界状态的能力,从而导致世界模型的不一致性。这两个问题往往相互加剧,使得长期一致的生成变得困难。

受大语言模型(LLM)[Achiam et al., 2023][Touvron et al., 2023] 在处理长序列方面取得成功的启发,我们探究了将类似技术应用于视频生成的方法。扩展上下文窗口虽然在一定程度上可能缓解误差累积问题,但会带来巨大的计算和内存开销。更为关键的是,我们发现与 LLM 不同,当前的视频生成模型表现出较弱的上下文学习能力,这使得更长的上下文在解决根本性一致性问题方面效果有限。同样,检索增强生成(RAG)[Gao et al., 2023][Zhao et al., 2024] 作为一种在 LLM 中融入外部知识的强大技术,在我们对视频模型的实验中收益有限。无论是采用启发式采样的静态检索,还是基于相似度搜索的动态检索,都未能显著提升世界模型的一致性。

这些发现表明,仅依靠对像素或潜在表示进行自回归预测来隐式学习世界一致性是不够的。我们认为,明确的全局状态条件化是必要的。将世界

地图、物体状态或坐标系统等显式表示作为条件信息,可以为生成一致的长期交互式模拟提供必要的支撑。

此外,评估长视频生成的特定失败模式需要恰当的度量方法。现有度量往往将误差累积和长期一致性(记忆保真度)这两个截然不同的问题混为一谈,给出的是一种耦合的评估结果,掩盖了根本问题。为实现更清晰的分析,我们提倡并引入一种解耦评估策略,分别量化长期交互式视频生成中误差累积的严重程度以及记忆检索的保真度。

我们的主要贡献如下:(1)我们系统性地解耦并分析了自回归视频生成在交互式世界建模中所面临的误差累积和记忆不足的挑战。(2)我们提出了具有显式全局状态条件化的视频检索增强生成(VRAG)方法,该方法显著提升了交互式视频生成的长期时空一致性并减少了误差累积。(3)我们对从 LLM 技术改编而来的各种长上下文方法进行了全面比较,包括位置插值、神经记忆增强和历史帧检索,结果表明由于视频扩散模型固有的弱上下文学习能力,这些方法的效果有限。这项工作揭示了构建一致的交互式视频世界模型所面临的根本性障碍,并为该方向未来的研究提供了基准和评估框架。

9.2 相关工作

视频扩散模型 扩散生成建模显著推动了图像和视频生成领域的发展 [Blattmann et al., 2023b; Harvey et al., 2022; Esser et al., 2023; Blattmann et al., 2023a; Chen et al., 2024b; Ho et al., 2022; Singer et al., 2022b; Hong et al., 2022; Yang et al., 2024; Wang et al., 2023b; Ding and Jin, 2024; Zheng et al., 2024]。潜在视频扩散模型 [Blattmann et al., 2023a] 在由变分自编码器(VAE)[Kingma, 2013] 得到的潜在空间中的视频标记(token)上进行操作,

建立在先前潜在图像扩散模型工作的基础之上 [Rombach et al., 2022b]。Diffusion Transformer (DiT) [Peebles and Xie, 2023] 引入了 Transformer [Vaswani et al., 2017] 骨干网络,作为扩散模型中此前普遍使用的 U-Net 架构 [Ho et al., 2022; Blattmann et al., 2023a; Chen et al., 2024b] 的替代方案。

长视频生成 自回归视频生成 [Weissenborn et al., 2019; Harvey et al., 2022; Li et al., 2024e; Xie et al., 2024a; Hong et al., 2024; Wu et al., 2024; Kim et al., 2024; Feng et al., 2024; Sand-AI, 2025; Henschel et al., 2024] 通过基于前序帧进行条件生成,代表了长视频合成的一种自然方法,其灵感来源于大语言模型的成功经验。这可以通过掩码条件视频扩散 [Voleti et al., 2022; Hong et al., 2024] 或 Diffusion Forcing [Chen et al., 2024a] 等技术实现。Diffusion Forcing 为每一帧引入不同程度的随机噪声,以便在推理时基于帧进行自回归生成。此外,自回归框架通过允许在每一步引入动作输入来影响未来预测,天然地支持交互式世界模拟。然而,误差累积仍是长视频生成中的一个重大挑战,尤其是在自回归范式中,这一点将在后文讨论。

交互式视频世界模型 世界模型 [Watter et al., 2015; Ha and Schmidhuber, 2018a; Hafner et al., 2020] 是一类模拟系统,旨在根据当前状态和所选动作预测未来的轨迹。基于扩散的世界模型 [Ding et al., 2024; Alonso et al., 2024; Valevski et al., 2024] 有助于对高维分布进行建模,从而能够在像素空间中直接实现对多样化轨迹的高保真预测。Sora 模型 [Brooks et al., 2024] 提出了将视频生成模型用作世界模拟器的概念。为视频生成模型扩展交互能力,已在多个领域催生出富有前景的应用,包括游戏模拟,如 Genie [Bruce et al., 2024]、GameNGen [Valevski et al., 2024]、Oasis [Decart and Etched, 2024]、Gamegen-x [Che et al., 2024]、The Matrix [Feng et al., 2024]、Mineworld [Guo et al., 2025]、GameFactory [Yu et al., 2025b] 等 [Alonso et al., 2024],以及自动驾驶 [Hu et al., 2023a]、机器人操作 [Wu et al., 2024];

Azzolini et al., 2025],以及导航 [Bar et al., 2024]。尽管现有的交互式视频世界模型工作在工程层面已取得显著进展,但在系统性和解决长期一致性与误差累积这一根本性挑战方面,仍存在明显的空白。时空一致性的缺失是当前视频生成技术在构建内部世界模型时面临的主要瓶颈。其中一条研究路线致力于预测底层的三维世界结构,如 Genie2 [Parker-Holder et al., 2024]、Aether [Team et al., 2025]、Gen3C [Ren et al., 2025b] 以及其他工作 [Liu et al., 2024; Gao et al., 2024; Zhen et al., 2025];然而,由于三维表示的复杂性,这些方法往往相较于直接视频生成存在分辨率较低的问题,交互能力也较为有限,并且通常只能在局部区域内运行。因此,我们的工作致力于增强基于视频的世界模型 [Valevski et al., 2024; Hong et al., 2024; Xiao et al., 2025a] 的一致性。SlowFast-VGen [Hong et al., 2024] 采用双速学习系统,通过逐步训练的 LoRA 模块实现记忆回溯,利用语义动作,但交互性有限。同期工作 [Xiao et al., 2025a] 通过整合补充性记忆模块来探索交互式世界模拟。

9.3 方法论

9.3.1 预备知识:潜在视频扩散模型

视频扩散模型已成为视频生成的一种强大框架。我们采用一种潜在视频扩散模型 [Blattmann et al., 2023a],它在压缩的潜在空间而非像素空间中运行,以提

a

$$\mathbf{q}_{\text{pt}}|\mathbf{z}_t\sim\mathcal{Q}(\cdot|\mathbf{z}_t,\mathbf{p}_{\text{pt}}); \quad \mathbf{1}^T\beta_t\mathbf{z}_t\sim\mathcal{1}, \beta_t\mathbf{l}_q \quad (9.1)$$

模型通过在每一步预测噪声 ϵ_θ 来学习逆转这一过程:

$$L = \mathbb{E}_{t, \epsilon, z} \|\epsilon - \epsilon_\theta(p_{zt}, t)\|_2^2 \quad (9.2)$$

其中 $z_t \sim \mathcal{N}(z_0, \frac{1}{\beta_t} \sigma_\epsilon^2)$, $p_{zt}, t \sim \mathcal{N}(p_0, I_q)$.

在推理阶段,我们可以从随机噪声 $z_T \sim \mathcal{N}(p_0, I_q)$ 出发,通过迭代去噪来采样生成新视频:

1 β_t

$$z_t \sim \mathcal{N}(p_{zt}, \frac{1}{\beta_t} \sigma_\epsilon^2) \quad (9.3)$$

$\alpha_t \sim \mathcal{N}(z_t, \frac{1}{\beta_t} \sigma_\epsilon^2)$

其中 $\alpha_t \sim \mathcal{N}(z_t, \frac{1}{\beta_t} \sigma_\epsilon^2)$, $\alpha_t \sim \mathcal{N}(z_t, \frac{1}{\beta_t} \sigma_\epsilon^2)$ 。最终的潜在序列 z_0 通过解码器 D 解码回像素空间,从而得到生成的视频。

图 9.2:我们用于交互式视频生成的 VRAG 框架概览。该框架结合了全局状态条件设定与记忆检索机制,以确保时空一致性并缓解误差累积。在训练和推理过程中,检索到的记忆均作为时空 DiT 模块中联合自注意力的上下文。该模型在训练时采用逐帧噪声注入,以便在推理时实现自回归采样。

9.3.2 交互式长视频生成

为实现基于动作序列条件设定的交互式长视频生成,我们对基础扩散模型增加了两项技术:(1) 附加的动作条件设定,采用自适应层归一化(AdaLN),以及

(2) 用于自回归建模的随机帧噪声,如图 9.2 所示。

动作条件设定 为实现基于动作序列条件设定的交互式视频生成,我们通过自适应层归一化(AdaLN)对基础扩散模型进行增强。给定动作序列 $a \in \mathbb{R}^L$ (其中 L 为动作序列长度),我们采用自回归方法,依次生成各帧。在每一步,我们以固定长度的上下文窗口 L_c (由先前生成的帧组成)为条件。然而,采用教师强制(teacher forcing)的朴素自回归生成会遭受较大的误差累积问题,即误差会随时间不断累积。我们在训练中应用 Diffusion Forcing [Chen et al., 2024a] 技术。

具体而言,在训练过程中,我们根据扩散调度,对整个输入视频序列中的每一帧随机添加噪声: $z_t = \alpha_t z_0 + \sqrt{1 - \alpha_t} \epsilon_t$, $\epsilon_t \sim \mathcal{N}(0, I_d)$, 其中 z_t 表示第 t 帧的加噪潜变量。

噪声预测模型 ϵ_θ 同时以动作序列 a 和加噪帧 z_t 为条件。

其中 r_t 是一个由 L 个时间步组成的向量,每帧具有不同的 $t \in [1, L]$ 。噪声预测模型 ϵ_θ 同时以动作序列 a 和加噪帧 z_t 为条件。

架构 我们采用扩散 Transformer(DiT)进行视频生成建模。我们采用具有独立空间和时间注意力模块的时空 DiT 模块。旋转位置编码(RoPE)[Su et al., 2024] 应用于这两个注意力模块,时间注意力则采用因果掩码实现。

9.3.3 具有全局状态的检索增强视频世界模型

尽管第 9.3.2 节中的基础模型为交互式视频生成提供了基础,但它缺乏用于维持长期一致性和世界模型连贯性的稳健机制。为解决这些局限,我们借鉴大语言模型的思路,整合了记忆检索与上下文增强机制,并纳入了针对视频的特定方法,如历史帧缓冲区 and 全局状态条件设定。这些增强机制通过为模型提供更好的历史上下文访问能力和空间感知能力,使自回归视频生成更加一致和连贯。

全局状态条件设定 为增强视频生成中的空间一致性,我们引入全局状态信息----具体而言是角色当前的坐标和姿态----作为额外的条件信号。全局状态向量 s 视频检索增强生成(VRAG) 除全局状态条件设定之外,我们还提出了记忆检索增强生成,以增强模型利用历史上下文的能力,同时保持时间连贯性,即视频检索

增强生成(VRAG)。对于 VRAG,我们将拼接后的历史帧和当前帧与其对应的动作序列 $\tilde{a} \in \mathcal{P}(\mathcal{R})^A$ 及全局状态序列 $\tilde{s} \in \mathcal{rshist}, \mathcal{ss} \in \mathcal{P}(\mathcal{R})^S$ 结合起来,作为模型输入 $\tilde{x} = [\tilde{s}, \tilde{a}, s_t]$ (9.5)

其中 fsim 是历史帧与待预测的最后一帧 s_{t-1} 之间的距离度量(例如欧几里得距离), $w \in \mathcal{P}(\mathcal{R})^S$ 是一个权重向量,用于调节不同状态分量的重要性。选取并排序前 L_h 个最相似的历史状态和帧,构成检索到的上下文。与大语言模型中利用强大的上下文学习能力的 RAG 不同,视频扩散模型表现出较弱的上下文学习能力,这使得直接以历史帧作为上下文进行推理效果不佳,这一点将在后文实验中加以证明。为解决这一局限,我们提出了对标准 RAG 方法进行关键改进的 VRAG 训练方案,从而实现有效的记忆增强视频生成。

在训练过程中,我们检索历史帧 $zhist \in \mathcal{P}(\mathcal{R})^{L_h}$,并将其与当前上下文窗口 $z \in \mathcal{P}(\mathcal{R})^{L_c}$ 拼接,形成扩展上下文 $\tilde{z} = [zhist, z]$ 。为使 VRAG 有效,我们做出了若

对于检索帧,我们仅以其全局状态 $\text{shist} \in \mathbb{R}^{L_h \times S}$ 为条件,而对动作条件 $\text{ahist} \in \mathbb{R}^{L_h \times A}$ 进行掩码处理,以避免动作序列中出现时间不连续性。这种选择性条

$$\text{LVRAG} = \text{Ert} \cdot \text{rt} \cdot \text{s} \cdot \epsilon \cdot \tilde{z} \cdot \text{a} \cdot \text{s} \cdot \text{r} \cdot \text{et} \cdot \epsilon \cdot \theta \cdot \text{p} \cdot \tilde{z} \cdot \tilde{t} \cdot \tilde{a} \cdot \tilde{s} \cdot \text{q} \cdot 22 \cdot \text{d} \cdot \text{ms}, \quad (9.6)$$

$$\tilde{z} \cdot \tilde{t} = \text{rzhist} \cdot \text{rt} \cdot \text{s} \cdot \text{zrts} \cdot \tilde{a} = \text{r} \odot \text{Lh} \cdot \text{as} \cdot \tilde{s} = \text{rshist} \cdot \text{ss} \cdot \text{m} = \text{r0Lh} \cdot \text{1Lc} \cdot \text{s}, \quad (9.7)$$

其中 $\tilde{t}$ 是 $\text{rt} \cdot \text{s}$ 和 rts 的拼接,且 $\text{t} \cdot \tilde{a} = \text{t} \cdot \text{t} \cdot \text{t} \cdot \text{PrT} \cdot \text{s}$ 。

9.3.4 长上下文扩展基线方法

为探究大语言模型中已有的长上下文扩展技术是否能够有效增强视频生成模型,我们基于第 9.3.2 节中的基础模型,设计了三种互补方法,分别利用显式帧上下文或神经记忆隐藏状态。这些方法作为我们主要方法的基线对比,专门用于评估模型在长视频生成中维持空间连贯性和时间一致性的能力。通过这些基线方法,我们旨在验证视频扩散模型的上下文学习能力,并评估其在处理扩展序列时的有效性。

长上下文增强 我们使用 YaRN [Peng et al., 2023] 对时间注意力中的 RoPE 进行修改,以扩展时间上下文窗口。RoPE 通过复数值旋转对相对位置进行编码,其中查询 q_m 与键 k_n 之间的内积取决于相对距离 $p_m - n_q$ 。YaRN 通过对旋转位置编码施加频率变换来扩展上下文窗口。这种变换以一种保留相对位置信息的方式缩放旋转角度,同时使模型在对较长视频片段进行小规模微调后,能够处理更长的视频序列。

历史缓冲区帧检索 我们实现了一个固定长度的缓冲区 B , 用于存储历史潜在帧, 并采用启发式采样策略。该缓冲区被划分为 NS “5 个呈指数递减的分段 G_j , 其中 $L_j = L_1 \cdot \alpha^{j-1}$ 。我们从每个分段 G_j 中采样 k 帧, 构成子集 F_j 。检索到的记忆 $z_{mem} = \{r_{f1}, \dots, r_{fNS}\}$ 与当前帧窗口 z 拼接, 作为额外上下文: $z^* = [z; z_{mem}]$, 随后输入时空 DiT 模块。这种设计确保了对近期帧具有更高的采样密度, 在强调近期视觉信息的同时, 仍保留对历史上下文的访问以维持时间一致性。

神经记忆增强注意力 与上述两种方法使用显式帧作为上下文不同, 我们探索了一种神经记忆机制, 用于存储和检索隐藏状态。这一方法受到大语言模型中 In-Context Learning (ICL) 的启发, 我们使用 q_{ns}^i 和 k_{ns}^i 来检索历史帧的隐藏状态 h_{ns}^i 。随后更新记忆 M_{ns} 和归一化向量 n_{ns} 。最终的注意力输出通过可学习的门控机制, 结合检索到的隐藏状态 A_{mem} 与标准注意力, 以在长视频序列中保持长期依赖。

帧打包 (Frame Pack) 作为另一个基线方法, 我们参照 Frame Pack [Zhang and Agrawala, 2024], 将历史帧压缩为上下文。采用三种不同核大小的输入压缩核---(2, 4, 4)、(4, 8, 8) 和 (8, 16, 16)---将历史帧压缩为固定长度的上下文。该方法本质上是通过带有近期偏好的重要性采样实现帧压缩, 从而在保持较低计算成本的同时获得更大的视野范围。然而, 对最近帧的优先处理在许多情况下对于长视频生成而言可能并非最优, 尤其是在考虑记忆问题时。我们基于帧相关性的 VRAG 在理论上提供了更优的历史信息检索能力。此外, 我们的方法实际上与

Frame Pack 中的帧压缩技术是正交的。我们将两种方法的结合留作未来工作。上述方法的更多细节可见于补充材料。

9.4 实验

图 9.3:VRAG 在世界连贯性评估中与真实视频(ground truth)的视觉对比。以 100 帧初始帧作为历史缓冲区,VRAG 预测后续 200 帧。

9.4.1 数据集与评估协议

在训练阶段,我们使用 MineRL [Guss et al., 2019] 收集了 1000 段 Minecraft 长时程游戏视频(总计 17 小时)。所有视频的分辨率均固定为 640×360 像素。每段序列跨越 1200 帧,并标注有动作向量(前进/后退移动、跳跃、摄像机旋转)和世界坐标(x、y、z 位置及偏航角)。在评估阶段,我们构建了两个不同的测试集:(1) 用于误差累积评估,我们使用 20 段带有随机动作和位置的 1200 帧长视频;(2) 用于世界连贯性评估,我们使用 60 段精心策划的 300 帧视频序列,专门设计用于

以系统性评估时空一致性。这些精心构建的序列包含受控的运动模式,包括原地旋转、方向反转和循环轨迹跟随。每个序列的前100帧作为需要缓冲帧的方法的初始化缓冲区,或对于其他方法则不参与评估。每个模型以步幅1自回归地生成下一单帧,直至达到所需长度。

我们使用若干指标将模型与真实测试集进行对比评估:结构相似性指数(SSIM)[Wang et al., 2004]用于衡量空间一致性,峰值信噪比(PSNR)用于评估像素级重建质量,学习感知图像块相似度(LPIPS)[Zhang et al., 2018]用于评估感知相似性。对于累积误差评估,我们发现SSIM能更准确地反映长序列中帧的忠实度。

9.4.2 训练细节

为公平比较,模型训练和评估均采用一致的20帧窗口大小。对于原始Diffusion Forcing,我们额外训练了一个窗口大小为10帧的变体,用于上下文长度评估。对于我们的VRAG方法,图9.4:不同方法的可视化比较,用于世界一致性分析评估。我们在训练和推理时将10个检索到的帧与10个当前帧相结合。我们使用全局状态向量 $s = [r_x, y, z, \text{yaw}]$ 在训练期间表示智能体的状态,该向量在需要时可扩展为完整的三维姿态表示。为便于训练收敛,这些数值

相对于初始状态进行归一化,从而降低扩散过程的复杂度。YaRN实现方式是在原始模型(窗口大小20)的基础上,将位置编码替换为YaRN,并采用拉伸因子4,随后在80帧序列上微调104步。在评估YaRN时,我们使用40帧窗口。带神经记忆的Infini-attention采用滑动窗口大小20、步幅10,使用前10帧进行记忆状态更新,后10帧进行局部注意力计算。History Buffer方法维护一个124帧的缓冲区,划分为5个呈指数递减的分段($L1 = 2, \alpha = 2$),每个分段采样2帧,构成与10个当前帧拼接的10个历史帧。所有模型均在数据集上训练3个epoch,批大小为32,使用8块A100 GPU。

9.4.3 世界一致性结果

我们通过评估给定初始帧和动作序列所生成的预测视频,来研究内部世界模型的时空一致性。如图9.4所示,我们的VRAG提供了一种有效方法,可增强模型利用历史上下文以提升世界一致性的能力。图9.3展示了VRAG与真实视频的更多可视化比较。我们使用多个指标评估不同方法的世界一致性。图9.5展示了随时间变化的SSIM分数,而表9.1呈现了所有指标的综合比较。我们的VRAG方法在所有指标上均取得最佳性能,表明其在生成视频中保持世界一致性方面具有卓越能力。我们的实验结果表明,将基线DF模型的窗口大小从10扩展到20帧可提升世界一致性,说明更长的上下文窗口能增强一致性。然而,使用YaRN进一步扩展上下文并未在原始DF模型基础上显示出改进。这表明YaRN的上下文扩展能力虽然在语言模型中有效,但并不能有效迁移到视频生成中以维持世界一致性。类似地,History Buffer方法在没有显式的上下文内训练的情况下,也无法有效利用历史帧来实现时空一致性。YaRN和

History Buffer两种方法的这些发现表明,当前规模的视频扩散模型具备的上下文内学习能力有限,使其无法有效利用历史帧来维持长期一致性。Neural Memory方法由于模型训练的不稳定性而表现不佳。

方法 SSIM ↓ PSNR ↓ LPIPS ↓ DF(窗口10) 0.455 16.161 0.509 DF(窗口20) 0.466 16.643 0.538 YaRN 0.462 16.567 0.532 History Buffer 0.459 16.922 0.543 Frame Pack 0.421 16.372 0.574 VRAG 0.506 17.097 0.506

表9.1:不同方法在世界一致性上的定量比较, 图9.5:不同方法在世界一致性评估中随时间变化的SSIM分数。评估基于300帧的视频。

9.4.4 累积误差结果

图9.6:不同方法在长期视频预测(1200帧)上的可视化比较,用于累积误差分析评估。

我们使用SSIM指标评估不同方法在长视频生成中的累积误差。如图9.7和表9.2所示,我们的VRAG方法取得了优异性能,SSIM分数为0.349,相较于基线方法展现出更好的结构相似性保持能力。将DF的窗口大小从10增加到20帧可提升SSIM,说明更长的上下文有助于缓解累积误差。然而,这一改进仍不及VRAG的性能,这表明我们的检索增强方法能提供更有效的长期一致性。如图9.6所示,我们的VRAG方法在长序列上生成了更连贯、更一致的帧,而其他方法则表现出明显的伪影和不一致。History Buffer方法表现不佳,SSIM分数仅为0.188,说明缺乏有效上下文内训练的朴素历史帧检索无法维持长期一致性。鉴于Neural Memory方法在世界一致性实验中的表现有限(第9.4.3节),我们在此更长视频预测的可视化中将其排除。

方法 SSIM DF(窗口10) 0.297 DF(窗口20) 0.321 YaRN 0.316 History Buffer 0.188 Neural Memory 0.283 VRAG 0.349

表9.2:累积误差评估中所有帧的平均SSIM分数 图9.7:累积误差评估中随时间变化的SSIM分数

9.4.5 VBench评估

我们使用VBench [Huang et al., 2023a]中的五项视频质量指标,对第9.4.4节中生成的长视频进行评估。VBench的评估结果(数值越高越好)见表10.3。结果表明,我们的方法在时间质量和视频帧质量两方面的所有指标上均优于所有基线方法。Neural Memory基线的美学质量(Aesthetic Quality)为0.343,成像质量(Imaging Quality)

分数分别为0.3597,明显低于其他基线方法,因此未在此列出。我们的VRAG相较于所有基线方法展现出更好的时间一致性,且较高的视频帧质量表明结果并未出现过度平滑。

9.4.6 扩展:真实世界场景

图9.8:在RealEstate10K数据集上的可视化视频帧。红色方框表示真实帧(ground-truth frames)。

Background	Temporal	Motion	Aesthetic	Imaging	方法	指标	DFoT	VRAG	Consistency	Flickering	Smoothness	Quality	Quality	SSIM																					
0.4436	0.9116	DF20	0.9668	0.9485	0.9582	0.5272	0.6058	PSNR	13.03	32.21	YaRN	0.9686	0.9401	0.9523	0.5252	0.6323	LPIPS	0.4469	0.1146	History	Buffer	0.9664	0.9475	0.9579	0.5167	0.6253	FVD	337.5	221	VRAG	0.9686	0.9511	0.9603	0.5295	0.6444

表9.3:VBench中五项视频质量指标的评估结果。 表9.4:在RealEstate10K数据集上的 定量比较。

我们在Minecraft模拟之外的真实世界场景中进行了额外实验,以展示我们方法的泛化能力。具体而言,遵循Diffusion Forcing Transformer(DFoT)[Song et al., 2025]的实验设置,我们的VRAG模型由预训练的DFoT初始化,并在RealEstate10K数据集[Zhou et al., 2018]上,结合第9.3.3节所述的额外检索历史上下文进行微调。

仅经过2个epoch(原始训练步数的10%)的微调后,我们的方法在记忆能力方面显著优于DFoT基线。可视化帧展示于图9.8,定量结果汇总于表9.4。这些结果有效证明了我们方法在Minecraft之外的泛化能力,可用于解决长视频预测中的记忆问题。

9.4.7 消融实验:VRAG的记忆与训练

方法 SSIM ↓ PSNR ↓ LPIPS ↑ VRAG 0.506 17.097 0.506 VRAG(无训练) 0.455 16.670 0.528 VRAG(无记忆) 0.436 16.372 0.547

表9.5:VRAG各组件的消融实验研究。我们将完整模型与移除记忆组件(仅保留 图9.9:VRAG各变体随时间变化的 额外全局状态条件)或训练组件(仅进行上下文内学习) SSIM分数比较。 的变体进行比较。

我们对VRAG方法的关键设计进行了消融实验,包括记忆组件和训练组件。消融结果见图9.9和表9.5。我们将完整的VRAG模型与两个变体进行比较:(1)不含记忆组件的VRAG,仅使用额外的全局状态条件;(2)不含训练组件的VRAG,即在推理阶段使用检索增强生成进行上下文内学习的原始模型。消融研究在世界一致性评估数据集上进行。

消融结果揭示了关于VRAG组件的若干关键见解。首先,移除记忆组件导致所有指标上性能下降幅度最大,SSIM下降13.8%,LPIPS上升8.1%。这表明记忆机制对于维持时空一致性和质量至关重要。其次,移除训练组件同样导致显著性能下降,SSIM下降10.1%,LPIPS上升4.3%,这凸显了当前视频模型在上下文内学习方面的能力较弱。

完整的VRAG模型在所有指标上均取得最佳性能,表明这两个组件能够协同作用,共同提升视频生成质量。

9.5 基线方法细节

针对第9.3.4节中的基线方法,我们实施了以下技术来扩展视频生成模型的时间上下文窗口。

长上下文增强 为扩展视频生成模型的时间上下文窗口,我们在时间注意力模块中应用了YaRN [Peng et al., 2023]对RoPE的改进,以提升外推能力。RoPE通过复值旋转对相对位置进行编码,使得第 m 个查询 q_m 与第 n 个键 k_n 之间的内积仅取决于相对距离 $p_m - n_q$:

$$\langle q_m, k_n \rangle = \langle fW_q p_{zm}, fW_k p_{zn} \rangle \quad (9.8)$$

、~

$$= \text{Re} \langle pW_q z_m, pW_k z_n \rangle \quad (9.9)$$

、~

$$= \text{Re} \langle pW_q z_m, qW_k z_n \rangle \quad (9.10)$$

$$\langle p_{zm}, z_n \rangle \quad (9.11)$$

其中 $\text{Re}\{s\}$ 表示复数的实部, $\bar{p}$ 表示复数的共轭, $z_m, z_n \in \mathbb{R}^D$ 为输入向量, W_q, W_k 为可学习的投影矩阵, $\theta \in \mathbb{R}^D$ 编码每个维度的旋转频率: $\theta_d \in [0, 2\pi D]$,其中

YaRN通过应用频率变换来修改旋转后的输入向量 $fW p_{zm}, m, \theta_d$:

$$fW p_{zm}, m, \theta_d = fW p_{zm}, gpmq, hp\theta_d q \quad (9.12)$$

其中 $\mathbf{gpmq} = \mathbf{m}$, 频率变换函数为:

$$\theta d$$
$$hp\theta d q = p1 \cdot \gamma prd q q \cdot \gamma prd q \cdot \theta d \quad (9.13)$$

s

此处,s为拉伸因子, $rd = Lc / \lambda d$ 为上下文长度与波长之比,其中 λd

D

$2\pi(\theta d + 2\pi pb1 q2d(D, b1 = bs D)^2$ 。斜坡函数 $\gamma p \cdot q$ 对低频维度进行插值以提升外推能力,同时保留高频成分。

历史缓冲区帧检索 我们还实验了一种固定长度缓冲区B,用于存储先前生成的潜变量帧的历史,并采用启发式采样策略进行检索。遵循[Jiang et al., 2024]的方法,该策略将B划分为NS = 5个分段 $G_j(j = 1, \dots, NS)$,按从最旧(G1)到最新(GNS)排序。缓冲区中的总帧数为 $NB = N \cdot rS_j = 1/|G_j|$ 。这些分段的长度 $L_j = |G_j|$ 呈指数递减(例如, $L_j = L1 \cdot \alpha^{j-1}$,其中 $\alpha > 1$ 为基数, $L1$ 为最旧分段G1的长度),确保较新分段更短。从每个分段 G_j 中随机采样k帧,构成子集 $F_j \subseteq G_j$ (其中 $|F_j| = k$)。检索到的记忆zmem由这些采样帧拼接而成, $zmem = [F1, F2, \dots, FNS]$,总计 $NS \cdot k$ 帧。这种带有近期偏好的设计意味着采样密度 k/L_j 在较新分段中更高,从而更强调近期信息。检索到的信息zmem沿时间维度与当前帧窗口z拼接,作为额外上下文: $z' = [zmem, z]$,随后作为输入传递给时空DiT模块,使模型能够同时关注近期帧和历史帧。

神经记忆增强 为了在保留过去场景记忆的同时,将视频生成能力扩展到超出固定注意力窗口的更长序列,我们将Infini-attention [Munkhdalai et al., 2024]作为一种神经记忆机制,改造后应用于我们的视频扩散模型。Infini-attention是一种循环机制,它在标准点积注意力(局部上下文)的基础上,增加了存储在不断演化的记忆中的过去上下文的压缩摘要(全局上下文)。

该模型使用滑动窗口按分段处理视频。为保持对视频生成至关重要的高度时间连续性,我们采用了重叠分段。这是对原始Infini-attention的一种修改,原始方

$$\sigma_{pq} s \mathbf{q} \mathbf{M}_s^T$$

Amem “ (9.14)

$$\sigma_{pq} s \mathbf{q} \mathbf{n}_s^T$$

其中 σ_{pq} 为逐元素的非线性激活函数(例如 $\text{ELU}(\sigma_{pq} - 1)$), $\mathbf{n}_s^T$ 为归一化向量(累积至分段 $s - 1$ 为止)。

接下来,使用当前分段 s 的键值条目更新压缩记忆 $\mathbf{M}_s$ 和归一化向量 $\mathbf{n}_s$:

$$\mathbf{A}^T$$

$$\mathbf{T} \sigma_{pk} s \mathbf{q} \mathbf{M}_s^T$$

$$\mathbf{M}_s \leftarrow \mathbf{M}_s^T \mathbf{1}^T \sigma_{pk} s \mathbf{q} \mathbf{v}_s^T$$

$$\sigma_{pk} s \mathbf{q} \mathbf{n}_s^T \quad (9.15)$$

T

$$\mathbf{n}_s \leftarrow \mathbf{n}_s^T \mathbf{1}^T \sigma_{pk} s \mathbf{q} \mathbf{1}^T \mathbf{N}$$

其中 $\mathbf{N}$ 为当前分段 s 的长度。 σ_{pq} 按逐元素方式应用, $\mathbf{1}^T \mathbf{N}$ 为 $\mathbf{N}^T$ 的全1向量。

分段 s 的最终注意力输出 $\mathbf{A}_s$,通过可学习的门控标量 $\beta \in \mathbb{R}$,将标准点积注意力输出 $\mathbf{A}_{\text{local}}$ (当前分段的局部上下文)与检索到的记忆状态 $\mathbf{A}_{\text{mem}}$ (过去分段的全局上下文)相结合: $\mathbf{A}_s = \text{sigmoid}(\beta \mathbf{q}^T \mathbf{A}_{\text{mem}}) \mathbf{p}_1^T + \text{sigmoid}(\beta \mathbf{q}^T \mathbf{A}_{\text{local}})$ (9.16)

与标准多头注意力一样,对 $\mathbf{A}_s$ 应用最终的线性投影,得到Infini-attention层的输出。

9.6 补充实验

9.6.1 累积误差评估指标分析

图9.10:SSIM、PSNR、LPIPS与判别器指标的比较。所有指标均归一化至[0,1]范围,数值越高表示所有分数的性能越好。判别器分数能够准确捕捉生成图像质量的变化,而其他指标则受分布偏移的影响,无法恰当地反映累积误差。

诸如SSIM、PSNR和LPIPS等传统指标衡量原始图像与生成图像之间的像素级或特征级差异。然而,当生成的视频样本明显偏离原始视频样本时----尤其是在累积误差评估中----即便它们落在同一分布内且视觉上看起来合理,这些指标也会失效。如图9.10所示,我们将所有指标归一化至0-1的量表,数值越高表示生成质量越好(SSIM分数取反)。尽管所有指标在初始帧(索引0)上表现良好,为真实值赋予高分,但其数值在第100帧之后开始恶化。

为解决这一局限性,我们开发了一种基于判别器的评估指标。我们使用来自原始DF模型(窗口大小20)生成的1000个视频训练一个判别器,每个视频包含1000帧。这产生了一个包含106个真实帧的数据集

以及106帧生成帧作为假样本。我们将判别器实现为一个使用轻量级架构的二分类器,该架构包含4个ResNet模块。过大的判别器架构会导致判别信号的意义降低。每个模块包含两个卷积层,并配有批归一化和激活函数。这种设计在保持计算效率的同时提供了具有判别力的输出。如图9.10所示,判别器数值的下降忠实地反映了生成图像中的失真,而其他指标的下降则源于两个原因:图像质量退化和相对于原始视频的分布偏移。这种偏移使得传统指标无法准确评估复合误差方面的生成性能。例如,尽管第270帧的生成质量明显优于第1080帧,但SSIM、PSNR和LPIPS却对二者给出了相似的分數。这表明分布偏移已成为降低指标分数的主导因素,使得这些指标在评估长程视频生成中的复合误差时变得不可靠。与传统指标不同,判别器的评估对分布偏移保持稳健,因为它并不依赖于原始图像,而只依赖于生成图像本身的失真程度。这使得判别器分数在这种情况下成为评估复合误差更可靠的指标。然而,判别器方法也存在若干局限性。首先,训练需要从预训练的扩散模型中采样,这会带来额外的计算开销。其次,判别器的训练严重依赖于人工判断。我们发现,即使是浅层的ResNet架构也能有效区分真实图像和生成图像。这表明,过于复杂的模型可能会对所有生成内容给出统一偏低的分數,从而使判别器指标失去参考意义。最后,判别器表现出有限的泛化能力。在评估由新方法或新数据集生成的视频时,判别器可能被误导,给出不恰当的高分。因此,我们在正文中未报告判别器分数,并倡导未来工作对长程视频生成中复合误差的忠实评估进行更多研究。

9.6.2 原始长上下文扩展 vs. YaRN

为确保公平比较,我们将YaRN与一个基线方法进行了对比,该基线方法在训练时直接将原始模型的窗口大小从20外推至推理时的40,以匹配第9.4节实验中YaRN的推理窗口长度。图9.11、9.12和9.13所示的定量指标LPIPS、SSIM和PSNR的评估结果表明,YaRN在长视频生成(1100帧)中保持了更低的复合误差。这证明了YaRN在经过最小程度的微调后,能够有效将扩散视频模型的上下文窗口扩展到40帧。而DF模型的原始上下文长度扩展由于推理时窗口大小超出分布范围,表现较差。尽管YaRN能够有效扩展上下文窗口,但其性能提升受到扩散模型在上下文学习方面固有局限性的制约。如图9.14所示,该模型在有效利用长程依赖关系方面存在困难,导致相对于真实数据的时空一致性欠佳。此外,与我们在第9.4节实验中的其他方法相比,YaRN由于窗口更大,推理时还需要更大的计算开销,使其不太适合用于实时游戏应用。

图9.11:DF模型的原始长上下文扩展与推理窗口长度为40帧的YaRN的比较。LPIPS分数越低越好。

图9.12:DF模型的原始长上下文扩展与推理窗口长度为40帧的YaRN的比较。SSIM分数越高越好。

图9.13:DF模型的原始长上下文扩展与推理窗口长度为40帧的YaRN的比较。PSNR分数越高越好。

图9.14:DF模型的原始长上下文扩展与YaRN的视觉效果比较。两个模型均以40帧窗口进行推理。

9.6.3 主要结果的更多讨论

针对第9.4.3节和第9.4.4节的主要结果,我们在此提供更多讨论。Infini-attention模型由于其全局注意力机制,面临着显著的训练挑战。如图9.15所示,该模型在训练过程中难以收敛。对于不带记忆组件的VRAG,我们将全局状态条件(具体为 $[x, y, z, yaw]$)纳入输入。然而,与原始扩散模型相比,训练过程变得明显更加困难。这可能是由于空间条件具有更高的维度和更大的取值范围,而动作条件大多由二值状态($[0, 1]$)组成,这使得模型的学习更加困难,并增加了困惑度。

图9.15:训练损失曲线

9.6.4 预测的全局状态

在本文中,我们的主要实验是在训练和推理阶段均可获取真实全局状态作为条件的情况下进行的。然而,实际应用中可能需要基于历史状态和动作来预测全局状态。为消融此影响,我们训练了一个位姿(全局状态)预测模型,该模型以当前帧和动作作为输入,输出预测的位姿变化。下一个位姿可以通过将预测的位姿变化加到当前位姿上得到。其架构仅

由少量卷积层和全连接层组成,推理时间开销极小。在评估阶段,我们应用这个训练好的预测器来预测下一步的全局状态,并基于预测的全局状态生成视频。遵循与第9.4.3节相同的设置,实验结果(针对300帧预测)汇总在表9.6中。

方法 SSIM ↑ PSNR ↑ LPIPS ↓ DF20 0.466 16.643 0.538 VRAG(预测位姿) 0.500 17.116 0.506 VRAG 0.506 17.097 0.506

表9.6:用训练好的位姿预测器所预测的位姿替代真实全局状态的消融研究

如表所示,无论是否使用位姿预测,评估结果几乎相同,这是因为相比于视频生成,位姿预测是一项相对简单的任务。这证明了在不显著降低视频性能的情况下使用预测的全局状态是可行的。

9.6.5 内存与时间开销

我们还比较了VRAG与若干基线方法的内存使用和推理时间:分别使用10帧和20帧上下文的diffusion forcing,以及使用40帧上下文的YaRN。

方法 DF10 DF20 YaRN VRAG 上下文长度(帧) 10 20 40 20 内存使用(MB) 4420 4448 4543 4452 推理时间(分钟) 9 12 23 12

表9.7:不同方法的内存使用和推理时间

如表9.7所示,VRAG的GPU内存使用和推理时间开销与DF20几乎相同。推理时间是针对600帧的自回归生成计算得出的。同时,检索操作的计算可以完全在CPU上执行,其内存占用在我们的实验中仅为 $\text{num_frame} \times \text{action_dim} \times 4 \text{ 字节} = 9.4 \text{ KB}$,几乎可以忽略不计。总而言之,VRAG与标准diffusion forcing相比几乎不产生额外的推理时间开销。检索机制引入的内存和计算成本

原文第 266 页

可以忽略不计,因为它只涉及一组向量之间的相似度计算。

第10章

带记忆的世界模型

本节基于论文《循环自回归扩散:全局记忆与局部注意力的结合》[Chen et al., 2025a],该论文与Taiye Chen、Anjian Li、Christina Zhang、Zeqi Xiao、Yisen Wang和Chi Jin合作完成。

10.1 引言

世界模型因其在数据合成、基于模型的规划、仿真等方面的关键作用而引起了研究界的广泛关注。得益于生成模型近期取得的巨大进展,视频扩散模型已成为高效且可扩展世界模型最有前景的方法之一。与学习环境潜在表示的表征世界模型[Bardes et al., 2024; Assran et al., 2025]不同,视频扩散模型[Brooks et al., 2024; Parker-Holder et al., 2024; DeepMind, 2025]直接在高维视频像素空间中实现世界建模。尽管取得了显著进展,现有的视频世界模型在保持时空一致性方面仍面临重大挑战。例如,以Minecraft游戏为核心的Oasis模型[Decart and Etched, 2024]以及诸如Cosmos[NVIDIA et al., 2025]之类的基础模型都存在严重的遗忘问题。这一困难主要源于

diffusion transformer (DiT) [Peebles and Xie, 2023] 架构固有的有限注意力窗口。鉴于视频数据的信息密度较低,对视频序列进行分词往往导致上下文长度迅速超出这些注意力限制。因此,活动注意力窗口之外的帧会被有效忽略,从而导致明显的时间和空间不一致。尽管近期研究探索了缓解这一问题的替代方法,例如利用 3D 表征 [Wang et al., 2025a] 来增强时空一致性,但这些方法通常缺乏基于像素的视频扩散模型所具备的交互性和可扩展性。将 DiT 应用于长视频生成的一个核心挑战在于,标准注意力机制中通过键值 (KV) 缓存对历史上下文的压缩效率低下。随着视频序列的延长,这种方法会导致可扩展性瓶颈,因为内存和计算需求会随序列长度成比例增长。近期的努力试图通过将循环神经网络 (RNN) ---- 包括 Mamba [Gu and Dao, 2023; Dao and Gu, 2024; Wang et al., 2025b; Po et al., 2025] 和测试时训练 (Test-Time Training, TTT) 方法 [Sun et al., 2024; Zhang et al., 2025a] ---- 引入 DiT 框架来解决这一局限。然而,这些整合方案通常存在两个主要问题: (1) TTT 类模型中常见的分块自回归处理严重依赖隐藏状态的传播,导致模型无法直接获取近期帧中密集存在的上下文信息,从而在分块边界处产生像素级不一致; 以及 (2) 循环机制的引入破坏了注意力在训练过程中的可并行性,从而增加了计算低效性,并加剧了训练与推理流程之间的差距。在本工作中,我们将一种记忆机制引入用于视频世界模型的 DiT 架构,并提出若干新颖设计以克服上述挑战。我们主张采用显式的逐帧自回归展开,这样能更好地保持上下文完整性,并引入隐藏状态预取机制以恢复注意力计算中的并行性。基于这些洞见,我们提出了循环自回归扩散 (Recurrent Autoregressive Diffusion, RAD) ---- 一个统一框架,能够同时解决长视频扩散中历史压缩的局限性以及循环机制带来的效率约束问题。在本文中,我们的主要贡献如下:

* 我们提出了循环自回归扩散(RAD),一个用于长时程视频生成的统一框架,结合全局记忆和局部注意力,并系统地比较了不同的RNN架构----包括LSTM[Hochreiter and Schmidhuber, 1997]、Mamba2和TTT----在自回归视频生成框架下的表现。我们的结果表明,LSTM尽管结构简单,却能提供稳健的性能,并在具有挑战性的长视频基准测试中往往超越更新近的RNN变体。

* 我们对分块式和逐帧式自回归范式进行了全面分析,证明具有上下文重叠的显式逐帧生成通过利用即时上下文信息大幅提升了时空一致性,并减少了对持续隐藏状态传播的依赖----从而减轻了像素级不一致性,提高了长视频合成的保真度。

* 我们设计并实现了一种隐藏状态预取机制,该机制克服了循环性与训练并行性之间固有的权衡问题。该机制在保留循环历史压缩优势的同时,实现了训练过程中完全并行的注意力计算,显著提升了大规模长序列建模的效率。

10.2 相关工作

视频扩散模型 扩散模型的显著成功最初起源于图像合成[Rombach et al., 2022a; Ramesh et al., 2022],后来扩展到视频生成[Singer et al., 2022a; Zheng et al., 2024; Tech, 2025; Blattmann et al., 2023b; Hong et al., 2022; Yang et al., 2024]。当前最先进的视频扩散模型通常采用VAE[Kingma and Welling, 2013]将视频从像素级映射到潜在空间,而模型架构已从UNet[Ronneberger et al., 2015b; Chen et al., 2024b; Blattmann et al., 2023a]演变为diffusion transformer(DiT)[Peebles and Xie, 2023]。

视频世界模型 世界模型[Watter et al., 2015; Ha and Schmidhuber, 2018a; Hafner et al., 2020]基于当前状态和输入动作预测未来状态。它们在自动驾驶[Hu et al., 2023a; Ren et al., 2025a]、导航[Bar et al., 2024]、机器人操作[Wu et al., 2024; Azzolini et al., 2025; Ge et al., 2025]以及游戏[Valevski et al., 2024; Decart and Etched, 2024; Che et al., 2024; Guo et al., 2025; Yu et al., 2025b]等领域具有广泛的应用前景。视频扩散模型合成高质量视频的能力使其成为视频世界模型的有力候选。自Sora[Brooks et al., 2024]提出"视频生成模型作为世界模拟器"以来,一系列基础世界模型陆续涌现----例如Genie2[Parker-Holder et al., 2024]、Genie3[DeepMind, 2025]和Cosmos[NVIDIA et al., 2025]----展现出卓越的视频生成质量和交互性。为实现视频世界模型,近期进展从多个方面改进了视频生成模型:用于延长视频时长的自回归推理[Chen et al., 2024a; Huang et al., 2025; Cui et al., 2025; Yin et al., 2025]、用于提升时空一致性的记忆机制[Chen et al., 2025c; Xiao et al., 2025b; Yu et al., 2025a]、物理建模[Kang et al., 2024]等。

带记忆的扩散模型 尽管基于DiT的可扩展视频扩散模型在短片段上表现出强劲性能,但对长视频应用全注意力会带来高昂的计算和内存成本。为在不使用全局注意力的情况下实现长视频生成,需要额外的记忆机制,这些机制可分为上下文记忆、隐藏状态记忆和权重记忆。上下文记忆方法将过去的帧作为自回归(AR)预测的条件。在有限的上下文窗口下,它们既可以采用偏重近期的帧选择方式----如diffusion forcing[Chen et al., 2024a; Song et al., 2025]和self-forcing[Huang et al., 2025; Cui et al., 2025]----也可以采用基于相似度的检索方式,如VRAG[Chen et al., 2025c]和WorldMem[Xiao et al., 2025b]。隐藏状态记忆方法将历史信息压缩为循环状态,例如在注意力之前插入Mamba层[Po et al., 2025]。然而,这种循环结构破坏了原始DiT的时间并行性,增加了额外的计算开销。权重记忆技术

使用内循环损失在滑动经过小块数据的同时更新特定权重作为记忆,如测试时训练(TTT)模块中所见[Dalal et al., 2023; Zhang et al., 2025a]。由于相邻的注意力窗口不重叠,更新后的权重必须完整编码来自先前分块的所有必要历史信息,这对存储和检索的记忆容量提出了很高的要求。相比之下,我们的方法只对超出上下文窗口的历史部分执行高效的记忆压缩。它通过自回归生成过程中的逐帧滑动,在上下文记忆和隐藏状态记忆之间建立了桥梁,兼顾了效率与长程一致性。

10.3 预备知识

10.3.1 循环神经网络

RNN 假设 $x_t \in \mathbb{R}^D$ 表示时刻 t 的输入,循环神经网络(RNN)的一般形式可以描述为:

$$h_t = f(h_{t-1}, x_t, y_{t-1}; \theta)$$

$$y_t = g(h_t, x_t)$$

其中 h_t 是时刻 t 的隐藏状态, x_t 是输入, y_t 是输出, $f(\cdot)$ 是一个参数化的非线性函数(通常涉及仿射变换和激活函数), θ 表示所有可训练参数, $g(\cdot)$ 将隐藏状态映射到输出。

LSTM 中， f_t 、 i_t 、 o_t 、 g_t 分别表示 t 时刻的遗忘门、输入门、输出门和候选细胞状态， σ 、 $\tanh$ 分别表示 sigmoid 激活函数和双曲正切激活函数， W 表示各门对应的权重和偏置矩阵。向量 C_t 和 y_t 分别表示 t 时刻的细胞状态和输出，二者共同封装了截至并包括该时间步所累积的全部压缩记忆信息。

$C_t = \sigma(f_t) \odot \tanh(g_t)$

$y_t = o_t \odot \tanh(C_t)$

其中 f_t 、 i_t 、 o_t 、 g_t 分别表示 t 时刻的遗忘门、输入门、输出门和候选细胞状态， σ 、 $\tanh$ 分别表示 sigmoid 激活函数和双曲正切激活函数， W 表示各门对应的权重和偏置矩阵。向量 C_t 和 y_t 分别表示 t 时刻的细胞状态和输出，二者共同封装了截至并包括该时间步所累积的全部压缩记忆信息。

其他 RNN 除 LSTM 之外，Mamba 和 TTT 是另外两种随时间更新隐藏状态的常见循环架构。它们之间的关键差异在于新状态的计算方式。LSTM 使用三和

10.3.2 视频扩散模型

潜在视频扩散模型 我们采用一种潜在视频扩散模型 [Blattmann et al., 2023a]，该模型首先使用一个预训练的变分自编码器 (VAE) 将像素空间编码为潜在表示 z 。前向过程根据方差调度 β_t 逐步向潜在变量添加高斯噪声：

a

$$q(z_t | z_{t-1}, q) = N(z_t | \mu_t, \beta_t I) \quad (10.1)$$

该模型通过在每一步预测噪声 ϵ_θ 来学习逆转这一过程:

$$L = \mathbb{E}_{t, \epsilon, z} \mathbb{E}_{\epsilon \sim \epsilon_\theta(p(z_t, t))} \|\epsilon - \epsilon_\theta(p(z_t, t))\|_2^2 \quad (10.2)$$

??

其中 $z_t \sim \mathcal{N}(z_0, \sigma_t^2)$ 从 z_0, σ_0 出发, 通过迭代去噪来采样新的视频:

在推理阶段, 我们可以从随机噪声 $z_T \sim \mathcal{N}(0, I)$ 出发, 通过迭代去噪来采样新的视频:

$$z_t = \beta_t z_{t-1} + \sqrt{\alpha_t - \beta_t} \epsilon$$

$$z_t \sim \mathcal{N}(z_{t-1}, \sigma_t^2) \quad (10.3)$$

$$\alpha_t = 1 - \beta_t$$

St

其中 $\alpha_t = 1 - \beta_t$, $\alpha_t \sim \mathcal{N}(0, 1)$ 。最终的潜在序列 z_0 通过解码器 D 解码回像素空间, 从而得到生成的视频。

扩散强制(Diffusion Forcing) 为实现长视频生成, 我们采用了 Diffusion Forcing [Chen et al., 2024a] 技术。在训练过程中, 我们按照扩散调度, 对整个输入视频序列中的每一帧随机加入噪声: $z_t \sim \mathcal{N}(z_0, \sigma_t^2)$

$\tilde{z}_t = z_t + \epsilon$, 其中 z_t 表示第 i 帧的带噪潜在变量, 此时动作条件自回归视频模型的训练目标变为:

$$L_{DF} = \mathbb{E}_{t, \epsilon, z, a} \mathbb{E}_{\epsilon \sim \epsilon_\theta(p(z_t, t))} \|\epsilon - \epsilon_\theta(p(z_t, t), a)\|_2^2$$

$$\epsilon \sim \mathcal{N}(0, I), z_t \sim \mathcal{N}(z_0, \sigma_t^2)$$

其中 r_t 是一个长度为 L 的时间步向量, 每一帧对应不同的 $t \in \{1, \dots, L\}$, a 是动作序列 $a \in \mathcal{A}^L$ 。噪声预测模型 ϵ_θ 同时以动作序列 a 和带噪帧 z_t 为条件。

10.4 方法

为解决视频扩散模型中固定大小上下文窗口的局限性, 我们提出将全局记忆与局部注意力机制相结合。这一

预取隐藏状态 RAD 用于并行注意力的分块式 RNN 空间注意力 窗口大小 l 窗口大小 l

HW RAD 模型 ($l = 1$) RNN 模块 RAD 模型 时间注意力 h_{li} h_{2i} h_{3i} ...

T 逐帧式 RNN 隐藏状态预取

窗口大小 l RNN 模块 RAD 模型 ... 空间注意力 h_{li} h_{2i} h_{3i} 时间注意力 RAD 模型 RNN 模块

并行注意力

图 10.1:具有全局记忆与局部注意力的循环自回归扩散(Recurrent Autoregressive Diffusion)训练范式:该模型在每个 DiT 模块中包含三个组成部分,即空间注意力、时间注意力和 RNN 记忆模块。它支持采用不同注意力机制的分块式和逐帧式自回归生成。为实现逐帧式 RNN 的高效训练,我们(1)从干净样本序列中预取隐藏状态,以支持对整条长序列进行并行注意力计算,并(2)以标准方式进行扩散模型的前向计算以得到扩散损失。这提高了大规模、长序列视频建模的效率与保真度。 h_{ij} 表示第 i 层、帧索引为 j 时的隐藏状态。

方法使模型能够有效捕捉视频序列中的长期依赖关系,同时在生成帧中保持高保真度。

10.4.1 循环自回归扩散

我们提出了循环自回归扩散(Recurrent Autoregressive Diffusion,RAD)模型,该模型将一个循环神经网络(RNN)模块整合进扩散 Transformer(DiT)架构 [Peebles and Xie, 2023] 中,以承载全局记忆信息。RAD 模型的整体架构如图 10.2 所示。遵循先前工作 [Zheng et al., 2024; Decart and Etched, 2024] 的设计,我们将 DiT 中的注意力机制分解为两个独立的模块:空间轴注意力(Spatial Axis Attention)和时间轴注意力(Temporal Axis Attention)。因此,DiT 的每一层

包含三个主要组成部分----作为标准架构的两个注意力模块,以及一个额外的 RNN 模块。旋转位置编码(Rotary Position Embedding, RoPE)[Su et al., 2024] 被同时应用于空间和时间维度,以增强两个注意力模块捕捉位置依赖关系的能力。条件信息,具体是时间步和动作条件,通过自适应层归一化(adaLN)被引入 RAD 模型。除了在注意力模块中,RAD 还额外将动作条件应用于 RNN 模块。这一设计经实验验证能够有效增强动作控制并提升视频保真度,相关实验见第 10.6.3 节。对于 RNN 模块,我们比较了三种方案,包括 LSTM、TTT 和 Mamba 的 SSM 模块 [Dalal et al., 2023],发现 LSTM 在我们的 RAD 模型中表现最佳,详见第 10.5 节的实验。在接下来的章节中,我们讨论为优化 RNN 与 DiT 框架整合而做出的两个关键设计选择:(a) 逐帧自回归以获得更好的上下文一致性(第 10.4.2 节),以及 (b) 用于并行注意力计算的隐藏状态预取(第 10.4.3 节)。

分块化 AdaLN AdaLN AdaLN

条件 空间 时间 RNN 轴 轴 模块 $\times N$ 注意力 注意力 MLP 缩放 空间轴 缩放 缩放 模块

AdaLN AdaLN 时间轴 模块 FFN FFN

缩放 缩放

RNN 模块

图 10.2:循环自回归扩散模型架构:RAD 模型由 N 个 DiT 模块组成,每个模块包含三个子模块:一个空间轴模块、一个时间轴模块和一个 RNN 模块。时间步和动作条件经过一个 MLP 处理后,用于通过 adaLN 控制每个模块。

10.4.2 分块式与逐帧式自回归

分块式 RNN 逐帧式 RNN 0 1 2 3 4 5 6 7 8 0 1 2 3 4 5 6 7 8 2 2 3 3 4 4 5 5 6 6 7 7 8 8

: 干净帧 : 预测帧 : 隐藏状态更新

图 10.3:分块式与逐帧式 RNN 的有效时间注意力图对比,分块大小为 3,初始上下文帧数为 2。图中横轴表示帧索引,纵轴表示当前预测帧的索引。

在 RAD 框架中,RNN 模块以自回归方式处理时间信息,可通过两种不同模式实现:分块式和逐帧式自回归。这两种方式均与 DiT 架构的时间注意力机制无缝集成,如图 10.1 和图 10.3 所示。对于分块式自回归,注意力窗口彼此不重叠。在训练过程中,输入视频序列根据模型的窗口大小被划分为若干块。在每个块内计算局部注意力,同时通过在块边界间传递 RNN 隐藏状态来保留全局时间依赖关系。为确保模型在每个块内以自回归方式运行,我们对注意力机制的时间维度施加因果掩码。值得注意的是,这一方法与现有方法 [Dalal et al., 2023; Zhang et al., 2025a] 形成对比,后者侧重于对为全序列去噪而训练的扩散模型进行微调。相反,我们的方法确保了训练与推理之间在架构和流程上的一致性:模型在推理时处理数据的方式与训练时完全相同,从而支持稳健的时间泛化能力和忠实的序列建模。对于逐帧式 RNN 模式,基于 Diffusion Forcing [Chen et al., 2024a],我们采用逐帧自回归生成方案。这种方法使

阶段 1: 阶段 2: 滑动窗口训练 预取 并行全注意力

: 干净帧 : 带噪帧 : 隐藏状态更新 : 隐藏状态传递

图 10.4: 标准前向过程(左)与隐藏状态预取过程(右)在逐帧式滑动窗口下的对比。

模型能够充分利用注意力机制在帧间传递像素级信息。然而,若直接采用基于滑动窗口的训练方式,这将带来显著的计算开销。为规避这一低效问题,我们引入了一种隐藏状态预取(Hidden State Pre-fetching)策略,详见后续章节。在推理阶段,如图 10.3 所示,我们每次将窗口滑动一帧,且每个窗口中仅由第一帧负责更新 RNN 的隐藏状态。这一流程直接对应于训练时隐藏状态预取步骤中所使用的窗口大小 1。此外,隐藏状态的更新仅在 DDIM 过程的最后一步执行,以确保所有记忆输入均为干净帧----从而与训练过程保持一致,并提升生成质量的稳定性。

10.4.3 用于并行注意力的隐藏状态预取

我们针对逐帧式 RNN 的训练策略如图 10.1 右侧面板和图 10.4 所示。RNN 的一个众所周知的局限性在于其固有的顺序依赖性:每个时间步的输出都与前一时刻的隐藏状态紧密相关。这种严格的时间递归阻碍了并行化,并大大拖慢了训练速度,对长序列和大规模数据集尤其构成挑战。当 RNN 进一步与注意力机制结合时,后续的注意力计算在训练期间需要沿输入序列遵循相同的递归方式,导致

计算瓶颈过重。在训练期间维持完全顺序化的时间循环变得代价高昂。为缓解这些问题,我们采用了一种隐藏状态预取方案,将 RNN 与注意力模块部分解耦。在该框架中,RNN 保持其时间递归性,但注意力运算可以并行进行,从而显著提高训练效率。若不采用此技术,对于长度为 L 的序列,窗口大小为 l 的逐帧式滑动需要进行 $pL \cdot l^q$ 次顺序注意力计算,而在我们的方案中,并行注意力在训练时仅需计算一次。该方法的另一个关键优势在于,预取过程中输入到 RNN 记忆中的仅是干净帧,而非来自扩散过程的带噪帧 [Po et al., 2025],这一设计选择可加速训练收敛(实证分析见第 10.6.1 节)。具体而言,对于逐帧式自回归,我们首先在每个训练序列的干净帧上,以窗口大小为 1 的方式应用 RAD 模型。此步骤独立地 dPrDs 计算序列及 DiT 各层中所有帧对应的 RNN 隐藏状态 $h_{t,d}$,其中每个 $h_{t,d}$ 汇聚了截至第 t 帧、第 d 个 DiT 层的上下文信息。假设前一隐藏状态已堆叠了足够的上下文信息,则预取得到的隐藏状态在效果上等同于采用滑动窗口方式的标准 DiT-RNN 结构。然而,在预取完所有隐藏状态之后,尽管 RNN 更新本质上是顺序的,每个窗口内的注意力模块现在都可以并行处理。通过在相应位置设置预取得到的 RNN 层隐藏状态,我们可以像往常一样以正常窗口大小计算 RAD 中的注意力模块,从而得到整条序列的扩散损失。在我们的实验中,我们并未针对给定样本(长度 L)中的所有子序列(长度 l)都计算扩散损失,而是随机采样其中一部分以降低计算成本。

10.5 实验

10.5.1 数据集与评估协议

迷宫数据集 在小规模实验设置中,我们使用了 Memory Maze 数据集 [Pasukonis et al., 2022],该数据集包含约 30000 个训练视频,每个视频描绘智能体在 15×15 迷宫环境中的导航过程,共包含 1000 帧。对于迷宫数据集,所有模型均在训练过程中从头开始训练。为进行评估,我们额外收集了 200 段迷宫视频,帧数在 100 到 300 之间。在每个序列中,智能体从指定的起始位置出发前往目标位置,然后沿相同路径返回。我们将前 60% 的帧---对应外出轨迹---定义为上下文输入,其余 40% 作为模型评估的预测目标。这一设置专门用于严格检验模型的长期记忆能力和上下文利用能力。迷宫数据的俯视可视化图见附录。

Minecraft 数据集 在大规模实验中,我们使用 MineRL [Guss et al., 2019] 按照 VRAG [Chen et al., 2025c] 的协议生成了 20000 条训练序列,每段视频包含 1200 帧。为进行评估,我们收集了 60 段包含不同动作模式(例如原地旋转)的序列,旨在探究模型在多样化行为下的记忆能力。为确保比较的公平性,我们首先在完整的 20000 个样本的训练集上训练一个标准的扩散强制模型。随后,将这一预训练的基础模型分别用不同的 RAD-RNN 架构和训练范式进行微调,每种设置均训练相同的 5000 个优化步。该流程使我们能够在一致的预训练条件下,对架构和方法上的差异进行受控评估。

评估指标 我们采用三种广泛使用的指标来定量评估模型性能:结构相似性指数(SSIM)[Wang et al., 2004],该指标

用于评估生成帧的空间一致性;峰值信噪比(PSNR),用于衡量像素级重建保真度;以及学习感知图像块相似度(LPIPS)[Zhang et al., 2018],用于评估感知相似度。值得注意的是,SSIM----在将生成输出与真实结果直接比较时----更加强调记忆信息与时空一致性的保持,而 PSNR 和 LPIPS 则更多地反映整体帧质量。

10.5.2 迷宫数据集结果

表 10.1:在迷宫数据集上的实验结果:分块式(“-c”) 表 10.2:在 Minecraft 数据集上的实验结果:分块式(“-c”)和逐帧式(“-f”)自回归模式和逐帧式(“-f”)自回归模式

模型	RNN 类型	PSNR	SSIM	LPIPS	DF 无
Mamba2-c	13.80	0.31	0.58	12.72	0.33
Mamba2-f	15.35	0.41	0.51	16.70	0.46
LSTM-c	14.24	0.39	0.55	15.50	0.41
LSTM-f	15.50	0.41	0.45	16.59	0.46

RNN 方法比较 我们在 RAD 架构中比较了 LSTM、Mamba2 和 TTT,如第 10.4.2 节所述,在相同设置下训练三个 epoch。表 10.1 汇总了分块式(“-c”)和逐帧式(“-f”)自回归模式下的结果。

在分块式设置下,LSTM 在所有指标上均取得了最强的表现,相较于扩散强制(DF)基线和其他循环变体均有显著提升。相比之下,Mamba2-c 和 TTT-c 在更侧

TTT---其设计目标是全局压缩而非细粒度的像素级信息传输---相比之下表现较差。

在逐帧(frame-wise)设置下,所有循环变体的表现都较为接近。当滑动步长为1时,连续帧共享同一个注意力窗口,使得像素级信息可以直接通过注意力传播,而不必依赖隐藏状态。这消除了Mamba2-c和TTT-c的主要失败模式,使Mamba2-f、TTT-f和LSTM-f能够达到相近的质量水平,指标差异很小。相应地,LSTM的结构优势也被削弱,因为当注意力已经提供了强有力的局部连续性时,其显式的短期记忆通路 y_{t-1} 就变得不那么必要了。

逐帧自回归与分块自回归的比较 表10.1中的跨范式比较凸显了自回归设计如何决定各循环架构的相对优势。

在分块(chunk-wise)模式下,由于缺乏跨块的局部注意力,隐藏状态被迫成为传递像素级信息的唯一通道。这对其信息存储和检索能力提出了过高的要求:记

在逐帧模式下,注意力跨越所有连续帧,恢复了直接的像素级通信。因此,隐藏状态主要专注于全局信息,减少了保留局部细节的需求。一旦卸下这一负担,Mamba2和TTT的性能大幅提升,并与LSTM趋于一致。LSTM的循环通路变得部分冗余,这解释了其先前优势的消失。

总体而言,结果表明RNN是否适用于自回归扩散,强烈依赖于注意力的时间粒度:当注意力无法跨越边界时(分块式),具有显式局部-全局分离特性的架构表现出色;当注意力是连续的时(逐帧式),架构差异的影响则大为减弱。我们认为,隐藏状态应更多地聚焦于全局信息,而高频的时间局部信息则应主要通过局部注意力来传输。

10.5.3 Minecraft结果

图10.5:Minecraft数据集上的可视化结果。逐帧RNN能够有效记忆和重建场景,与真实值保持一致。相比之下,由于缺乏重叠注意力,分块RNN在信息密度更高的数据集中无法重建历史场景

逐帧自回归与分块自回归 表10.2展示了将分块式和逐帧式RAD应用于不同RNN变体在Minecraft数据集上的结果。总体趋势与Maze数据集上观察到的相符,但由于Minecraft视频在视觉和结构复杂度上显著更高,存在一些重要差异。

在分块式设置下,LSTM再次优于Mamba2和TTT,这与其在短期和长期记忆之间平衡的结构性能相一致。然而,与Maze实验不同的是,所有RNN变体的表现都不及标准的Diffusion Forcing基线。Minecraft场景包含密集的纹理、丰富的几何结构以及快速的视角变化,这使得任何循环隐藏状态都难以在跨块边界时完全压缩和传输像素级信息。因此,仅依赖隐藏状态进行块间通信的架构局限性变得更加明显,导致重建质量下降、场景一致性变弱。

在逐帧式设置下,采用三种不同RNN类型的RAD模型表现相近,显著优于分块式对应模型,因为局部视觉信息可以直接通过注意力传播,不再需要在隐藏状态中编码高频细节。在这种更有利的机制下,三种RNN类型都比其分块式对应模型取得了明显更好的性能。

图10.5中的定性结果凸显了这一对比。逐帧式TTT和逐帧式LSTM生成的视频保持了较强的记忆能力,并在长时间跨度内与真实值保持一致。相反,分块式TTT和分块式LSTM在记忆场景布局和物体配置方面表现出明显的失败,进一步印证了仅依赖隐藏状态来承载丰富的Minecraft级细节跨块传递的困难性。

总体而言,这些实验进一步支持了如下结论:自回归窗口的粒度起着关键作用。分块式自回归对于视觉复杂的环境施加了不切实际的压缩负担,而逐帧式自回归则利用注意力来维持局部保真度,使所有RNN架构都能在更全局的信号上运作,从而取得明显更好的结果。

性能曲线 为了更详细地比较分块式和逐帧式RNN方法,我们在图10.6中展示了SSIM和PSNR值随帧索引变化的曲线。值得注意的是,两种方法在第60帧处均表现出性能下降,该帧对应模型预测的第一帧。然而,这种下降对分块式RNN而言更为显著,凸显了其在跨块边界有效传递像素级信息方面能力的局限。

(a) 分块式TTT和逐帧式TTT的SSIM (b) 分块式TTT和逐帧式TTT的PSNR

图10.6:分块式TTT与逐帧式TTT的比较

10.5.4 VBench结果

除了像素相似度之外,我们还加入了VBench [Huang et al., 2023b]评估结果。如表10.3所示,VBench上的结果与第10.5.2节和第10.5.3节中的结果趋势相似。基于LSTM的RNN取得了更好的性能,能够生成更高质量的视频。

表10.3:不同自回归模型变体的VBench评估结果。 Maze数据集 Minecraft数据集 背景一致性 时间闪烁 运动流畅度 美学质量 图像质量 背景一致性 时间闪烁 运动流畅度 美学质量 图像质量 指标 LSTM-c 91.29 93.49 71.4 26.29 51.44 97.42 93.96 95.12 57.58 69.34 Mamba-c 91.07 93.07 69.98 25.51 50.7 97.27 93.79 94.96 57.22 69.27 TTT-c 90.89 92.21 70.71 26 51.6 97.44 93.91 95.06 57.55 69.15 LSTM-f 91.16 93.01 71.48 26.11 50.94 97.38 94.21 95.3 55.71 66.72 Mamba-f 90.78 92.97 70.78 25.76 50.94 97.37 94.17 95.26 55.73 66.4 TTT-f 90.99 93.2 70.87 25.82 51.08 97.38 94.14 95.25 55.64 66.26

10.5.5 计算资源分析

直接比较不同RNN架构的计算效率本质上具有挑战性,因为它们的实现方式和底层优化程度可能存在很大差异。例如,LSTM得益于大量的底层优化,如cuDNN提供的优化,而TTT目前缺乏此类专门的增强。尽管存在这些差异,我们仍报告了计算资源开销的汇总数据以供参考。所有报告结果均来自在Minecraft数据集上的实验,使用8块NVIDIA L40 GPU,批大小为3。

如表10.4所示,尽管LSTM架构的参数量最高,但其GPU内存消耗和每步训练时间与Mamba2相当。这些参数量上的差异反映了模型设计上的内在区别,而非不公平的实验条件,因此可以进行合理的评估。

表10.4:不同RNN模块类型间计算 表10.5:在分块式设置下,将条件动 资源需求的比较。"Time"表示在指 作信息整合进RNN输入的影响的消 定RNN类型下(相同的DiT)一个 融实验。 RAD模型单步训练所需的时长,而 模型 +动作 PSNR Ò SSIM Ò LPIPS Ó "Params"表示该RNN模块中的参 N 14.00 0.35 0.57 数总量。 RAD-Mamba2 Y 13.80 0.31 0.58 N 14.44 0.37 0.53 RAD-TTT RNN类型 GPU内存(GB) 时间(s/step) 参数量(M) Y 14.36 0.35 0.53 LSTM 10.0 3.7 195 N 14.88 0.38 0.52 RAD-LSTM Mamba2 11.1 3.6 153 Y 15.64 0.43 0.47 TTT 16.0 12.9 118

10.6 消融实验

10.6.1 记忆帧的噪声水平

指标 加噪 干净 PSNR Ò 14.70 15.30 SSIM Ò 0.38 0.41 LPIPS Ó 0.52 0.50

表10.6:逐帧式LSTM在干净 帧记忆与加噪帧记忆条件下, 图10.7:逐帧式LSTM在干净帧记忆 不同指标上的评估结果 与加噪帧记忆条件下的训练损失曲线。

我们进行了一项消融实验,以评估训练期间输入RNN隐藏状态的记忆帧所施加的噪声水平的影响。我们的发现表明,引入更高的噪声水平会使模型优化更加困难,导致训练损失增加、 评估指标下降,如图10.7和

表10.6所示。这些结果凸显了我们设计选择的重要性:所有用作记忆的干净帧均在预取阶段进行预处理,由窗口大小为1的DiT模型滑动处理,并与去噪阶段解耦。

这种方法与Po等人[Po et al., 2025]的策略有一定相似之处,该策略将每个训练序列的最初n帧保持干净,并将其排除在扩散损失计算之外。然而,我们的方法更为全面,传入RNN隐藏状态的所有帧都保持干净,而不仅仅是最初的一部分。此外,我们的方法不会干扰扩散损失的计算,进一步提高了训练效率。

10.6.2 带步长的分块自回归

我们进一步研究了分块式RNN中部分重叠的注意力窗口,保持与第10.5.2节相同的模型规模和训练数据,同时将滑动窗口的步长设置为10帧(训练和推理阶段保持一致)。换言之,对于逐帧式RNN,步长为1;对于分块式RNN,步长等于窗口大小(20)。如表10.7所示的实验结果表明,该方法相较于逐帧式RNN和分块式RNN均未显示出优势。

表10.7:LSTM在Maze数据集上的带步长分块自回归

RNN类型	步长	PSNR	SSIM	LPIPS	LSTM-c	LSTM-f	部分重叠
	20	15.64	0.43	0.47	15.50	0.41	0.45
	10	15.06	0.40	0.55			

10.6.3 动作条件设计

我们进行了一项消融实验,以研究RNN模块设计的影响,重点关注条件信息的引入方式。在标准方法中,时间步和动作条件信息通过AdaLN引入DiT模块,这可能限制RNN模块利用这些信号的能力。为解决这一问题,我们进行了实验,

将动作条件通过线性投影传递,并将其沿隐藏维度直接与注意力模块的输出拼接,将这一组合作为RNN模块的输入。

如表10.5所示,这一改动为LSTM带来了显著提升,但对TTT和Mamba架构而言仅带来有限的增益。值得注意的是,即使不进行这种整合,LSTM仍然优于TTT和Mamba,这凸显了其在建模序列信息方面的内在优越性。

10.7 实现细节

10.7.1 Mamba和TTT

Mamba

$$H_t \leftarrow A H_t + B X_t; X_t \leftarrow C H_t + D X_t$$

其中 H_t 为潜在状态,A、B、C、D均为输入 X_t 的线性投影,即 $A_t := \text{Linear}_{\theta_A}(X_t)$, B_t 、 C_t 和 D_t 依此类推。这是Mamba用于自回归任务的数学表述。在多层设置中,上一时间步的输出并不作为下一时间步的输入,而是作为下一层的输入。因此,可表示为:

$$H_t \leftarrow A H_t + B X_t; Y_t \leftarrow C H_t + D X_t$$

TTT

$$W_t \leftarrow W_t + \eta \nabla_{\theta} W_t; x_t \leftarrow$$

$$y_t \leftarrow f_{\theta_Q}(x_t; W_t)$$

其中自监督损失 L 通常定义为 $L_p W; x_t q \parallel f p \theta K x_t; W q ' \theta V x_t \parallel^2, \theta Q、\theta K、\theta V$ 为可训练参数, W_t 为 t 时刻的隐藏状态矩阵。TTT-linear学习逐实例权重 W_t 比较如表10.8所示,我们提供了不同RNN类型的比较,其中 h 为LSTM隐藏层大小, n 为Mamba SSM状态大小。

表10.8:对于输入张量 pB, H, W, L, dq , LSTM、Mamba(SSM)与测试时训练(TTT-linear,采用 $d \wedge d$ 记忆权重)之间计算复杂度的比较。方面 LSTM Mamba (SSM) TTT 计算量(FLOPs) $OpBHW Lpdh \cdot h^2 qq OpBHW Ldnq OpBHW Lpd2 \cdot Cqq$ 参数量 $Ophpd \cdot hqq Opdnq \cdot Opd2 q$ (若有投影) $Opd2 q$ 训练内存 $OpBHW Lhq OpBHW Lpd \cdot nqq OpBHW dq$ 推理内存 $OpBHW hq OpBHW pd \cdot nqq OpBHW dq$ 序列扩展性 随 L 线性 随 L 线性 随 L 线性

10.7.2 逐帧式RNN中注意力的高效并行化

为实现逐帧式RNN中注意力计算的高效并行化,我们针对两个关键阶段采用了不同的并行化策略:(1)隐藏状态的预取,(2)扩散前向计算。

在第一阶段,我们在每个DiT层内沿序列的时间维度并行化注意力计算。我们采用窗口大小 $l \ll 1$ 的滑动窗口来预取隐藏状态。以Maze设置为例,由于每个训练样本包含 $L \ll 1001$ 帧,我们拥有大量的块。为实现并行计算,我们将每个样本的这些块进一步划分为较小的块批次,共计 $b \ll 128$ 个批次,以满足GPU内存限制。请注意,这与全局批大小 $B \ll 64$ 不同,后者指每个批次中长度为 L 的样本数量。训练数据据此被划分为批次,对于每一层,我们依次处理样本批次,使空间注意力和时间注意力在批次内的各样本间并行计算。之后,RNN前向

操作在每一层内随后进行。至关重要的是,空间注意力(rBL, H, W, ds)机制在各帧之间独立运作,而时间注意力($rBHW, l, l, ds$)采用窗口大小 $l = 1$,从而如因果注意力一样保持时间上的因果性。通过这一过程,我们最终得到每一帧和每一层的所有隐藏状态。

在第二阶段,我们沿整个时间维度进行并行化,以计算扩散前向过程从而获得扩散损失。我们在注意力计算中采用正常窗口大小 $l = 20$ 。为降低GPU内存耗尽的风险,我们从完整序列中随机采样 N 个子序列用于评估扩散损失。所选取的 N 个隐藏状态及其相关联的加噪帧输入沿批次维度拼接(空间维度为 rBN, l, H, W, ds , 时间维度为 $rBHW, N, l, ds$),从而实现注意力的同时计算。这种双阶段并行化框架旨在在不超出内存限制的前提下最大化内存效率,唯一的循环计算发生在RNN前向传播过程中。

Bibliography

Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Floren- cia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anad- kat, et al. Gpt-4 technical report. arXiv preprint arXiv:2303.08774, 2023. Rishabh Agarwal, Dale Schuurmans, and Mohammad Norouzi. An optimistic perspective on offline reinforcement learning. In International Conference on Machine Learning, pages 104-114. PMLR, 2020. Ashkan Aghdai, Cing-Yu Chu, Yang Xu, David H Dai, Jun Xu, and H Jonathan Chao. Spotlight: Scalable transport layer load balancing for data center networks. arXiv preprint arXiv:1806.08455, 2018a. Ashkan Aghdai, Michael I-C Wang, Yang Xu, Charles H-P Wen, and H Jonathan Chao. In-network congestion-aware load balancing at transport layer. arXiv preprint arXiv:1811.09731, 2018b. Anurag Ajay, Yilun Du, Abhi Gupta, Joshua Tenenbaum, Tommi Jaakkola, and Pulkit Agrawal. Is conditional generative modeling all you need for decision-making? arXiv preprint arXiv:2211.15657, 2022. Mohammed Shabbir Ali, Pierre Coucheney, and Marceau Coupechoux. Reinforcement learning algorithm for load balancing in self-organizing networks, 2019. Eloi Alonso, Adam Jelley, Anssi Kanervisto, and Tim Pearce. Diffusion world models, 2024. URL <https://openreview.net/forum?id=bAXmvOLtjA>. Brandon Amos, Samuel Stanton, Denis Yarats, and Andrew Gordon Wilson. On the model-based stochastic value gradient for continuous reinforcement learning. In Learn- ing for Dynamics and Control, pages 6-20. PMLR, 2021. Marcin Andrychowicz, Anton Raichuk, Piotr Sta ń czyk, Manu Orsini, Sertan Girgin, Raphael Marinier, L é onard Hussenot, Matthieu Geist, Olivier Pietquin, Marcin Michal- ski, et al. What matters in on-policy reinforcement learning? a large-scale empirical study. arXiv preprint arXiv:2006.05990, 2020a. OpenAI: Marcin Andrychowicz, Bowen Baker, Maciek Chociej, Rafal Jozefowicz, Bob McGrew, Jakub Pachocki, Arthur Petron, Matthias Plappert, Glenn Powell, Alex Ray, et al. Learning dexterous in-hand manipulation. The International Journal of Robotics Research, 39(1):3-20, 2020b. Jo ˆ o Taveira Ara ú jo, Lorenzo Saino, Lennert Buytenhek, and Raul Landa. Balancing on the edge: Transport affinity without network state. page 111-124, 2018. ISBN 978-1-939133-01-4.

- Kai Arulkumaran, Marc Peter Deisenroth, Miles Brundage, and Anil Anthony Bharath. Deep reinforcement learning: A brief survey. *IEEE Signal Processing Magazine*, 34(6): 26-38, 2017.
- Kavosh Asadi, Dipendra Misra, Seungchan Kim, and Michel L Littman. Combating the compounding-error problem with a multi-step model. *arXiv preprint arXiv:1905.13320*, 2019.
- Mahmoud Assran, Quentin Fan, Adrien Bardes, et al. V-jepa 2: Self-supervised video models enable understanding, prediction, and planning. *arXiv preprint arXiv:2506.09985*, 2025.
- Nikhil Borse Azzolini, Ujval Bhatt, et al. Cosmos world foundation model platform for physical ai. *arXiv preprint arXiv:2501.12599*, 2025.
- Adrià Puigdomènech Badia, Bilal Piot, Steven Kapturowski, Pablo Sprechmann, Alex Vitvitskyi, Zhaohan Daniel Guo, and Charles Blundell. Agent57: Outperforming the atari human benchmark. In *International Conference on Machine Learning*, pages 507-517. PMLR, 2020.
- Yu Bai and Chi Jin. Provable self-play algorithms for competitive reinforcement learning. In *International Conference on Machine Learning*, pages 551-560. PMLR, 2020.
- Yu Bai, Chi Jin, and Tiancheng Yu. Near-optimal reinforcement learning with self-play. *Advances in Neural Information Processing Systems*, 2020.
- Yu Bai, Chi Jin, Song Mei, and Tiancheng Yu. Near-optimal learning of extensive-form games with imperfect information. *arXiv preprint arXiv:2202.01752*, 2022.
- James P Bailey and Georgios Piliouras. Multiplicative weights update in zero-sum games. In *Proceedings of the 2018 ACM Conference on Economics and Computation*, pages 321-338, 2018.
- Bowen Baker, Ingmar Kanitscheider, Todor Markov, Yi Wu, Glenn Powell, Bob McGrew, and Igor Mordatch. Emergent tool use from multi-agent autocurricula. In *International Conference on Learning Representations*, 2019.
- Philip J Ball, Laura Smith, Ilya Kostrikov, and Sergey Levine. Efficient online reinforcement learning with offline data. *arXiv preprint arXiv:2302.02948*, 2023.
- Amir Bar, Gaoyue Zhou, Danny Tran, Trevor Darrell, and Yann LeCun. Navigation world models. *arXiv preprint arXiv:2412.03572*, 2024.
- Omer Bar-Tal, Hila Chefer, Omer Tov, Charles Herrmann, Roni Paiss, Shiran Zada, Ariel Ephrat, Junhwa Hur, Guanhui Liu, Amit Raj, et al. Lumiere: A space-time diffusion model for video generation. *arXiv preprint arXiv:2401.12945*, 2024.
- Tom Barbette, Chen Tang, Haoran Yao, Dejan Kostić, Gerald Q Maguire Jr, Panagiotis Papadimitratos, and Marco Chiesa. A high-speed load-balancer design with guaranteed per-connection-consistency. In *17th USENIX Symposium on Networked Systems Design and Implementation (tNSDIu 20)*, pages 667-683, 2020.

- Nolan Bard, Jakob N Foerster, Sarath Chandar, Neil Burch, Marc Lanctot, H Francis Song, Emilio Parisotto, Vincent Dumoulin, Subhodeep Moitra, Edward Hughes, et al. The hanabi challenge: A new frontier for ai research. *Artificial Intelligence*, 280:103216, 2020.
- Adrien Bardes, Jean Ponce, and Yann LeCun. Revisiting feature prediction for learning visual representations from video. *arXiv preprint arXiv:2404.08471*, 2024.
- Charles Beattie, Thomas K Ö ppe, Edgar A Du é ñ ez-Guzm á n, and Joel Z Leibo. Deepmind lab2d. *arXiv preprint arXiv:2011.07027*, 2020.
- M. G. Bellemare, Y. Naddaf, J. Veness, and M. Bowling. The arcade learning envi- ronment: An evaluation platform for general agents. *Journal of Artificial Intelli- gence Research*, 47:253-279, Jun 2013. ISSN 1076-9757. doi: 10.1613/jair.3912. URL <http://dx.doi.org/10.1613/jair.3912>.
- Christopher Berner, Greg Brockman, Brooke Chan, Vicki Cheung, Przemys ł aw D e biak, Christy Dennison, David Farhi, Quirin Fischer, Shariq Hashme, Chris Hesse, et al. Dota 2 with large scale deep reinforcement learning. *arXiv preprint arXiv:1912.06680*, 2019.
- Kevin Black, Michael Janner, Yilun Du, Ilya Kostrikov, and Sergey Levine. Training diffusion models with reinforcement learning. *arXiv preprint arXiv:2305.13301*, 2023.
- Andreas Blattmann, Tim Dockhorn, Sumith Kulal, Daniel Mendelevitch, Maciej Kilian, Dominik Lorenz, Yam Levi, Zion English, Vikram Voleti, Adam Letts, et al. Stable video diffusion: Scaling latent video diffusion models to large datasets. *arXiv preprint arXiv:2311.15127*, 2023a.
- Andreas Blattmann, Robin Rombach, Huan Ling, Tim Dockhorn, Seung Wook Kim, Sanja Fidler, and Karsten Kreis. Align your latents: High-resolution video synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 22563-22575, 2023b.
- David Brandfonbrener, Will Whitney, Rajesh Ranganath, and Joan Bruna. Offline rl without off-policy evaluation. *Advances in neural information processing systems*, 34: 4933-4946, 2021.
- Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym. *arXiv preprint arXiv:1606.01540*, 2016.
- Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, et al. Rt-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022.
- Tim Brooks, Bill Peebles, Connor Holmes, Will DePue, Yufei Guo, Li Jing, David Schnurr, Joe Taylor, Troy Luhman, Eric Luhman, et al. Video generation models as world simulators. *OpenAI Blog*, 2024.
- George W Brown. Iterative solution of games by fictitious play. *Activity analysis of production and allocation*, 13(1):374-376, 1951.

Noam Brown and Tuomas Sandholm. Superhuman ai for heads-up no-limit poker: Li-bratus beats top professionals. *Science*, 359(6374):418-424, 2018.

Noam Brown and Tuomas Sandholm. Superhuman ai for multiplayer poker. *Science*, 365 (6456):885-890, 2019.

Jake Bruce, Michael D Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, et al. Ge-nie: Generative interactive environments. In *Forty-first International Conference on Machine Learning*, 2024.

Ozan Candogan, Ishai Menache, Asuman Ozdaglar, and Pablo A Parrilo. Flows and decompositions of games: Harmonic and potential games. *Mathematics of Operations Research*, 36(3):474-503, 2011.

Haoxuan Che, Xuanhua He, Quande Liu, Cheng Jin, and Hao Chen. Gamegen-x: Interactive open-world game video generation. *arXiv preprint arXiv:2411.00769*, 2024.

Boyuan Chen, Bogdan Mazouze, Shang-Wen Lin, Yann Drouhard, Gabriel Goh, Jaewoo Yoon, Anirudh Srivastava, Chelsea Finn, Vincent Vanhoucke, Tim Salimans, et al. Diffusion forcing: Next-token prediction meets full-sequence diffusion. *arXiv preprint arXiv:2407.01392*, 2024a.

Chang Chen, Yi-Fu Wu, Jaesik Yoon, and Sungjin Ahn. Transdreamer: Reinforcement learning with transformer world models. *arXiv preprint arXiv:2202.09481*, 2022.

Haoxin Chen, Yong Zhang, Xiaodong Cun, Menghan Xia, Xintao Wang, Chao Weng, and Ying Shan. Videocrafter2: Overcoming data limitations for high-quality video diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 7310-7320, 2024b.

Li Chen, Justinas Lingys, Kai Chen, and Feng Liu. Auto: Scaling deep reinforcement learning for datacenter-scale automatic traffic optimization. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*, pages 191- 205. ACM, 2018.

Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Michael Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. Decision transformer: Reinforcement learning via sequence modeling. In *Thirty-Fifth Conference on Neural Information Processing Systems*, 2021. URL <https://openreview.net/forum?id=a7APmM4B9d>.

Taiye Chen, Zihan Ding, Anjian Li, Christina Zhang, Zeqi Xiao, Yisen Wang, and Chi Jin. Recurrent autoregressive diffusion: Global memory meets local attention. *arXiv preprint arXiv:2511.12940*, 2025a.

Taiye Chen, Xun Hu, Zihan Ding, and Chi Jin. Learning world models for interactive video generation. *arXiv preprint arXiv:2505.21996*, 2025b.

Taiye Chen, Xun Hu, Zihan Ding, and Chi Jin. Learning world models for interactive video generation. *arXiv preprint arXiv:2501.07891*, 2025c.

- Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. arXiv preprint arXiv:2303.04137, 2023.
- Rohan Chitnis, Yingchen Xu, Bobak Hashemi, Lucas Lehnert, Urun Dogan, Zheqing Zhu, and Olivier Delalleau. Iql-td-mpc: Implicit q-learning for hierarchical model predictive control. arXiv preprint arXiv:2306.00867, 2023.
- Hyungjin Chung, Jeongsol Kim, Michael T Mccann, Marc L Klasky, and Jong Chul Ye. Diffusion posterior sampling for general noisy inverse problems. arXiv preprint arXiv:2209.14687, 2022.
- Junyoung Chung, Caglar Gulcehre, KyungHyun Cho, and Yoshua Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling. arXiv preprint arXiv:1412.3555, 2014.
- Kevin Clark, Paul Vicol, Kevin Swersky, and David J Fleet. Directly fine-tuning diffusion models on differentiable rewards. arXiv preprint arXiv:2309.17400, 2023.
- Reuven Cohen, Matty Kadosh, Alan Lo, and Qasem Sayah. Lb scalability: Achieving the right balance between being stateful and stateless. IEEE/ACM Transactions on Networking, 2021.
- Yilun Cui et al. Self-forcing for video generation. arXiv preprint arXiv:2503.09543, 2025.
- Karan Dalal et al. Diffusion models with learned adaptive noise. arXiv preprint arXiv:2312.13236, 2023.
- Tri Dao and Albert Gu. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality. arXiv preprint arXiv:2405.21060, 2024.
- Constantinos Daskalakis and Ioannis Panageas. Last-iterate convergence: Zero-sum games and constrained min-max optimization. arXiv preprint arXiv:1807.04252, 2018.
- Constantinos Daskalakis, Dylan J Foster, and Noah Golowich. Independent policy gradient methods for competitive reinforcement learning. Advances in neural information processing systems, 33:5527-5540, 2020.
- Christian Schroeder de Witt, Tarun Gupta, Denys Makoviichuk, Viktor Makovychuk, Philip HS Torr, Mingfei Sun, and Shimon Whiteson. Is independent learning all you need in the starcraft multi-agent challenge? arXiv preprint arXiv:2011.09533, 2020.
- Sarah Dean, Horia Mania, Nikolai Matni, Benjamin Recht, and Stephen Tu. On the sample complexity of the linear quadratic regulator. Foundations of Computational Mathematics, 20(4):633-679, 2020.
- Decart and Etched. Oasis: A universe in a transformer. 2024. <https://oasis-model.github.io/>.
- DeepMind. Genie 3, 2025. <https://deepmind.google/>.
- Marc Peter Deisenroth, Gerhard Neumann, Jan Peters, et al. A survey on policy search for robotics. Foundations and Trends[®] in Robotics, 2(1-2):1-142, 2013.

- Yoann Desmouceaux, Pierre Pfister, Jérôme Tollet, Mark Townsley, and Thomas Clausen. 6lb: Scalable and application-aware load balancing with segment routing. *IEEE/ACM Transactions on Networking*, 26(2):819-834, 2018.
- Zihan Ding and Chi Jin. Consistency models as a rich and efficient policy class for reinforcement learning. *arXiv preprint arXiv:2309.16984*, 2023.
- Zihan Ding and Chi Jin. Dollar: Few-step video generation via distillation and latent reward optimization. *arXiv preprint arXiv:2412.15689*, 2024.
- Zihan Ding, Pablo Hernandez-Leal, Gavin Weiguang Ding, Changjian Li, and Ruitong Huang. Cdt: Cascading decision trees for explainable reinforcement learning. *arXiv preprint arXiv:2011.07553*, 2020.
- Zihan Ding, Dijia Su, Qinghua Liu, and Chi Jin. A deep reinforcement learning approach for finding non-exploitable strategies in two-player atari games. *arXiv preprint arXiv:2207.08894*, 2022.
- Zihan Ding, Amy Zhang, Yuandong Ma, Hantian Bhatt, et al. Diffusion world model. *arXiv preprint arXiv:2402.03570*, 2024.
- Le Cong Dinh, Yaodong Yang, Zheng Tian, Nicolas Perez Nieves, Oliver Slumbers, David Henry Mguni, Haitham Bou Ammar, and Jun Wang. Online double oracle. *arXiv preprint arXiv:2103.07780*, 2021.
- Alexander Domahidi, Eric Chu, and Stephen Boyd. Ecos: An socp solver for embedded systems. In *2013 European Control Conference (ECC)*, pages 3071-3076. IEEE, 2013.
- Carles Domingo-Enrich, Michal Drozdal, Brian Karrer, and Ricky TQ Chen. Adjoint matching: Fine-tuning flow and diffusion generative models with memoryless stochastic optimal control. *arXiv preprint arXiv:2409.08861*, 2024.
- Hao Dong, Hao Dong, Zihan Ding, Shanghang Zhang, and Chang. *Deep Reinforcement Learning*. Springer, 2020.
- Nicola Dragoni, Saverio Giallorenzo, Alberto Lluch Lafuente, Manuel Mazzara, Fabrizio Montesi, Ruslan Mustafin, and Larisa Safina. Microservices: yesterday, today, and tomorrow. In *Present and Ulterior Software Engineering*, pages 195-216. Springer, 2017.
- Melvin Dresher, Lloyd S Shapley, and Albert William Tucker. *Advances in Game Theory*. (AM-52), Volume 52, volume 52. Princeton University Press, 2016.
- Yilun Du, Mengjiao Yang, Bo Dai, Hanjun Dai, Ofir Nachum, Josh Tenenbaum, Dale Schuurmans, and Pieter Abbeel. Learning universal policies via text-guided video generation. *arXiv preprint arXiv:2302.00111*, 2023.
- Daniel E Eisenbud, Cheng Yi, Carlo Contavalli, Cody Smith, Roman Kononov, Eric Mann-Hielscher, Ardas Cilingiroglu, Bin Cheyney, Wentao Shang, and Jinnah Dylan Hosein. Maglev: A fast and reliable software network load balancer. In *NSDI*, pages 523-535, 2016.

Arpad E Elo and Sam Sloan. The rating of chessplayers: Past and present. (No Title), 1978.

Scott Emmons, Benjamin Eysenbach, Ilya Kostrikov, and Sergey Levine. Rvs: What is essential for offline rl via supervised learning? arXiv preprint arXiv:2112.10751, 2021.

Patrick Esser, Johnathan Chiu, Parmida Atighehchian, Jonathan Granskog, and Anastasis Germanidis. Structure and content-guided video synthesis with diffusion models. In 2023 IEEE/CVF International Conference on Computer Vision (ICCV), pages 7312- 7322, 2023. doi: 10.1109/ICCV51070.2023.00675.

Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, et al. Scaling rectified flow transformers for high-resolution image synthesis. In Forty-first international conference on machine learning, 2024.

Gabriele Farina, Christian Kroer, and Tuomas Sandholm. Stochastic regret minimization in extensive-form games. In International Conference on Machine Learning, pages 3018-3028. PMLR, 2020.

Vladimir Feinberg, Alvin Wan, Ion Stoica, Michael I Jordan, Joseph E Gonzalez, and Sergey Levine. Model-based value estimation for efficient model-free reinforcement learning. arXiv preprint arXiv:1803.00101, 2018.

Ruili Feng, Han Zheng, Zhantao Zhong, Jie Wu, Evelyn Peng, Qiwei Lu, Song Peng, Xing Qin, et al. The matrix: Infinite-horizon world generation with real-time moving control. arXiv preprint arXiv:2412.03568, 2024.

Jerzy Filar and Koos Vrieze. Competitive Markov decision processes. Springer Science & Business Media, 2012.

Jakob Foerster, Gregory Farquhar, Triantafyllos Afouras, Nantas Nardelli, and Shimon Whiteson. Counterfactual multi-agent policy gradients. In Proceedings of the AAAI conference on artificial intelligence, volume 32, 2018.

Roy Fox, Stephen M McAleer, Will Overman, and Ioannis Panageas. Independent natural policy gradient always converges in markov potential games. In International Conference on Artificial Intelligence and Statistics, pages 4414-4425. PMLR, 2022.

Dan Friedman and Adjani Bousso Dieng. The vendi score: A diversity evaluation metric for machine learning. arXiv preprint arXiv:2210.02410, 2022.

Nicholas Frosst and Geoffrey Hinton. Distilling a neural network into a soft decision tree. arXiv preprint arXiv:1711.09784, 2017.

Justin Fu, Aviral Kumar, Ofir Nachum, George Tucker, and Sergey Levine. D4rl: Datasets for deep data-driven reinforcement learning. arXiv preprint arXiv:2004.07219, 2020.

Drew Fudenberg, Fudenberg Drew, David K Levine, and David K Levine. The theory of learning in games, volume 2. MIT press, 1998.

Scott Fujimoto and Shixiang Gu. A minimalist approach to offline reinforcement learning. In Thirty-Fifth Conference on Neural Information Processing Systems, 2021. URL <https://openreview.net/forum?id=Q32U7dzWXpc>.

Scott Fujimoto, Herke Hoof, and David Meger. Addressing function approximation error in actor-critic methods. In International conference on machine learning, pages 1587- 1596. PMLR, 2018.

Scott Fujimoto, David Meger, and Doina Precup. Off-policy deep reinforcement learning without exploration. In International Conference on Machine Learning, pages 2052- 2062. PMLR, 2019.

Ruiqi Gao, Aleksander Holynski, Philipp Henzler, Arthur Brussee, Ricardo Martin- Brualla, Pratul Srinivasan, Jonathan T Barron, and Ben Poole. Cat3d: Create anything in 3d with multi-view diffusion models. arXiv preprint arXiv:2405.10314, 2024.

Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. arXiv preprint arXiv:2312.10997, 2023.

Divyansh Garg, Joey Hejna, Matthieu Geist, and Stefano Ermon. Extreme q-learning: Maxent rl without entropy. arXiv preprint arXiv:2301.02328, 2023.

Honghao Ge et al. Robust world models: Measuring and improving robustness of video diffusion world models. arXiv preprint arXiv:2502.14800, 2025.

Kamyar Ghasemipour, Shixiang Shane Gu, and Ofir Nachum. Why so pessimistic? es- timating uncertainties for offline rl through ensembles, and why their independence matters. Advances in Neural Information Processing Systems, 35:18267-18281, 2022.

Shao-Xiang Go, Yu Jiang, and Desmond K Loke. A phase-change memristive reinforce- ment learning for rapidly outperforming champion street-fighter players. Advanced Intelligent Systems, 5(11):2300335, 2023.

Wonjoon Goo and Scott Niekum. Know your boundaries: The necessity of explicit be- havioral cloning in offline rl. arXiv preprint arXiv:2206.00695, 2022.

Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. Advances in neural information processing systems, 27, 2014.

Guy Goren, Shay Vargaftik, and Yoram Moses. Distributed dispatching in the parallel server model. arXiv:2008.00793 [cs], Aug 2020. arXiv: 2008.00793.

Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. arXiv preprint arXiv:2312.00752, 2023.

Chuanxiong Guo, Lihua Yuan, Dong Xiang, Yingnong Dang, Ray Huang, Dave Maltz, Zhaoyi Liu, Vin Wang, Bin Pang, Hua Chen, et al. Pingmesh: A large-scale system for data center network latency measurement and analysis. In Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication, pages 139-152, 2015.

Junliang Guo, Yang Ye, Tianyu He, Haoyu Wu, Yushu Jiang, Tim Pearce, and Jiang Bian. Mineworld: a real-time and open-source interactive world model on minecraft. arXiv preprint arXiv:2504.08388, 2025.

William H Guss, Brandon Houghton, Nicholay Topin, Phillip Wang, Cayden Codel, Manuela Veloso, and Ruslan Salakhutdinov. Minerl: A large-scale dataset of minecraft demonstrations. arXiv preprint arXiv:1907.13440, 2019.

David Ha. Slime volleyball gym environment. <https://github.com/hardmaru/slimevolleygym>, 2020.

David Ha and Jürgen Schmidhuber. Recurrent world models facilitate policy evolution. Advances in neural information processing systems, 31, 2018a.

David Ha and Jürgen Schmidhuber. World models. arXiv preprint arXiv:1803.10122, 2018b.

Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In International Conference on Machine Learning, pages 1861-1870, 2018.

Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. arXiv preprint arXiv:1912.01603, 2019a.

Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In International conference on machine learning, pages 2555-2565. PMLR, 2019b.

Danijar Hafner, Timothy Lillicrap, Mohammad Norouzi, and Jimmy Ba. Mastering atari with discrete world models. arXiv preprint arXiv:2010.02193, 2020.

Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. arXiv preprint arXiv:2301.04104, 2023.

Nicklas Hansen, Yixin Lin, Hao Su, Xiaolong Wang, Vikash Kumar, and Aravind Rajeswaran. Modem: Accelerating visual model-based reinforcement learning with demonstrations. arXiv preprint arXiv:2212.05698, 2022a.

Nicklas Hansen, Xiaolong Wang, and Hao Su. Temporal difference learning for model predictive control. arXiv preprint arXiv:2203.04955, 2022b.

Nicklas Hansen, Hao Su, and Xiaolong Wang. Td-mpc2: Scalable, robust world models for continuous control. arXiv preprint arXiv:2310.16828, 2023.

Philippe Hansen-Estruch, Ilya Kostrikov, Michael Janner, Jakub Grudzien Kuba, and Sergey Levine. Idql: Implicit q-learning as an actor-critic method with diffusion policies. arXiv preprint arXiv:2304.10573, 2023.

William Harvey, Søren Nørskov, Niklas Kölsch, and George Vogiatzis. Flexible diffusion modeling of long videos. arXiv preprint arXiv:2205.11495, 2022.

- Xuan He, Dongfu Jiang, Ge Zhang, Max Ku, Achint Soni, Sherman Siu, Haonan Chen, Abhramil Chandra, Ziyang Jiang, Aaran Arulraj, et al. Videoscore: Building automatic metrics to simulate fine-grained human feedback for video generation. arXiv preprint arXiv:2406.15252, 2024.
- Johannes Heinrich and David Silver. Deep reinforcement learning from self-play in imperfect-information games, 2016.
- Johannes Heinrich, Marc Lanctot, and David Silver. Fictitious self-play in extensive-form games. In International conference on machine learning, pages 805-813. PMLR, 2015a.
- Johannes Heinrich, Marc Lanctot, and David Silver. Fictitious self-play in extensive-form games. In Francis Bach and David Blei, editors, Proceedings of the 32nd International Conference on Machine Learning, volume 37 of Proceedings of Machine Learning Research, pages 805-813, Lille, France, 07-09 Jul 2015b. PMLR. URL <https://proceedings.mlr.press/v37/heinrich15.html>.
- Roberto Henschel, Levon Khachatryan, Daniil Hayrapetyan, Hayk Poghosyan, Vahram Tadevosyan, Zhangyang Wang, Shant Navasardyan, and Humphrey Shi. Streamingt2v: Consistent, dynamic, and extendable long video generation from text. arXiv preprint arXiv:2403.14773, 2024.
- Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. arXiv preprint arXiv:2207.12598, 2022.
- Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In Advances in Neural Information Processing Systems, volume 33, pages 6840-6851, 2020.
- Jonathan Ho, Tim Salimans, Alexey Gritsenko, William Chan, Mohammad Norouzi, and David J. Fleet. Video diffusion models. arXiv preprint arXiv:2204.03458, 2022.
- Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. Neural computation, 9(8):1735-1780, 1997.
- Yining Hong, Beide Zhang, Maxine Zheng, Tianyu Chen, Yizhou Chen, Longshen Li, Kevin Lin, Jianfeng Wang, Silvio Savarese, Caiming Li, et al. Slowfast-vgen: Slow-fast learning for action-driven long video generation. arXiv preprint arXiv:2410.23277, 2024.
- Yu Hong, Jing Wei, Xing Liu, Xiaodi Wang, Yutong Bai, Haitao Li, Ming Zhang, and Hao Xu. Cogvideo: Large-scale pretraining for text-to-video generation with transformers. arXiv preprint arXiv:2205.15868, 2022.
- Anthony Hu, Lloyd Russell, Hudson Yeo, Zak Murez, George Fedoseev, Alex Kendall, Jamie Shotton, and Gianluca Corrado. Gaia-1: A generative world model for autonomous driving. arXiv preprint arXiv:2309.17080, 2023a.
- Jian Hu, Siyang Jiang, Seth Austin Harding, Haibin Wu, and Shih-wei Liao. Rethinking the implementation tricks and monotonicity constraint in cooperative multi-agent reinforcement learning. arXiv preprint arXiv:2102.03479, 2021.

Junling Hu and Michael P Wellman. Nash q-learning for general-sum stochastic games. *Journal of machine learning research*, 4(Nov):1039-1069, 2003.

Siyi Hu, Yifan Zhong, Minquan Gao, Weixun Wang, Hao Dong, Xiaodan Liang, Zhihui Li, Xiaojun Chang, and Yaodong Yang. Marllib: A scalable and efficient multi-agent reinforcement learning library. *Journal of Machine Learning Research*, 24(315):1-23, 2023b.

Baihe Huang, Jason D Lee, Zhaoran Wang, and Zhuoran Yang. Towards general function approximation in zero-sum markov games. *arXiv preprint arXiv:2107.14702*, 2021.

Xun Huang, Chitwan Saharia, Tim Salimans, and Mohammad Norouzi. Self- forcing: Bridging the train-test gap in autoregressive video diffusion. *arXiv preprint arXiv:2506.01713*, 2025.

Ziqi Huang, Yinan He, Jiashuo Yu, Fan Zhang, Chenyang Si, Yuming Jiang, Yuanhan Zhang, Tianxing Wu, Qingyang Jin, Nattapol Chanpaisit, et al. Vbench: Comprehensive benchmark suite for video generative models, 2023a.

Ziqi Huang, Yinan He, Jiashuo Yu, Fan Zhang, Chenyang Si, Yuming Jiang, Yuanhan Zhang, Tianxing Wu, Qingyang Jin, Nattapol Chanpaisit, et al. Vbench: Comprehensive benchmark suite for video generative models. *arXiv preprint arXiv:2311.17982*, 2023b.

Ziqi Huang, Yinan He, Jiashuo Yu, Fan Zhang, Chenyang Si, Yuming Jiang, Yuanhan Zhang, Tianxing Wu, Qingyang Jin, Nattapol Chanpaisit, et al. Vbench: Comprehensive benchmark suite for video generative models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 21807-21818, 2024.

Matthew Thomas Jackson, Michael Tryfan Matthews, Cong Lu, Benjamin Ellis, Shimon Whiteson, and Jakob Foerster. Policy-guided diffusion. *arXiv preprint arXiv:2404.06356*, 2024.

Robert A Jacobs, Michael I Jordan, Steven J Nowlan, and Geoffrey E Hinton. Adaptive mixtures of local experts. *Neural computation*, 3(1):79-87, 1991.

Eric Jang, Shixiang Gu, and Ben Poole. Categorical reparameterization with gumbel- softmax. *arXiv preprint arXiv:1611.01144*, 2016.

Michael Janner, Justin Fu, Marvin Zhang, and Sergey Levine. When to trust your model: Model-based policy optimization. *Advances in neural information processing systems*, 32, 2019.

Michael Janner, Igor Mordatch, and Sergey Levine. gamma-models: Generative temporal difference learning for infinite-horizon prediction. *Advances in Neural Information Processing Systems*, 33:1724-1735, 2020.

Michael Janner, Qiyang Li, and Sergey Levine. Offline reinforcement learning as one big sequence modeling problem. In *Thirty-Fifth Conference on Neural Information Processing Systems*, 2021. URL <https://openreview.net/forum?id=wgeK563QgSw>.

Michael Janner, Yilun Du, Joshua B Tenenbaum, and Sergey Levine. Planning with diffusion for flexible behavior synthesis. *arXiv preprint arXiv:2205.09991*, 2022.

- Natasha Jaques, Asma Ghandeharioun, Judy Hanwen Shen, Craig Ferguson, Agata Lapedriza, Noah Jones, Shixiang Gu, and Rosalind Picard. Way off-policy batch deep reinforcement learning of implicit human preferences in dialog. arXiv preprint arXiv:1907.00456, 2019.
- Zhiwei Jia, Fangchen Liu, Vineet Thummuluri, Linghao Chen, Zhiao Huang, and Hao Su. Chain-of-thought predictive control. arXiv preprint arXiv:2304.00776, 2023.
- Jianwen Jiang, Chao Liang, Jiaqi Yang, Gaojie Lin, Tianyun Zhong, and Yanbo Zheng. Loopy: Taming audio-driven portrait avatar with long-term motion dependency. arXiv preprint arXiv:2409.02634, 2024.
- Jiechuan Jiang and Zongqing Lu. Learning fairness in multi-agent systems. Advances in Neural Information Processing Systems, 32, 2019.
- Chi Jin, Qinghua Liu, Yuanhao Wang, and Tiancheng Yu. V-learning-a simple, efficient, decentralized algorithm for multiagent rl. arXiv preprint arXiv:2110.14555, 2021a.
- Chi Jin, Qinghua Liu, and Tiancheng Yu. The power of exploiter: Provable multi-agent rl in large state spaces. arXiv preprint arXiv:2106.03352, 2021b.
- Yang Jin, Zhicheng Sun, Ningyuan Li, Kun Xu, Hao Jiang, Nan Zhuang, Quzhe Huang, Yang Song, Yadong Mu, and Zhouchen Lin. Pyramidal flow matching for efficient video generative modeling. arXiv preprint arXiv:2410.05954, 2024.
- Lukasz Kaiser, Mohammad Babaeizadeh, Piotr Milos, Blazej Osinski, Roy H Campbell, Konrad Czechowski, Dumitru Erhan, Chelsea Finn, Piotr Kozakowski, Sergey Levine, et al. Model-based reinforcement learning for atari. arXiv preprint arXiv:1903.00374, 2019.
- Bingyi Kang, Yang Yue, Rui Lu, Zhijie Lin, Yang Zhao, Kaixin Wang, Gao Huang, and Jiashi Feng. How far is video generation from world model: A physical law perspective. arXiv preprint arXiv:2411.02385, 2024.
- Tero Karras, Miika Aittala, Timo Aila, and Samuli Laine. Elucidating the design space of diffusion-based generative models. Advances in Neural Information Processing Systems, 35:26565-26577, 2022.
- Levon Khachatryan, Adrien Davy, Baptiste Emond, and Jun Wang. Text2video-zero: Zero-shot text-to-video generation using pretrained text-to-image diffusion models. arXiv preprint arXiv:2302.01327, 2023.
- Ibrahim Khan, Thai Van Nguyen, Xincheng Dai, and Ruck Thawonmas. Darefighting- ice competition: A fighting game sound design and ai competition. In 2022 IEEE Conference on Games (CoG), pages 478-485. IEEE, 2022.
- Rahul Kidambi, Aravind Rajeswaran, Praneeth Netrapalli, and Thorsten Joachims. Morel: Model-based offline reinforcement learning. Advances in neural information processing systems, 33:21810-21823, 2020.

- Dongjun Kim, Chieh-Hsin Lai, Wei-Hsiang Liao, Naoki Murata, Yuhta Takida, Toshimitsu Uesaka, Yutong He, Yuki Mitsufuji, and Stefano Ermon. Consistency trajectory models: Learning probability flow ode trajectory of diffusion. arXiv preprint arXiv:2310.02279, 2023.
- Jihwan Kim, Junoh Park, Jinyoung Choi, and Bohyung Han. Fifo-diffusion: Generating infinite videos from text without training. arXiv preprint arXiv:2405.11473, 2024.
- Diederik Kingma, Tim Salimans, Ben Poole, and Jonathan Ho. Variational diffusion models. Advances in neural information processing systems, 34:21696-21707, 2021.
- Diederik P Kingma. Auto-encoding variational bayes. arXiv preprint arXiv:1312.6114, 2013.
- Diederik P Kingma and Max Welling. Auto-encoding variational bayes. arXiv preprint arXiv:1312.6114, 2013.
- Yuval Kirstain, Adam Polyak, Uriel Singer, Shahbuland Matiana, Joe Penna, and Omer Levy. Pick-a-pic: An open dataset of user preferences for text-to-image generation. Advances in Neural Information Processing Systems, 36:36652-36663, 2023.
- Vijay R Konda and John N Tsitsiklis. Actor-critic algorithms. In Advances in neural information processing systems, pages 1008-1014, 2000.
- Weijie Kong and et al Qi Tian. Hunyuanvideo: A systematic framework for large video generative models, 2024. URL <https://arxiv.org/abs/2412.03603>.
- Ilya Kostrikov, Ashvin Nair, and Sergey Levine. Offline reinforcement learning with implicit q-learning, 2021.
- Tadashi Kozuno, Pierre Ménard, Rémi Munos, and Michal Valko. Model-free learning for two-player zero-sum partially observable markov games with perfect recall. arXiv preprint arXiv:2106.06279, 2021.
- Kuaishou. Kling. <https://kling.kuaishou.com/en>, 2024. Accessed: [today's date].
- Adithya Kumar, IySwarya Narayanan, Timothy Zhu, and Anand Sivasubramaniam. The fast and the frugal: Tail latency aware provisioning for coping with load variations. In Proceedings of The Web Conference 2020, pages 314-326, 2020a.
- Aviral Kumar, Justin Fu, George Tucker, and Sergey Levine. Stabilizing off-policy q-learning via bootstrapping error reduction. arXiv preprint arXiv:1906.00949, 2019.
- Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. Conservative q-learning for offline reinforcement learning. arXiv preprint arXiv:2006.04779, 2020b.
- Karol Kurach, Anton Raichuk, Piotr Ściech, Michał Żajac, Olivier Bachem, Lasse Espeholt, Carlos Riquelme, Damien Vincent, Marcin Michalski, Olivier Bousquet, et al. Google research football: A novel reinforcement learning environment. In Proceedings of the AAAI conference on artificial intelligence, volume 34, pages 4501-4510, 2020.
- Nathan Lambert, Kristofer Pister, and Roberto Calandra. Investigating compounding prediction errors in learned dynamics models. arXiv preprint arXiv:2203.09637, 2022.

- Marc Lanctot, Kevin Waugh, Martin Zinkevich, and Michael Bowling. Monte carlo sampling for regret minimization in extensive games. *Advances in neural information processing systems*, 22, 2009.
- Marc Lanctot, Vinicius Zambaldi, Audrunas Gruslys, Angeliki Lazaridou, Karl Tuyls, Julien P érolat, David Silver, and Thore Graepel. A unified game-theoretic approach to multiagent reinforcement learning. *Advances in neural information processing systems*, 30, 2017.
- Marc Lanctot, Edward Lockhart, Jean-Baptiste Lespiau, Vinicius Zambaldi, Satyaki Upadhyay, Julien P érolat, Sriram Srinivasan, Finbarr Timbers, Karl Tuyls, Shayegan Omidshafiei, et al. Openspiel: A framework for reinforcement learning in games. *arXiv preprint arXiv:1908.09453*, 2019.
- Kuang-Huei Lee, Ofir Nachum, Mengjiao Yang, Lisa Lee, Daniel Freeman, Winnie Xu, Sergio Guadarrama, Ian Fischer, Eric Jang, Henryk Michalewski, et al. Multi-game decision transformers. *arXiv preprint arXiv:2205.15241*, 2022a.
- Seunghyun Lee, Younggyo Seo, Kimin Lee, Pieter Abbeel, and Jinwoo Shin. Offline-to- online reinforcement learning via balanced replay and pessimistic q-ensemble. *arXiv preprint arXiv:2107.00591*, 2022b.
- Stefanos Leonardos, Will Overman, Ioannis Panageas, and Georgios Piliouras. Global convergence of multi-agent policy gradient in markov potential games. *arXiv preprint arXiv:2106.01969*, 2021.
- Hepeng Li and Haibo He. Multi-agent trust region policy optimization. *arXiv preprint arXiv:2010.07916*, 2020.
- Jiachen Li, Weixi Feng, Wenhui Chen, and William Yang Wang. Reward guided latent consistency distillation. *arXiv preprint arXiv:2403.11027*, 2024a.
- Jiachen Li, Weixi Feng, Tsu-Jui Fu, Xinyi Wang, Sugato Basu, Wenhui Chen, and William Yang Wang. T2v-turbo: Breaking the quality bottleneck of video consistency model with mixed reward feedback. *arXiv preprint arXiv:2405.18750*, 2024b.
- Jiachen Li, Qian Long, Jian Zheng, Xiaofeng Gao, Robinson Piramuthu, Wenhui Chen, and William Yang Wang. T2v-turbo-v2: Enhancing video generation model post- training through data, reward, and conditional guidance design. *arXiv preprint arXiv:2410.05677*, 2024c.
- Junjie Li, Sotetsu Koyamada, Qiwei Ye, Guoqing Liu, Chao Wang, Ruihan Yang, Li Zhao, Tao Qin, Tie-Yan Liu, and Hsiao-Wuen Hon. Suphx: Mastering mahjong with deep reinforcement learning. *arXiv preprint arXiv:2003.13590*, 2020.
- Wenzhe Li, Zihan Ding, Seth Korten, and Chi Jin. Fightladder: A benchmark for competitive multi-agent reinforcement learning. *arXiv preprint arXiv:2406.02081*, 2024d.
- Zongyi Li, Shujie Hu, Shujie Liu, Long Zhou, Jeongsoo Choi, Lingwei Meng, Xun Guo, Jinyu Li, Hefei Ling, and Furu Wei. Arlon: Boosting diffusion transformers with autoregressive models for long video generation. *arXiv preprint arXiv:2410.20502*, 2024e.

Timothy P Lillicrap, Jonathan J Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. arXiv preprint arXiv:1509.02971, 2015.

Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. arXiv preprint arXiv:2210.02747, 2022.

Michael L Littman. Markov games as a framework for multi-agent reinforcement learning. In Machine learning proceedings 1994, pages 157-163. Elsevier, 1994.

Fangfu Liu, Wenqiang Sun, Hanyang Wang, Yikai Wang, Haowen Sun, Junliang Ye, Jun Zhang, and Yueqi Duan. Reconx: Reconstruct any scene from sparse views with video diffusion model. arXiv preprint arXiv:2408.16767, 2024.

Qinghua Liu, Tiancheng Yu, Yu Bai, and Chi Jin. A sharp analysis of model-based reinforcement learning with self-play. In International Conference on Machine Learning, pages 7001-7010. PMLR, 2021.

Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. arXiv preprint arXiv:2209.03003, 2022.

Xingchao Liu, Xiwen Zhang, Jianzhu Ma, Jian Peng, et al. InstafLOW: One step is enough for high-quality diffusion-based text-to-image generation. In The Twelfth International Conference on Learning Representations, 2023.

Ryan Lowe, Yi I Wu, Aviv Tamar, Jean Harb, OpenAI Pieter Abbeel, and Igor Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. Advances in neural information processing systems, 30, 2017.

Chao Lu, Yuhao Zhou, Fan Bao, Jianfei Chen, Chongxuan Li, and Jun Zhu. DPM-Solver: A fast ode solver for diffusion probabilistic model sampling in around 10 steps. arXiv preprint arXiv:2206.00927, 2022.

Cheng Lu and Yang Song. Simplifying, stabilizing and scaling continuous-time consistency models. arXiv preprint arXiv:2410.11081, 2024.

Cheng Lu, Huayu Chen, Jianfei Chen, Hang Su, Chongxuan Li, and Jun Zhu. Contrastive energy prediction for exact energy-guided diffusion sampling in offline reinforcement learning. arXiv preprint arXiv:2304.12824, 2023a.

Cong Lu, Philip J Ball, and Jack Parker-Holder. Synthetic experience replay. arXiv preprint arXiv:2303.06614, 2023b.

Eric Luhman and Tobias Luhman. Knowledge distillation for generative models. arXiv preprint arXiv:2106.05237, 2021.

Simian Luo, Yiqin Tan, Longbo Huang, Jian Li, and Hang Zhao. Latent consistency models: Synthesizing high-resolution images with few-step inference. arXiv preprint arXiv:2310.04378, 2023.

Weijian Luo, Tianyang Hu, Shifeng Zhang, Jiacheng Sun, Zhenguo Li, and Zhihua Zhang. Diff-instruct: A universal approach for transferring knowledge from pre-trained diffusion models. Advances in Neural Information Processing Systems, 36, 2024.

- Sergio Valcarcel Macua, Javier Zazo, and Santiago Zazo. Learning parametric closed-loop policies for markov potential games. arXiv preprint arXiv:1802.00899, 2018.
- Hongzi Mao, Malte Schwarzkopf, Shaileshh Bojja Venkatakrishnan, Zili Meng, and Mohammad Alizadeh. Learning scheduling algorithms for data processing clusters. arXiv preprint arXiv:1810.01963, 2018.
- Jason R Marden, Gürdal Arslan, and Jeff S Shamma. Cooperative control and potential games. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, 39(6):1393-1407, 2009.
- Stephen McAleer, John Lanier, Pierre Baldi, and Roy Fox. Xdo: A double oracle algorithm for extensive-form games. arXiv preprint arXiv:2103.06426, 2021.
- H Brendan McMahan, Geoffrey J Gordon, and Avrim Blum. Planning in the presence of cost functions controlled by an adversary. In *Proceedings of the 20th International Conference on Machine Learning (ICML-03)*, pages 536-543, 2003.
- Linghui Meng, Muning Wen, Yaodong Yang, Chenyang Le, Xiyun Li, Weinan Zhang, Ying Wen, Haifeng Zhang, Jun Wang, and Bo Xu. Offline pre-trained multi-agent decision transformer: One big sequence model conquers all starcraftii tasks. arXiv preprint arXiv:2112.02845, 2021.
- David H Mguni, Yutong Wu, Yali Du, Yaodong Yang, Ziyi Wang, Minne Li, Ying Wen, Joel Jennings, and Jun Wang. Learning in nonzero-sum stochastic games with potentials. In *International Conference on Machine Learning*, pages 7688-7699. PMLR, 2021.
- Rui Miao, Hongyi Zeng, Changhoon Kim, Jeongkeun Lee, and Minlan Yu. Silkroad: Making stateful layer-4 load balancing fast and cheap using switching ASICs. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication, SIGCOMM '17*, page 15-28. ACM, 2017. ISBN 978-1-4503-4653-5. event-place: Los Angeles, CA, USA.
- Vincent Micheli, Eloi Alonso, and François Fleuret. Transformers are sample efficient world models. arXiv preprint arXiv:2209.00588, 2022.
- Misra D Mish. A self regularized non-monotonic activation function [j]. arXiv preprint arXiv:1908.08681, 2019.
- Utkarsh A Mishra and Yongxin Chen. Reorientdiff: Diffusion model based reorientation for object manipulation. arXiv preprint arXiv:2303.12700, 2023.
- Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing atari with deep reinforcement learning. arXiv preprint arXiv:1312.5602, 2013.
- Sharada Mohanty, Erik Nygren, Florian Laurent, Manuel Schneider, Christian Scheller, Nilabha Bhattacharya, Jeremy Watson, Adrian Egli, Christian Eichenberger, Christian Baumberger, et al. Flatland-rl: Multi-agent reinforcement learning on trains. arXiv preprint arXiv:2012.05893, 2020.

- Dov Monderer and Lloyd S Shapley. Potential games. *Games and economic behavior*, 14 (1):124-143, 1996.
- Matej Moravčík, Martin Schmid, Neil Burch, Viliam Lisý, Dustin Morrill, Nolan Bard, Trevor Davis, Kevin Waugh, Michael Johanson, and Michael Bowling. Deepstack: Expert-level artificial intelligence in heads-up no-limit poker. *Science*, 356(6337):508- 513, 2017.
- Igor Mordatch and Pieter Abbeel. Emergence of grounded compositional language in multi-agent populations. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32, 2018.
- Tsendsuren Munkhdalai, Manaal Faruqui, and Siddharth Gopal. Leave no context behind: Efficient infinite context transformers with infinite-attention, 2024.
- David Murphy. Hacking public memory: Understanding the multiple arcade machine emulator. *Games and Culture*, 8(1):43-53, 2013.
- Anusha Nagabandi, Gregory Kahn, Ronald S Fearing, and Sergey Levine. Neural network dynamics for model-based deep reinforcement learning with model-free fine-tuning. In *2018 IEEE international conference on robotics and automation (ICRA)*, pages 7559- 7566. IEEE, 2018.
- Ashvin Nair, Murtaza Dalal, Abhishek Gupta, and Sergey Levine. Accelerating online reinforcement learning with offline datasets. *CoRR*, abs/2006.09359, 2020. URL <https://arxiv.org/abs/2006.09359>.
- Mitsuhiko Nakamoto, Yuexiang Zhai, Anikait Singh, Max Sobol Mark, Yi Ma, Chelsea Finn, Aviral Kumar, and Sergey Levine. Cal-ql: Calibrated offline rl pre-training for efficient online fine-tuning. *arXiv preprint arXiv:2303.05479*, 2023.
- John F Nash et al. Equilibrium points in n-person games. *Proceedings of the national academy of sciences*, 36(1):48-49, 1950.
- Tung Nguyen, Qingqing Zheng, and Aditya Grover. Conserweightive behavioral cloning for reliable offline reinforcement learning. *arXiv preprint arXiv:2210.05158*, 2022.
- Alex Nichol, Vicki Pfau, Christopher Hesse, Oleg Klimov, and John Schulman. Gotta learn fast: A new benchmark for generalization in rl. *arXiv preprint arXiv:1804.03720*, 2018.
- Alexander Quinn Nichol and Prafulla Dhariwal. Improved denoising diffusion probabilistic models. In *International Conference on Machine Learning*, pages 8162-8171. PMLR, 2021.
- NVIDIA et al. Cosmos world foundation model platform for physical ai. *arXiv preprint arXiv:2501.12599*, 2025.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730-27744, 2022.

Alessandro Palmas. Diambra arena: a new reinforcement learning platform for research and experimentation. arXiv preprint arXiv:2210.10595, 2022.

Xuehai Pan, Mickel Liu, Fangwei Zhong, Yaodong Yang, Song-Chun Zhu, and Yizhou Wang. Mate: Benchmarking multi-agent reinforcement learning in distributed target coverage control. Advances in Neural Information Processing Systems, 35:27862-27879, 2022.

Georgios Papoudakis, Filippos Christianos, Lukas Sch ¨ a fer, and Stefano V Albrecht. Benchmarking multi-agent deep reinforcement learning algorithms in cooperative tasks. arXiv preprint arXiv:2006.07869, 2020.

Jack Parker-Holder, Philip Ball, et al. Genie 2: A large-scale foundation world model. 2024.
<https://deepmind.google/discover/blog/genie-2-a-large-scale-foundation-world-model/>.

Jurgis Pasukonis, Timothy Lillicrap, and Daniil Hafner. Evaluating long-term memory in 3d mazes. arXiv preprint arXiv:2210.13383, 2022.

Tim Pearce, Tabish Rashid, Anssi Kanervisto, Dave Bignell, Mingfei Sun, Raluca Georgescu, Sergio Valcarcel Macua, Shan Zheng Tan, Ida Momennejad, Katja Hofmann, et al. Imitating human behaviour with diffusion models. arXiv preprint arXiv:2301.10677, 2023.

William Peebles and Saining Xie. Scalable diffusion models with transformers. In Proceedings of the IEEE/CVF International Conference on Computer Vision, pages 4195- 4205, 2023.

Bei Peng, Tabish Rashid, Christian Schroeder de Witt, Pierre-Alexandre Kamienny, Philip Torr, Wendelin B ¨ o hmer, and Shimon Whiteson. Facmac: Factored multi-agent centralised policy gradients. Advances in Neural Information Processing Systems, 34: 12208-12221, 2021.

Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. Yarn: Efficient context window extension of large language models. arXiv preprint arXiv:2309.00071, 2023.

Xue Bin Peng, Aviral Kumar, Grace Zhang, and Sergey Levine. Advantage-weighted regression: Simple and scalable off-policy reinforcement learning. arXiv preprint arXiv:1910.00177, 2019.

Jan Peters and Stefan Schaal. Natural actor-critic. Neurocomputing, 71(7-9):1180-1190, 2008.

Pika Labs. Pika Labs. <https://www.pika.art/>. Accessed: September 25, 2023.

Ryan Po et al. Long-context state space video world models. arXiv preprint arXiv:2503.11071, 2025.

Antonio Polino, Razvan Pascanu, and Dan Alistarh. Model compression via distillation and quantization. arXiv preprint arXiv:1802.05668, 2018.

Adam Polyak and et al Amit Zohar. Movie gen: A cast of media foundation models, 2025. URL <https://arxiv.org/abs/2410.13720>.

Adam Polyak, Amit Zohar, Andrew Brown, Andros Tjandra, Animesh Sinha, Ann Lee, Apoorv Vyas, Bowen Shi, Chih-Yao Ma, Ching-Yao Chuang, et al. Movie gen: A cast of media foundation models. arXiv preprint arXiv:2410.13720, 2024.

Ben Poole, Ajay Jain, Jonathan T Barron, and Ben Mildenhall. Dreamfusion: Text-to-3d using 2d diffusion. arXiv preprint arXiv:2209.14988, 2022.

Mihir Prabhudesai, Russell Mendonca, Zheyang Qin, Katerina Fragkiadaki, and Deepak Pathak. Video diffusion alignment via reward gradients. arXiv preprint arXiv:2407.08737, 2024.

Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. Journal of Machine Learning Research, 22(268):1-8, 2021. URL <http://jmlr.org/papers/v22/20-1364.html>.

Aditya Ramesh, Prafulla Dhariwal, Alex Nichol, Casey Chu, and Mark Chen. Hierarchical text-conditional image generation with clip latents. arXiv preprint arXiv:2204.06125, 2022.

Tabish Rashid, Mikayel Samvelyan, Christian Schroeder, Gregory Farquhar, Jakob Foerster, and Shimon Whiteson. Qmix: Monotonic value function factorisation for deep multi-agent reinforcement learning. In International Conference on Machine Learning, pages 4295-4304. PMLR, 2018.

Tabish Rashid, Mikayel Samvelyan, Christian Schroeder De Witt, Gregory Farquhar, Jakob Foerster, and Shimon Whiteson. Monotonic value function factorisation for deep multi-agent reinforcement learning. The Journal of Machine Learning Research, 21(1):7234-7284, 2020.

Allen Z Ren, Justin Lidard, Lars L Ankile, Anthony Simeonov, Pulkit Agrawal, Anirudha Majumdar, Benjamin Burchfiel, Hongkai Dai, and Max Simchowitz. Diffusion policy optimization. arXiv preprint arXiv:2409.00588, 2024.

Jie Ren, Yewen Li, Zihan Ding, Wei Pan, and Hao Dong. Probabilistic mixture-of-experts for efficient deep reinforcement learning. arXiv preprint arXiv:2104.09122, 2021.

Xuanchi Ren, Yifan Lu, Tianshi Cao, Ruiyuan Gao, Shengyu Huang, Amirmojtaba Sabour, Tianchang Shen, Tobias Pfaff, Jay Zhangjie Wu, Runjian Chen, Seung Wook Kim, Jun Gao, Laura Leal-Taixe, Mike Chen, Sanja Fidler, and Huan Ling. Cosmos-drive-dreams: Scalable synthetic driving data generation with world foundation models, 2025a. URL <https://arxiv.org/abs/2506.09042>.

Xuanchi Ren, Tianchang Shen, Jiahui Huang, Huan Ling, Yifan Lu, Merlin Nimier-David, Thomas Müller, Alexander Keller, Sanja Fidler, and Jun Gao. Gen3c: 3d-informed world-consistent video generation with precise camera control. arXiv preprint arXiv:2503.03751, 2025b.

Cinjon Resnick, Wes Eldridge, David Ha, Denny Britz, Jakob Foerster, Julian Togelius, Kyunghyun Cho, and Joan Bruna. Pommerman: A multi-agent playground. arXiv preprint arXiv:1809.07124, 2018.

Moritz Reuss, Maximilian Li, Xiaogang Jia, and Rudolf Lioutikov. Goal-conditioned imitation learning using score-based diffusion policies. arXiv preprint arXiv:2304.02532, 2023.

Marc Rigter, Jun Yamada, and Ingmar Posner. World models via policy-guided trajectory diffusion. arXiv preprint arXiv:2312.08533, 2023.

Jan Robine, Marc Höftmann, Tobias Uelwer, and Stefan Harmeling. Transformer-based world models are happy with 100k interactions. arXiv preprint arXiv:2303.07109, 2023.

Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. pages 10684-10695, 2022a.

Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pages 10684-10695, 2022b.

Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In Medical Image Computing and Computer-Assisted Intervention-MICCAI 2015: 18th International Conference, Munich, Germany, October 5-9, 2015, Proceedings, Part III 18, pages 234-241. Springer, 2015a.

Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. pages 234-241, 2015b.

Arjun Roy, Hongyi Zeng, Jasmeet Bagga, George Porter, and Alex C. Snoeren. Inside the social network's (datacenter) network. In Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication, SIGCOMM '15, page 123-137. ACM, 2015. ISBN 978-1-4503-3542-3. doi: 10.1145/2785956.2787472. URL <http://doi.acm.org/10.1145/2785956.2787472>. event-place: London, United Kingdom.

Tim Salimans and Jonathan Ho. Progressive distillation for fast sampling of diffusion models. arXiv preprint arXiv:2202.00512, 2022.

Tim Salimans, Thomas Mensink, Jonathan Heek, and Emiel Hooeboom. Multistep distillation of diffusion models via moment matching. arXiv preprint arXiv:2406.04103, 2024.

Mikayel Samvelyan, Tabish Rashid, Christian Schroeder de Witt, Gregory Farquhar, Nantas Nardelli, Tim G. J. Rudner, Chia-Man Hung, Philip H. S. Torr, Jakob Foerster, and Shimon Whiteson. The starcraft multi-agent challenge, 2019.

Sand-AI. Magi-1: Autoregressive video generation at scale, 2025. URL https://static.magi.world/static/files/MAGI_1.pdf.

William H Sandholm. Potential games with continuous player sets. Journal of Economic theory, 97(1):81-108, 2001.

Bidipta Sarkar, Aditi Talati, Andy Shih, and Dorsa Sadigh. Pantheonrl: A marl library for dynamic training interactions. In Proceedings of the AAAI Conference on Artificial Intelligence, volume 36, pages 13221-13223, 2022.

- Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. arXiv preprint arXiv:1511.05952, 2015.
- Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, et al. Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 588(7839):604-609, 2020.
- Christoph Schuhmann, Romain Beaumont, Richard Vencu, Cade Gordon, Ross Wightman, Mehdi Cherti, Theo Coombes, Aarush Katta, Clayton Mullis, Mitchell Wortsman, et al. Laion-5b: An open large-scale dataset for training next generation image-text models. *Advances in Neural Information Processing Systems*, 35:25278-25294, 2022.
- John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In *International conference on machine learning*, pages 1889-1897. PMLR, 2015a.
- John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. arXiv preprint arXiv:1506.02438, 2015b.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347, 2017.
- Siddhartha Sen, David Shue, Sunghwan Ihm, and Michael J Freedman. Scalable, optimal flow routing in datacenters via local link balancing. In *Proceedings of the ninth ACM conference on Emerging networking experiments and technologies*, pages 151-162, 2013.
- Jack Serrino, Max Kleiman-Weiner, David C Parkes, and Joshua B Tenenbaum. Finding friend and foe in multi-agent games. arXiv preprint arXiv:1906.02330, 2019.
- Lloyd S Shapley. Stochastic games. *Proceedings of the national academy of sciences*, 39 (10):1095-1100, 1953.
- Shouqian Shi, Ye Yu, Minghao Xie, Xin Li, Xiaozhou Li, Ying Zhang, and Chen Qian. Concurry: A fast and light-weight software cloud load balancer. page 14, 2020.
- David Silver, Guy Lever, Nicolas Heess, Thomas Degris, Daan Wierstra, and Martin Riedmiller. Deterministic policy gradient algorithms. In *International conference on machine learning*, pages 387-395. Pmlr, 2014.
- David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of go with deep neural networks and tree search. *nature*, 529(7587):484, 2016.
- David Silver, Julian Schrittwieser, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, Yutian Chen, Timothy Lillicrap, Fan Hui, Laurent Sifre, George Driessche, Thore Graepel, and Demis Hassabis. Mastering the game of go without human knowledge. *Nature*, 550:354-359, 10 2017. doi: 10.1038/nature24270.

David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharmashan Kumaran, Thore Graepel, et al. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419):1140-1144, 2018.

Uriel Singer, Adam Polyak, Tom Hayes, Xi Yin, Jie An, Songyang Zhang, Qiuyuan Hu, Harry Yang, Oron Blum, Qi Huang, et al. Make-a-video: Text-to-video generation without text-video data. arXiv preprint arXiv:2209.14792, 2022a.

Uriel Singer, Adam Polyak, Eliya Nachmani, Guy Dahan, Eli Shechtman, and Haggai Hachohen. Make-a-video: Text-to-video generation without text-video data. arXiv preprint arXiv:2209.14792, 2022b.

Viswanath Sivakumar, Tim Rocktäschel, Alexander H Miller, Heinrich Kottler, Nantas Nardelli, Mike Rabbat, Joelle Pineau, and Sebastian Riedel. Mvfst-rl: An asynchronous rl framework for congestion control with delayed actions. arXiv preprint arXiv:1910.04054, 2019.

Jascha Sohl-Dickstein, Eric A. Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *International Conference on Machine Learning*, pages 2256-2265, 2015.

Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *International Conference on Learning Representations*, 2021a.

Kiwhan Song, Matthias Böhler, Stefan Bauer, Jiajun Zhu, Sanja Fidler, Karsten Kreis, Amlan Kar, and Marc ' Aurelio Ranzato. History-guided video diffusion. arXiv preprint arXiv:2502.06764, 2025.

Yang Song and Stefano Ermon. Generative modeling by estimating gradients of the data distribution. *Advances in neural information processing systems*, 32, 2019.

Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. arXiv preprint arXiv:2011.13456, 2020a.

Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations*, 2021b.

Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models. arXiv preprint arXiv:2303.01469, 2023.

Yuda Song, Yifei Zhou, Ayush Sekhari, Drew Bagnell, Akshay Krishnamurthy, and Wen Sun. Hybrid rl: Using both offline and online data can make rl efficient. arXiv preprint arXiv:2210.06718, 2022.

Yuhang Song, Andrzej Wojcicki, Thomas Lukasiewicz, Jianyi Wang, Abi Aryan, Zhenghua Xu, Mai Xu, Zihan Ding, and Lianlong Wu. Arena: A general evaluation platform and building toolkit for multi-agent intelligence. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 7253-7260, 2020b.

- Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Ro- former: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568: 127063, 2024.
- Joseph Suarez, Yilun Du, Clare Zhu, Igor Mordatch, and Phillip Isola. The neural mmo platform for massively multiagent research. *arXiv preprint arXiv:2110.07594*, 2021.
- Sainbayar Sukhbaatar, Rob Fergus, et al. Learning multiagent communication with back- propagation. *Advances in neural information processing systems*, 29, 2016.
- Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, et al. Learning to learn at test time: Rnns with expressive hidden states. *arXiv preprint arXiv:2407.04620*, 2024.
- Peter Sunehag, Guy Lever, Audrunas Gruslys, Wojciech Marian Czarnecki, Vinicius Zam- baldi, Max Jaderberg, Marc Lanctot, Nicolas Sonnerat, Joel Z Leibo, Karl Tuyls, et al. Value-decomposition networks for cooperative multi-agent learning. *arXiv preprint arXiv:1706.05296*, 2017.
- Richard S Sutton. Integrated architectures for learning, planning, and reacting based on approximating dynamic programming. In *Machine learning proceedings 1990*, pages 216-224. Elsevier, 1990.
- Richard S Sutton. Dyna, an integrated architecture for learning, planning, and reacting. *ACM Sigart Bulletin*, 2(4):160-163, 1991.
- Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT press, 2018.
- Richard S Sutton, David A McAllester, Satinder P Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. In *Advances in neural information processing systems*, pages 1057-1063, 2000.
- Ming Tan. Multi-agent reinforcement learning: Independent vs. cooperative agents. In *Proceedings of the tenth international conference on machine learning*, pages 330-337, 1993.
- Aether Team, Haoyi Zhu, Yifan Wang, Jianjun Zhou, Wenzheng Chang, Yang Zhou, Zizun Li, Junyi Chen, Chunhua Shen, Jiangmiao Pang, et al. Aether: Geometric- aware unified world modeling. *arXiv preprint arXiv:2503.18945*, 2025.
- HPC-AI Tech. Open-sora 2.0: Training a commercial-level video generation model in \$200k, 2025.
- J Terry, Benjamin Black, Nathaniel Grammel, Mario Jayakumar, Ananth Hari, Ryan Sullivan, Luis S Santos, Clemens Dieffendahl, Caroline Horsch, Rodrigo Perez-Vicente, et al. Pettingzoo: Gym for multi-agent reinforcement learning. *Advances in Neural Information Processing Systems*, 34:15032-15043, 2021a.
- Justin K. Terry, Benjamin Black, Mario Jayakumar, Ananth Hari, Ryan Sullivan, Luis Santos, Clemens Dieffendahl, Niall L. Williams, Yashas Lokesh, Caroline Horsch, and Praveen Ravi. Pettingzoo: Gym for multi-agent reinforcement learning, 2021b.

- Gerald Tesauro et al. Temporal difference learning and td-gammon. *Communications of the ACM*, 38(3):58-68, 1995.
- The Fast Data Project (fd.io). Vector Packet Processing (VPP). <https://wiki.fd.io/view/VPP/>.
- Theo Julienne. GLB: GitHub 's open source load balancer, 2018.
- Sebastian Thrun and Anton Schwartz. Issues in using function approximation for reinforcement learning. In *Proceedings of the 1993 connectionist models summer school*, pages 255-263. Psychology Press, 2014.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timoth é e Lacroix, Baptiste Rozi è re, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- Guido Urdaneta, Guillaume Pierre, and Maarten van Steen. Wikipedia workload analysis for decentralized hosting. *Elsevier Computer Networks*, 53(11):1830-1845, July 2009.
- Dani Valevski, Yaniv Leviathan, Moab Arar, and Shlomi Fruchter. Diffusion models are real-time game engines. *arXiv preprint arXiv:2408.14837*, 2024.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Ł ukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- Siddarth Venkatraman, Shivesh Khaitan, Ruslan Salakhutdinov, and J Andrew Bag- nell. Reasoning with latent diffusion in offline reinforcement learning. *arXiv preprint arXiv:2309.06599*, 2023.
- Ruben Villegas, Jiahui Yang, Sergey Tulyakov, Jan Kautz, and Seungjun Hong. Phenaki: Variable length video generation from open domain textual descriptions. *arXiv preprint arXiv:2210.02399*, 2022.
- Oriol Vinyals, Timo Ewalds, Sergey Bartunov, Petko Georgiev, Alexander Sasha Vezhn- evets, Michelle Yeo, Alireza Makhzani, Heinrich K ü ttler, John Agapiou, Julian Schrit- twieser, et al. Starcraft ii: A new challenge for reinforcement learning. 2017.
- Oriol Vinyals, Igor Babuschkin, Wojciech M Czarnecki, Micha ë l Mathieu, Andrew Dudzik, Junyoung Chung, David H Choi, Richard Powell, Timo Ewalds, Petko Georgiev, et al. Grandmaster level in starcraft ii using multi-agent reinforcement learn- ing. *Nature*, 575(7782):350-354, 2019.
- Vikram Voleti, Alexia Jolicoeur-Martineau, and Chris Pal. Mcvd-masked conditional video diffusion for prediction, generation, and interpolation. *Advances in neural infor- mation processing systems*, 35:23371-23385, 2022.
- John Von Neumann and Oskar Morgenstern. *Theory of games and economic behavior*. Princeton University Press Princeton, NJ, 1945.

Fu-Yun Wang, Zhaoyang Huang, Xiaoyu Shi, Weikang Bian, Guanglu Song, Yu Liu, and Hongsheng Li. Animatelcm: Accelerating the animation of personalized diffusion models and adapters with decoupled consistency learning. arXiv preprint arXiv:2402.00769, 2024a.

Haochen Wang, Xiaodan Du, Jiahao Li, Raymond A Yeh, and Greg Shakhnarovich. Score jacobian chaining: Lifting pretrained 2d diffusion models for 3d generation. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pages 12619-12629, 2023a.

Jiuniu Wang, Hangjie Yuan, Dayou Chen, Yingya Zhang, Xiang Wang, and Shiwei Zhang. Modelscope text-to-video technical report. arXiv preprint arXiv:2308.06571, 2023b.

Tingwu Wang, Xuchan Bao, Ignasi Clavera, Jerrick Hoang, Yeming Wen, Eric Langlois, Shunshi Zhang, Guodong Zhang, Pieter Abbeel, and Jimmy Ba. Benchmarking model-based reinforcement learning, 2019.

Xiang Wang, Shiwei Zhang, Han Zhang, Yu Liu, Yingya Zhang, Changxin Gao, and Nong Sang. Videolcm: Video latent consistency model. arXiv preprint arXiv:2312.09109, 2023c.

Yaohui Wang, Xinyuan Chen, Xin Ma, Shangchen Zhou, Ziqi Huang, Yi Wang, Ceyuan Yang, Yanan He, Jiashuo Yu, Peiqing Yang, et al. Lavie: High-quality video generation with cascaded latent diffusion models. arXiv preprint arXiv:2309.15103, 2023d.

Yi Wang, Yanan He, Yizhuo Li, Kunchang Li, Jiashuo Yu, Xin Ma, Xinhao Li, Guo Chen, Xinyuan Chen, Yaohui Wang, et al. Internvid: A large-scale video-text dataset for multimodal understanding and generation. arXiv preprint arXiv:2307.06942, 2023e.

Yi Wang, Kunchang Li, Xinhao Li, Jiashuo Yu, Yanan He, Guo Chen, Baoqi Pei, Rongkun Zheng, Jilan Xu, Zun Wang, et al. Internvideo2: Scaling video foundation models for multimodal video understanding. arXiv preprint arXiv:2403.15377, 2024b.

Yida Wang et al. Marble: Spatial 3d world model with global-local geometry. arXiv preprint, 2025a.

Yiming Wang et al. Lingen: Long video generation through temporally consistent spatial-semantic correspondence. arXiv preprint arXiv:2503.04522, 2025b.

Zhendong Wang, Jonathan J Hunt, and Mingyuan Zhou. Diffusion policies as an expressive policy class for offline reinforcement learning. arXiv preprint arXiv:2208.06193, 2022.

Zhengyi Wang, Cheng Lu, Yikai Wang, Fan Bao, Chongxuan Li, Hang Su, and Jun Zhu. Prolificdreamer: High-fidelity and diverse text-to-3d generation with variational score distillation. Advances in Neural Information Processing Systems, 36, 2024c.

Zhou Wang, Alan C Bovik, Hamid R Sheikh, and Eero P Simoncelli. Image quality assessment: from error visibility to structural similarity. volume 13, pages 600-612. IEEE, 2004.

Christopher JCH Watkins and Peter Dayan. Q-learning. Machine learning, 8:279-292, 1992.

Manuel Watter, Jost Springenberg, Joshka Boedecker, and Martin Riedmiller. Embed to control: A locally linear latent dynamics model for control from raw images. *Advances in neural information processing systems*, 28, 2015.

Dirk Weissenborn, Oscar Täckström, and Jakob Uszkoreit. Scaling autoregressive video models. *arXiv preprint arXiv:1906.02634*, 2019.

Grady Williams, Andrew Aldrich, and Evangelos Theodorou. Model predictive path integral control using covariance variable importance sampling. *arXiv preprint arXiv:1509.01149*, 2015.

Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8:229-256, 1992.

Xiaoshi Wu, Yiming Hao, Keqiang Sun, Yixiong Chen, Feng Zhu, Rui Zhao, and Hongsheng Li. Human preference score v2: A solid benchmark for evaluating human preferences of text-to-image synthesis. *arXiv preprint arXiv:2306.09341*, 2023a.

Xiaoshi Wu, Keqiang Sun, Feng Zhu, Rui Zhao, and Hongsheng Li. Human preference score: Better aligning text-to-image models with human preference. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 2096-2105, 2023b.

Yifan Wu, George Tucker, and Ofir Nachum. Behavior regularized offline reinforcement learning. *arXiv preprint arXiv:1911.11361*, 2019.

Yuxin Wu and Kaiming He. Group normalization. In *Proceedings of the European conference on computer vision (ECCV)*, pages 3-19, 2018.

Zhengyuan Wu, Shreyas Jain, Jiachen Su, Hao Su, Wei Dong, Xi Chen, Yan Qiang, Zhiqiang Liu, et al. *videogpt: Interactive videogpts are scalable world models*. *arXiv preprint arXiv:2405.15223*, 2024.

Chenjun Xiao, Yifan Wu, Chen Ma, Dale Schuurmans, and Martin Müller. Learning to combat compounding-error in model-based reinforcement learning. *arXiv preprint arXiv:1912.11206*, 2019.

Tianyu Xiao, Dara Bahri, Pawel Lucjan Stanczuk, Duygu Ceylan, Julian McAuley, Arash Vahdat, and Jan Kautz. Tackling the generative learning trilemma with denoising diffusion gans. *arXiv preprint arXiv:2112.07804*, 2021.

Tong Xiao, Peng Liu, and Yi Yang. Dual diffusion models for high-fidelity video generation. *arXiv preprint arXiv:2301.06513*, 2023.

Zeqi Xiao, Yushi Lan, Yifan Zhou, Wenqi Ouyang, Shuai Yang, Yanhong Zeng, and Xingang Pan. Worldmem: Long-term consistent world simulation with memory. *arXiv preprint arXiv:2504.12369*, 2025a.

Zeqi Xiao, Yushi Lan, Yifan Zhou, Wenqi Ouyang, Shuai Yang, Yanhong Zeng, and Xingang Pan. Worldmem: Long-term consistent world simulation with memory, 2025b. URL <https://arxiv.org/abs/2504.12369>.

- Desai Xie, Zhan Xu, Yicong Hong, Hao Tan, Difan Liu, Feng Liu, Arie Kaufman, and Yang Zhou. Progressive autoregressive video diffusion models. arXiv preprint arXiv:2410.08151, 2024a.
- Qiaomin Xie, Yudong Chen, Zhaoran Wang, and Zhuoran Yang. Learning zero-sum simultaneous-move markov games using function approximation and correlated equilibrium. In Conference on Learning Theory, pages 3674-3682. PMLR, 2020.
- Sirui Xie, Zhisheng Xiao, Diederik P Kingma, Tingbo Hou, Ying Nian Wu, Kevin Patrick Murphy, Tim Salimans, Ben Poole, and Ruiqi Gao. Em distillation for one-step diffusion models. arXiv preprint arXiv:2405.16852, 2024b.
- Jiazheng Xu, Xiao Liu, Yuchen Wu, Yuxuan Tong, Qinkai Li, Ming Ding, Jie Tang, and Yuxiao Dong. Imagereward: Learning and evaluating human preferences for text-to-image generation. Advances in Neural Information Processing Systems, 36, 2024.
- Yucheng Xu, Nanbo Li, Arushi Goel, Zijian Guo, Zonghai Yao, Hamidreza Kasaei, Mohammadreza Kasaei, and Zhibin Li. Controllable video generation by learning the underlying dynamical system with neural ode. arXiv preprint arXiv:2303.05323, 2023.
- Yue Xu, Wenjun Xu, Zhi Wang, Jiaru Lin, and Shuguang Cui. Load balancing for ultra-dense networks: A deep reinforcement learning based approach. IEEE Internet of Things Journal, 6(6):9399-9412, Dec 2019. ISSN 2327-4662, 2372-2541. doi: 10.1109/JIOT.2019.2935010. arXiv: 1906.00767.
- Taku Yamagata, Ahmed Khalil, and Raul Santos-Rodriguez. Q-learning decision transformer: Leveraging dynamic programming for conditional sequence modelling in offline rl. In International Conference on Machine Learning, pages 38989-39007. PMLR, 2023.
- Mengjiao Yang, Yilun Du, Kamyar Ghasemipour, Jonathan Tompson, Dale Schuurmans, and Pieter Abbeel. Learning interactive real-world simulators. arXiv preprint arXiv:2310.06114, 2023.
- Yaodong Yang and Jun Wang. An overview of multi-agent reinforcement learning from game theoretical perspective. arXiv preprint arXiv:2011.00583, 2020.
- Zhuoyi Yang, Jiayan Teng, Wendi Zheng, Ming Ding, Shiyu Huang, Jiazheng Xu, Yuanming Yang, Wenyi Hong, Xiaohan Zhang, Guanyu Feng, et al. Cogvideox: Text-to-video diffusion models with an expert transformer. arXiv preprint arXiv:2408.06072, 2024.
- Zhiyuan Yao and Zihan Ding. Learning distributed and fair policies for network load balancing as markov potential game. Advances in Neural Information Processing Systems, 35:28815-28828, 2022.
- Zhiyuan Yao, Zihan Ding, and Thomas Clausen. Reinforced cooperative load balancing in data center. arXiv preprint arXiv:2201.11727, 2022.
- Weirui Ye, Shaohuai Liu, Thanard Kurutach, Pieter Abbeel, and Yang Gao. Mastering atari games with limited data. Advances in Neural Information Processing Systems, 34:25476-25488, 2021.

- Jie Yin et al. Slow-fast architecture for video multi-modal large language models. In CVPR, 2025.
- Tianwei Yin, Michaël Gharbi, Taesung Park, Richard Zhang, Eli Shechtman, Fredo Durand, and William T Freeman. Improved distribution matching distillation for fast image synthesis. arXiv preprint arXiv:2405.14867, 2024a.
- Tianwei Yin, Michaël Gharbi, Richard Zhang, Eli Shechtman, Fredo Durand, William T Freeman, and Taesung Park. One-step diffusion with distribution matching distillation. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pages 6613-6623, 2024b.
- Chao Yu, Akash Velu, Eugene Vinitsky, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of mappo in cooperative, multi-agent games, 2021.
- Chao Yu, Akash Velu, Eugene Vinitsky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of ppo in cooperative multi-agent games. Advances in Neural Information Processing Systems, 35:24611-24624, 2022.
- Fengda Yu et al. Cam: A causal-aware model for video generation with memory. arXiv preprint arXiv:2503.00440, 2025a.
- Jiwen Yu, Yiran Zhong, Jiawei Han, Runsen Cen, et al. Gamefactory: Creating new games with generative interactive videos. arXiv preprint arXiv:2501.08325, 2025b.
- Tianhe Yu, Garrett Thomas, Lantao Yu, Stefano Ermon, James Y Zou, Sergey Levine, Chelsea Finn, and Tengyu Ma. Mopo: Model-based offline policy optimization. Advances in Neural Information Processing Systems, 33:14129-14142, 2020.
- Zishun Yu and Xinhua Zhang. Actor-critic alignment for offline-to-online reinforcement learning. 2023.
- Daochen Zha, Jingru Xie, Wenye Ma, Sheng Zhang, Xiangru Lian, Xia Hu, and Ji Liu. Douzero: Mastering doudizhu with self-play deep reinforcement learning. arXiv preprint arXiv:2106.06135, 2021.
- Chi Zhang et al. Test-time training done right. arXiv preprint arXiv:2505.11320, 2025a.
- Huichu Zhang, Siyuan Feng, Chang Liu, Yaoyao Ding, Yichen Zhu, Zihan Zhou, Weinan Zhang, Yong Yu, Haiming Jin, and Zhenhui Li. Cityflow: A multi-agent reinforcement learning environment for large scale city traffic scenario. In The world wide web conference, pages 3620-3624, 2019.
- Lunjun Zhang, Yuwen Xiong, Ze Yang, Sergio Casas, Rui Hu, and Raquel Urtasun. Learning unsupervised world models for autonomous driving via discrete diffusion. arXiv preprint arXiv:2311.01017, 2023.
- Lvmin Zhang and Maneesh Agrawala. Packing input frame context in next-frame prediction models for video generation. arXiv preprint arXiv:2412.02191, 2024.
- Peiyuan Zhang, Yongqi Chen, Runlong Su, Hangliang Ding, Ion Stoica, Zhenghong Liu, and Hao Zhang. Fast video generation with sliding tile attention. arXiv preprint arXiv:2502.04507, 2025b.

- Richard Zhang, Phillip Isola, Alexei A Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In Proceedings of the IEEE conference on computer vision and pattern recognition, pages 586-595, 2018.
- Zhixing Zhang, Yanyu Li, Yushu Wu, Yanwu Xu, Anil Kag, Ivan Skorokhodov, Willi Menapace, Aliaksandr Siarohin, Junli Cao, Dimitris Metaxas, et al. Sf-v: Single for- ward video generation model. arXiv preprint arXiv:2406.04324, 2024.
- Penghao Zhao, Hailin Zhang, Qinhan Yu, Zhengren Wang, Yunteng Geng, Fang Fu, Ling Yang, Wentao Zhang, and Bin Cui. Retrieval-augmented generation for ai-generated content: A survey. arXiv preprint arXiv:2402.19473, 2024.
- Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, et al. Pytorch fsdp: experiences on scaling fully sharded data parallel. arXiv preprint arXiv:2304.11277, 2023.
- Haoyu Zhen, Qiao Sun, Hongxin Zhang, Junyan Li, Siyuan Zhou, Yilun Du, and Chuang Gan. Tesseract: Learning 4d embodied world models. arXiv preprint arXiv:2504.20995, 2025.
- Lianmin Zheng, Jiacheng Yang, Han Cai, Ming Zhou, Weinan Zhang, Jun Wang, and Yong Yu. Magent: A many-agent reinforcement learning platform for artificial collective intelligence. In Proceedings of the AAAI conference on artificial intelligence, volume 32, 2018.
- Qinqing Zheng, Amy Zhang, and Aditya Grover. Online decision transformer. In inter- national conference on machine learning, pages 27042-27059. PMLR, 2022.
- Qinqing Zheng, Mikael Henaff, Brandon Amos, and Aditya Grover. Semi-supervised offline reinforcement learning with action-free trajectories. In International Conference on Machine Learning, pages 42339-42362. PMLR, 2023a.
- Qinqing Zheng, Matt Le, Neta Shaul, Yaron Lipman, Aditya Grover, and Ricky TQ Chen. Guided flows for generative modeling and decision making. arXiv preprint arXiv:2311.13443, 2023b.
- Zangwei Zheng, Xiangyu Peng, Tianji Yang, Chenhui Shen, Shenggui Li, Hongxin Liu, Yukun Zhou, Tianyi Li, and Yang You. Open-sora: Democratizing efficient video production for all, March 2024. URL <https://github.com/hpcaitech/Open-Sora>.
- Ming Zhou, Jun Luo, Julian Vilella, Yaodong Yang, David Rusu, Jiayu Miao, Weinan Zhang, Montgomery Alban, Iman Fadarar, Zheng Chen, et al. Smarts: Scalable multi- agent reinforcement learning training school for autonomous driving. arXiv preprint arXiv:2010.09776, 2020.
- Ming Zhou, Ziyu Wan, Hanjing Wang, Muning Wen, Runzhe Wu, Ying Wen, Yaodong Yang, Yong Yu, Jun Wang, and Weinan Zhang. Malib: A parallel framework for population-based multi-agent reinforcement learning. Journal of Machine Learning Research, 24(150):1-12, 2023.
- Tinghui Zhou, Richard Tucker, John Flynn, Graham Fyffe, and Noah Snavely. Realestate10k dataset, 2018. <https://google.github.io/realestate10k/>.

Deyao Zhu, Yuhui Wang, Jürgen Schmidhuber, and Mohamed Elhoseiny. Guiding online reinforcement learning with action-free offline pretraining. arXiv preprint arXiv:2301.12876, 2023.

Matthieu Zimmer, Claire Glanois, Umer Siddique, and Paul Weng. Learning fair policies in decentralized cooperative multi-agent reinforcement learning. In International Conference on Machine Learning, pages 12967-12978. PMLR, 2021.